

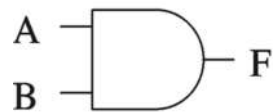
Basic Logic Review



Basic Concepts

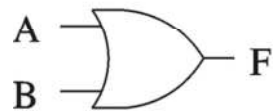
- Simple logic gates
 - AND $\rightarrow$ 0 if one or more inputs is 0
 - OR $\rightarrow$ 1 if one or more inputs is 1
 - NOT
 - NAND = AND + NOT
 - 1 if one or more inputs is 0
 - NOR = OR + NOT
 - 0 if one or more input is 1
 - XOR implements exclusive-OR function
- NAND and NOR gates require fewer transistors than AND and OR in standard CMOS
- Functionality can be expressed by a truth table
 - A truth table lists output for each possible input combination

Basic Logic Gates



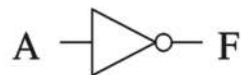
AND gate

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1



OR gate

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

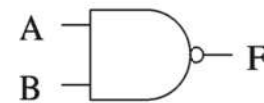


NOT gate

A	F
0	1
1	0

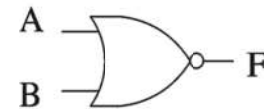
Logic symbol

Truth table



NAND gate

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0



NOR gate

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0



XOR gate

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

Logic symbol

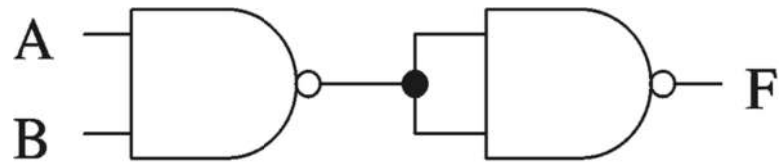
Truth table

Complete Set of Gates

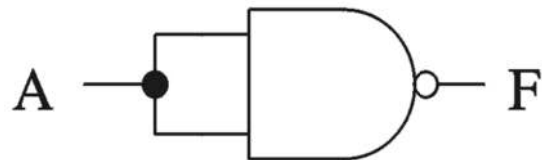
- Complete sets
 - A set of gates is complete
 - if we can implement any logical function using only the type of gates in the set
 - Some example complete sets
 - {AND, OR, NOT} ← Not a minimal complete set
 - {AND, NOT}
 - {OR, NOT}
 - {NAND}
 - {NOR}
 - Minimal complete set
 - A complete set with no redundant elements.

NAND as a Complete Set

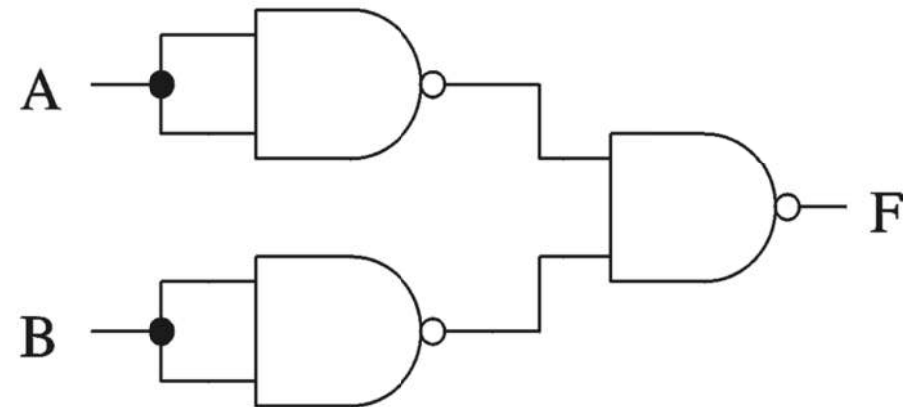
- Proving NAND gate is universal



AND gate



NOT gate



OR gate

Logic Functions

- Logical functions can be expressed in several ways:
 - Truth table
 - Logical expressions
 - Graphical form
 - HDL code
- Example:
 - Majority function
 - Output is one whenever majority of inputs is 1
 - We use 3-input majority function

Logic Functions (cont'd)

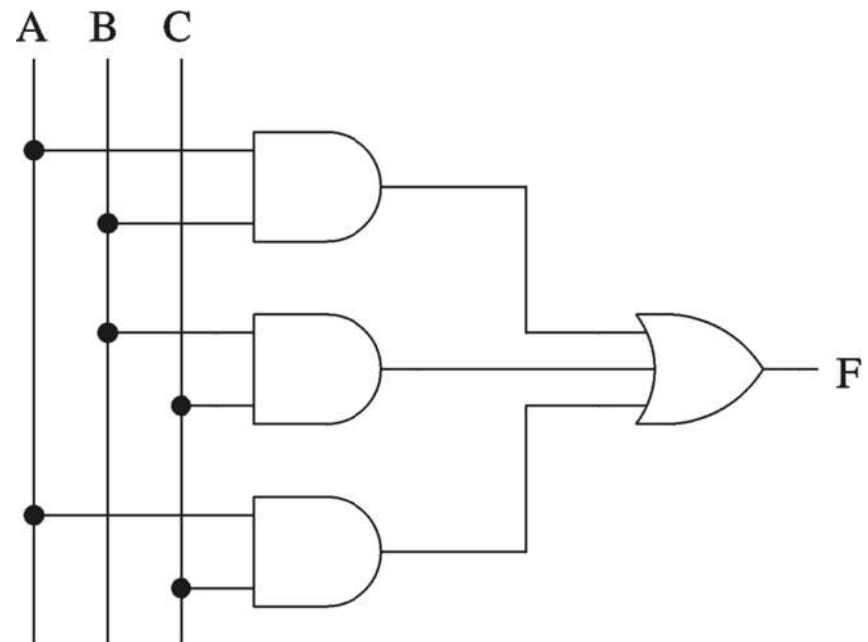
Truth table

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Logical expression form

$$F = A B + B C + A C$$

Graphical schematic form



Boolean Algebra

Boolean identities

Name	AND version	OR version
Identity	$x \cdot 1 = x$	$x + 0 = x$
Complement	$x \cdot x' = 0$	$x + x' = 1$
Commutative	$x \cdot y = y \cdot x$	$x + y = y + x$
Distribution	$x \cdot (y + z) = xy + xz$	$x + (y \cdot z) =$ $(x + y) (x + z)$
Idempotent	$x \cdot x = x$	$x + x = x$
Null	$x \cdot 0 = 0$	$x + 1 = 1$

Boolean Algebra (cont'd)

- Boolean identities (cont'd)

Name	AND version	OR version
Involution	$x = (x')'$	---
Absorption	$x \cdot (x + y) = x$	$x + (x \cdot y) = x$
Associative	$x \cdot (y \cdot z) = (x \cdot y) \cdot z$	$x + (y + z) = (x + y) + z$
de Morgan	$(x \cdot y)' = x' + y'$	$(x + y)' = x' \cdot y'$

(de Morgan's law in particular is very useful)

Majority Function Using Other Gates

- Using NAND gates
 - Get an equivalent expression

$$A B + C D = (A B + C D)''$$

- Using de Morgan's law

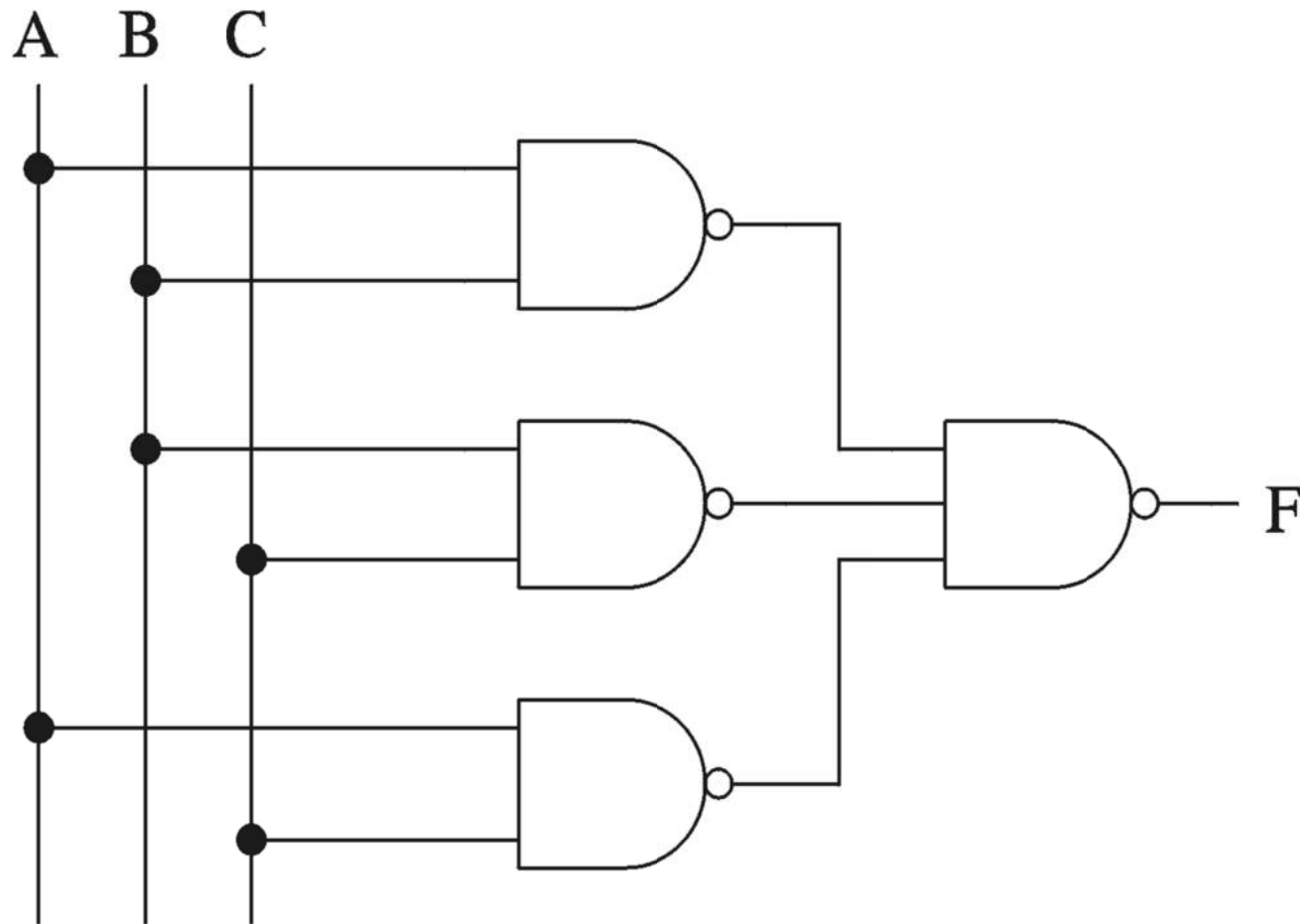
$$A B + C D = ((A B)' \cdot (C D)')'$$

- Can be generalized
 - Example: Majority function

$$A B + B C + A C = ((A B)' \cdot (B C)' \cdot (A C)')'$$

Majority Function Using Other Gates (cont'd)

- Majority function



Karnaugh Maps

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

		$x_1 x_2$			
		00	01	11	10
x_3	0	1	1	1	1
	1	0	0	0	1

$f = \bar{x}_3 + x_1 \bar{x}_2$

An example of three-variable Karnaugh maps

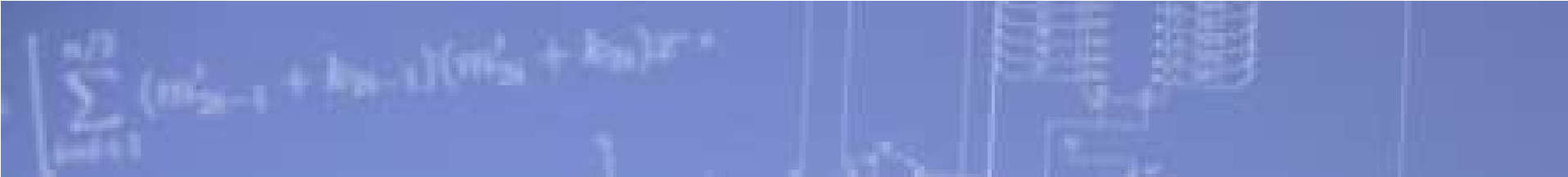
Numbers

Decimal	Binary	Octal	Hexadecimal
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10
17	10001	21	11
18	10010	22	12

Table 5.1 Interpretation of four-bit signed integers.

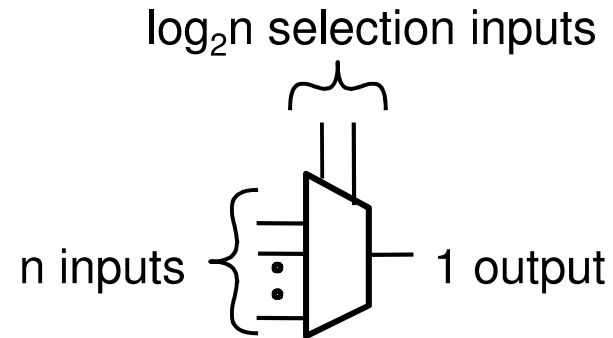
$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0111	+7	+7	+7
0110	+6	+6	+6
0101	+5	+5	+5
0100	+4	+4	+4
0011	+3	+3	+3
0010	+2	+2	+2
0001	+1	+1	+1
0000	+0	+0	+0
1000	-0	-7	-8
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	-0	-1

Numbers in different systems



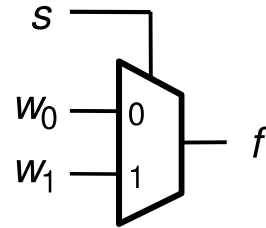
Combinational Logic Building Blocks

Multiplexers



- multiplexer
 - n binary inputs (binary input = 1-bit input)
 - $\log_2 n$ binary selection inputs
 - 1 binary output
 - Function: one of n inputs is placed onto output
 - Called **n-to-1** multiplexer

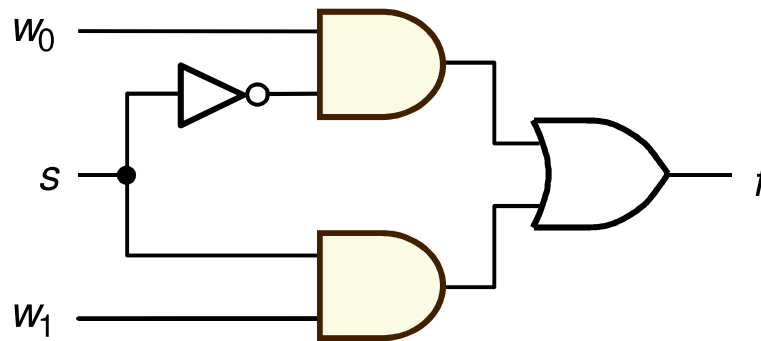
2-to-1 Multiplexer



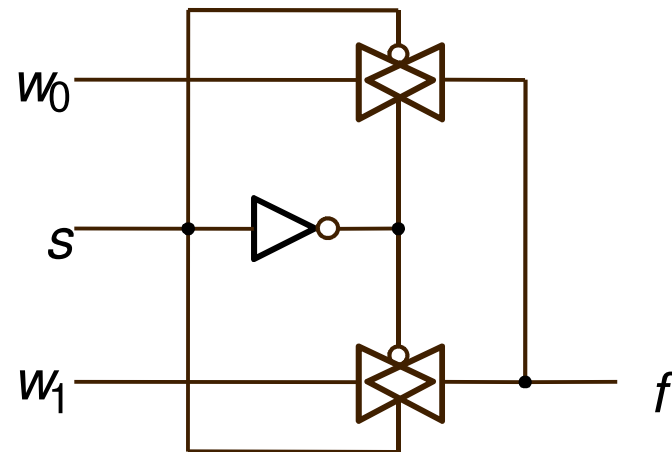
(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table

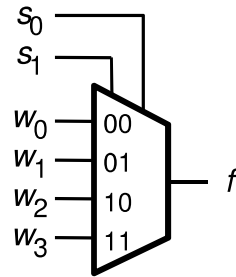


(c) Sum-of-products circuit



(d) Circuit with transmission gates

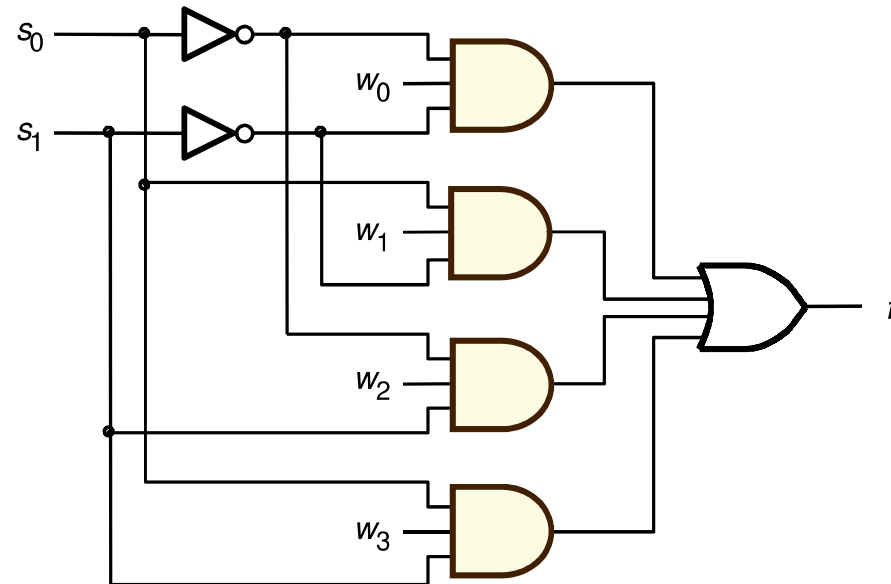
4-to-1 Multiplexer



(a) Graphic symbol

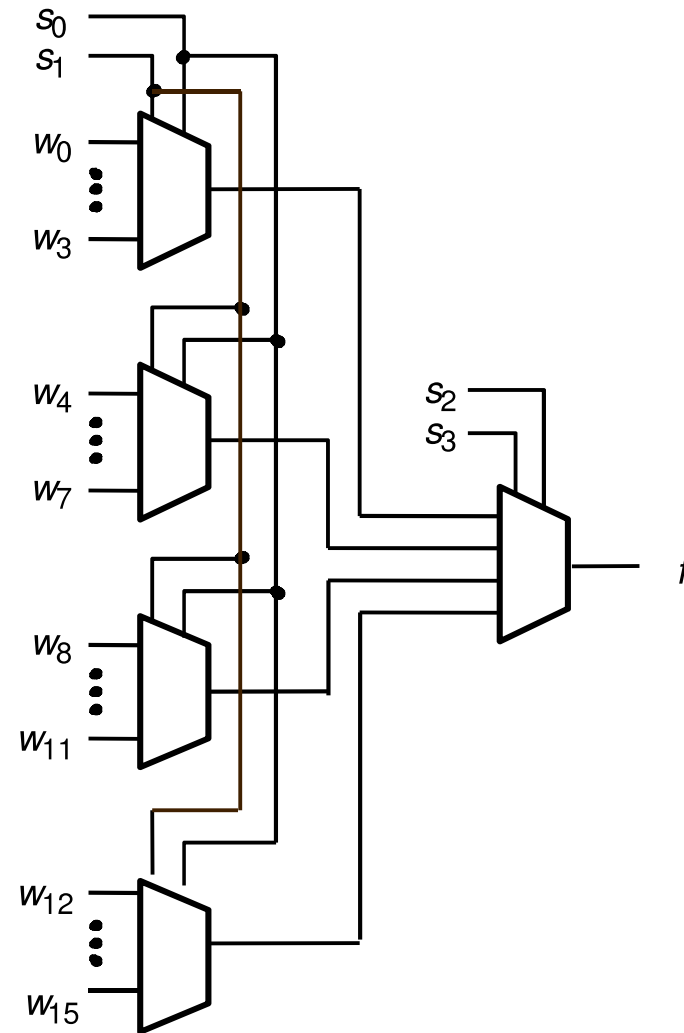
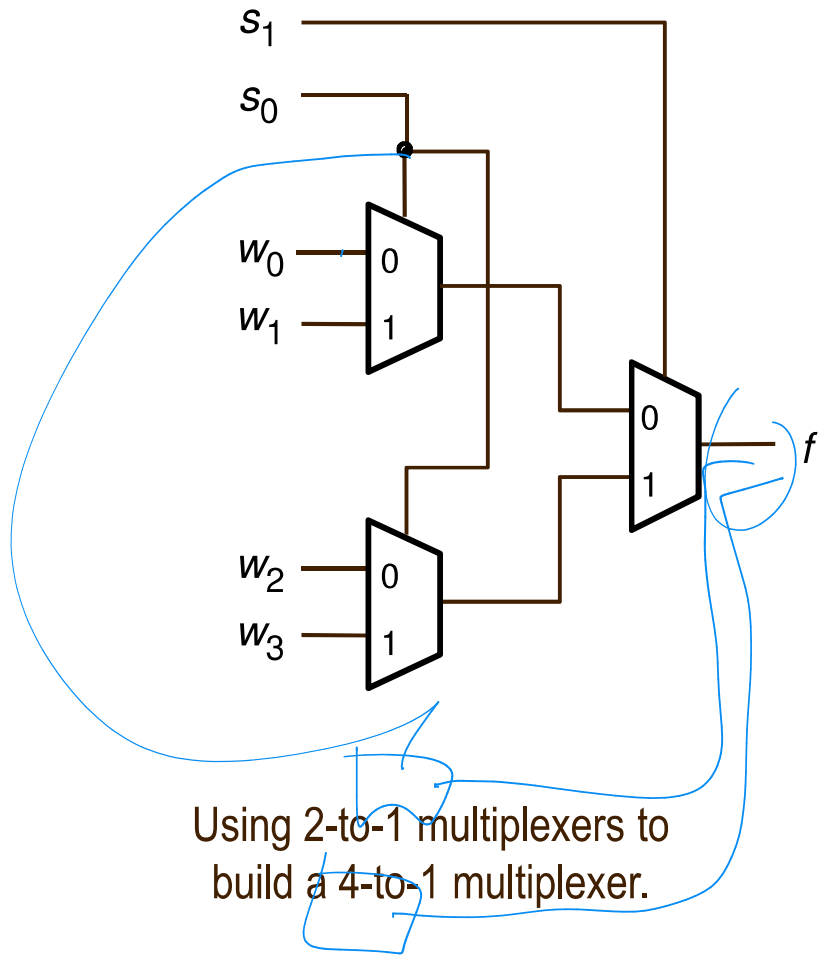
s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table



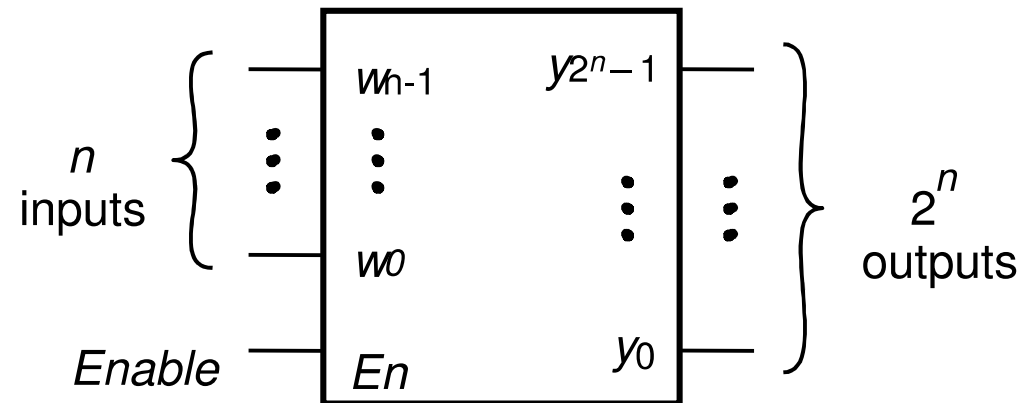
(c) Circuit

Multiplexer



A 16-to-1 multiplexer.

Decoders

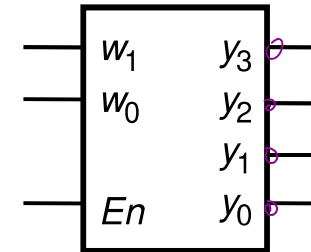


- Decoder
 - n binary inputs
 - 2^n binary outputs
 - Function: decode encoded information
 - If enable=1, one output is asserted high, the other outputs are asserted low
 - If enable=0, all outputs asserted low
 - Often, enable pin is not needed (i.e. the decoder is always enabled)
 - Called **n-to- 2^n** decoder
 - Can consider n binary inputs as a single n -bit input
 - Can consider 2^n binary outputs as a single 2^n -bit output
 - Decoders are often used for RAM/ROM addressing

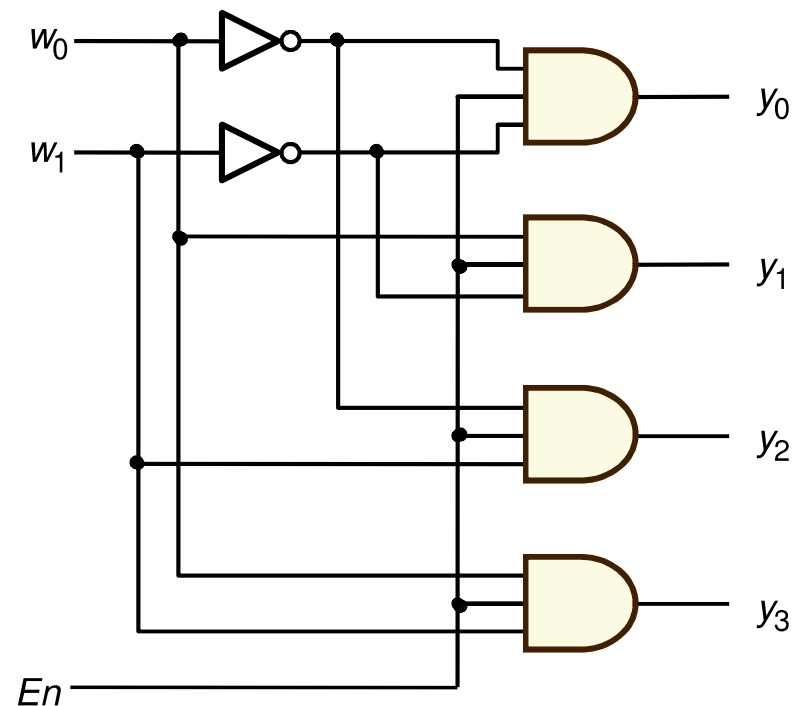
2-to-4 Decoder

En	w_1	w_0	y_3	y_2	y_1	y_0
1	0	0	0	0	1	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	-	-	0	0	0	0

(a) Truth table

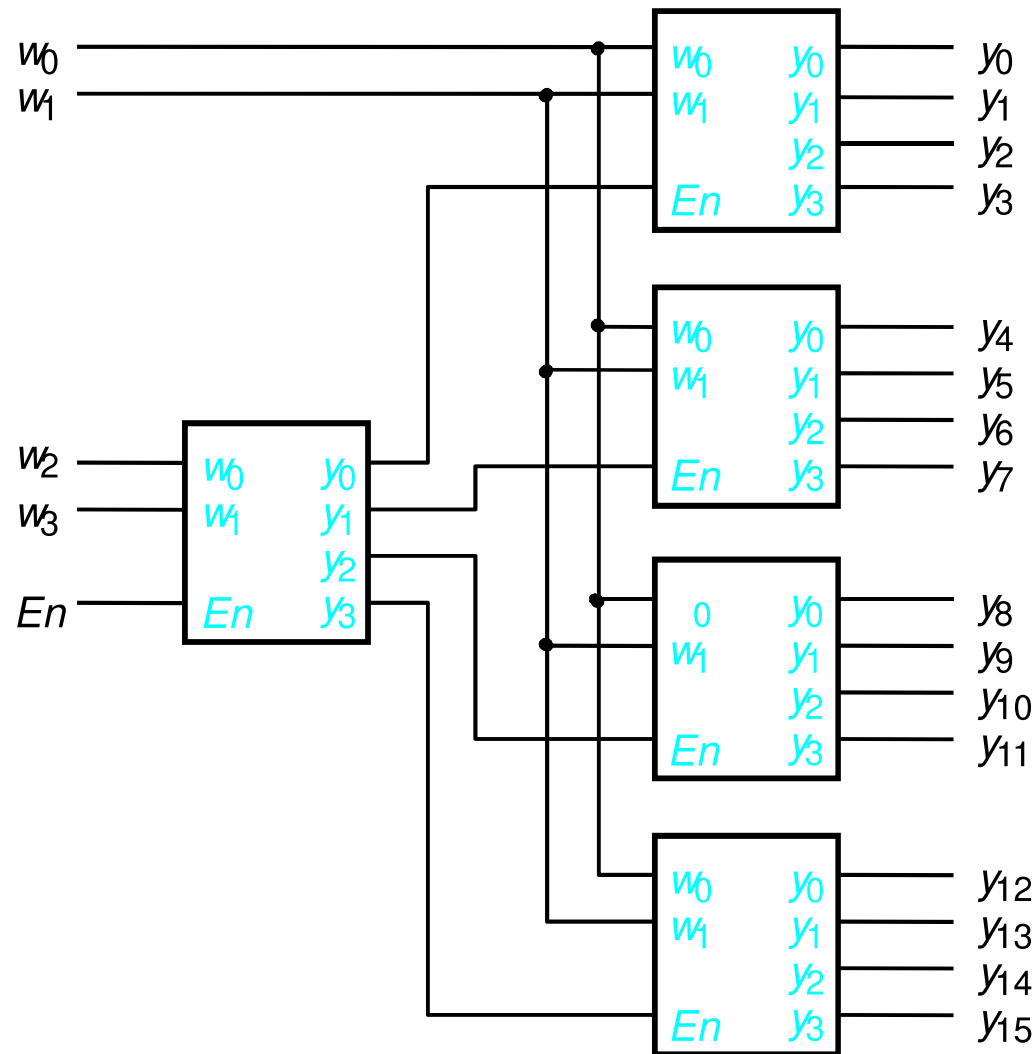


(b) Graphical symbol

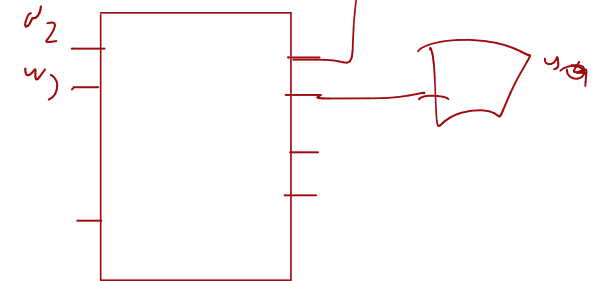


(c) Logic circuit

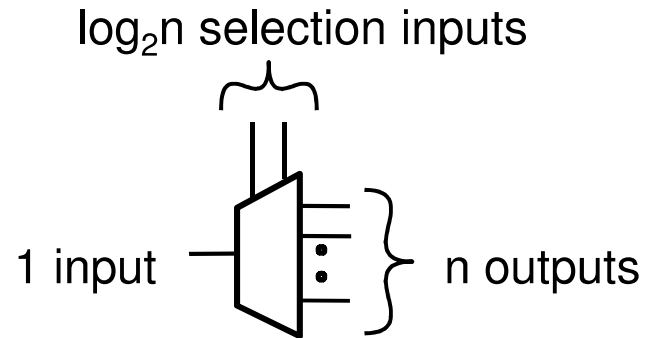
Decoders



A 4-to-16 decoder built using a decoder tree



Demultiplexers

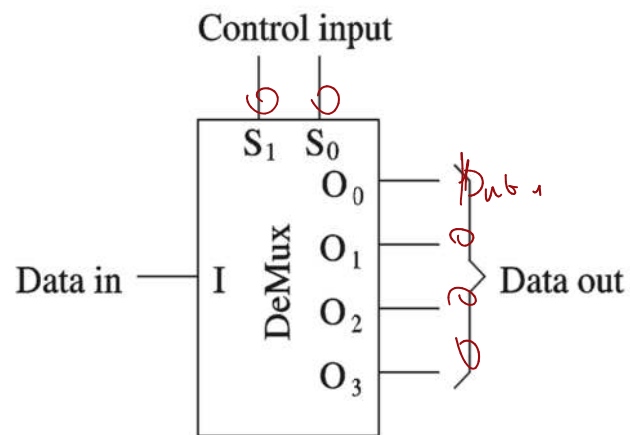


- Demultiplexer
 - 1 binary input
 - n binary outputs
 - $\log_2 n$ binary selection inputs
 - Function: places input onto one of n outputs, with the remaining outputs asserted low
 - Called **1-to-n** demultiplexer
- Closely related to decoder
 - Can build 1-to-n demultiplexer from $\log_2 n$ -to-n decoder by using the decoder's enable signal as the demultiplexer's input signal, and using decoder's input signals as the demultiplexer's selection input signals.

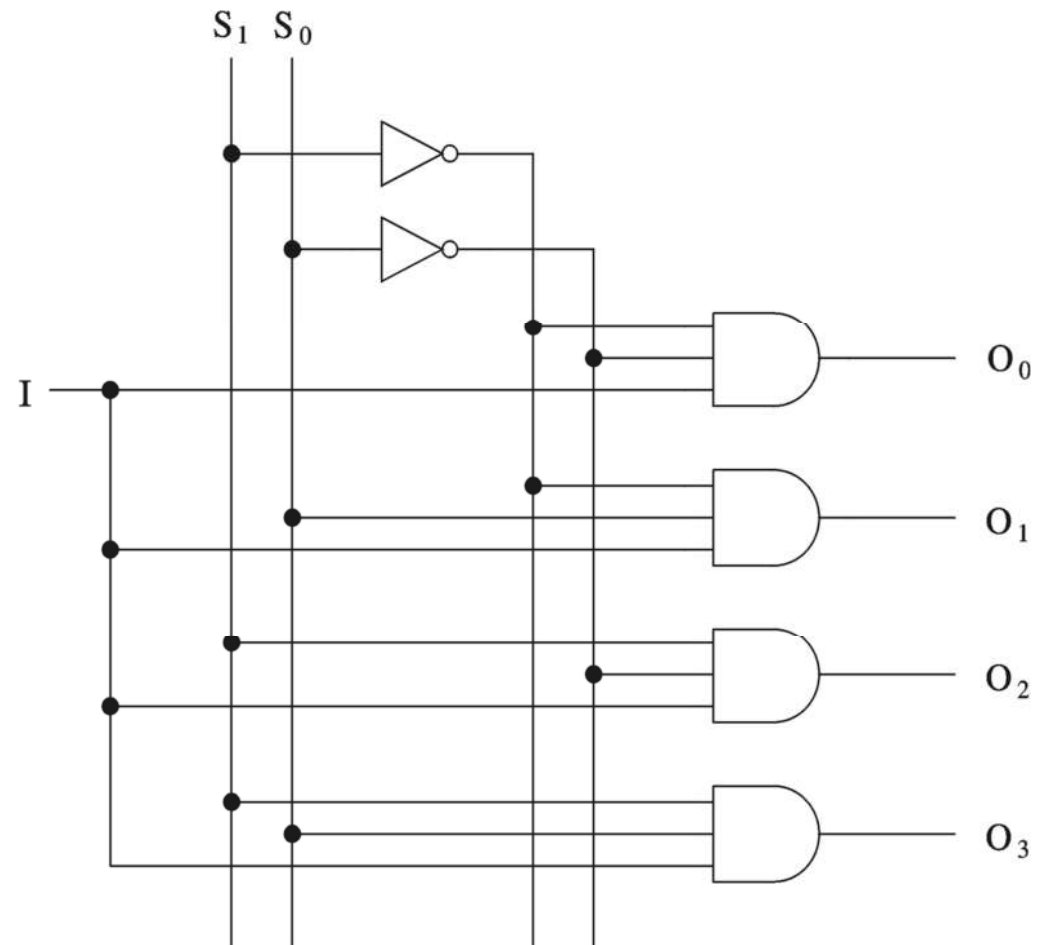
1-to-4 Demultiplexer

S_1	S_0	Q_3	Q_2	Q_1	Q_0
1	0	3	2	1	0
0	0	0	0	0	A
0	1	0	0	A	0
1	0	0	A	0	0
1	1	A	0	0	0
-	-	0	0	0	0

(a) Truth table

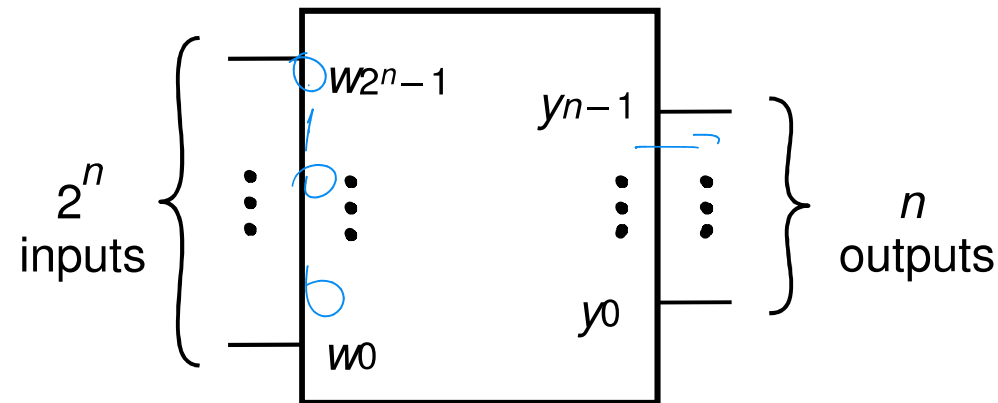


(b) Graphic symbol



(c) Circuit

Encoders



1 0 0 0

0 0 0 1
 0 0 0 0
 0 1 0 0

- Encoder
 - 2^n binary inputs
 - n binary outputs
 - Function: encodes information into an n -bit code
 - Called **2^n -to- n** encoder
 - Can consider 2^n binary inputs as a single 2^n -bit input
 - Can consider n binary output as a single n -bit output
- Encoders only work when **exactly one** binary input is equal to 1

4-to-2 Encoder

$$y_0 = f(w_3, w_2, w_1, w_0)$$

$$y_1 = f(w_3, w_2, w_1, w_0)$$

$y_1 = w_2 + w_3$

y_1

$w_3 \backslash w_2 w_1 w_0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

w_2

w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

w_2

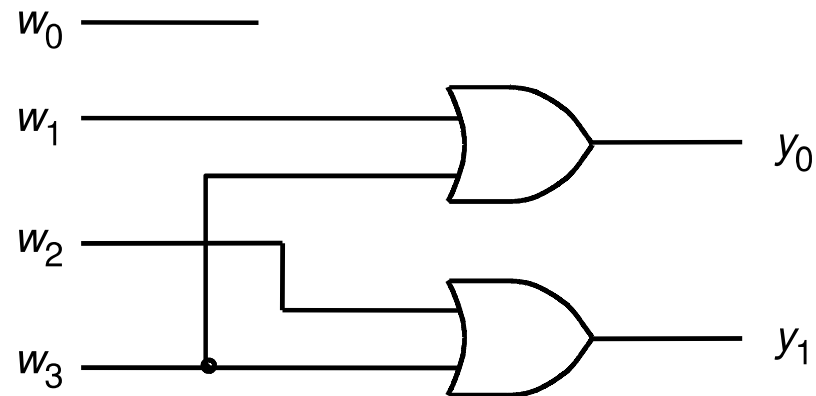
(a) Truth table

$y_0 = w_1 + w_3$

y_0

$w_3 \backslash w_2 w_1 w_0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

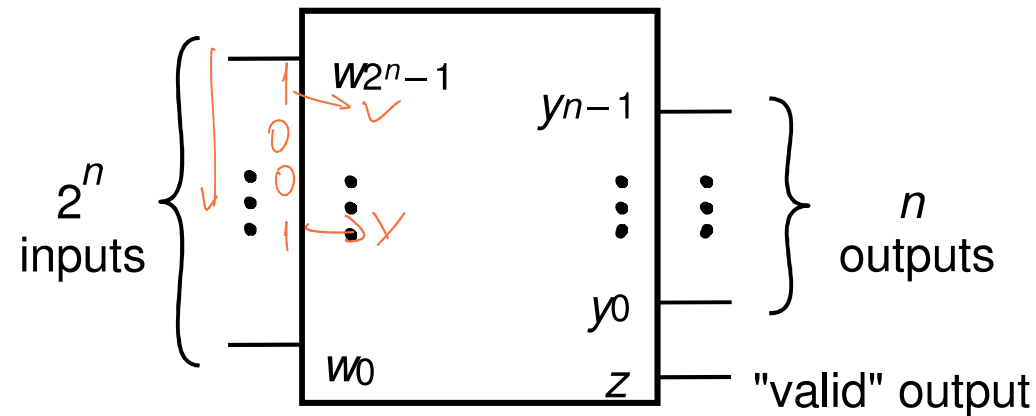
w_1



(b) Circuit

Priority Encoders

en yüksek
ilke 1: anından başlangıç
çalıştırır



1

- Priority Encoder
 - 2^n binary inputs
 - n binary outputs
 - 1 binary "valid" output
 - Function: encodes information into an n -bit code based on priority of inputs
 - Called **2^n -to- n** priority encoder
- Priority encoder allows for multiple inputs to have a value of '1', as it encodes the input with the highest priority (MSB = highest priority, LSB = lowest priority)
 - "valid" output indicates when priority encoder output is valid
 - Priority encoder is more common than an encoder

↓ dot curl

Single-Bit Adders

- Half-adder
 - Adds two binary (i.e. 1-bit) inputs A and B
 - Produces a *sum* and *carryout*
 - Problem: Cannot use it alone to build larger adders
- Full-adder
 - Adds three binary (i.e. 1-bit) inputs A , B , and *carryin*
 - Like half-adder, produces a *sum* and *carryout*
 - Allows building M -bit adders ($M > 1$)
 - Simple technique
 - Connect C_{out} of one adder to C_{in} of the next
 - These are called *ripple-carry adders*
 - Shown in next section

Single-Bit Adders (cont'd)

C_{out}

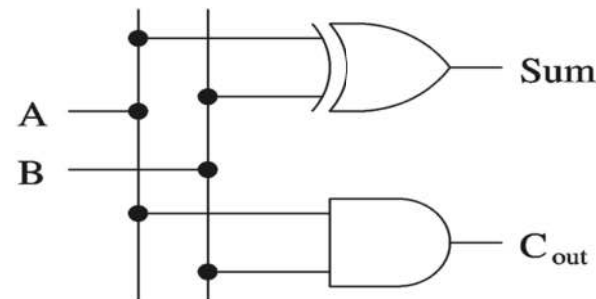
<i>A</i>	0	1
0	0	1
1	1	0

C_{in}

<i>A</i>	0	0	1	1
0	0	1	0	1
1	1	0	1	0

$\rightarrow A \oplus B \oplus C$

A	B	Sum	C _{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



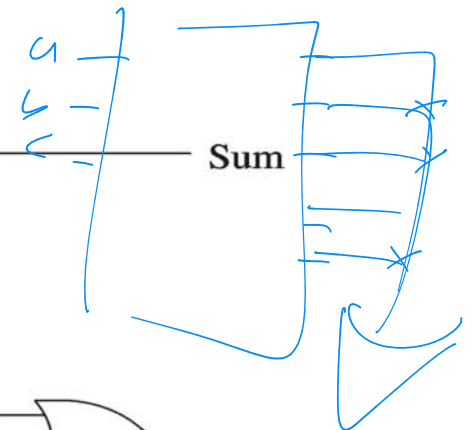
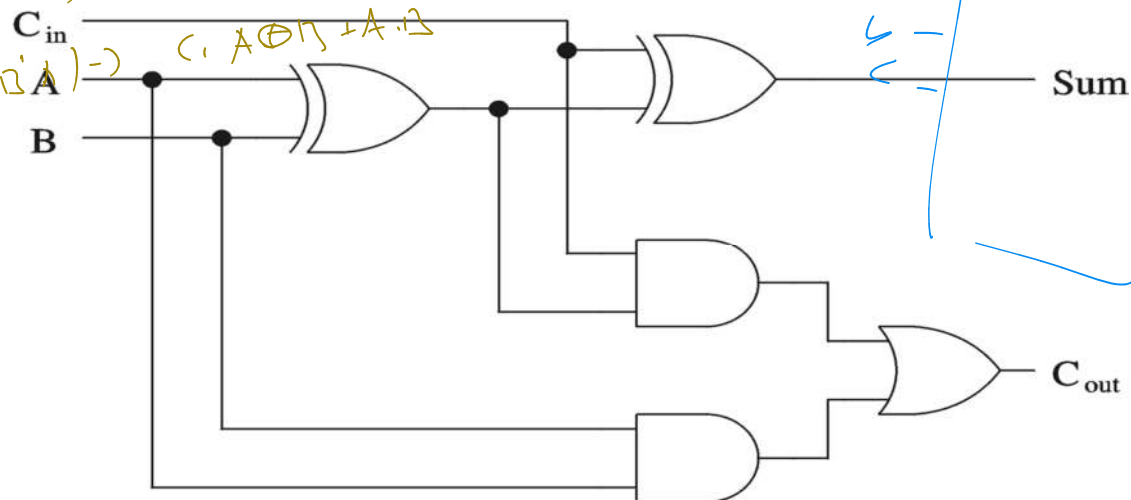
$A \cdot B$
 $A \oplus B$

$a'b'c'$
 $a'b'c$

(a) Half-adder truth table and implementation

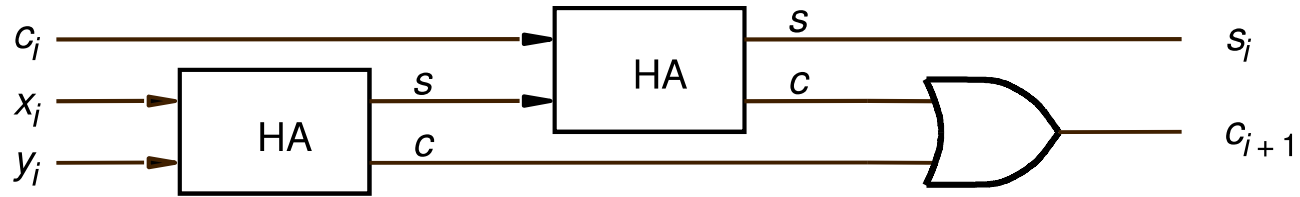
A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$A' \cdot B \cdot C + A \cdot B' \cdot C + A \cdot B \cdot C' + A \cdot B \cdot C$
 $C(A' \cdot B + B' \cdot A) \rightarrow C(A \oplus B)$

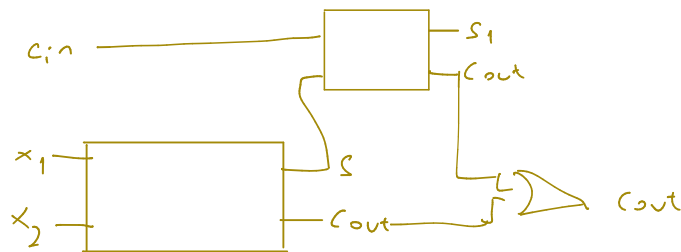


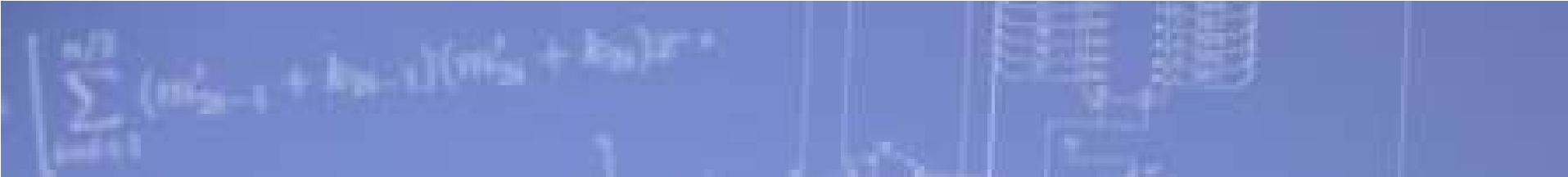
(b) Full-adder truth table and implementation

Adders



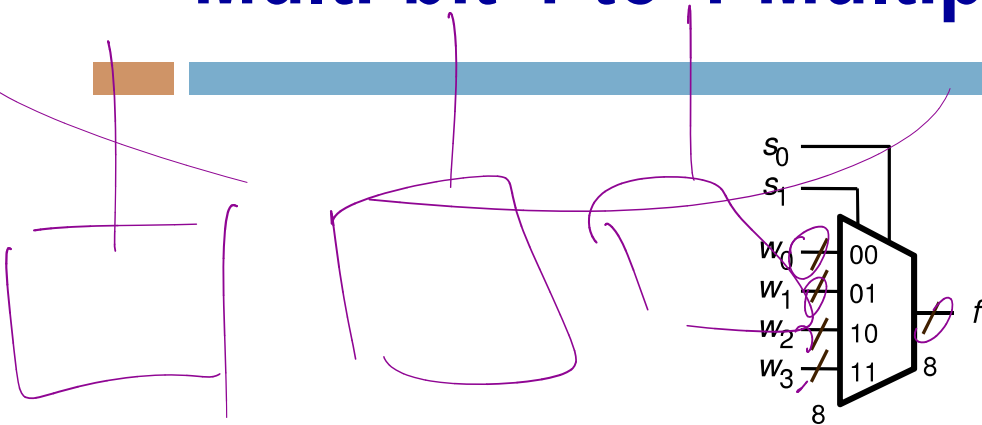
A decomposed implementation of the full-adder circuit





Multi-Bit Combinational Logic Building Blocks

Multi-bit 4-to-1 Multiplexer

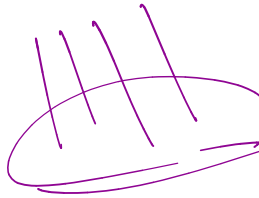


(a) Graphic symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table

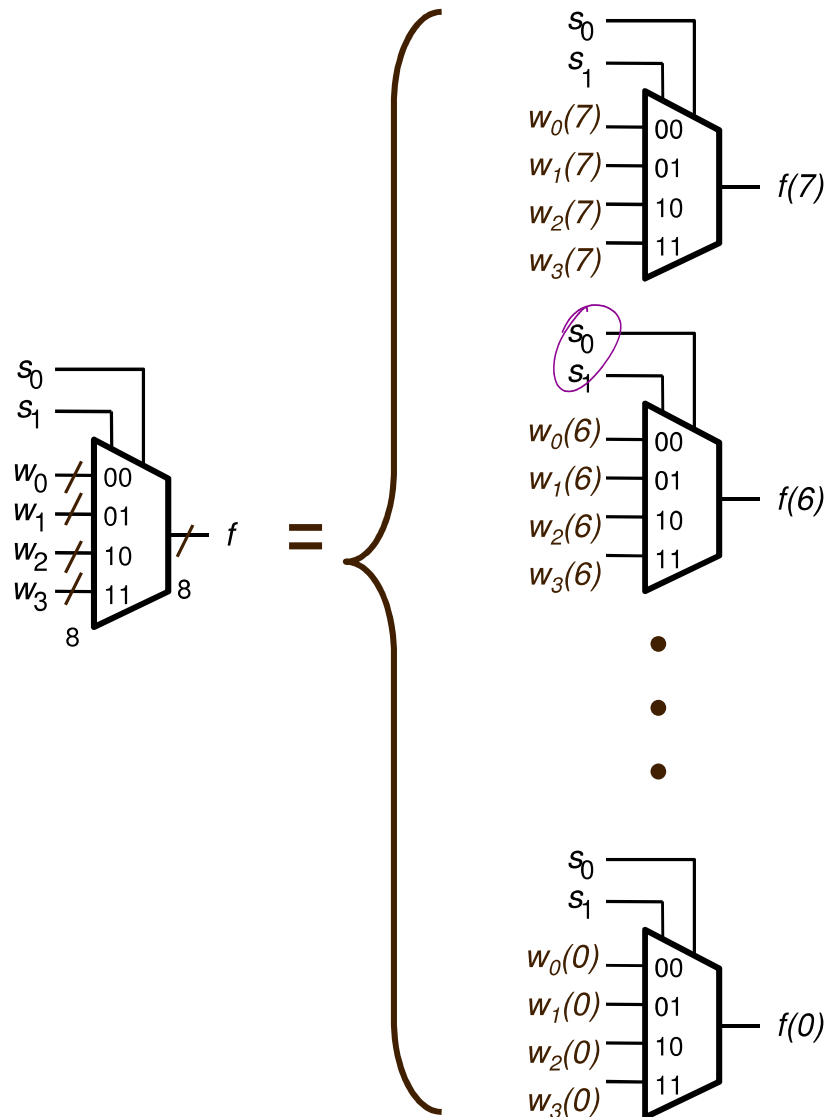
Handwritten notes: $f \rightarrow \text{bus}$ and a set of horizontal lines enclosed in curly braces.



- When drawing schematics, can draw **multi-bit** multiplexers
- Example: 4-to-1 (8 bit) multiplexer
 - 4 inputs (each 8 bits)
 - 1 output (8 bits)
 - 2 selection bits
- Can also have multi-bit 2-to-1 muxes, 16-to-1 muxes, etc.

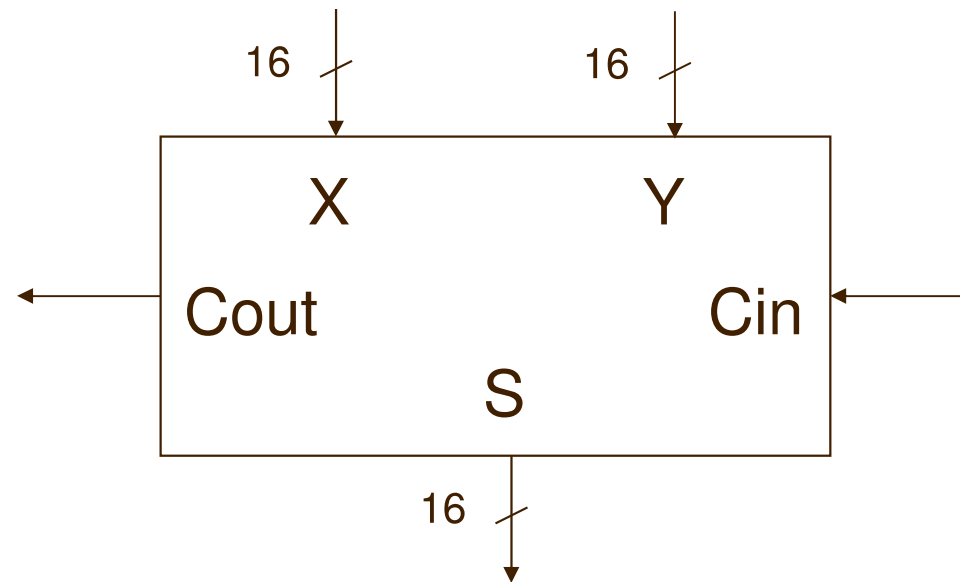


4-to-1 (8-bit) Multiplexer



A 4-to-1 (8-bit) multiplexer is composed of eight 4-to-1 (1-bit) multiplexers

16-bit Unsigned Adder

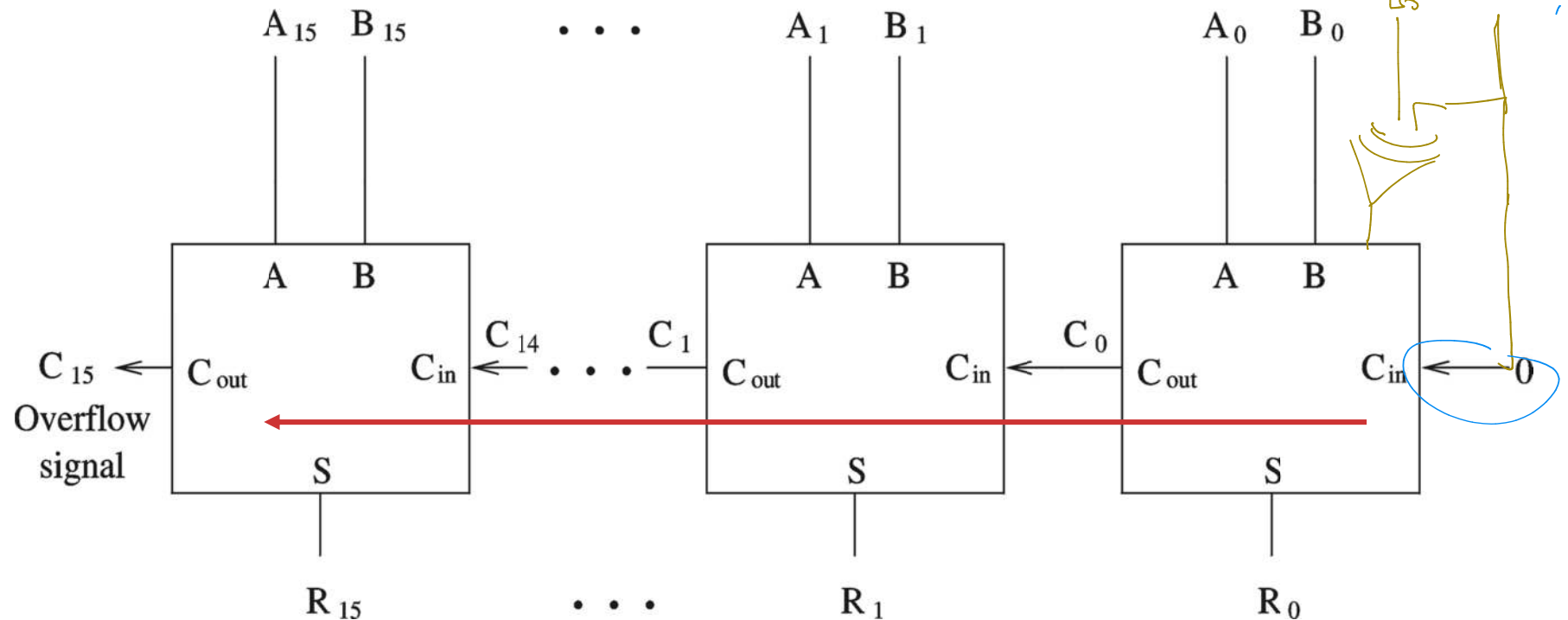


Multi-Bit Ripple-Carry Adder

A 16-bit ripple-carry adder is composed of 16 (1-bit) full adders

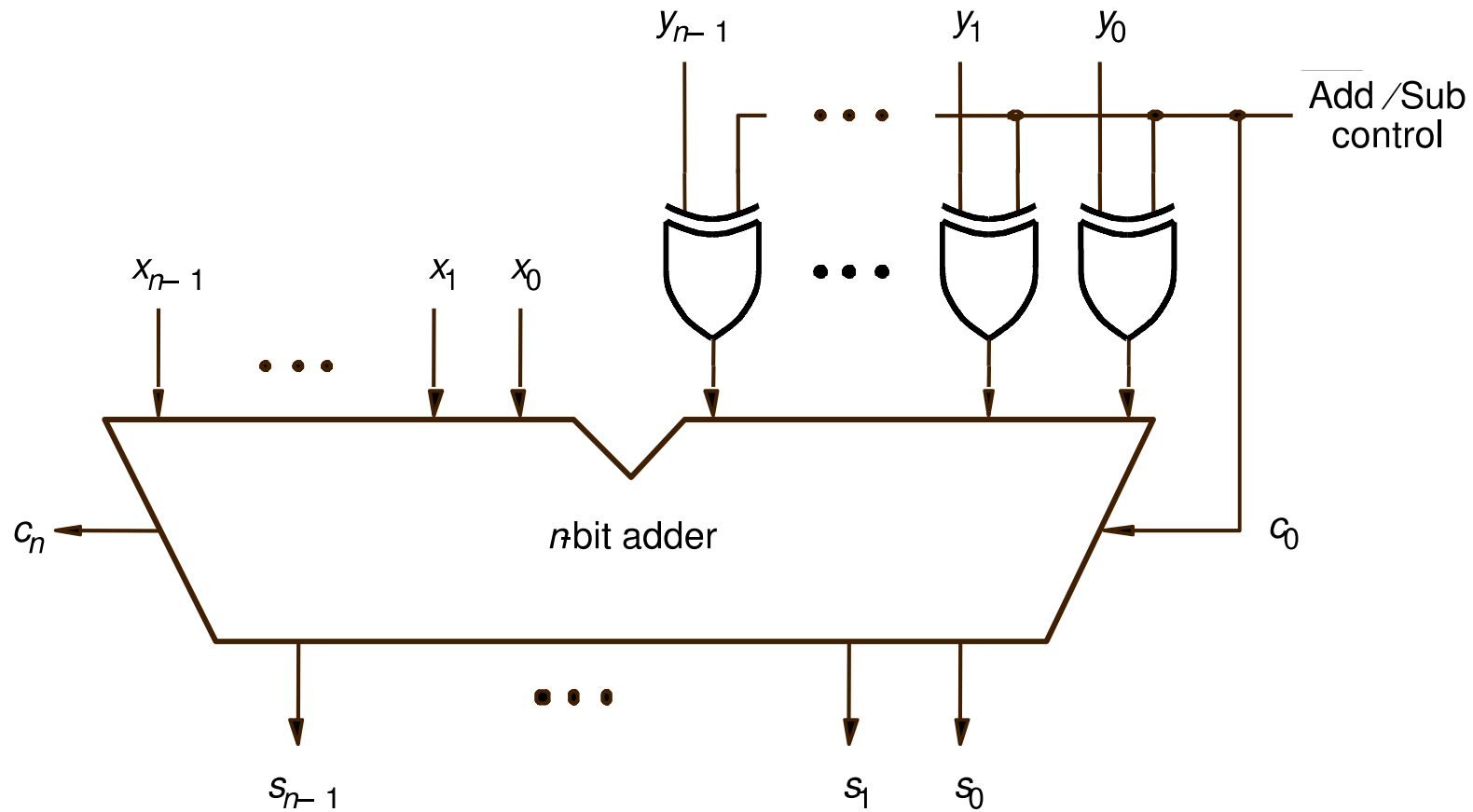
Inputs: 16-bit A, 16-bit B, 1-bit carryin (set to zero in the figure below)

Outputs: 16-bit sum R, 1-bit overflow



Called a ripple-carry adder because carry ripples from one full-adder to the next.
Critical path is 16 full-adders.

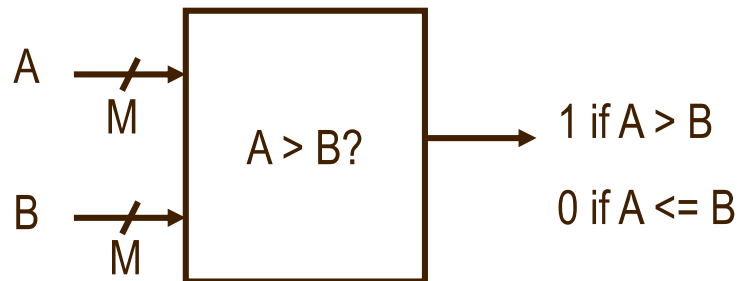
Adder/subtractor unit



Comparator

- Used to compare two M-bit numbers and produce a flag ($M > 1$)

- Inputs: M-bit input A, M-bit input B
- Output: 1-bit output flag
 - 1 indicates condition is met
 - 0 indicates condition is not met
- Can compare: $>$, $\geq$, $<$, $\leq$, $=$, etc.



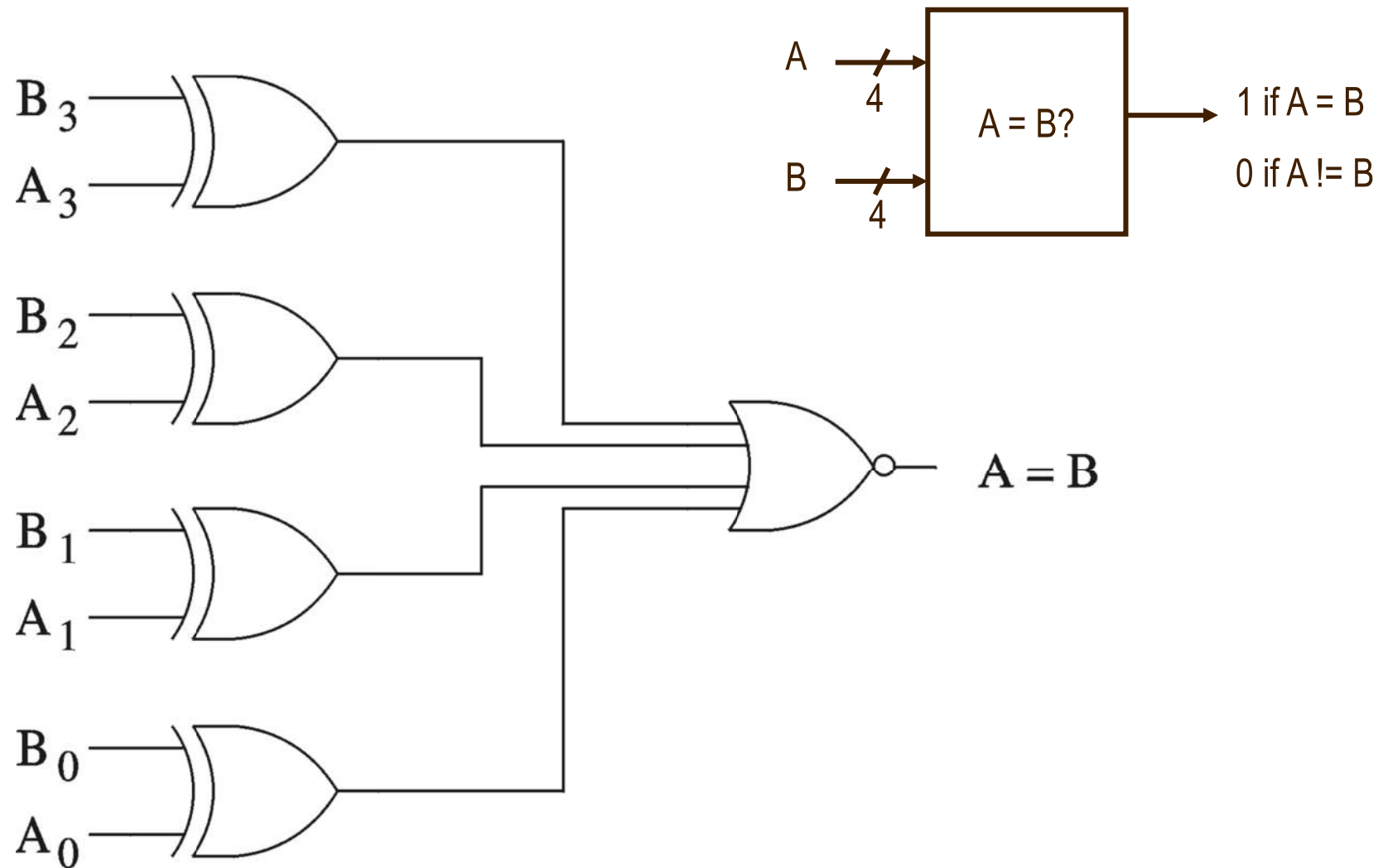
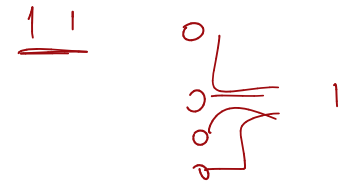
Handwritten truth table for a 4-bit comparator (A > B):

A ₃ A ₂ A ₁ A ₀	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0010	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0011	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1010	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1011	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

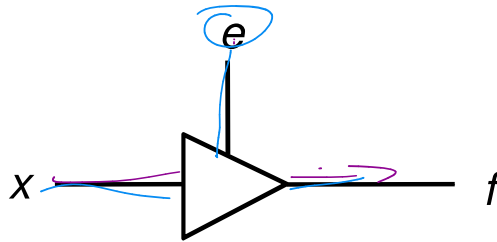
Handwritten truth table for a 4-bit comparator (A < B):

A ₃ A ₂ A ₁ A ₀	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0010	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0011	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1010	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1011	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

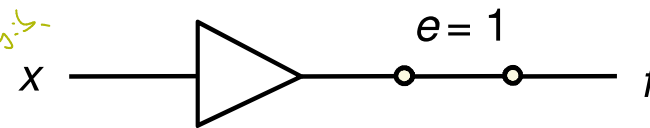
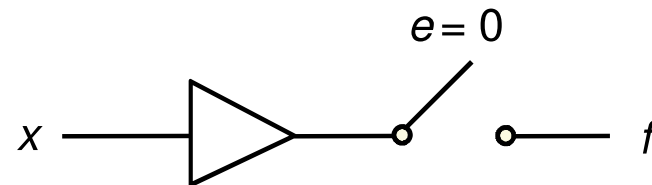
Example: 4-bit comparator (A = B)



Tri-state Buffer



(a) A tri-state buffer



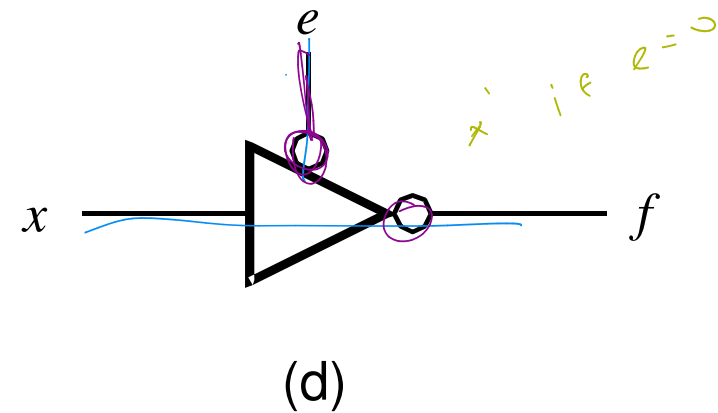
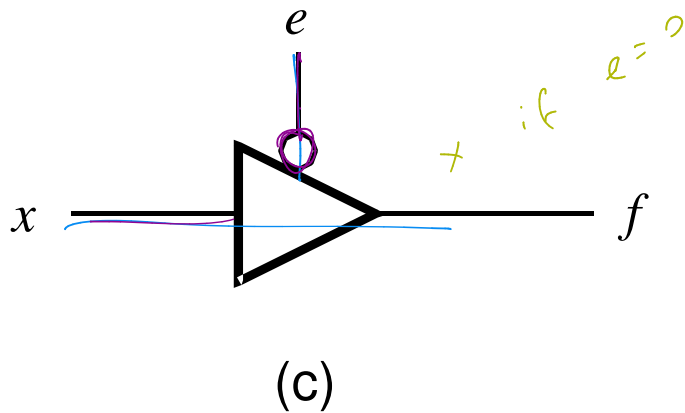
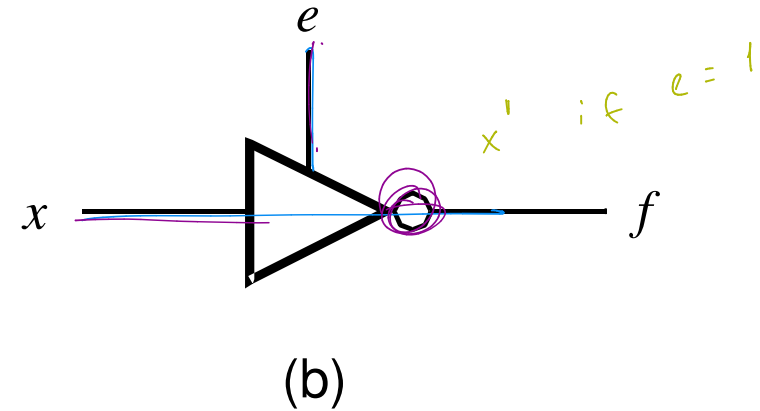
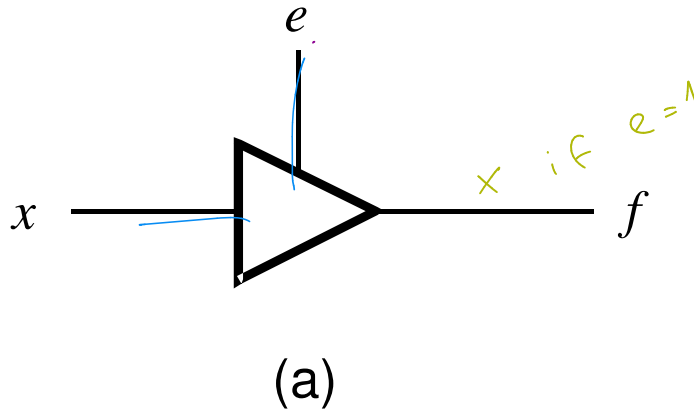
(b) Equivalent circuit

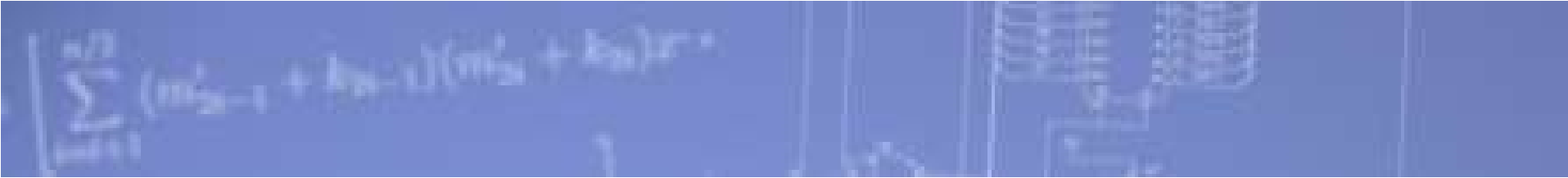
e	x	f
0	0	Z
0	1	Z
1	0	0
1	1	1

*güçsüz empedans
açık devre gib-*

(c) Truth table

Four types of Tri-state Buffers





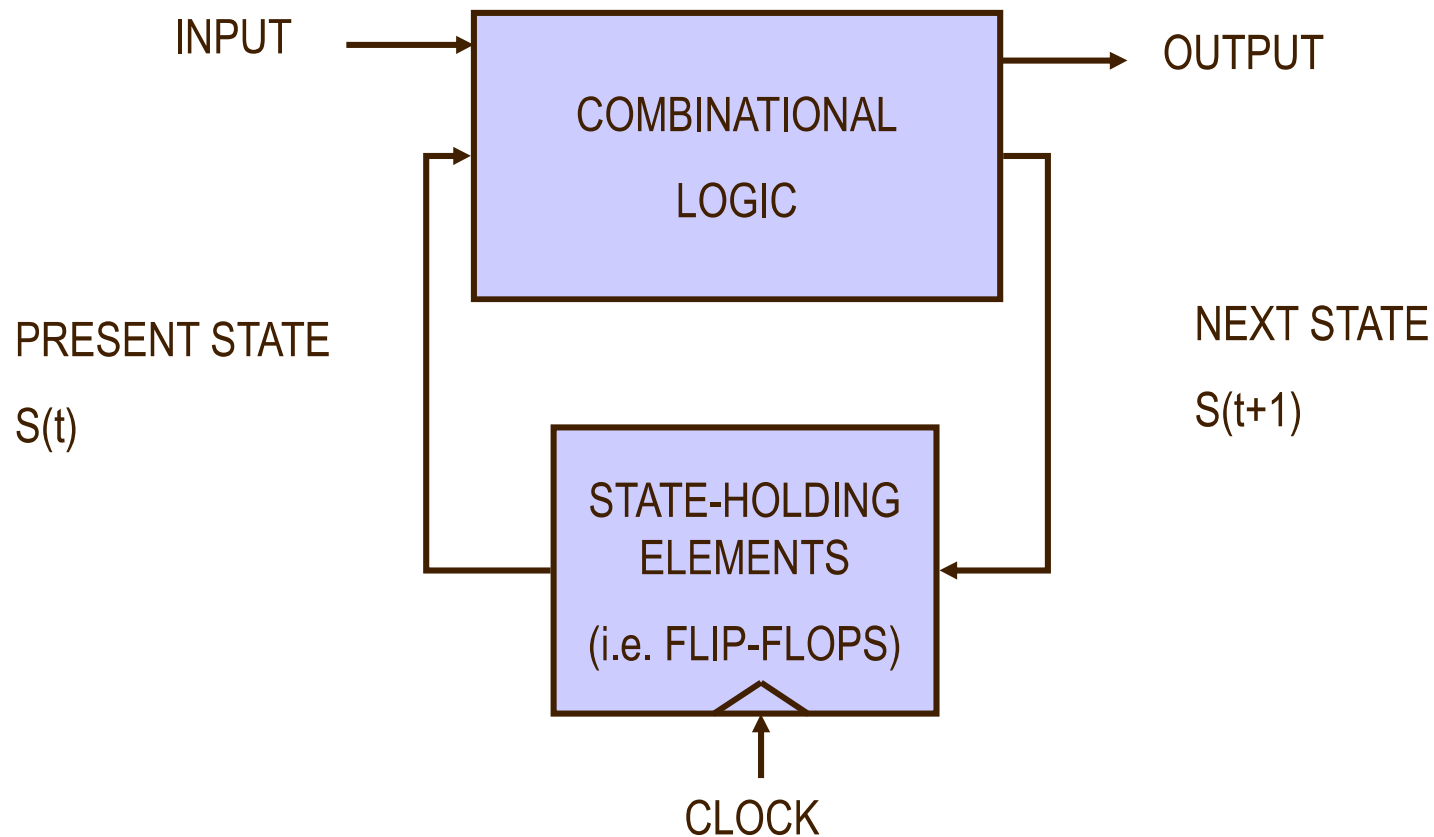
Sequential Logic Building Blocks

Introduction to Sequential Logic

- Output depends on current as well as past inputs
 - Depends on the history
 - Have “memory” property
- Sequential circuit consists of
 - Combinational circuit
 - Feedback circuit
- Past input is encoded into a set of state variables
 - Uses feedback (to feed the state variables)
 - Simple feedback
 - Uses flip flops

Introduction (cont'd)

Main components of a typical synchronous sequential circuit
(synchronous = uses a clock to keep circuits in lock step)



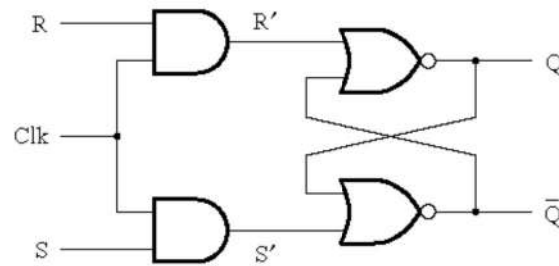
State-Holding Memory Elements

- Latch versus Flip Flop
 - Latches are level-sensitive: whenever clock is high, latch is transparent
 - Flip-flops are edge-sensitive: data passes through (i.e. data is sampled) only on a rising (or falling) edge of the clock
 - Latches cheaper to implement than flip-flops
 - Flip-flops are easier to design with than latches
- In this course, primarily use D flip-flops

Latches and Flip-Flops

→ *seviye* *tek:kleme*

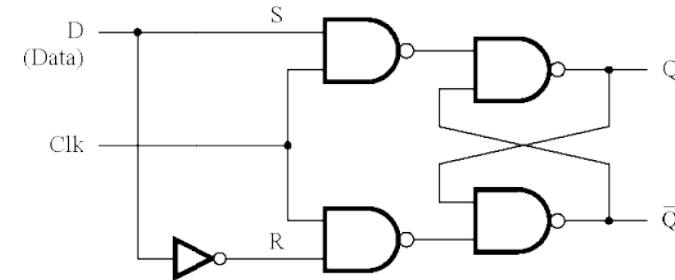
→ *kenar* *tek:kleme*



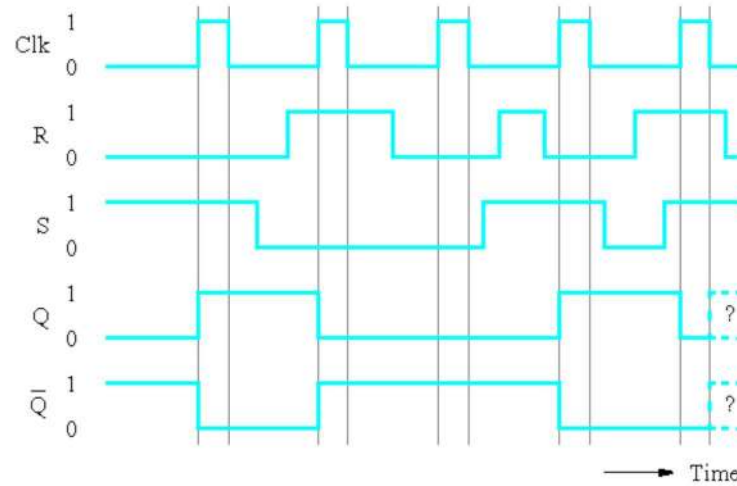
(a) Circuit

Clk	S	R	$Q(t+1)$
0	x	x	$Q(t)$ (no change)
1	0	0	$Q(t)$ (no change)
1	0	1	0
1	1	0	1
1	1	1	x

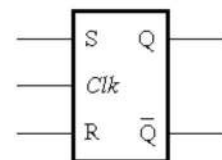
(b) Characteristic table



(a) Circuit



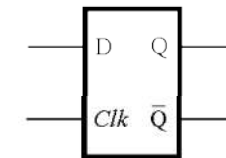
(c) Timing diagram



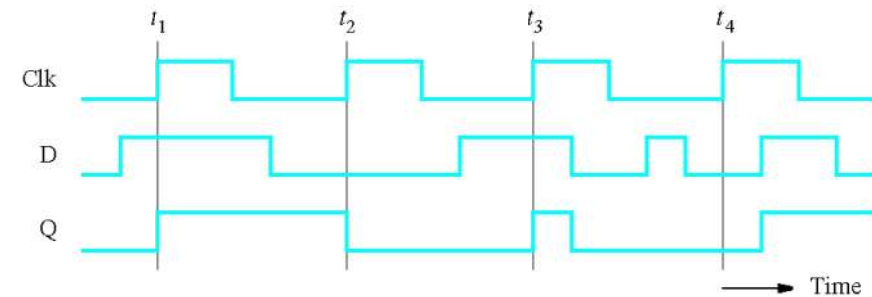
(d) Graphical symbol

Clk	D	$Q(t+1)$
0	x	$Q(t)$
1	0	0
1	1	1

(b) Characteristic table

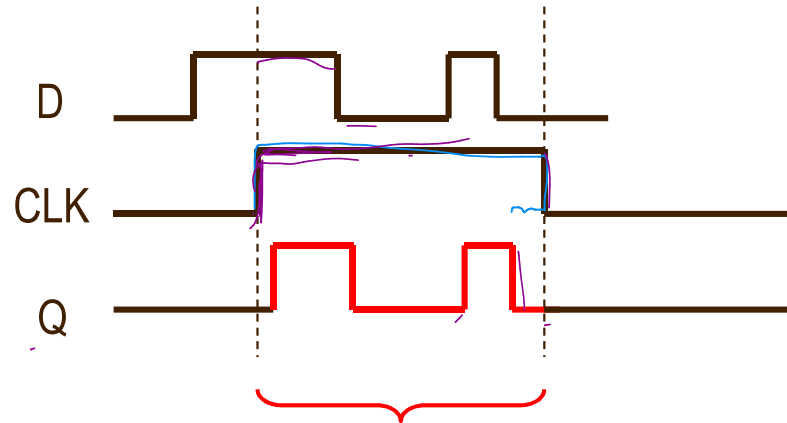
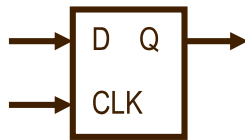


(c) Graphical symbol

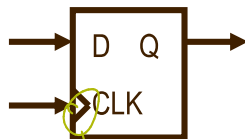


(d) Timing diagram

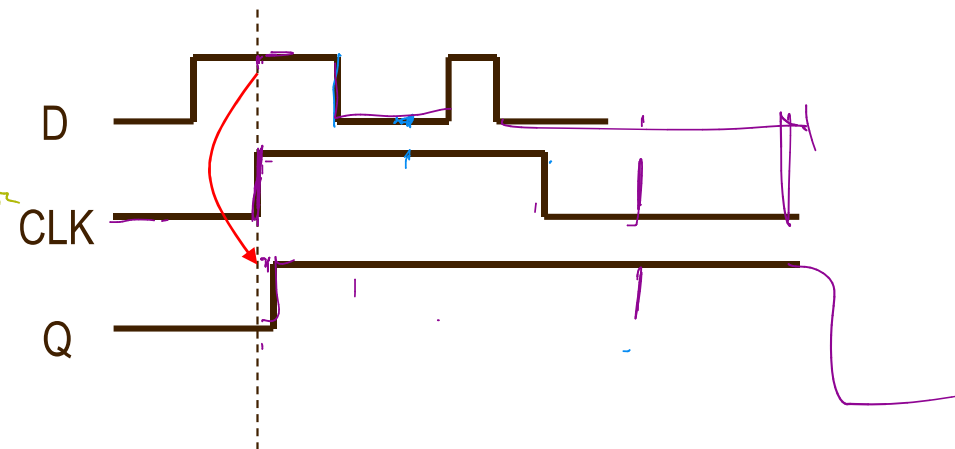
D Latch vs. D Flip-Flop



Latch transparent when clock is high

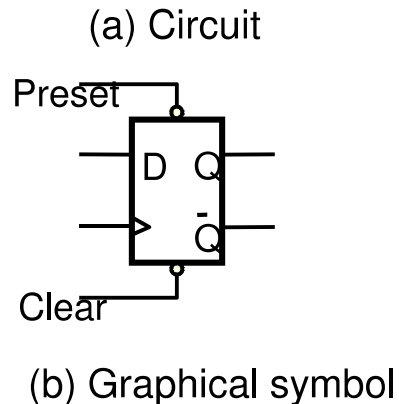


new clock; new



"Samples" D on rising edge of clock

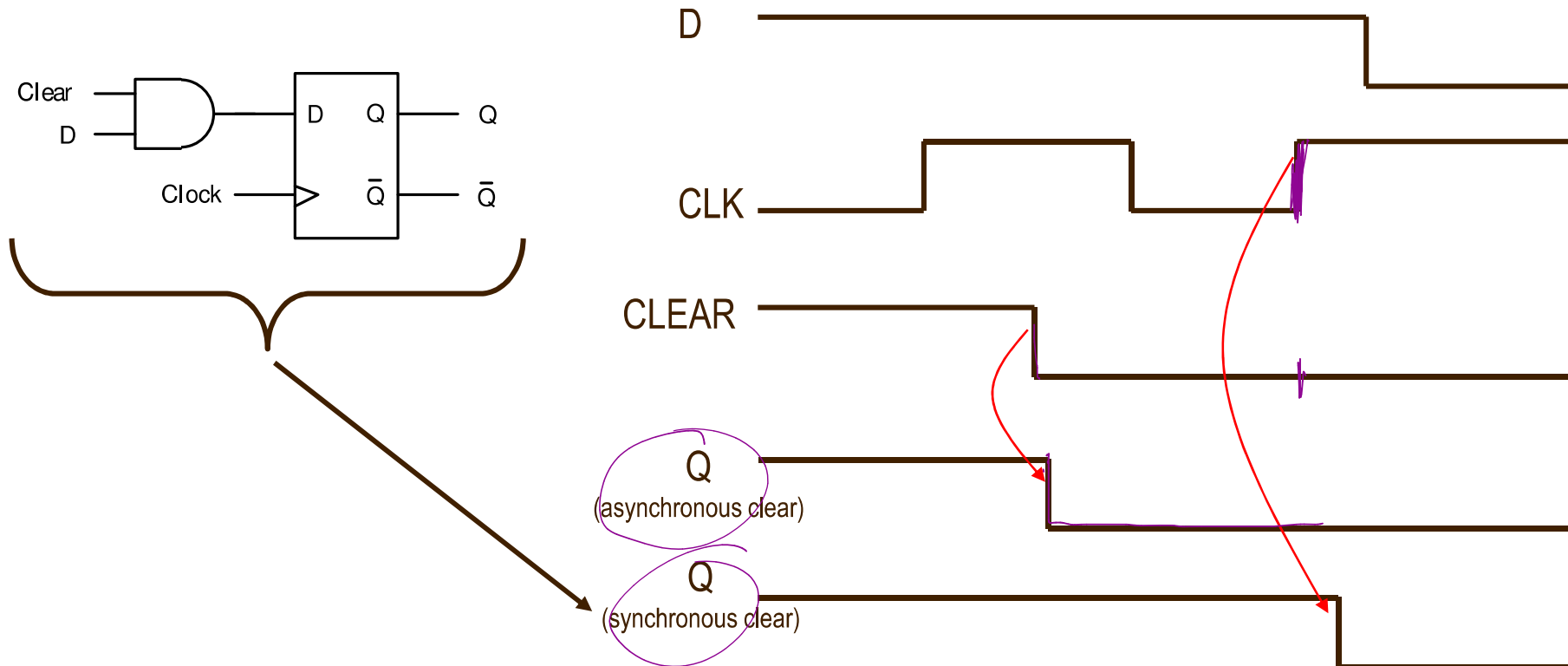
D Flip-Flop with Asynchronous Preset and Clear



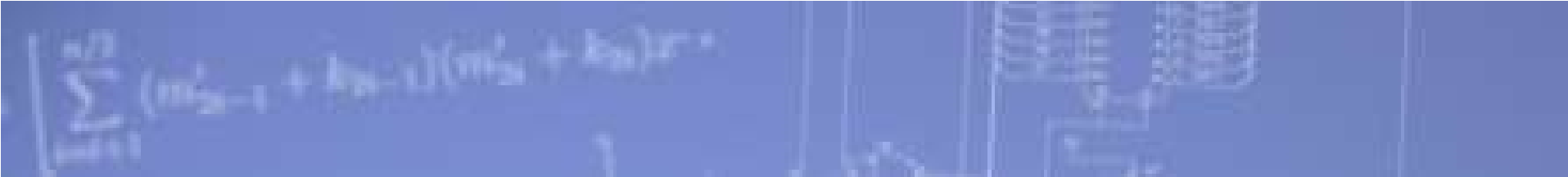
- Bubble on the symbol means “active-low”
 - When preset = 0, preset Q to 1
 - When preset = 1, do nothing
 - When clear = 0, clear Q to 0
 - When clear = 1, do nothing
- “Preset” and “Clear” also known as “Set” and “Reset” respectively
- In this circuit, preset and clear are asynchronous
 - Q changes immediately when preset or clear are active, regardless of clock

D Flip-Flop with Synchronous Clear

সেট্রন দেব্রেড 1 clock
হেসিং গি:৩৩৮

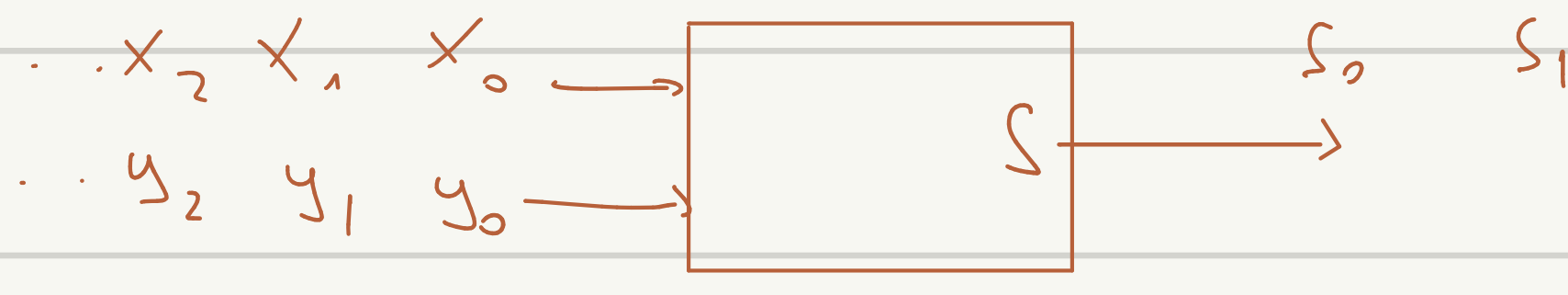


- Asynchronous active-low clear: Q immediately clears to 0
- Synchronous active-low clear: Q clears to 0 on rising-edge of clock



Sequential Logic Circuits

Seri toplama ardisil devre T-FF kullanarak

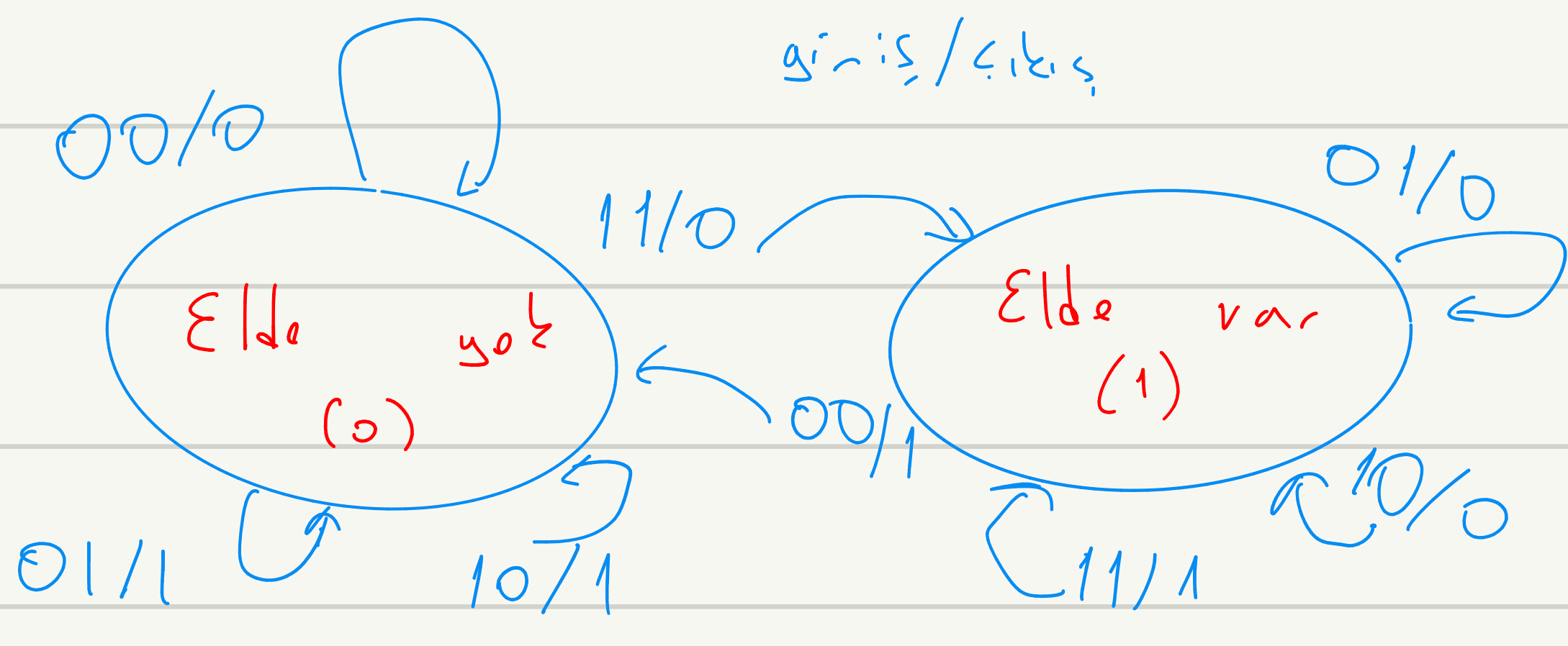


* Elde edilebilir $\log_2 2 = 1$
 * Elde edilebilir
 1 flip-flop

- Çıkış hem giriş hem de duruma bağlıysa Mealy

- Çıkış duruma bağlıysa moore

x	y	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0



önceki durum sonraki durum

x	y	q	Q	S	T
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	1	1	0	0
1	1	0	1	0	1
1	1	1	1	1	0

T	Q
0	q
1	q'

$T = f(x, y, q)$

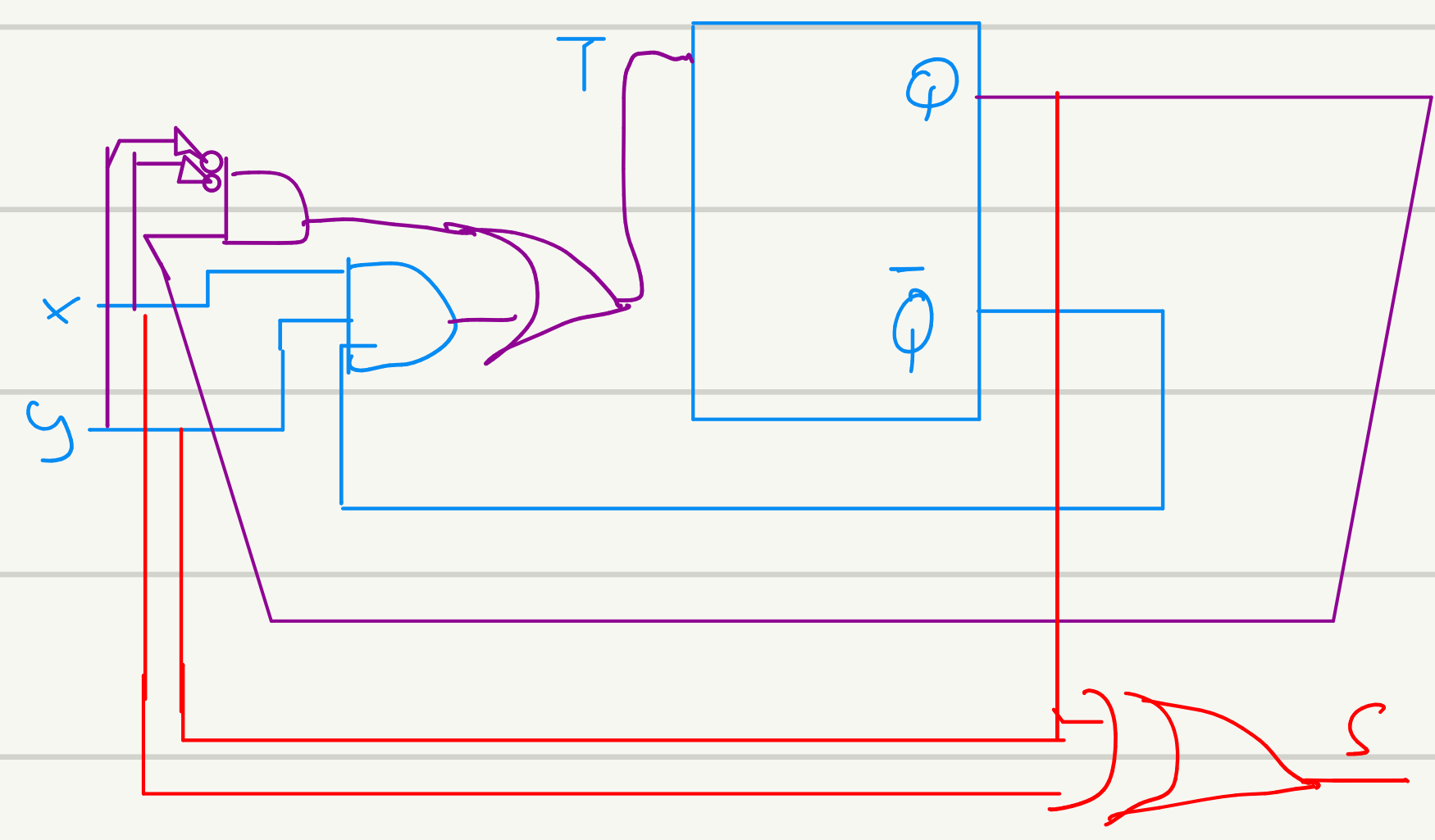
$S = f(x, y, q)$

q \ xy	00	01	11	10
0	0	1	0	1
1	1	0	1	0

q \ xy	00	01	11	10
0	0	0	1	0
1	1	0	0	0

$\rightarrow xyq' + x'1yq$

$\Rightarrow q \oplus x \oplus y$

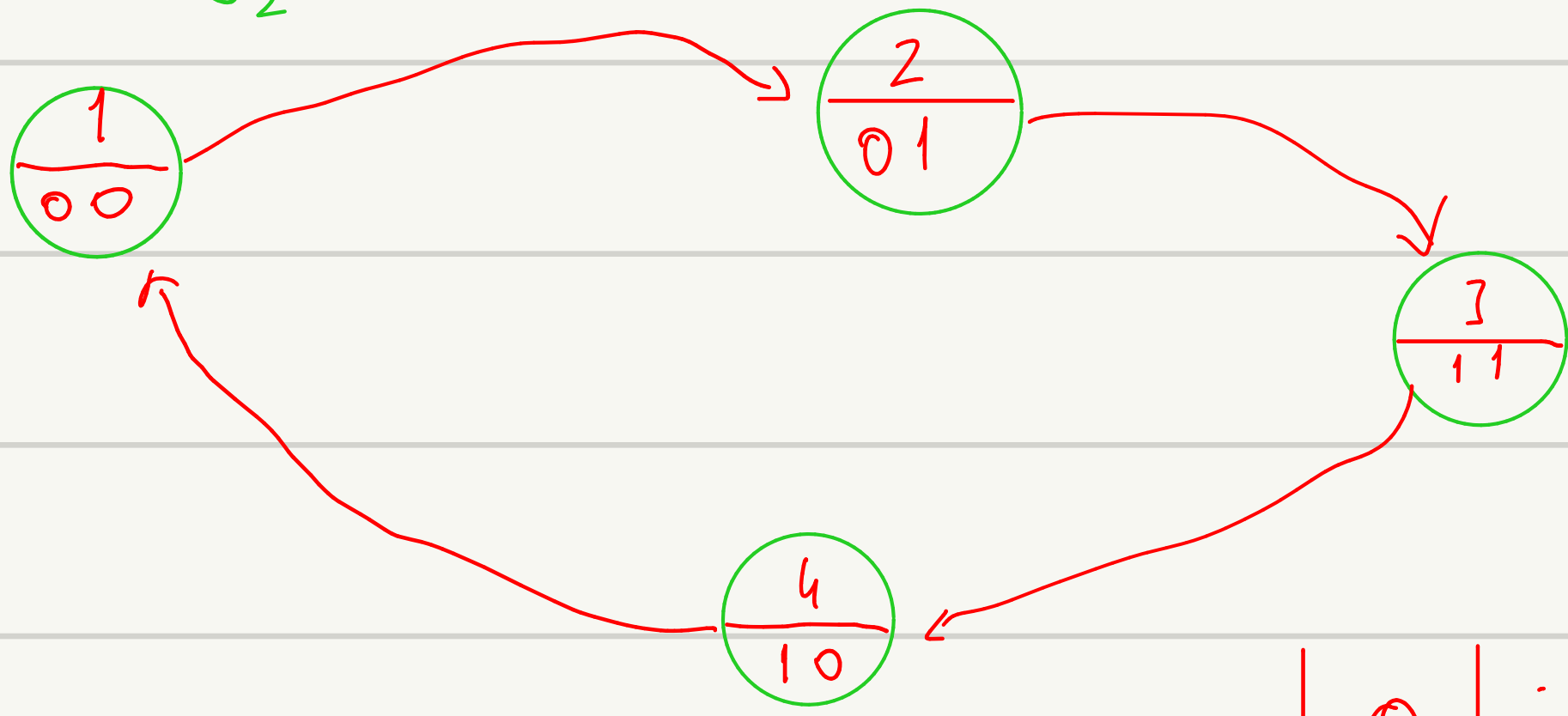


1, 2, 3, 4, 1, 2, 3, 4, 1 ... şeklinde sayılar serilen ardışıl devresi JK-flip flop

kullanarak tasarla

$\log_2 4 = 2$ flip flop gerekiyor,

★ C, kısı $\log_2 4 + 1$ olur



Clock	J	K	Q
┐	0	0	q
└	0	1	0
┐	1	0	1
└	1	1	q
→	x	x	q

q	Q	J	K
0	0	0	0
0	1	1	0
1	0	0	1
1	1	0	0

q ₀	q ₁	Q ₀	Q ₁	j ₀	k ₀	j ₁	k ₁	z ₀	z ₁	z ₂
0	0	0	1	0	0	1	0	0	0	1
0	1	1	1	1	0	0	0	0	1	0
1	0	0	0	0	1	0	0	1	0	0
1	1	1	0	0	0	0	1	0	1	1

q ₀	j ₀	k ₀
0	0	1
1	0	0

→ q₁

q ₁	j ₁	k ₁
0	1	0
1	0	1

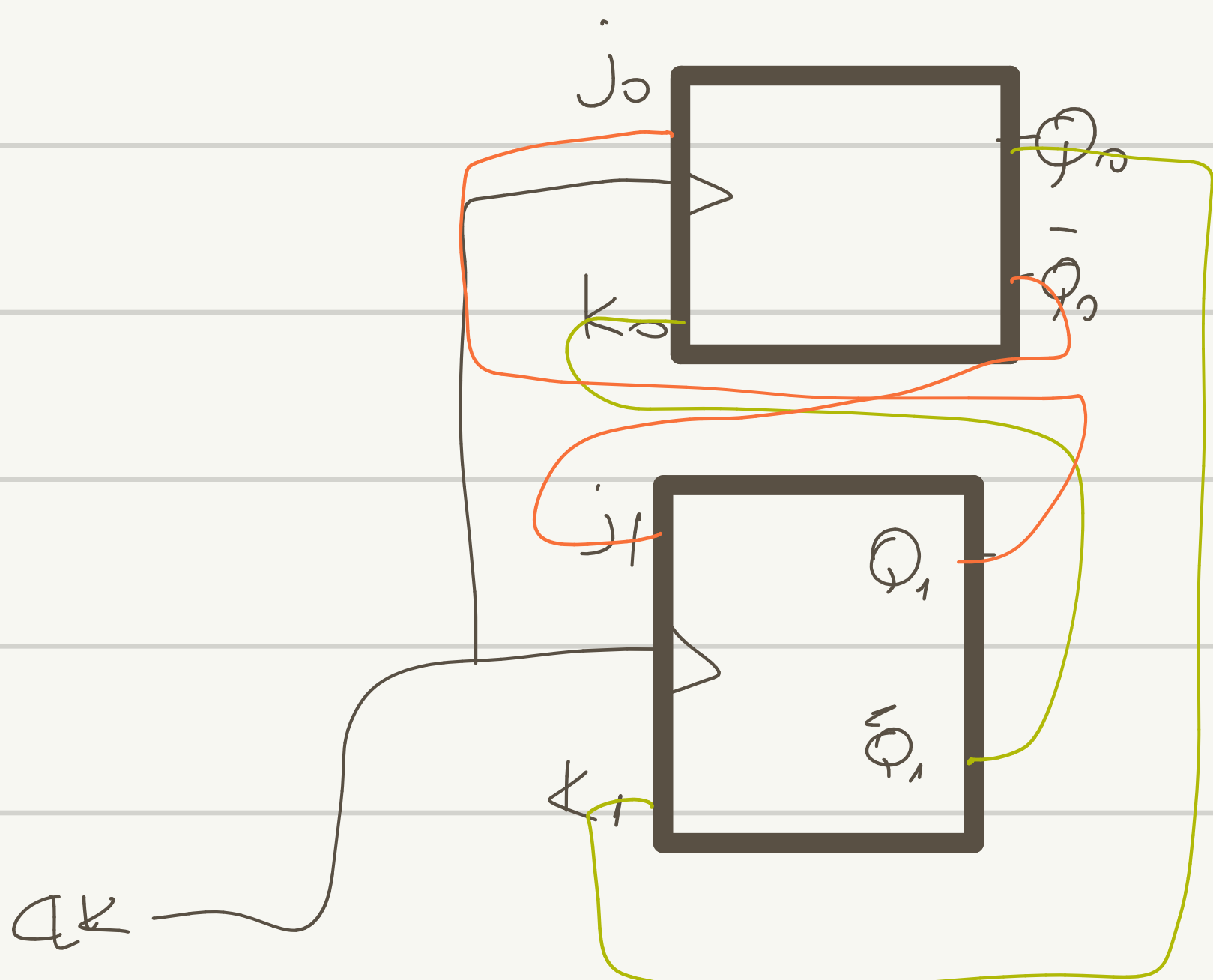
→ q₀

q ₁	j ₁	k ₁
0	1	0
1	0	1

→ q₀

q ₀	j ₀	k ₀
0	0	1
1	0	0

→ q₁



Asenkron sayıcı 0 ripple counter

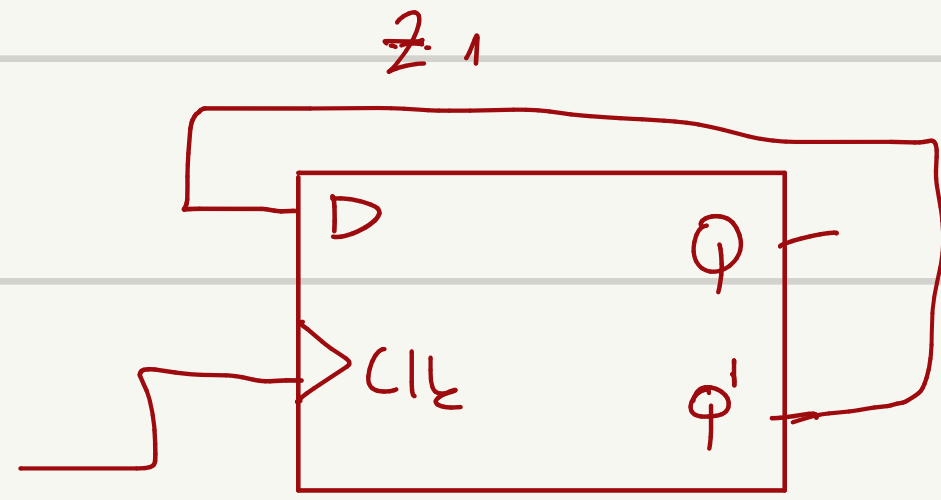
z_1	z_2	z_3
0	0	0
0	0	1
0	1	0
0	1	0
1	0	0
1	0	1

$z_1 \rightarrow 4Hz$

$z_2 \rightarrow 2Hz$

$z_3 \rightarrow 1Hz$

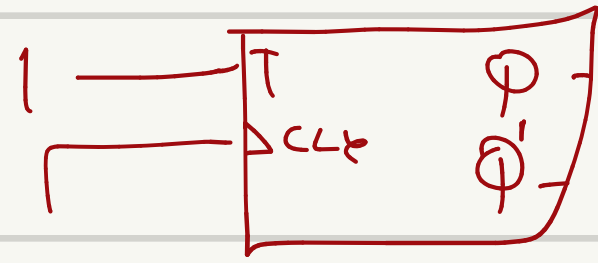
D-FF ile yap



Clk	Q
5	9

T-FF

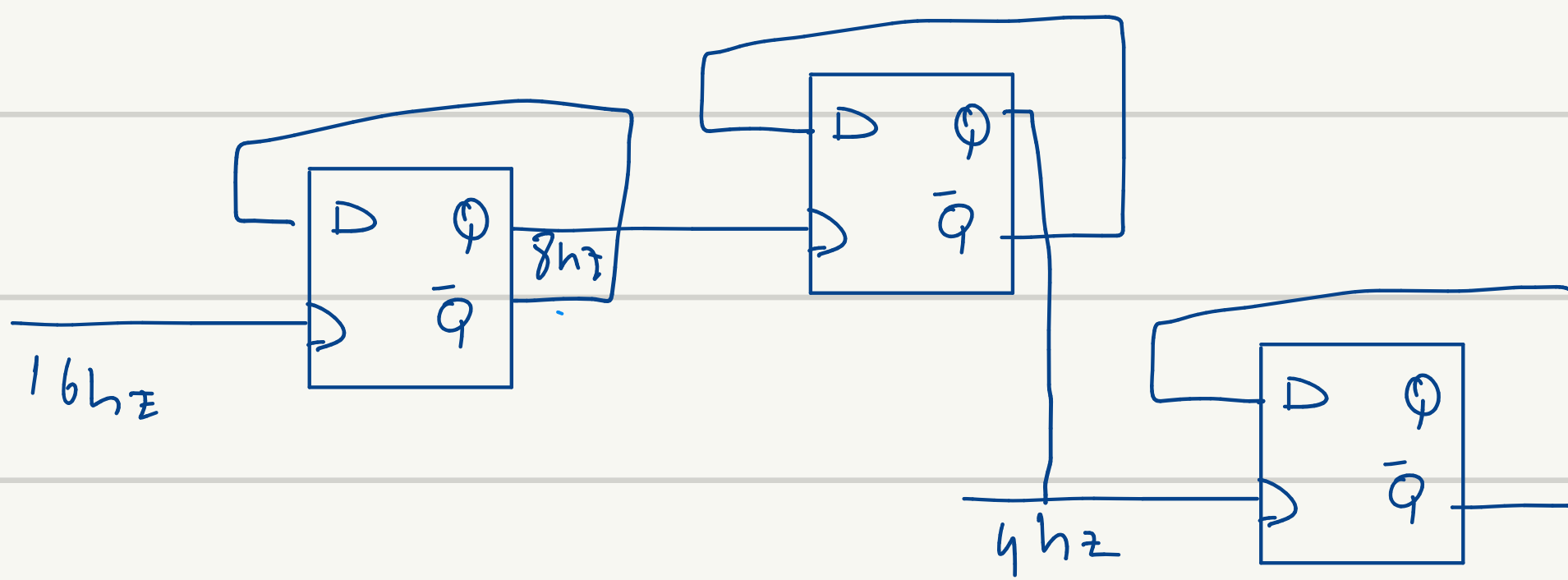
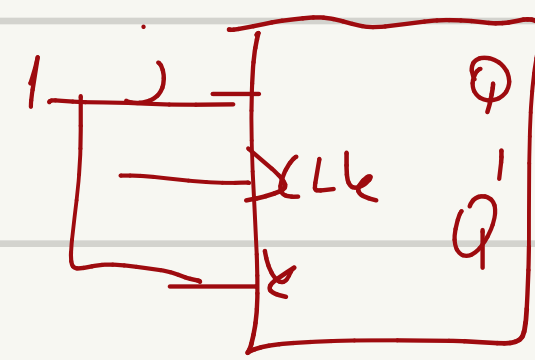
T	Q
0	0
1	0



JK-FF

q	Q	j	k
0	1	1	x
1	0	x	1

0-7 arası asenkron sayıcı



Clk

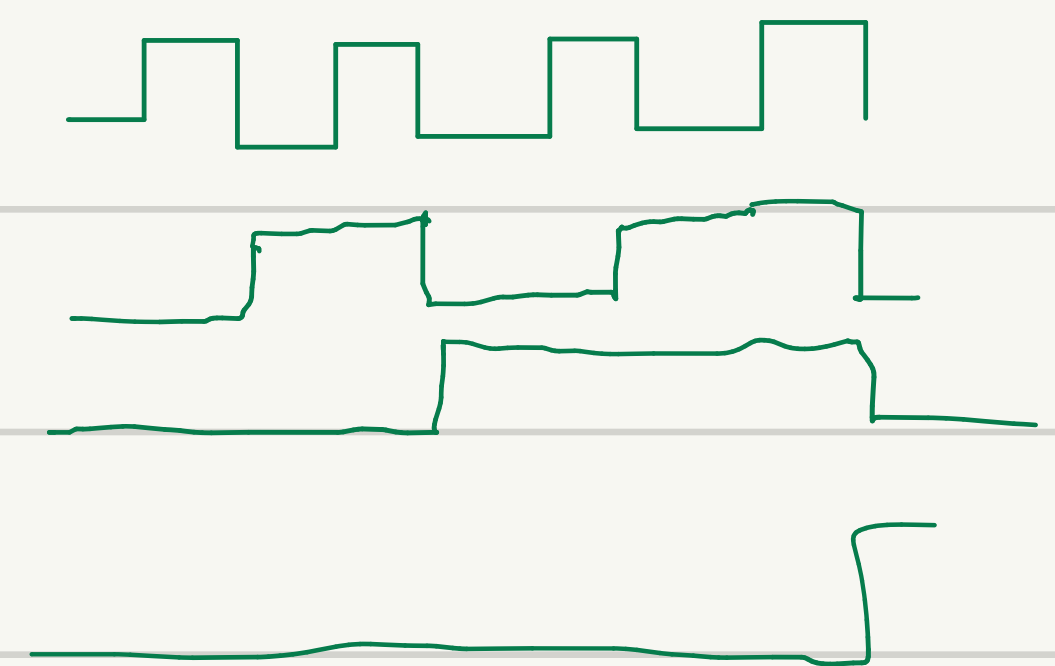
z_1

z_2

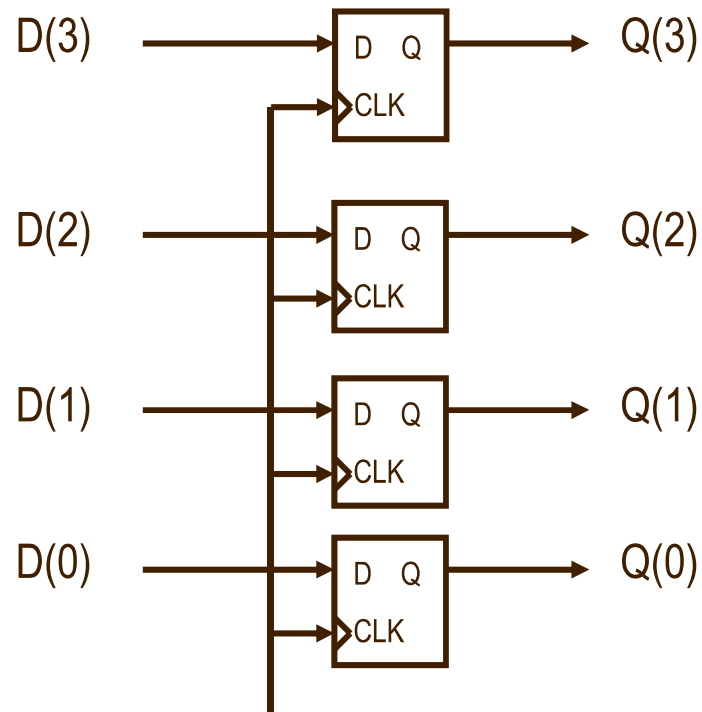
z_3

$\rightarrow$ bu gerçe doğru sayılar bundan dolayı $\bar{Q}$ 'leri kullan ya da diğer kenarları kullanmalısınız

1	1	1
1	1	0
1	0	1
1	0	0
0	1	1
0	1	0
0	0	1
0	0	0
1	1	1

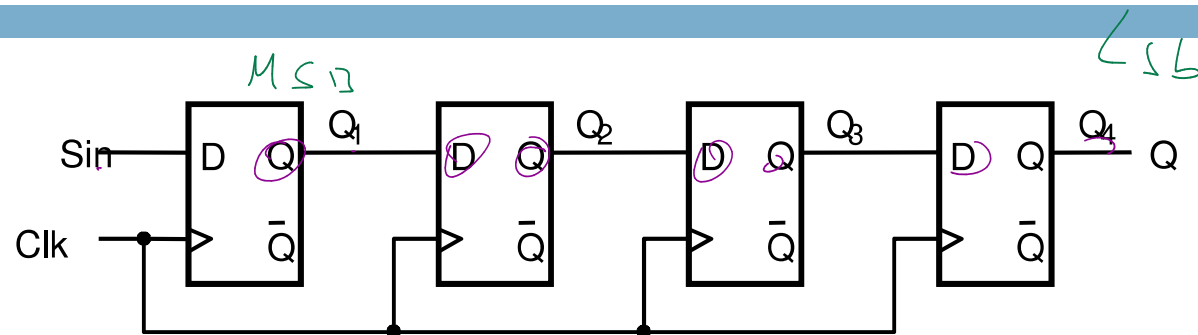


Register

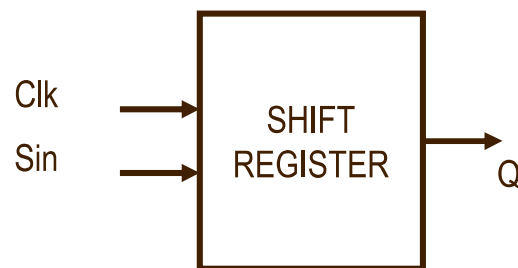


- In typical nomenclature, a register is a name for a collection of flip-flops used to hold a bus (i.e. `std_logic_vector`)

Shift Register



(a) Circuit



	Sin	Q ₁	Q ₂	Q ₃	Q ₄ = Q
t_0	1	0	0	0	0
t_1	0	1	0	0	0
t_2	1	0	1	0	0
t_3	1	1	0	1	0
t_4	1	1	1	0	1
t_5	0	1	1	1	0
t_6	0	0	1	1	1
t_7	0	0	0	1	1

(b) A sample sequence

$n \times \text{clock cycle}$

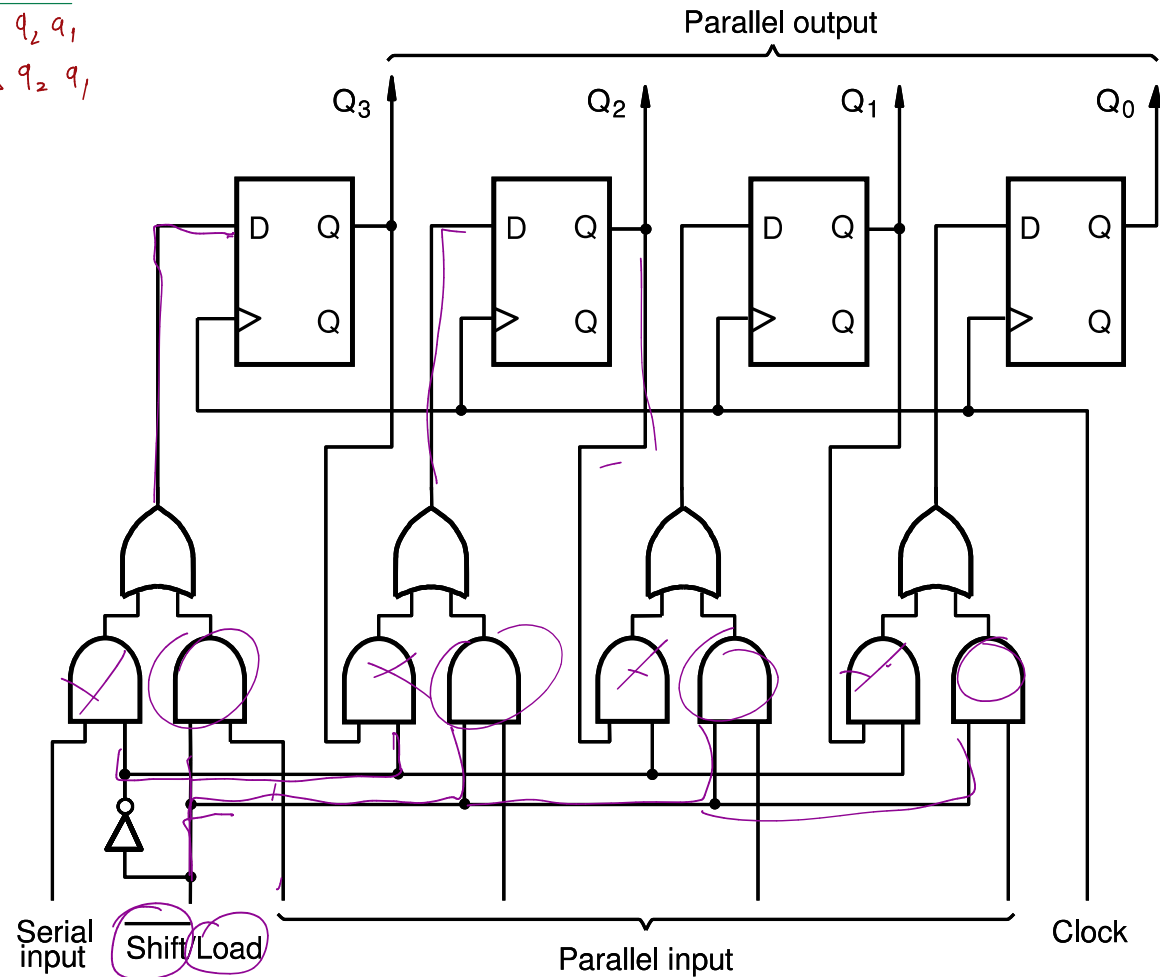
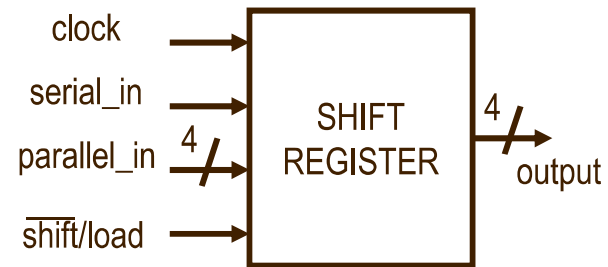
$a_2 \phi_1$

Parallel Access Shift Register

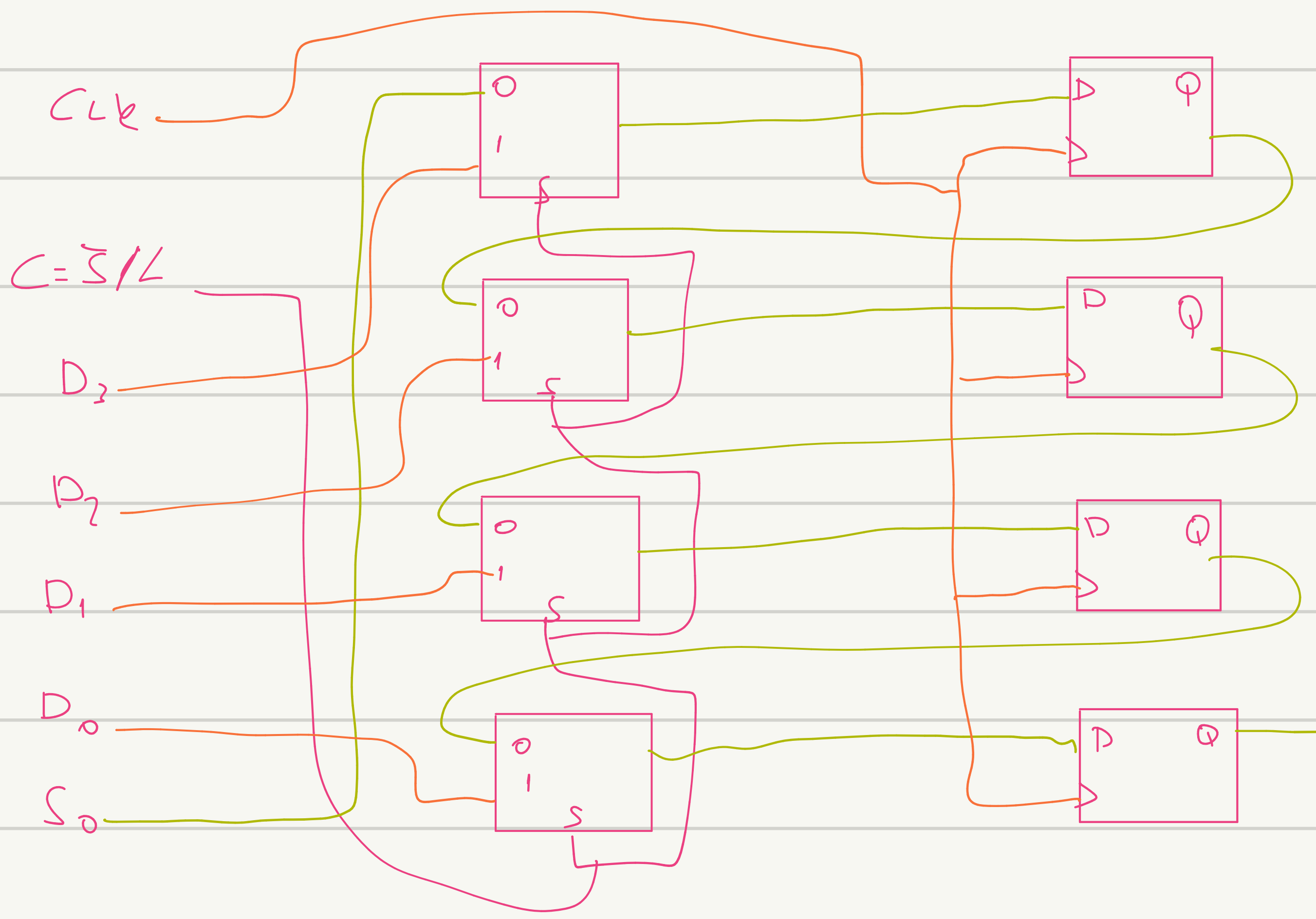
$$\begin{array}{c|c|c|c} \overline{SK} & D_i & S_i & D \\ \hline 0 & \overline{V} & \downarrow & S_i \\ 1 & \overline{V} & - & D_i \end{array}$$

$$C^1, S_i + C D_i$$

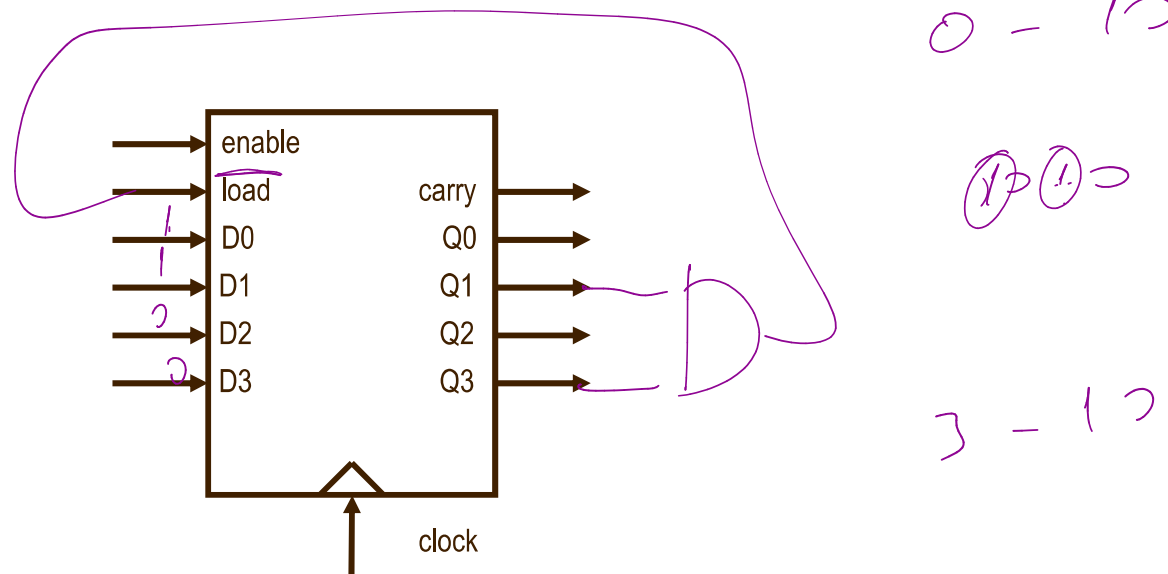
clk	$\overline{S/L}$	S_{in}		
1	0	0		0 $q_3 q_2 q_1$
1	0	1		1 $q_3 q_2 q_1$
1	1	x	a b c d	x



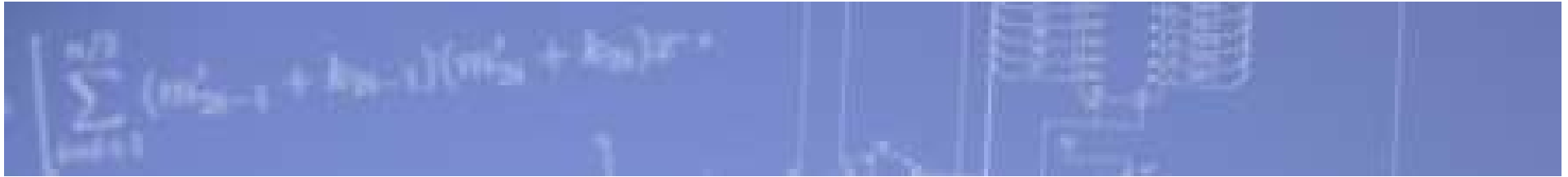
4 bitlik paralel g tlenmeli -  telenmeli yazma ı D-FF ve muxla tasarla.



Synchronous Up Counter



- Enable (synchronous): when high enables the counter, when low counter holds its value
- Load (synchronous) : when load = 1, load the desired value into the counter
- Output carry: indicates when the counter “rolls over”
- D3 down to D0, Q3 down to Q0 is how to interpret MSB to LSB



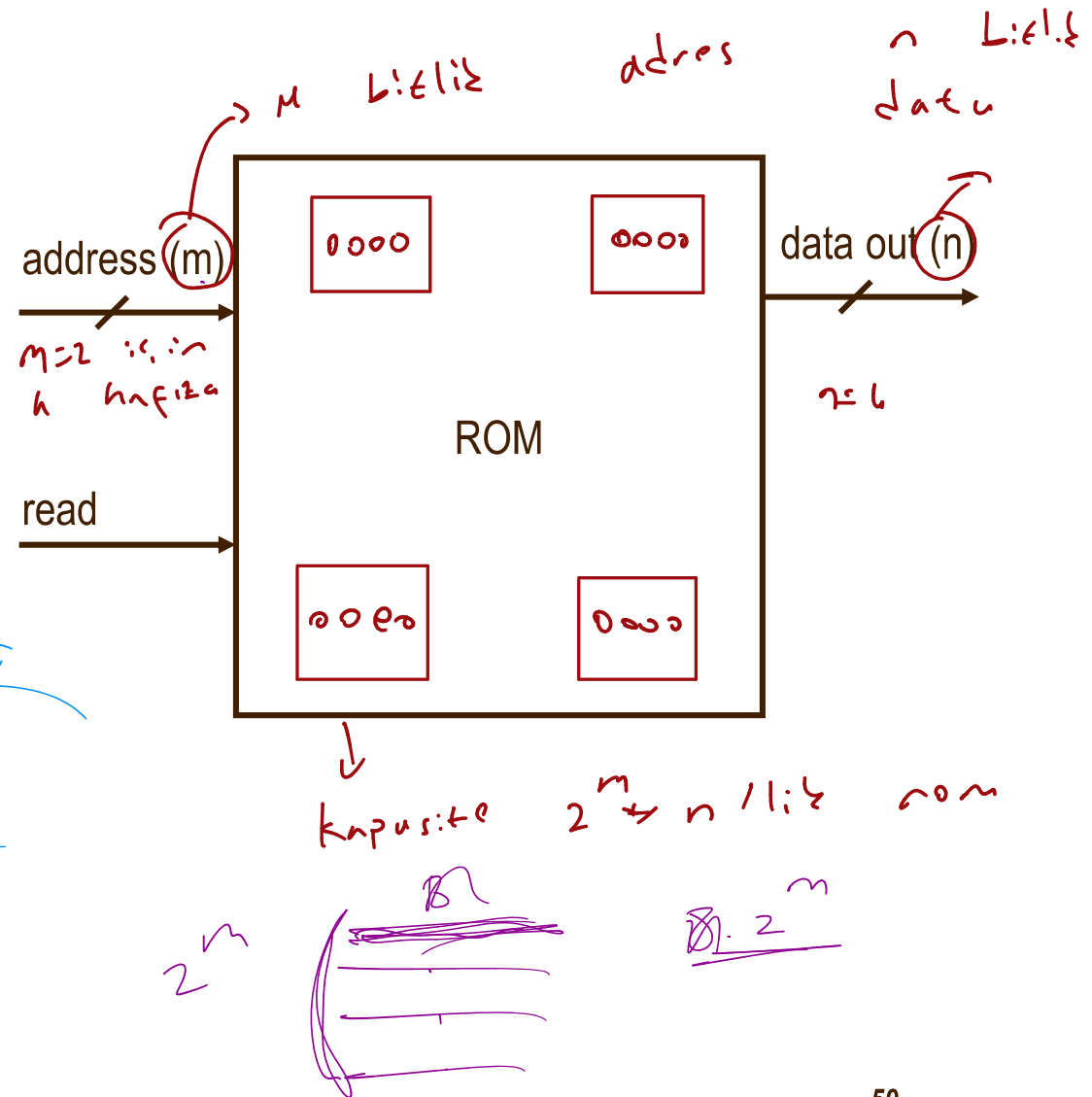
Memories



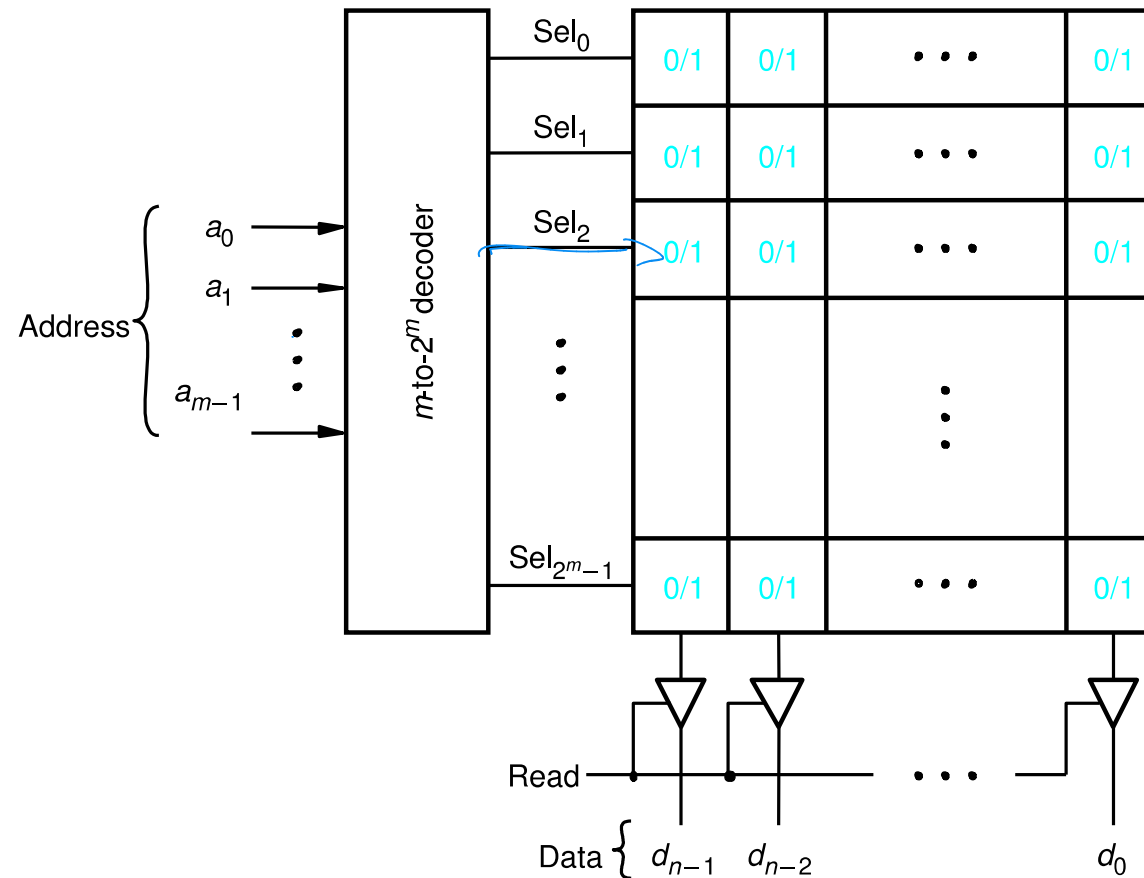
Read Only Memory (ROM)

2^m

- Similar to RAM except read only
- Addressable memory
- Can be synchronous (with clock) or asynchronous (no clock)

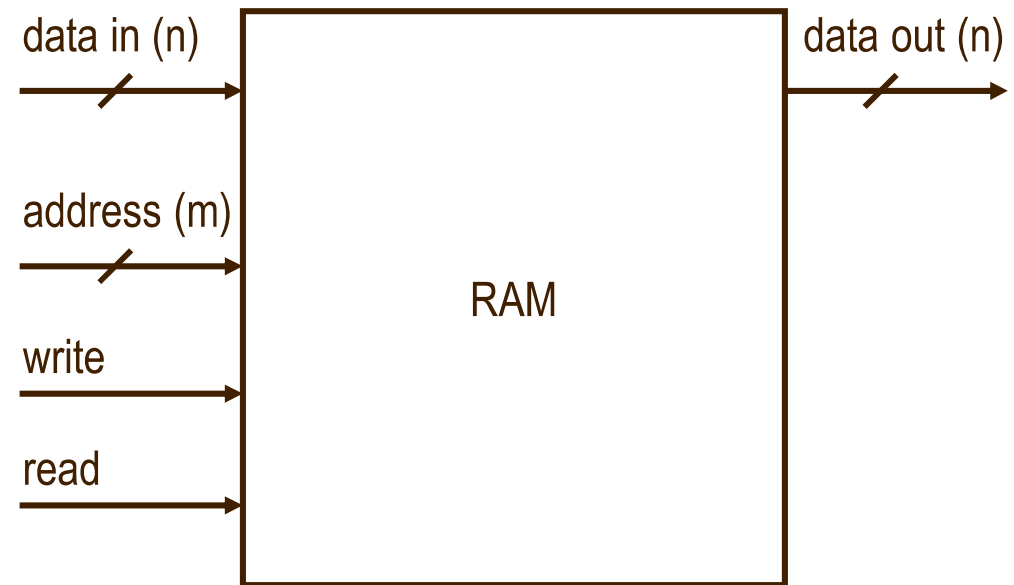


Read-Only Memory (ROM)

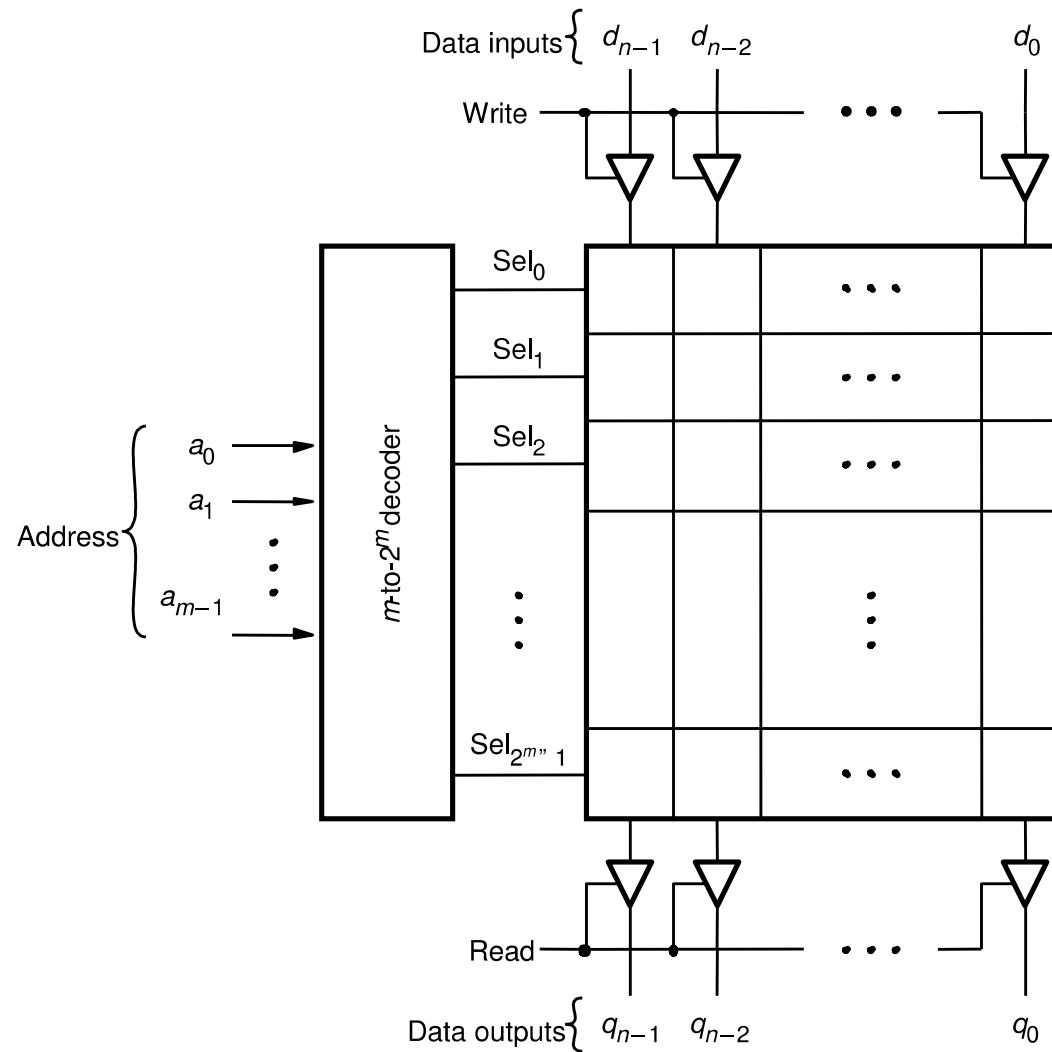


Random Access Memory (RAM)

- More efficient than registers for storing large amounts of data
- Can read and write to RAM
- Addressable memory
- Can be synchronous (with clock) or asynchronous (no clock)
- SRAM dimensions are:
 - (number of words) x (bits per word)SRAM
- Address is m bits, data is n bits
 - 2^m x n -bit RAM
- Example: address is 5 bits, data is 8 bits
 - 32 x 8-bit RAM
- Write
 - Data_in and address are stable
 - Assert write signal (then de-assert)
- Read
 - Address is stable
 - Assert read signal
 - Data_out is valid



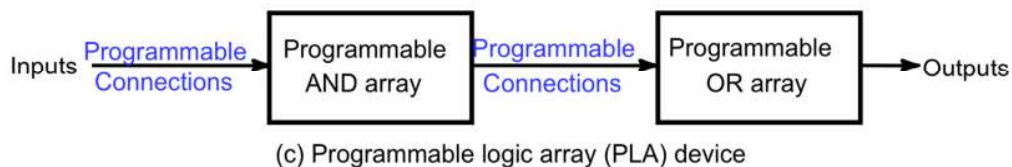
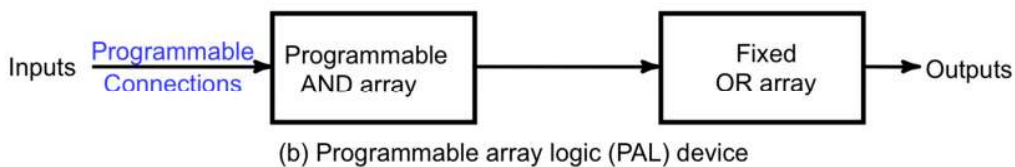
Random Access Memory (RAM)



PROGRAMMABLE LOGIC DEVICES

→ sabit sugdu and kapisi var bi? or eklenebilir

- **Read Only Memory (ROM)** - a fixed array of AND gates and a programmable array of OR gates
- **Programmable Array Logic (PAL)** - a programmable array of AND gates feeding a fixed array of OR gates.
- **Programmable Logic Array (PLA)** - a programmable array of AND gates feeding a programmable array of OR gates.
- **Complex Programmable Logic Device (CPLD) / Field- Programmable Gate Array (FPGA)** - complex enough to be called "architectures"



READ ONLY MEMORY

- Read Only Memories (ROM) or Programmable Read Only Memories (PROM) have:
 - N input lines,
 - M output lines, and
 - 2^N decoded minterms.
- Fixed AND array with 2^N outputs implementing all N-literal minterms.
- Programmable OR Array with M outputs lines to form up to M sum of minterm expressions.
- A program for a ROM or PROM is simply a multiple-output truth table
 - If a 1 entry, a connection is made to the corresponding minterm for the corresponding output
 - If a 0, no connection is made
- Can be viewed as a *memory* with the inputs as *addresses of data* (output values), hence ROM or PROM names!

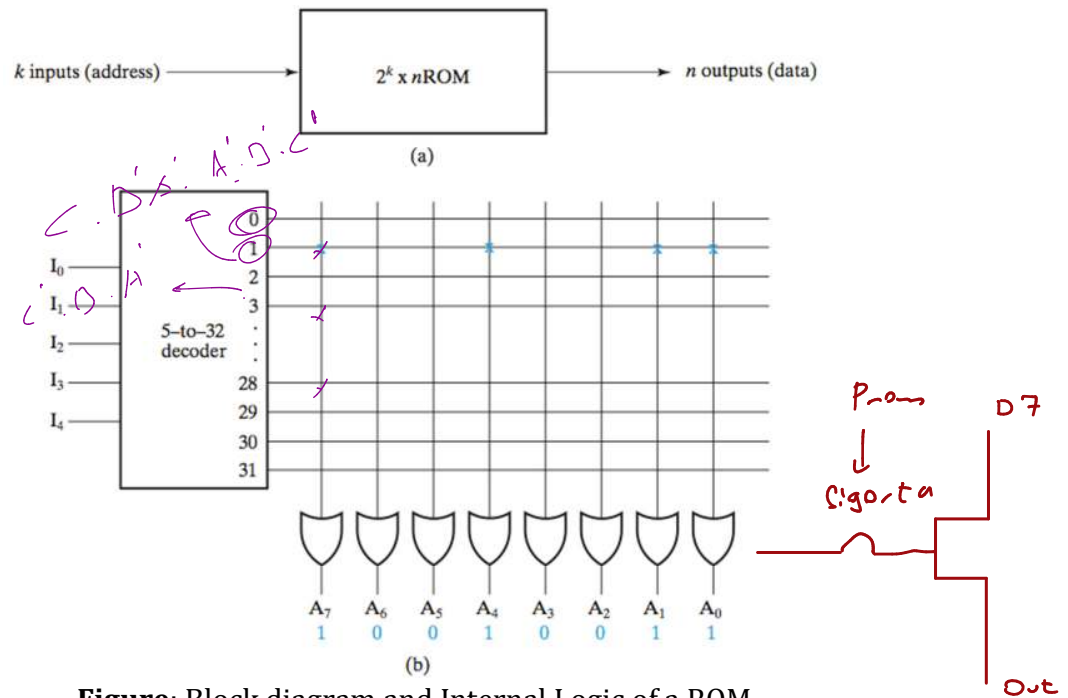
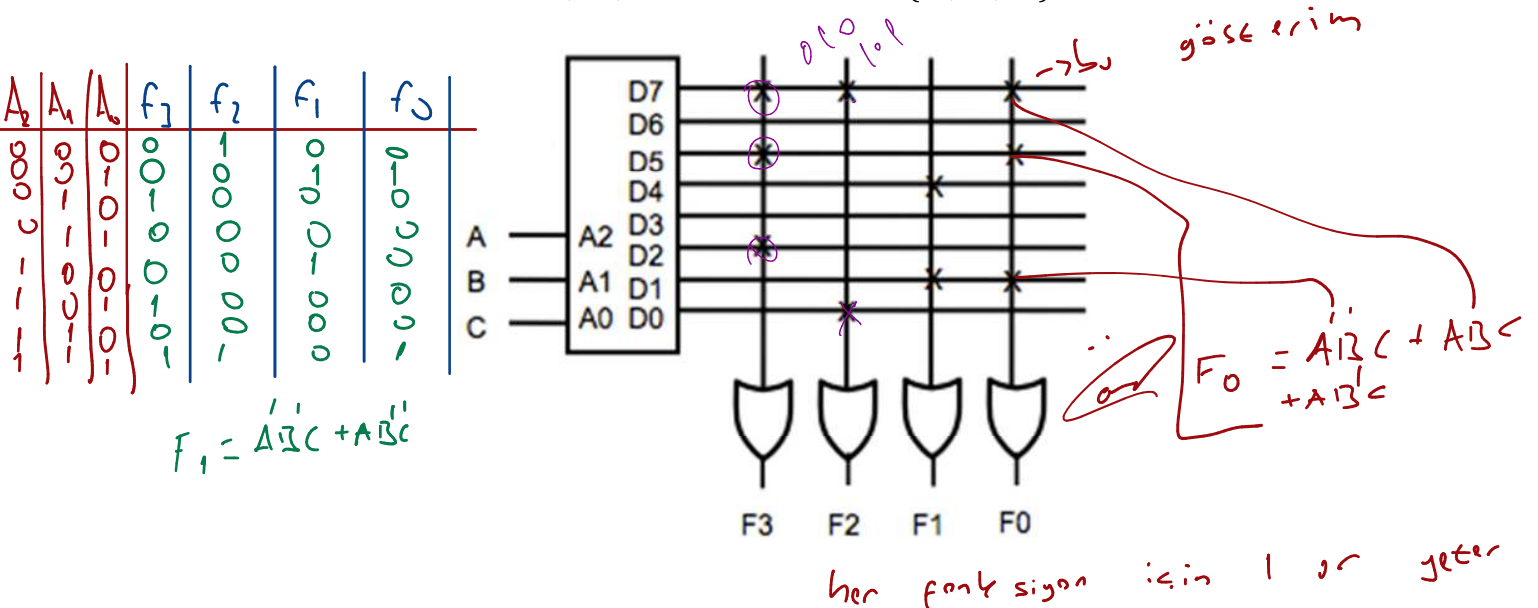


Figure: Block diagram and Internal Logic of a ROM

- Depending on the programming technology and approaches, read-only memories have different names
 - ROM – mask programmed → *defu programlanıyor*
 - PROM – fuse or antifuse programmed
 - EPROM – erasable floating gate programmed *pozitif yazılabilir elektronik olarak programlanıyor*
 - EEPROM or E²PROM – electrically erasable floating gate programmed
 - FLASH memory: electrically erasable floating gate with multiple erasure and programming modes.
- Example: A 8 X 4 ROM (N = 3 input lines, M= 4 output lines)
 - The fixed "AND" array is a "decoder" with 3 inputs and 8 outputs implementing minterms.
 - The programmable "OR" array uses a single line to represent all inputs to an OR gate. An "X" in the array corresponds to attaching the minterm to the OR
 - Read Example: For input (A₂,A₁,A₀) = 001, output is (F₃,F₂,F₁,F₀) = 0011.
 - What are functions F₃, F₂, F₁ and F₀ in terms of (A₂, A₁, A₀)?



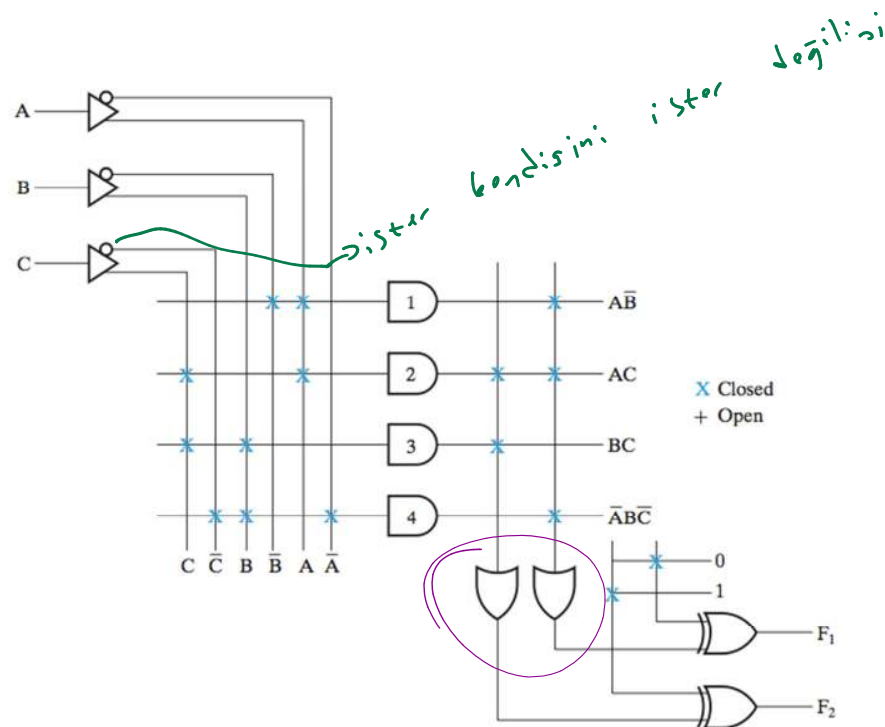
PROGRAMMABLE LOGIC ARRAY (PLA)

- Compared to a ROM and a PAL, a PLA is the most flexible having a programmable set of ANDs combined with a programmable set of ORs.
- Advantages
 - A PLA can have large N and M permitting implementation of equations that are impractical for a ROM (because of the number of inputs, N, required)
 - A PLA has all of its product terms connectable to all outputs, overcoming the problem of the limited inputs to the PAL Ors
 - Some PLAs have outputs that can be complemented, adding POS functions
- Disadvantages
 - Often, the product term count limits the application of a PLA.
 - Two-level multiple-output optimization is required to reduce the number of product terms in an implementation, helping to fit it into a PLA.
 - Multi-level circuit capability available in PAL not available in PLA. PLA requires external connections to do multi-level circuits.

Programmable Logic Array Example

$$F_1 = AB' + AC + A'BC'$$

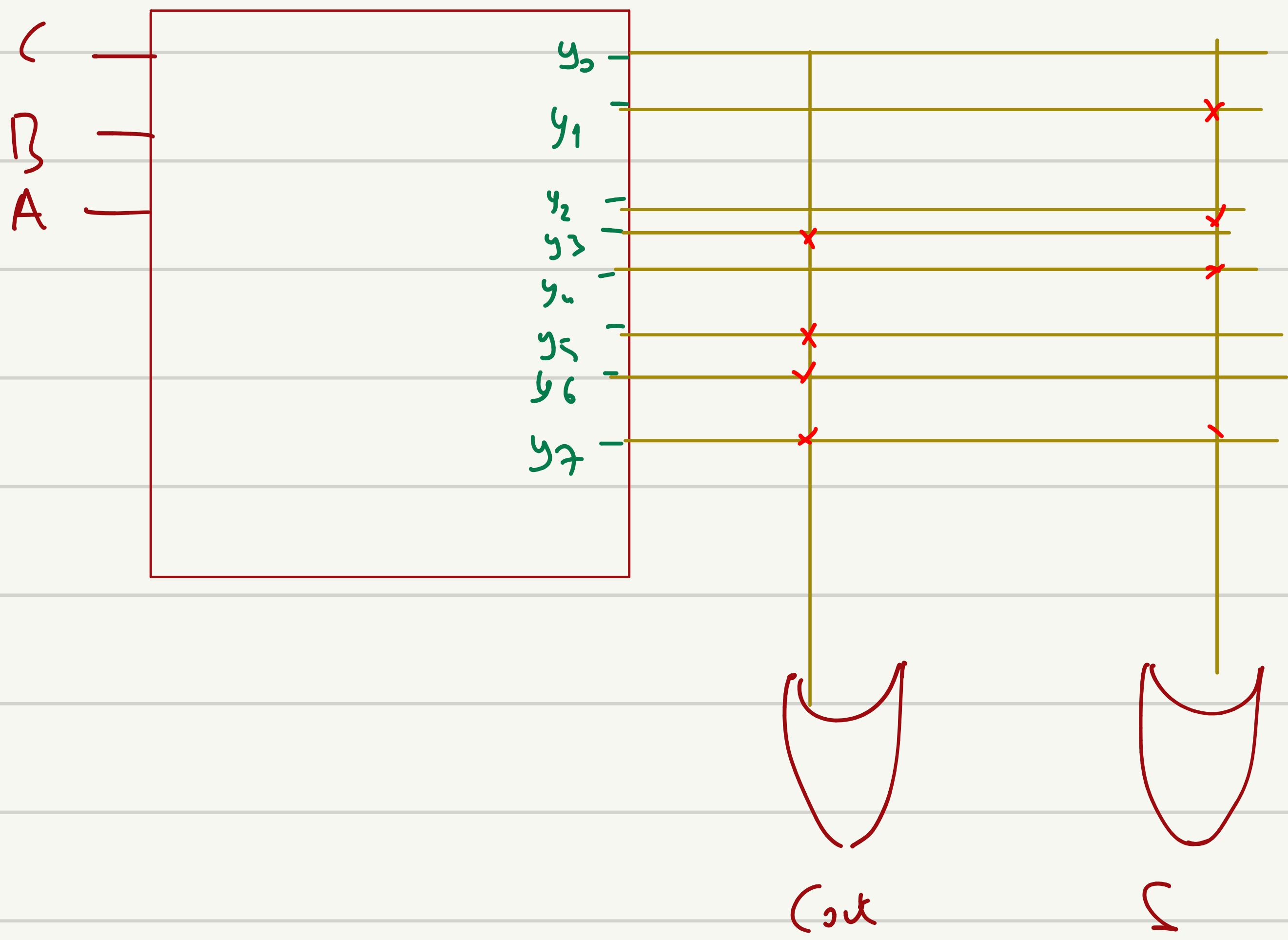
$$F_2 = (AC + BC)'$$



- What are the equations for F_1 and F_2 ?
- Could the PLA implement the functions without the XOR gates?
- 3-input, 3-output PLA with 4 product terms

Bir bitlik $F\hat{A}'$ ^{Full adder} uygun büyüklükteki bir rom ile gerçekleştiriniz.

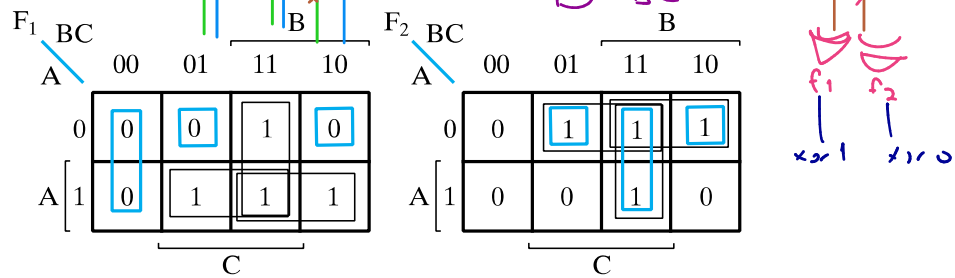
A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Example 6-3 from Mano: Implementing a Combinational Circuit Using a PLA

$$F_1(A,B,C) = \sum m(3,5,6,7)$$

$$F_2(A,B,C) = \sum m(1,2,3,7)$$



$$F_1 = AB + AC + BC$$

$$F_1 = \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{B}\overline{C}$$

$$F_2 = \overline{A}B + \overline{A}C + BC$$

$$F_2 = \overline{A}\overline{B}C + \overline{A}B\overline{C} + BC$$

The solution is:

$$F_1 = \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{B}\overline{C}$$

$$F_2 = \overline{A}\overline{B}C + \overline{A}B\overline{C} + BC$$

buradan AND kapın sonra aynı andler bulmaya çalışınız
buradan sonucu toplu alık

PROGRAMMABLE ARRAY LOGIC (PAL)

- The PAL is the opposite of the ROM, having a programmable set of ANDs combined with fixed ORs.
- Disadvantage
 - ROM guaranteed to implement any M functions of N inputs. PAL may have too few inputs to the OR gates.
- Advantages
 - For given internal complexity, a PAL can have larger N and M
 - Some PALs have outputs that can be complemented, adding POS functions
 - No multilevel circuit implementations in ROM (without external connections from output to input). PAL has outputs from OR terms as internal inputs to all AND terms, making implementation of multi-level circuits easier.

Programmable Array Logic Example

- 4-input, 3-output PAL with fixed, 3-input OR terms
- What are the equations for F1 through F4?

$$W(A,B,C,D) = \sum m(2,12,13)$$

$$X(A,B,C,D) = \sum m(7,8,9,10,11,12,13,14,15)$$

$$Y(A,B,C,D) = \sum m(0,2,3,4,5,6,7,8,10,11,15)$$

$$Z(A,B,C,D) = \sum m(1,2,8,12,13)$$

Simplifying the four function to a minimum number of terms results in the following Boolean functions

$$W = ABC' + A'B'CD'$$

$$X = A + BCD$$

$$Y = A'B + CD + B'D'$$

$$Z = ABC' + A'B'CD' + AC'D' + A'B'C'D = W + AC'D' + A'B'C'D$$

Overview



- Memory definitions
- Random Access Memory (RAM)
- Static RAM (SRAM) integrated circuits
 - ▣ Cells and slices
 - ▣ Cell arrays and coincident selection
- Arrays of SRAM integrated circuits
- Dynamic RAM (DRAM) integrated circuits
- DRAM Types

Memory Definitions

- Memory — A collection of storage cells together with the necessary circuits to transfer information to and from them.
- Memory Organization — the basic architectural structure of a memory in terms of how data is accessed.
- Random Access Memory (RAM) — a memory organized such that data can be transferred to or from any cell (or collection of cells) in a time that is not dependent upon the particular cell selected.
- Memory Address — A vector of bits that identifies a particular memory element (or collection of elements).

Sram
↓
Latch

Dram
↓
DDR

var $\hookrightarrow$ high $\rightarrow$ 0 $\leftarrow$ ans: scin var

↓
Pahuli

ve set
elektrik etiket?

Memory Definitions (Continued)

- Typical data elements are:
 - ▣ bit — a single binary digit
 - ▣ byte — a collection of eight bits accessed together
 - ▣ word — a collection of binary bits whose size is a typical unit of access for the memory. It is typically a power of two multiple of bytes (e.g., 1 byte, 2 bytes, 4 bytes, 8 bytes, etc.)
- Memory Data — a bit or a collection of bits to be stored into or accessed from memory cells.
- Memory Operations — operations on memory data supported by the memory unit. Typically, *read* and *write* operations over some data element (bit, byte, word, etc.).

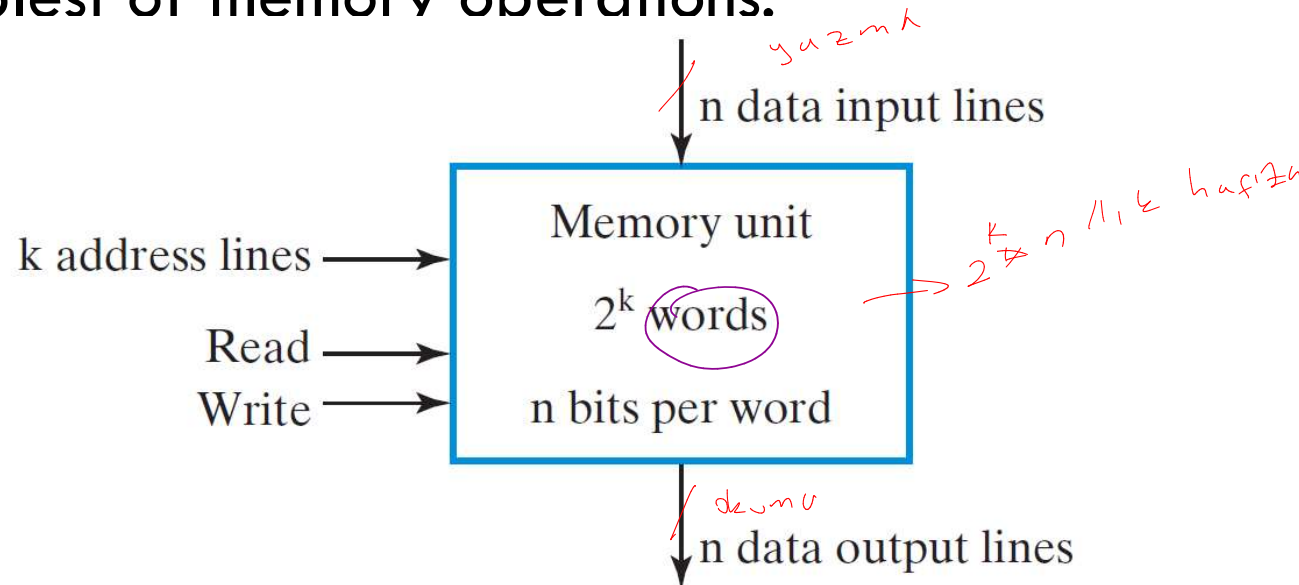
Donanma göre
değişik t defa-
da yazılabilen
hafıza 32-64 bit

Memory Organization

- Organized as an indexed array of words. Value of the index for each word is the memory address.
- Often organized to fit the needs of a particular computer architecture. Some historically significant computer architectures and their associated memory organization:
 - ▣ Digital Equipment Corporation PDP-8 – used a 12-bit address to address 4096 12-bit words.
 - ▣ IBM 360 – used a 24-bit address to address 16,777,216 8-bit bytes, or 4,194,304 32-bit words.
 - ▣ Intel 8080 – (8-bit predecessor to the 8086 and the current Intel processors) used a 16-bit address to address 65,536 8-bit bytes.

Memory Block Diagram

- A basic memory system is shown here:
- k address lines are decoded to address 2^k words of memory.
- Each word is n bits.
- Read and Write are single control lines defining the simplest of memory operations.



Memory Organization Example

Example memory contents:

- A memory with 3 address bits & 8 data bits has:
- $k = 3$ and $n = 8$ so $2^3 = 8$ addresses labeled 0 to 7.
- $2^3 = 8$ words of 8-bit data

$2^3 = 8$

<u>Memory address</u>		<u>Memory contents</u>
<u>Binary</u>	<u>Decimal</u>	
000000000	0	10110101 01011100
000000001	1	10101011 10001001
000000010	2	00001101 01000110
.	.	.
.	.	.
.	.	.
.	.	.
111111101	1021	10011101 00010101
111111110	1022	00001101 00011110
111111111	1023	11011110 00100100

$2^3 = 8$

16 bit

$2^{10} = 1024$ 16 bit bus

Basic Memory Operations

- Memory operations require the following:
 - ▣ *Data* — data written to, or read from, memory as required by the operation.
 - ▣ *Address* — specifies the memory location to operate on. The address lines carry this information into the memory. Typically: n bits specify locations of 2^n words.
 - ▣ An operation — Information sent to the memory and interpreted as control information which specifies the type of operation to be performed. Typical operations are READ and WRITE. Others are READ followed by WRITE and a variety of operations associated with delivering blocks of data. Operation signals may also specify timing info.

Control Inputs to a Memory Chip

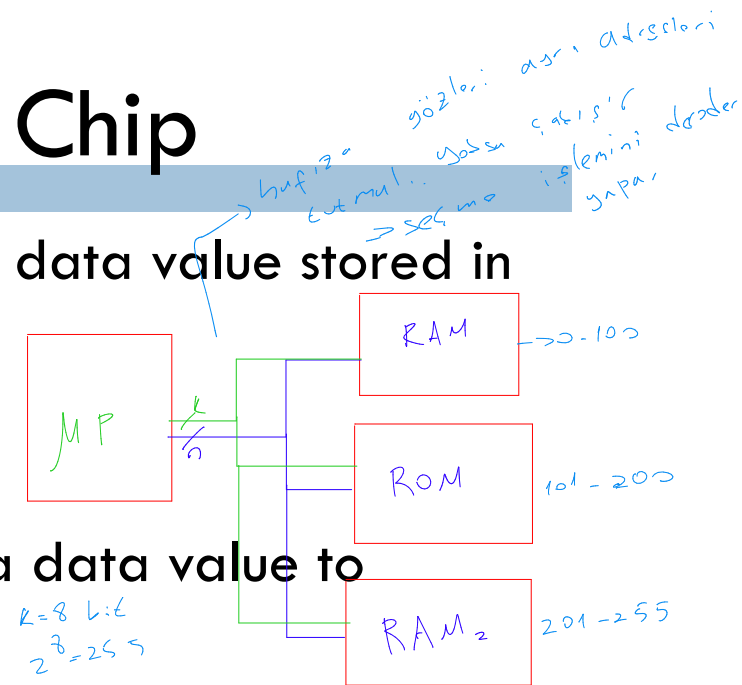
- Read Memory — an operation that reads a data value stored in memory:

- Place a valid address on the address lines.
- Wait for the read data to become stable.

- Write Memory — an operation that writes a data value to memory:

- Place a valid address on the address lines and valid data on the data lines.
- Toggle the memory write control line

- Chip Select — to enable the particular RAM chip or chips containing the word to select.



Control Inputs to a Memory Chip

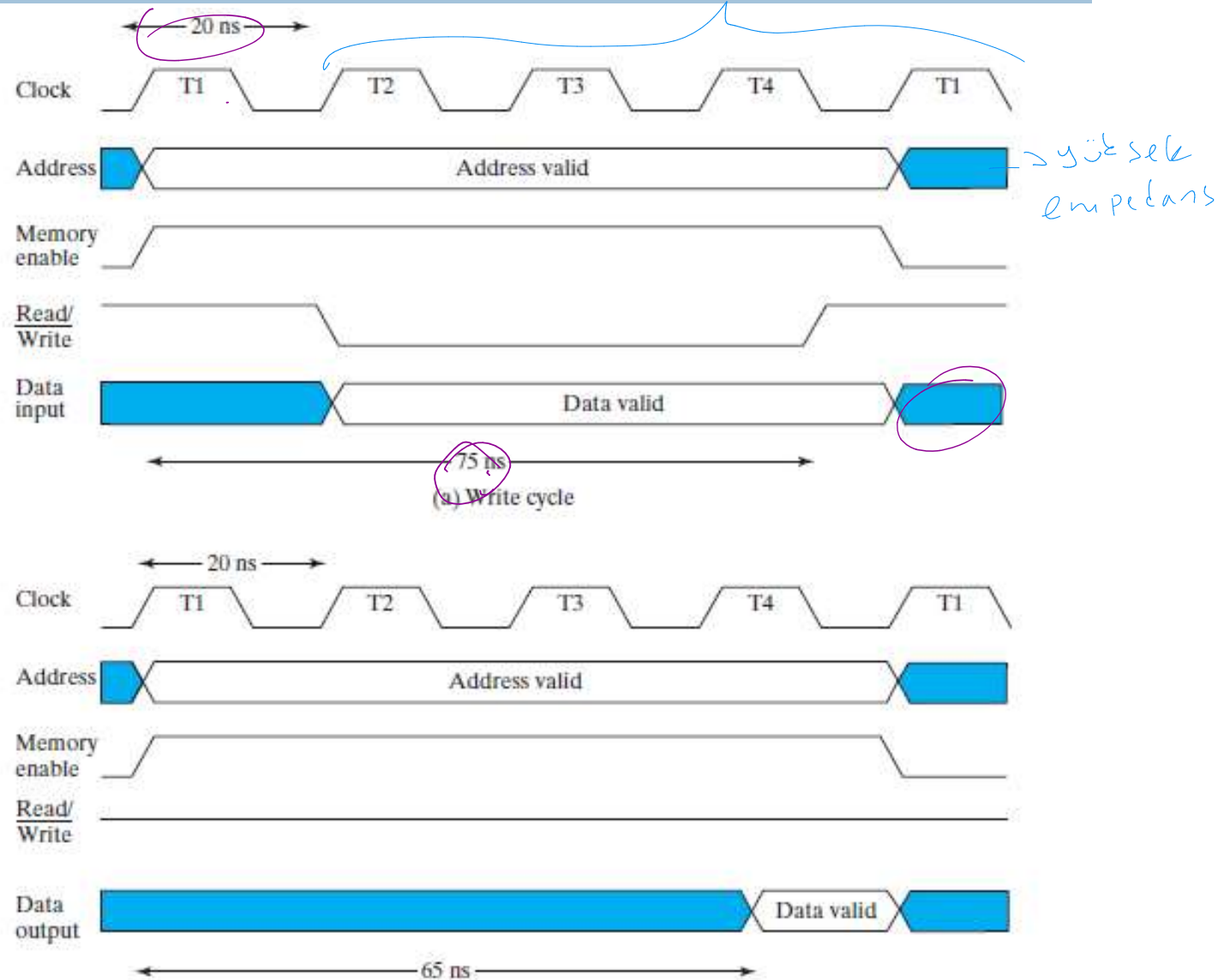
Chip select CS	Read/Write R/W	Memory operation
0	X	None
1	0	Write to selected word
1	1	Read from selected word

Memory Operation

- The operation of memory unit is controlled by an external device, such as a CPU. → *Özunda - Daznır sızış*
- The **access time** of a memory read operation is the maximum time from the application of the address to the appearance of the data at the Data Output.
- The **write cycle time** is the maximum time from the application of the address to the completion of all internal memory operations at the intervals of the cycle time.
- As an example: a CPU operates 50 MHz, giving a period of 20 ns for one clock pulse. The CPU communicates with a memory with an access time of 65 ns and write cycle time of 75 ns.

Memory Operation

Memory Cycle Timing Waveforms. Four pulses T1, T2, T3, and T4 with a cycle of 20 ns.



RAM Integrated Circuits

Chapter 8

SRAM → Static ram

□ Types of random access memory

□ Static – information stored in latches

↖ 6 transistör

Seviye

tektikleneli

□ Dynamic – information stored as electrical charges on capacitors

■ Charge “leaks” off

■ Periodic refresh of charge required

↳ Kondansatörler 10 yıl sonra
fulan boşalır ve bilgi tutmak
sıkıntı

□ Dependence on Power Supply

□ Volatile – loses stored information when power turned off

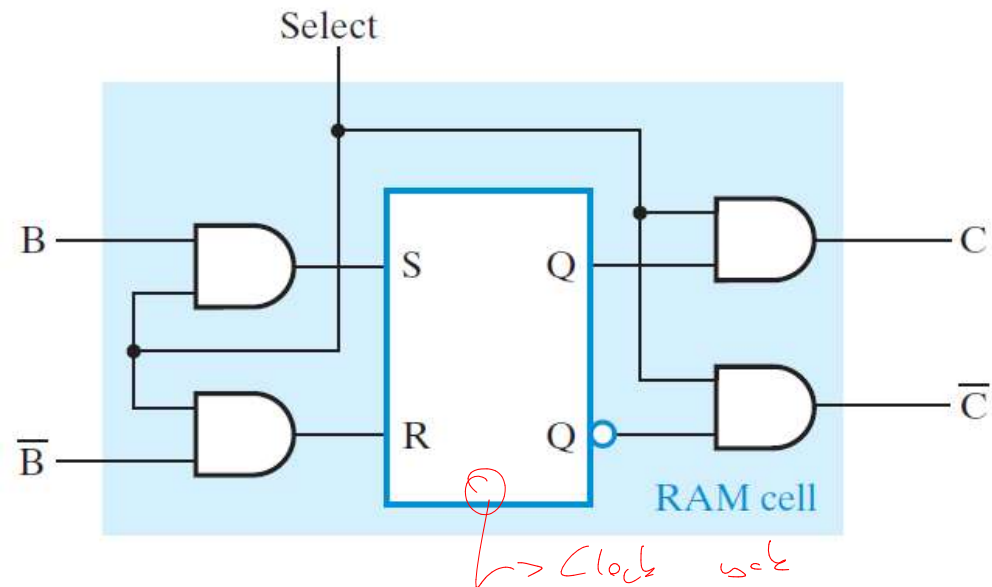
↖ ram

□ Non-volatile – retains information when power turned off

↳ rom

Static RAM Cell

- The internal structure of a RAM chip of m words with n bits per word consists of an array of mn binary storage cells and associated circuitry.
- The **RAM cell** is the basic binary storage element used in the RAM chip.
- Storage Cell
 - ▣ SR Latch
 - ▣ Select input for control
 - ▣ Dual Rail Data Inputs B and $\bar{B}$
 - ▣ Dual Rail Data Outputs C and $\bar{C}$

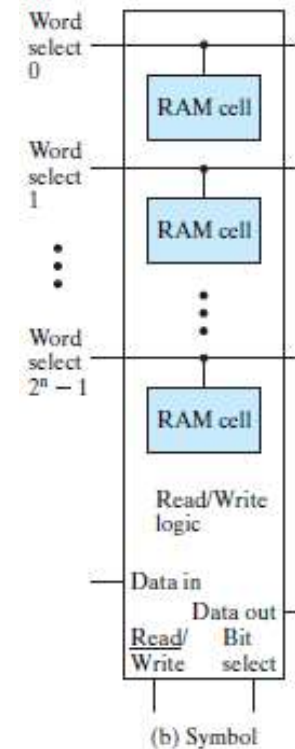
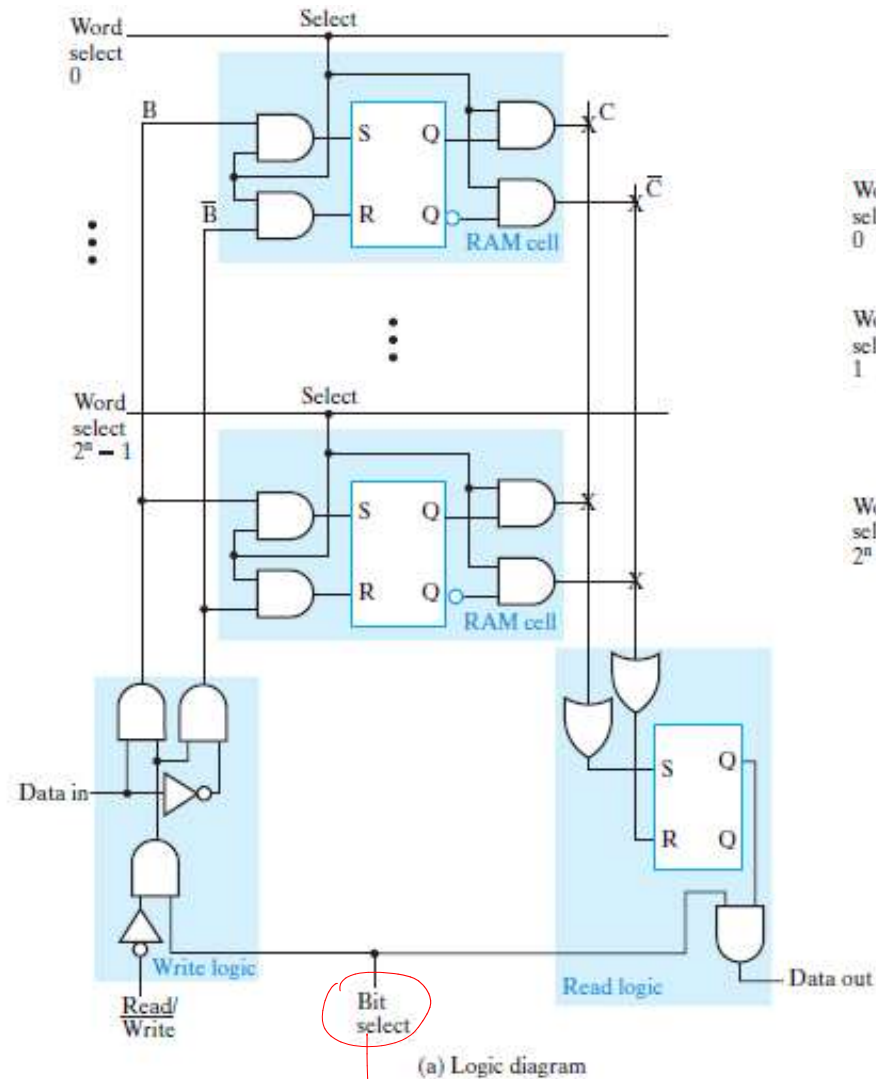


Static RAM Bit Slice

- Represents all circuitry that is required for $2^n - 1$ bit words

- Multiple RAM cells
- Control Lines:
 - Word select i – one for each word
 - Read/Write.
 - Bit Select

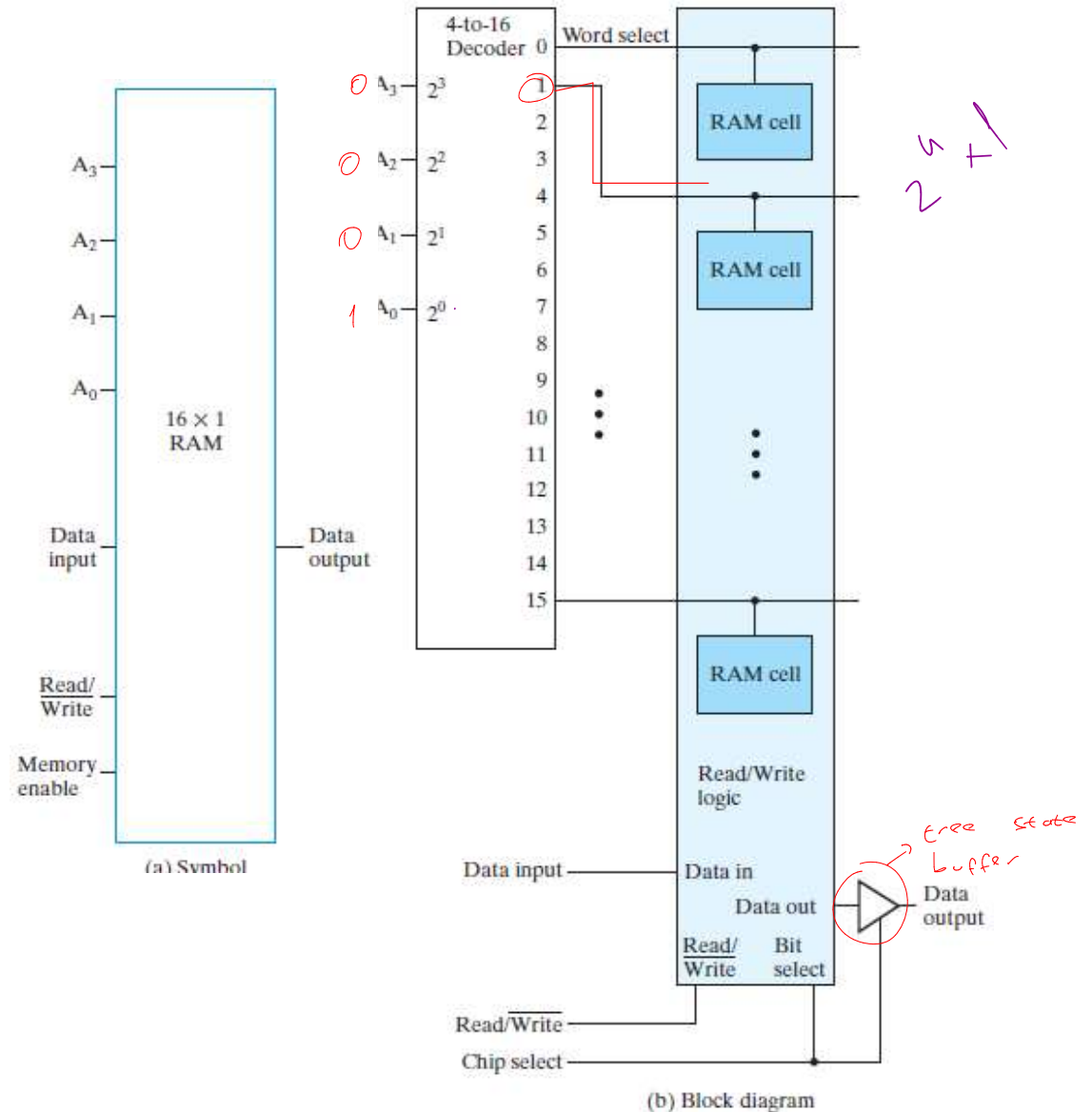
- Data Lines:
 - Data in
 - Data out



enable

2ⁿ-Word × 1-Bit RAM IC

- To build a RAM IC from a RAM slice, we need:
 - ▣ **Decoder** : decodes the n address lines to 2ⁿ word select lines
 - ▣ A **3-state buffer** on the permits RAM ICs to be combined into a RAM with c × 2ⁿ words



Cell Arrays and Coincident Selection

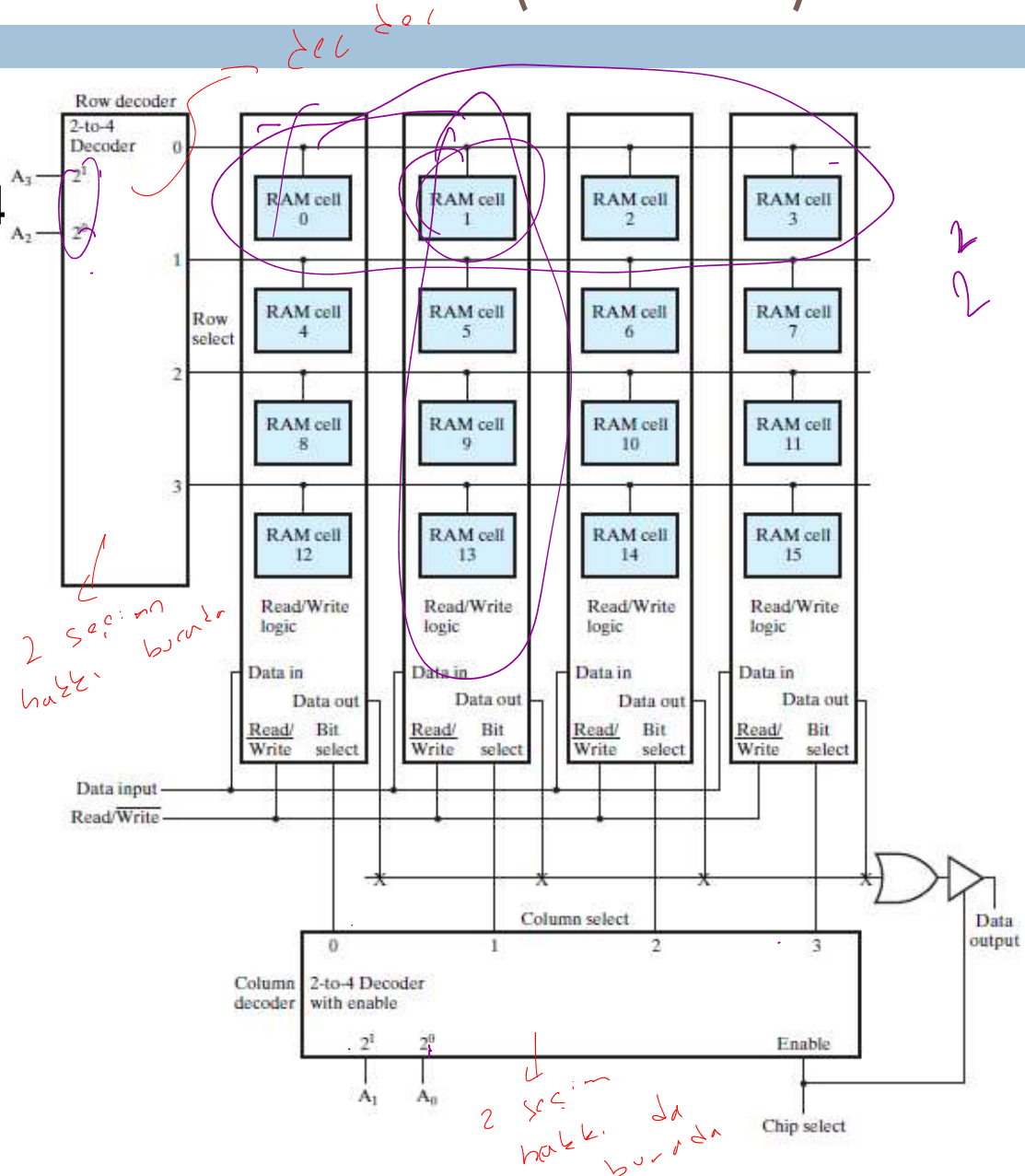
Chapter 8
16

- Memory arrays can be very large =>
 - Large decoders
 - Large fanouts for the bit lines
 - The decoder size and fanouts can be reduced by approximately $\sqrt{n}$ by using a coincident selection in a 2-dimensional array
 - Uses two decoders, one for words and one for bits
 - Word select becomes Row select
 - Bit select becomes Column select
- See next slide for example
 - A_3 and A_2 used for Row select
 - A_1 and A_0 for Column select

Cell Arrays and Coincident Selection (continued)

Diagram of 16x1 RAM Using a 4 x 4 RAM Cell Array

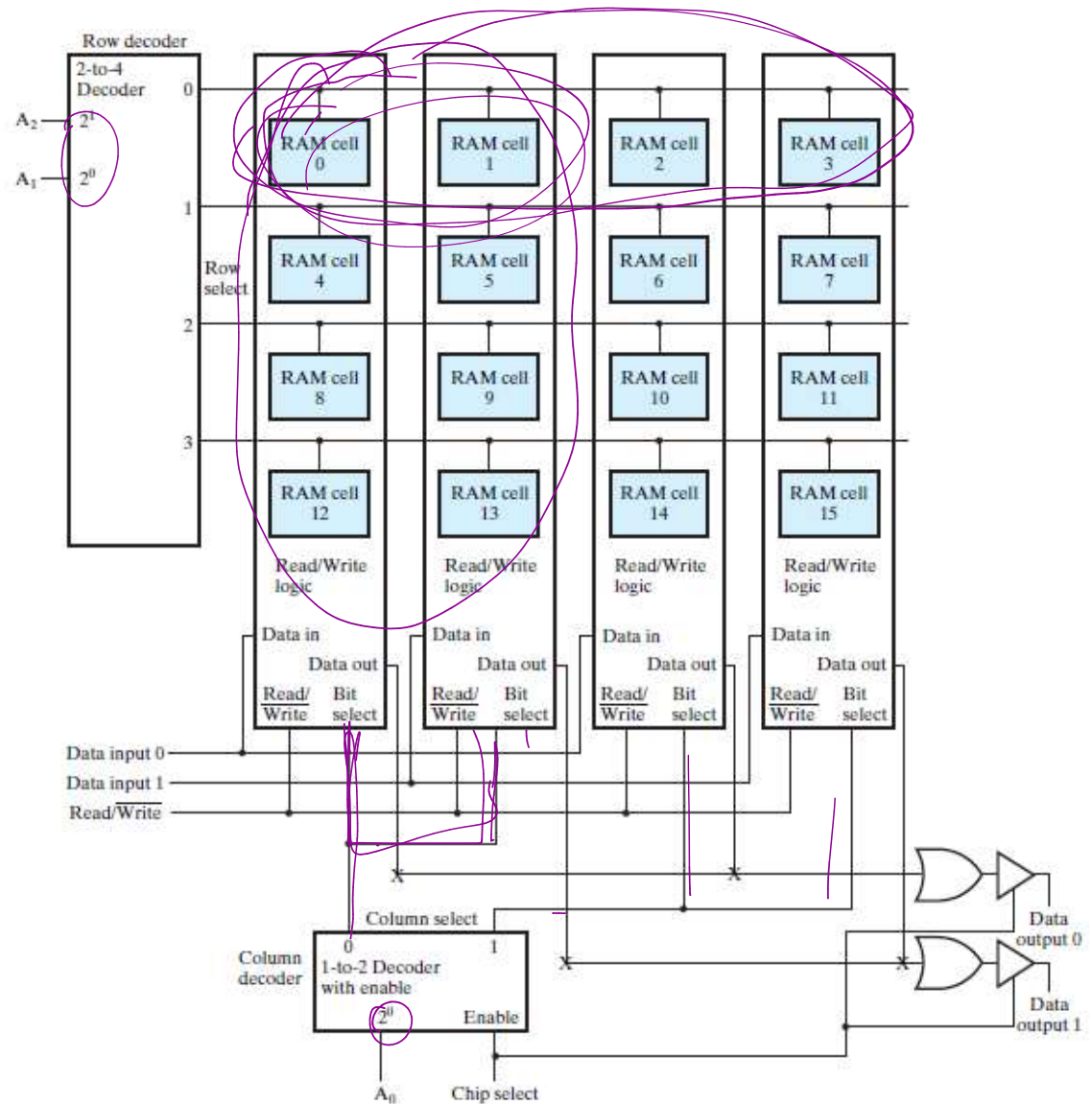
- For example, for the address 1001, the first two address bits are decoded to select row 10, the since two address bits are decoded to select column 01 of the array.



Cell Arrays and Coincident Selection (continued)

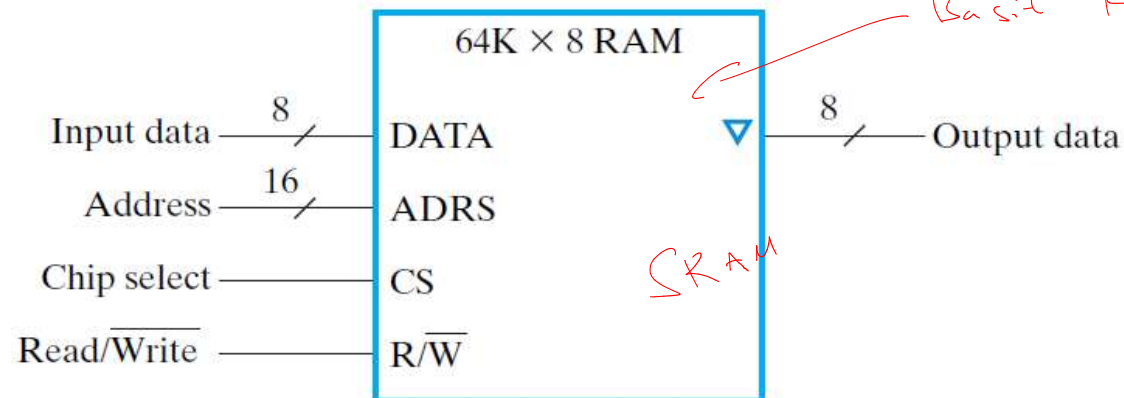
Diagram of 8x2 RAM Using a 4 x 4 RAM Cell Array

- For example, for the address 3 (011), the first two address bits are decoded to select row 1, the final bit, 1, access column 1, which consists of bit slices 2 and 3.



Array of SRAM ICs

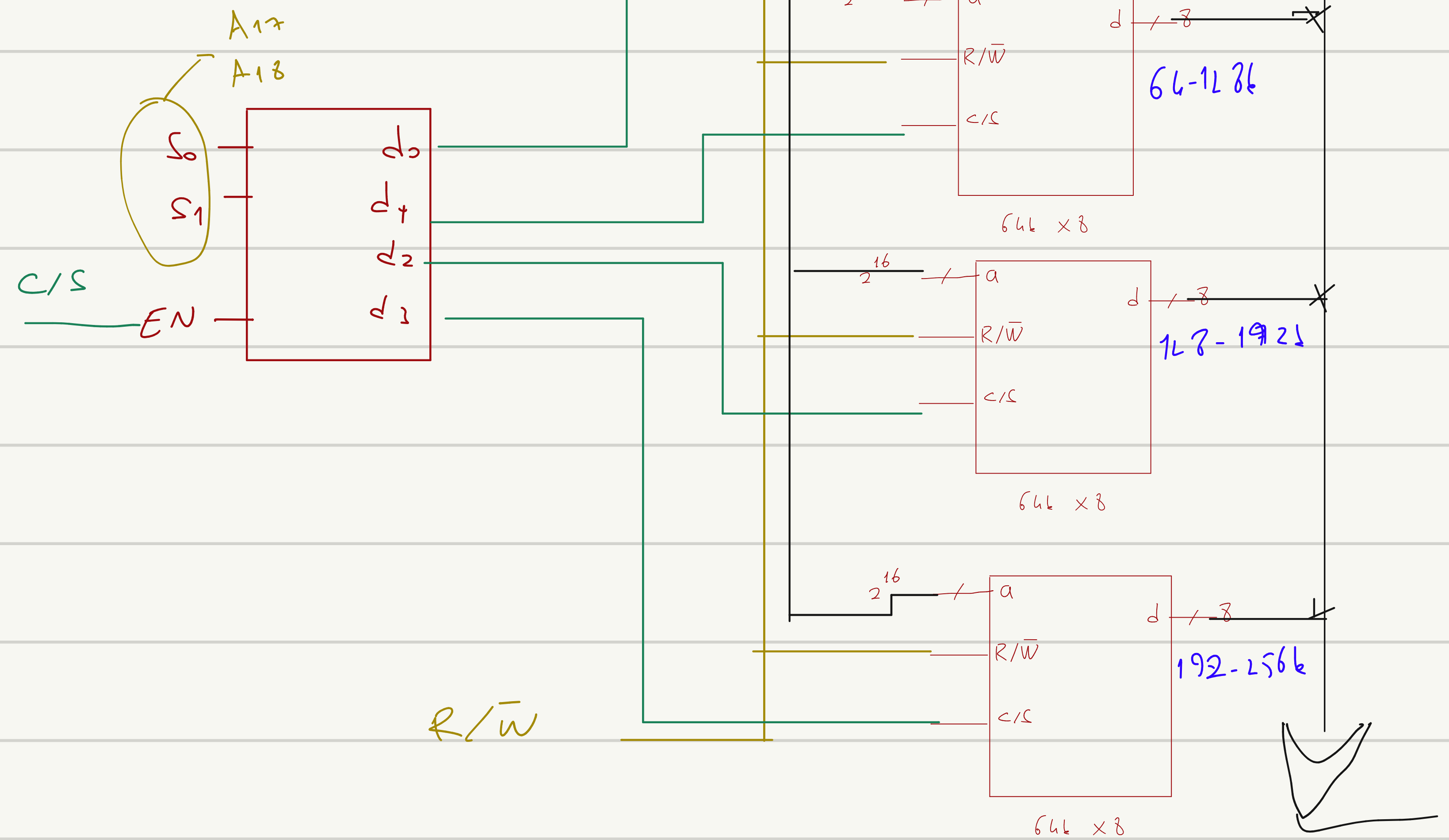
- If the memory unit need for an application is larger than the capacity of one chip, it is necessary to combine a number of chips in an array to form the required size of a memory.
 - ▣ An increase in the number of words requires that we increase the address length.
 - ▣ An increase in the number of bits per word requires that we increase the number of data input and output lines.
- Using the CS lines, we can make larger memories from smaller ones by trying all address, data, and R/W lines in parallel, and using the decoded higher order address bits to control CS.



64k x 8'lik SRAM bloklarını kullanarak

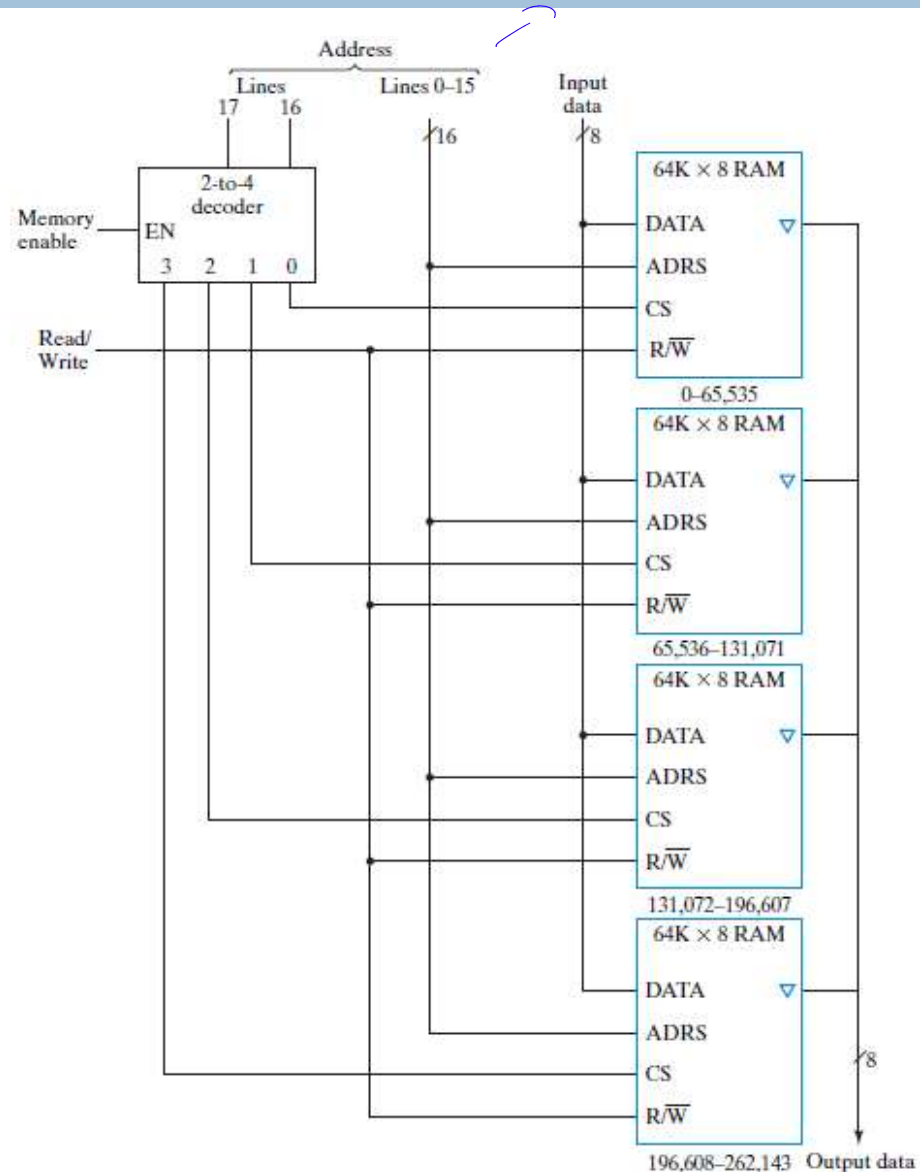
256k x 8'lik RAM bloğunu tasarla

$$\frac{256k \times 8}{64k \times 8} \rightarrow 4 \text{ tane lazım}$$



Making Larger Memories

- Constructing a 256K x 8 RAM with four 64K x 8 RAM chips.
- 256K-word memory requires an 18-bit address. The least 16 least significant bits are applied to the address of all four chips. The two most significant bits are applied to a 2x4 decoder.

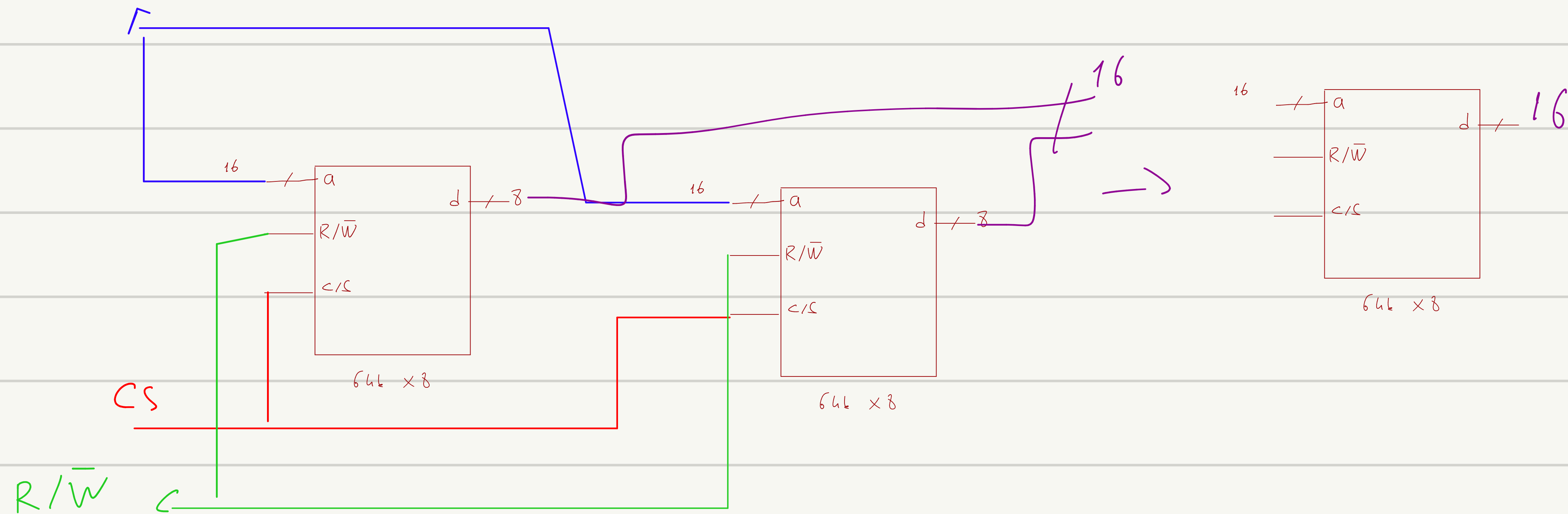


64k x 8'lik SRAM bloklarını kullanarak

64k x 16'lık RAM bloğunu tasarla

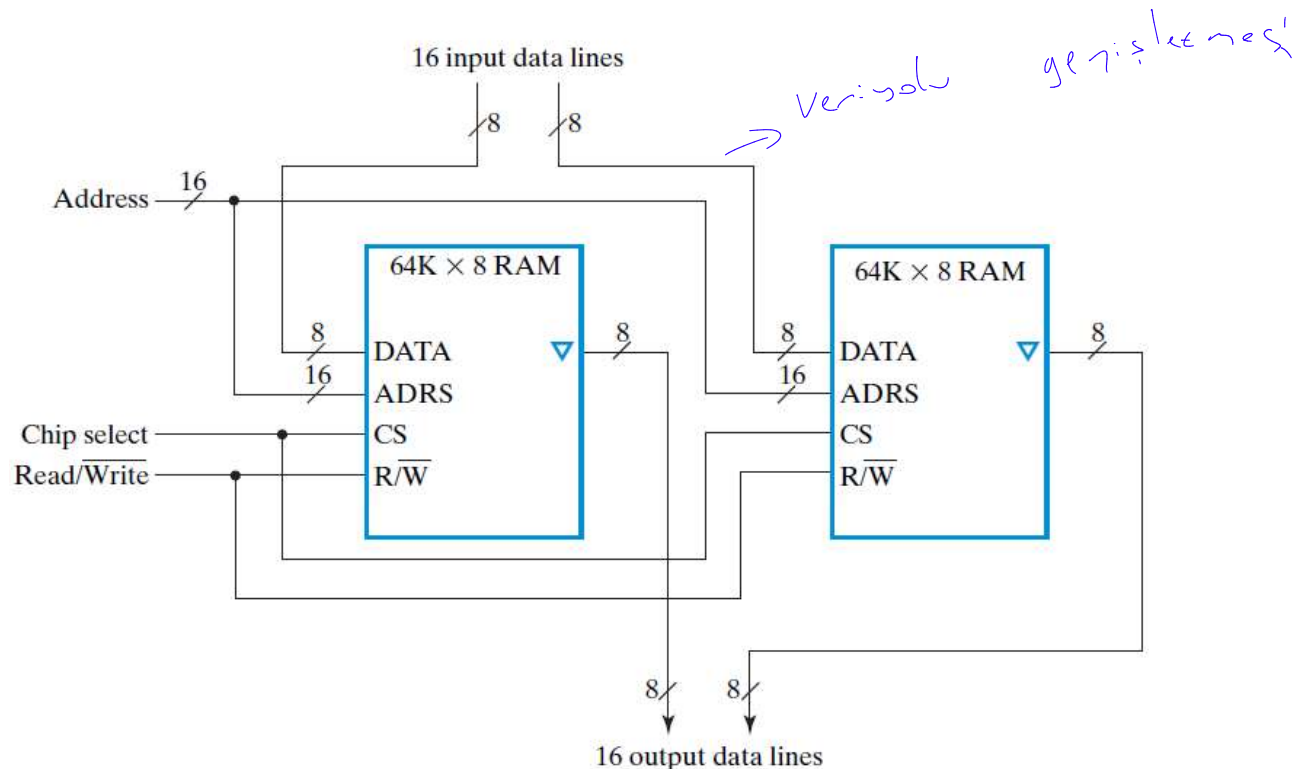
$$\frac{64k \times 16}{64k \times 8} \rightarrow 2$$

A0 - A15



Making Wider Memories

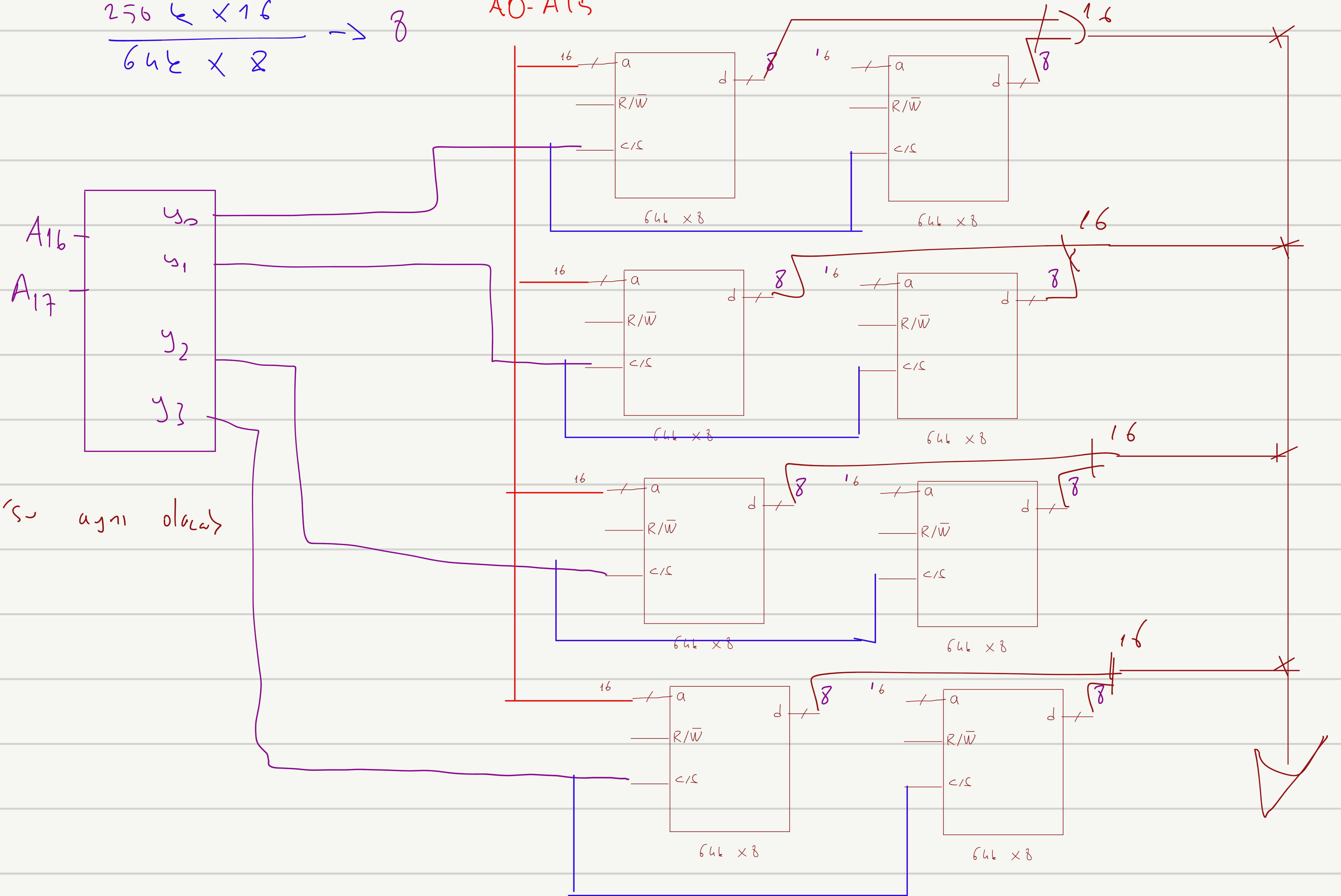
- To construct wider memories from narrow ones, we tie the address and control lines in parallel and keep the data lines separate.
- For example, two $64\text{K} \times 8$ chips can form a $64\text{K} \times 16$ memory.



64k x 8'lik SRAM bloklarını kullanarak 256 x 16'lık RAM bloğunu tasarla

$$\frac{256k \times 16}{64k \times 8} \rightarrow 8$$

AO-A15



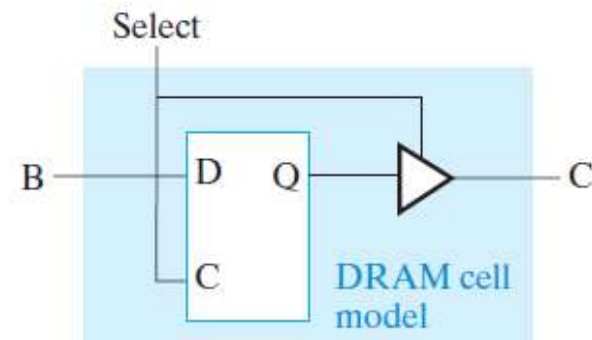
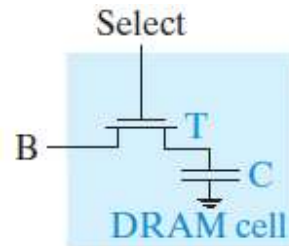
her birin R/W'su aynı olacak

Ge_cti

Dynamic RAM (DRAM)

Chapter 8
22

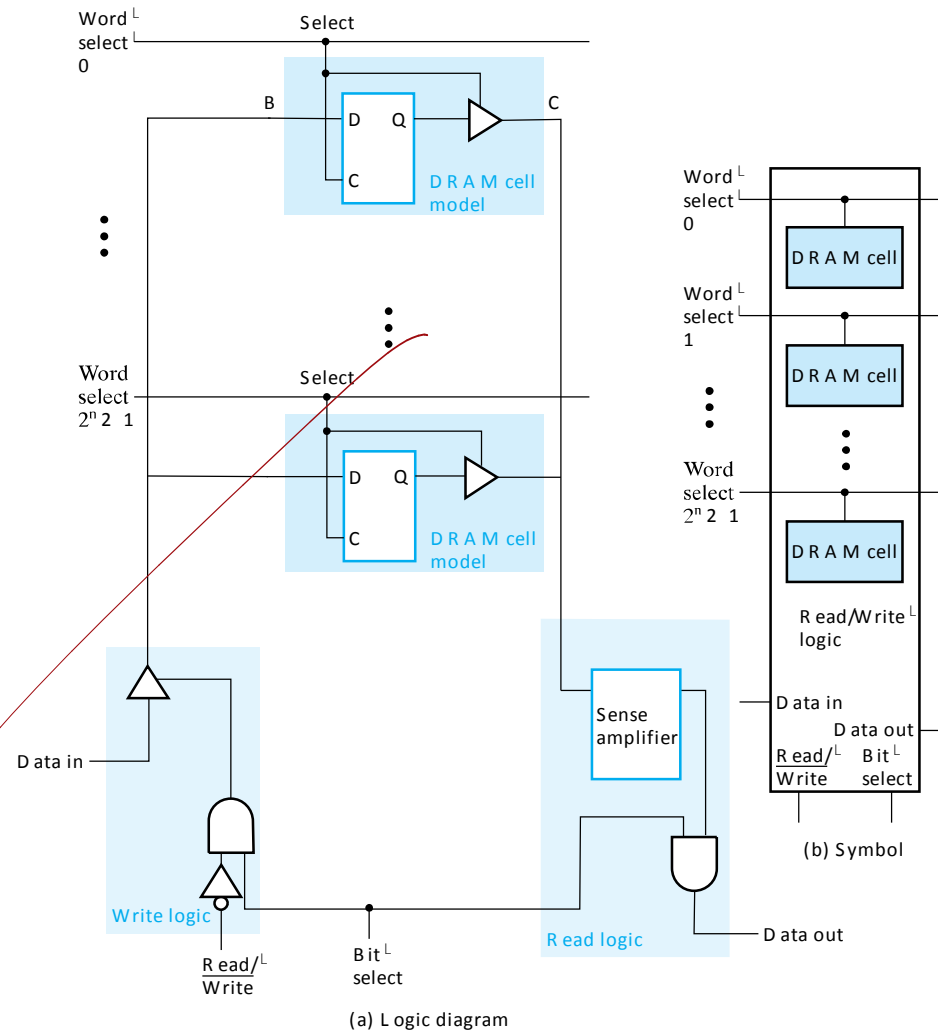
- Because of its ability to provide high storage capacity at low cost, DRAM dominates the high capacity memory applications.
- Basic Principle: Storage of information on capacitors.
- DRAM Cell consists of a capacitor C and a transistor T.
 - ▣ The capacitor is used to store electrical charge.
Sufficient charge → Logic 1, Insufficient charge → Logic 0
 - ▣ Use of transistor as “switch” to:
 - Store charge
 - Charge or discharge



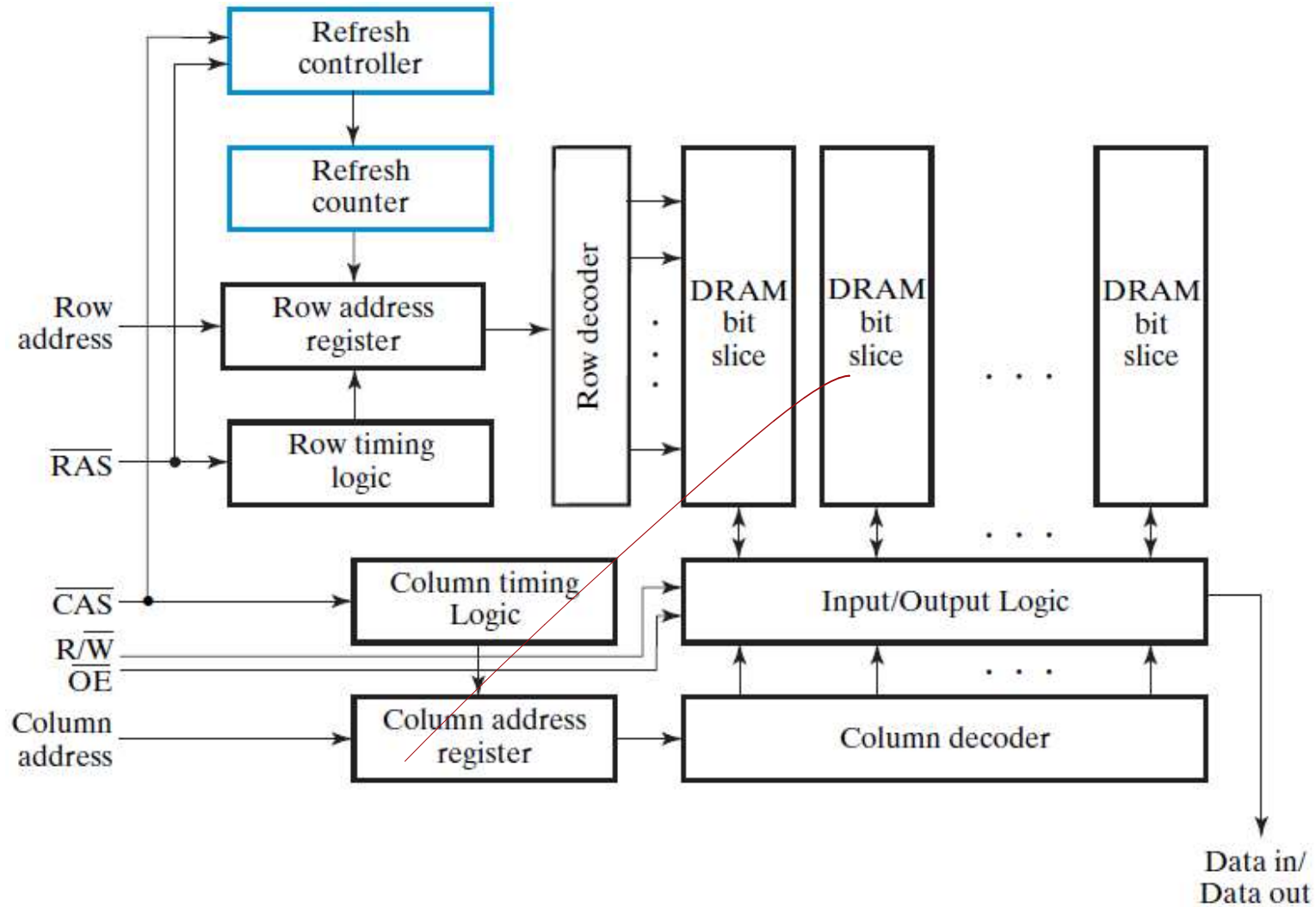
Dynamic RAM - Bit Slice

Chapter 8
21

- C is driven by 3-state drivers
- Sense amplifier is used to change the small voltage change on C into H or L
- In the electronics, B, C, and the sense amplifier output are connected to make destructive read into non-destructive read



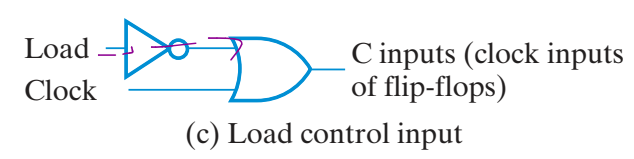
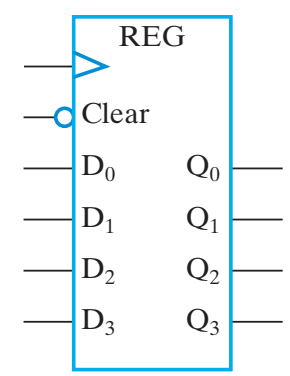
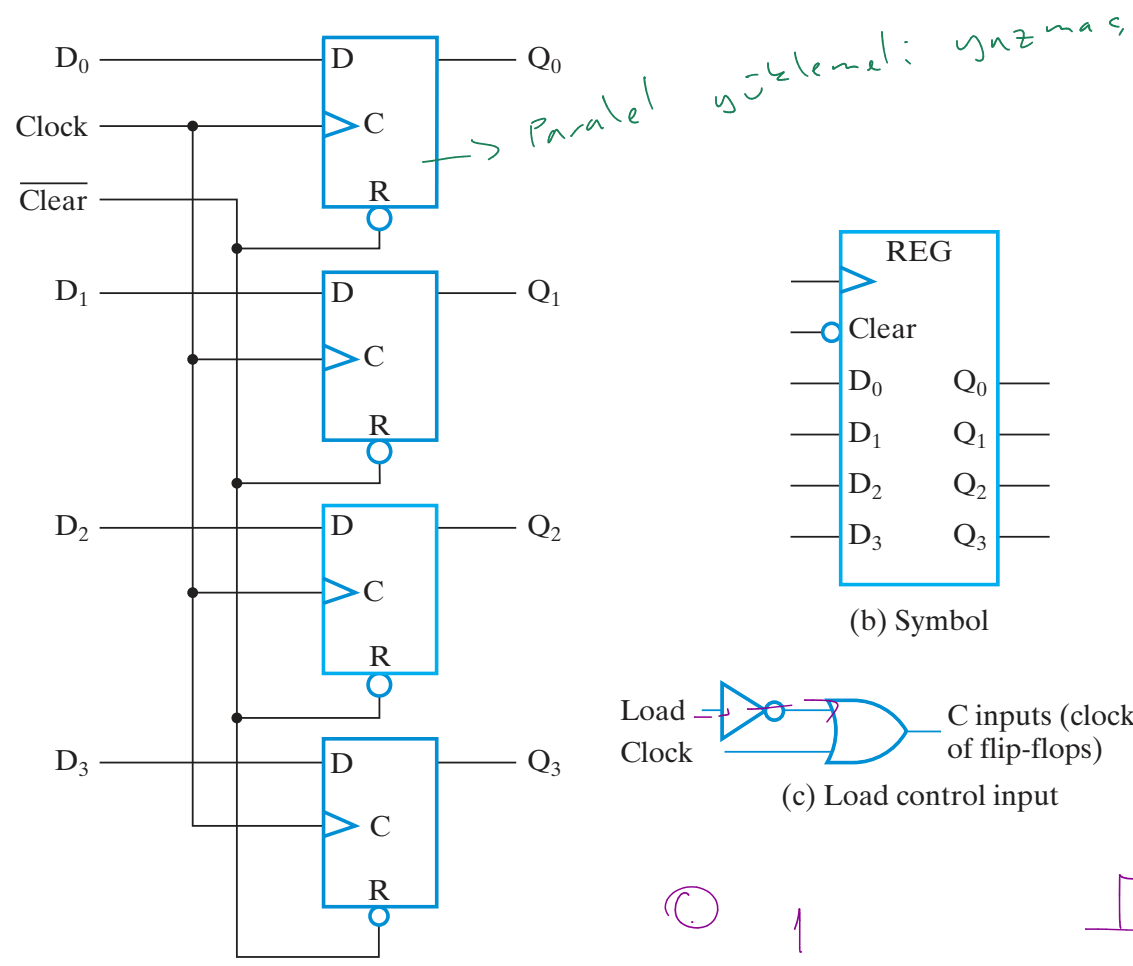
Dynamic RAM



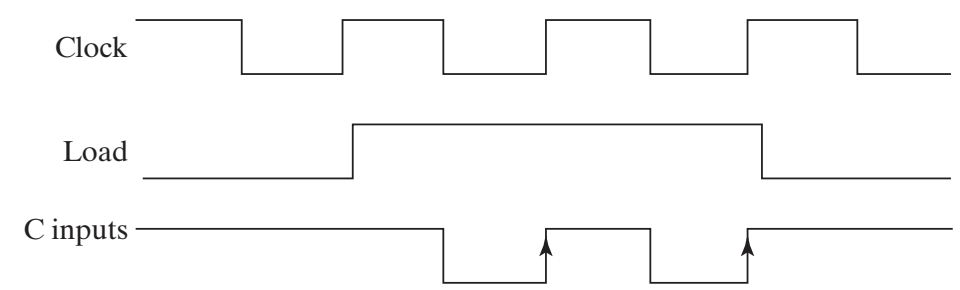
Block Diagram of a DRAM Including Refresh Logic

DRAM Types

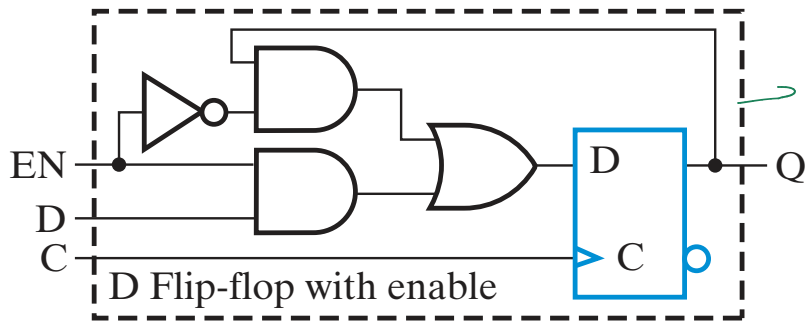
Type	Abbreviation	Description
Fast Page Mode DRAM	FPM DRAM	Takes advantage of the fact that, when a row is accessed, all of the row values are available to be read out. By changing the column address, data from different addresses can be read out without reapplying the row address and waiting for the delay associated with reading out the row cells to pass if the row portion of the addresses match.
Extended Data Output DRAM	EDO DRAM	Extends the length of time that the DRAM holds the data values on its output, permitting the CPU to perform other tasks during the access since it knows the data will still be available.
Synchronous DRAM	SDRAM	Operates with a clock rather than being asynchronous. This permits a tighter interaction between memory and CPU, since the CPU knows exactly when the data will be available. SDRAM also takes advantage of the row value availability and divides memory into distinct banks, permitting overlapped accesses.
Double Data Rate Synchronous DRAM	DDR SDRAM	The same as SDRAM except that data output is provided on both the negative and the positive clock edges.
Rambus DRAM	RDRAM	A proprietary technology that provides very high memory access rates using a relatively narrow bus.
Error-Correcting Code	ECC	May be applied to most of the DRAM types above to correct single bit data errors and often detect double errors.



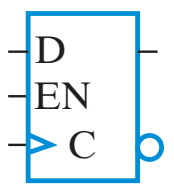
(a) Logic diagram



(d) Timing diagram



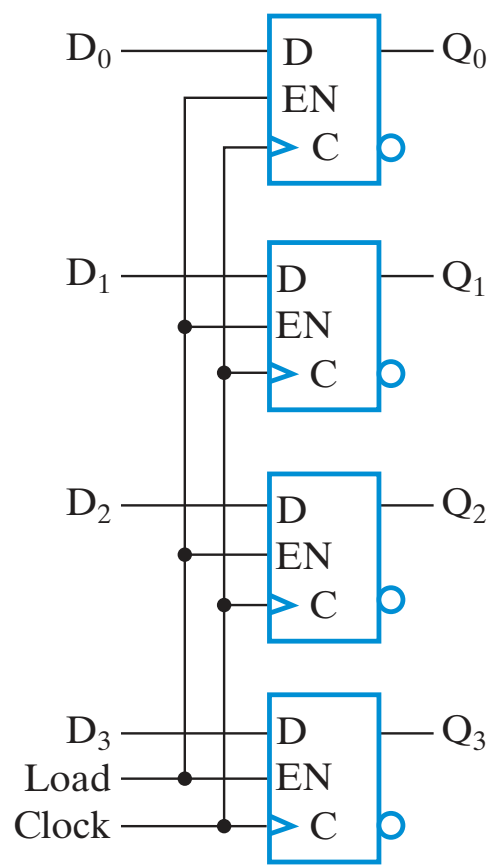
bu da load'ı
giriş vermes



(a)

(b) Q_i

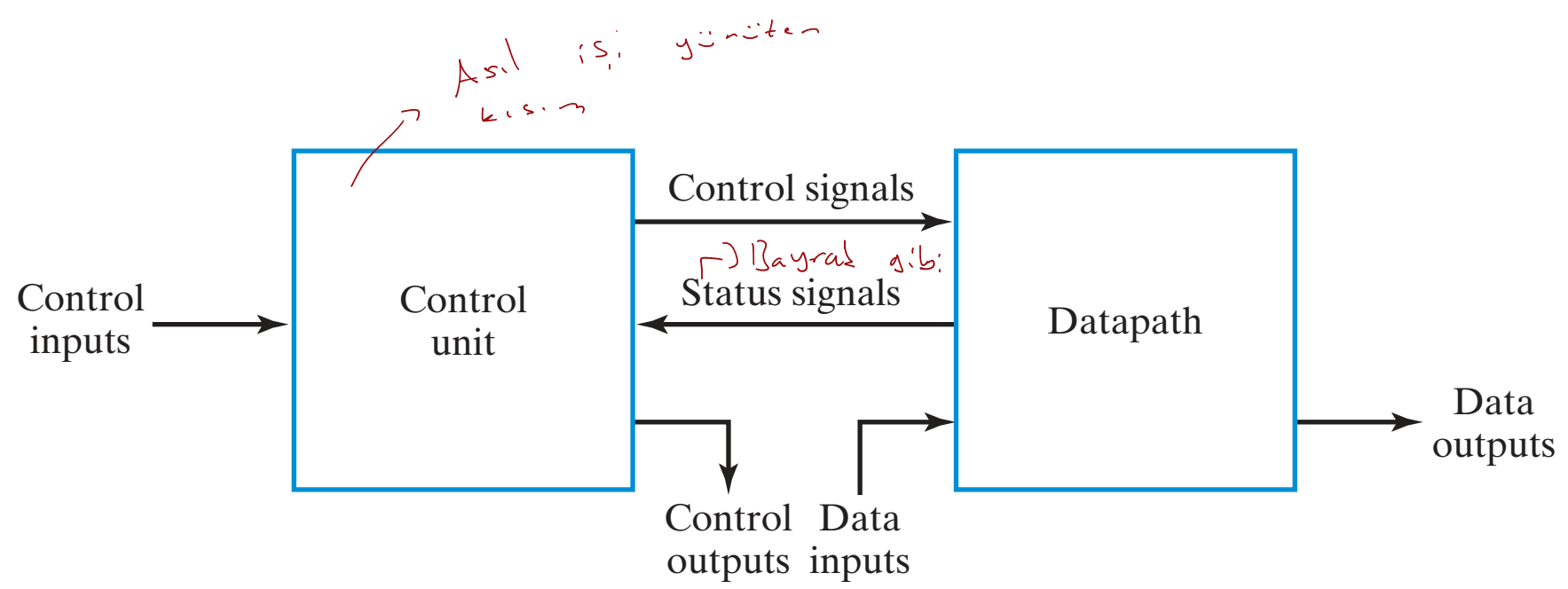
L	D_i	q_i	Q_i	DFF
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1



$L D_i$	0	0	1	1
q_i	0	0	0	1
	0	0	0	0
	1	1	1	0

$q_i L' + L D_i$

(c)





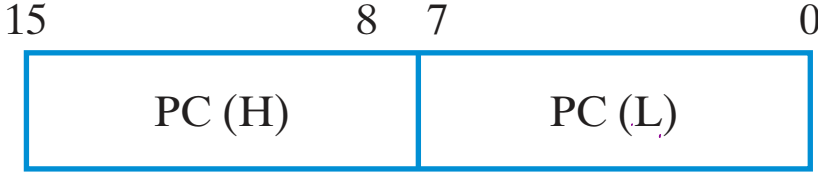
(a) Register R



(b) Individual bits of 8-bit register



(c) Numbering of 16-bit register

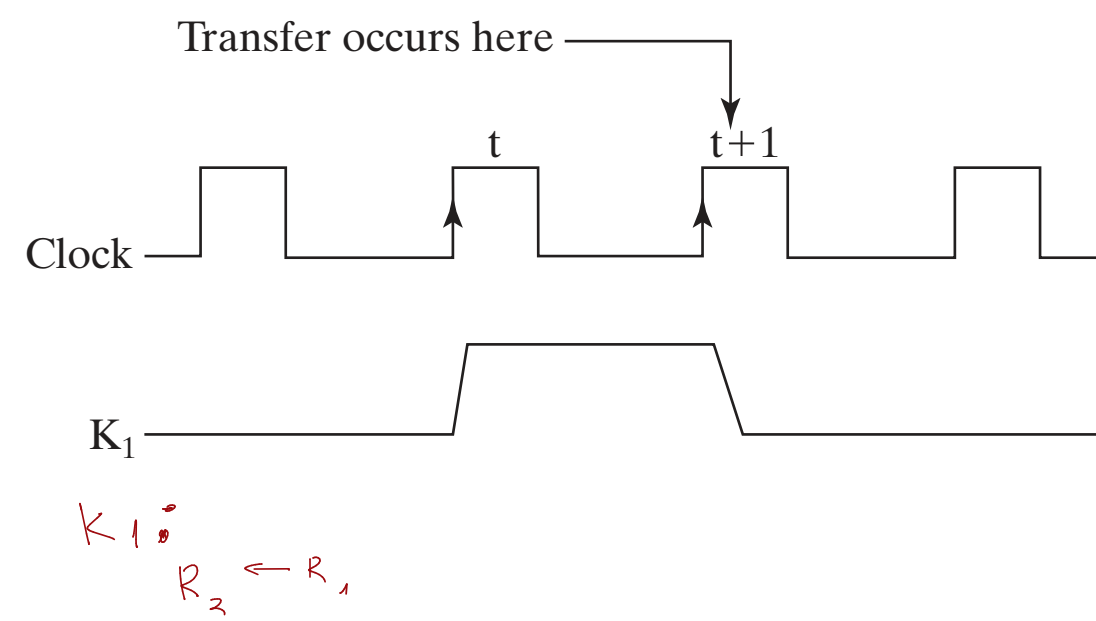
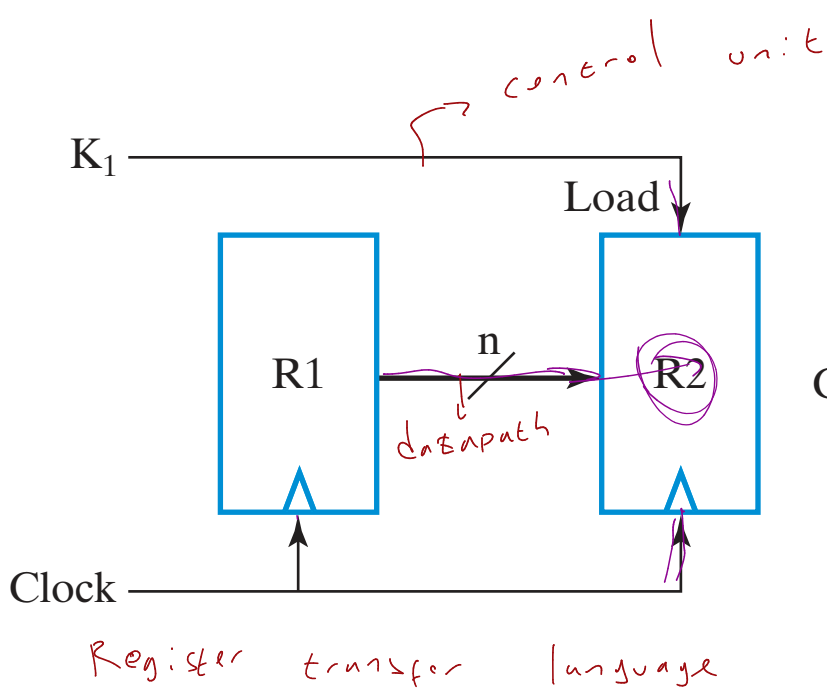


(d) Two-part 16-bit register

$$\frac{CLK}{5}$$

$$\frac{K_1}{1}$$

$$\frac{R_2}{R_1}$$



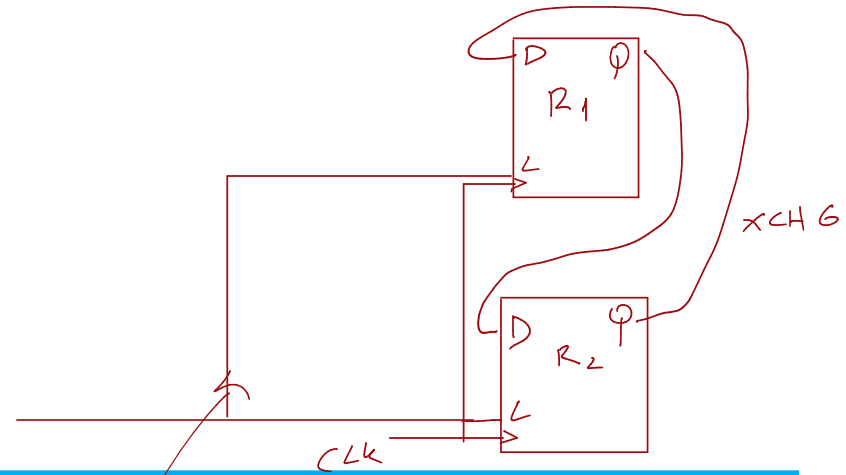


TABLE 7-1
Basic Symbols for Register Transfers

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	$AR, R2, DR, IR$
Parentheses	Denotes a part of a register	$R2(1), R2(7:0), AR(L)$
Arrow	Denotes transfer of data	$R1 \leftarrow R2$
Comma	Separates simultaneous transfers	$R1 \leftarrow R2, R2 \leftarrow R1$
Square brackets	Specifies an address for memory	$DR \leftarrow [M][AR]$

Handwritten notes:

- bit arithmetic
- low
- XCHG
- Address register
- Memory

111 111

111 111

□ **TABLE 7-2**

Textbook RTL, VHDL, and Verilog Symbols for Register Transfers

Operation	Text RTL <i>→ *</i>	VHDL <i>→ Don't Conc</i>	Verilog <i>→</i>
Combinational assignment	=	<= (concurrent)	assign = (nonblocking)
Register transfer	←	<= (concurrent)	<= (nonblocking)
Addition	+	+	+
Subtraction	−	−	−
Bitwise AND	^	and	&
Bitwise OR	∨	or	
Bitwise XOR	⊕	xor	^
Bitwise NOT	<u>¬</u> (overline)	not	~
Shift left (logical)	sl	sll	<<
Shift right (logical)	sr	srl	>>
Vectors/registers	A(3:0)	A(3 down to 0)	A[3:0]
Concatenation		&	{ , }

0000 || 1111 → 0000 1111

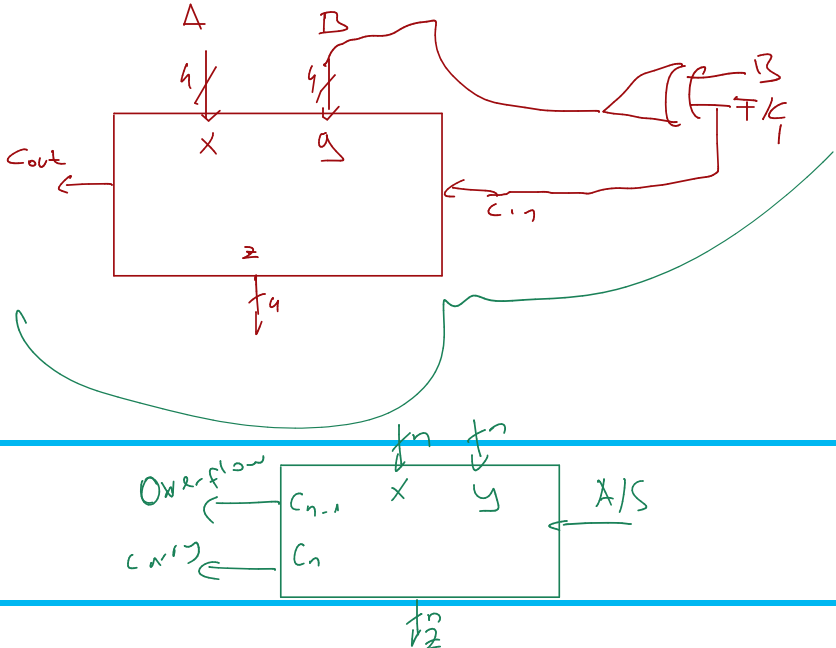
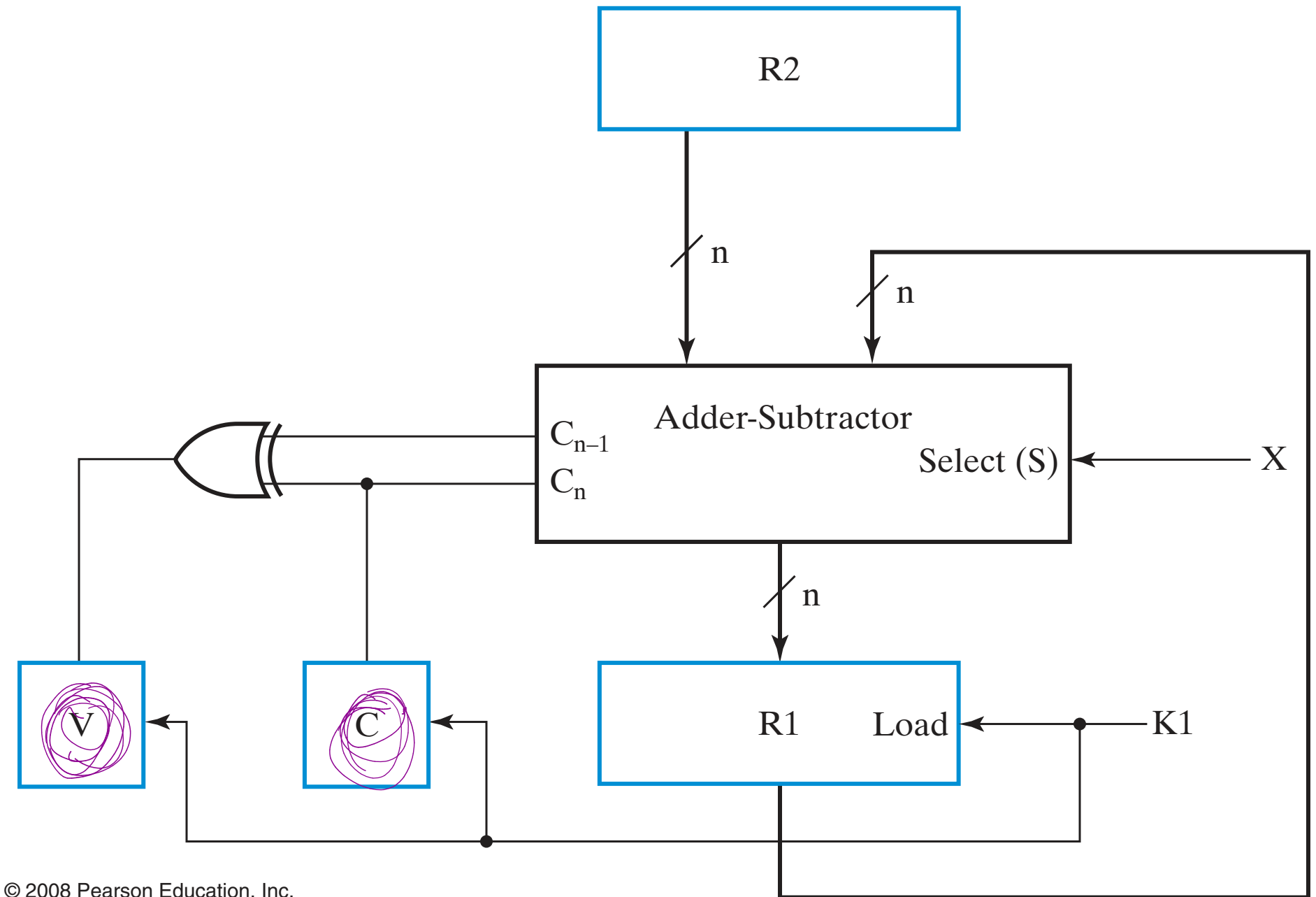
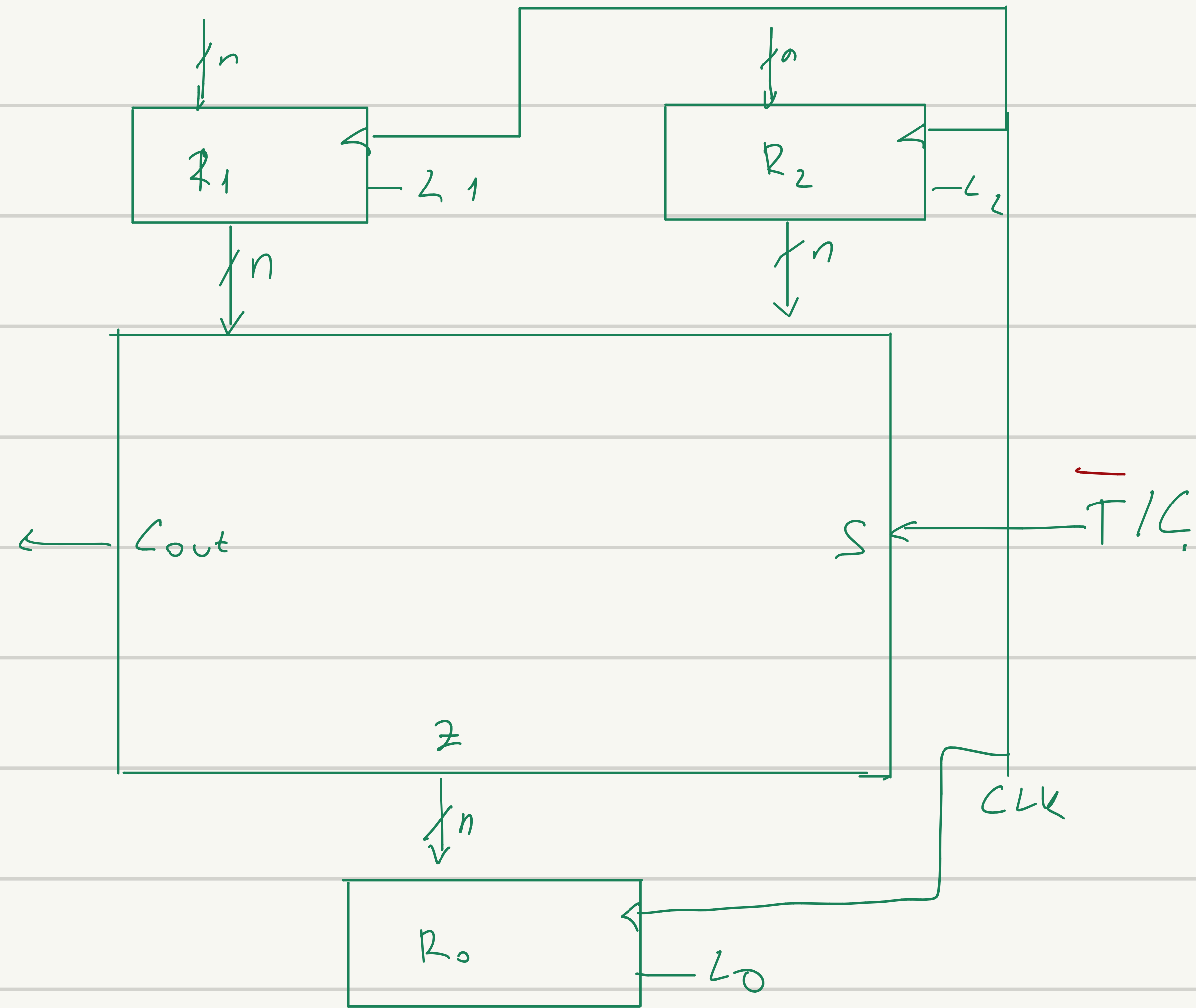


TABLE 7-3
Arithmetic Microoperations

Symbolic designation	Description
$R0 \leftarrow R1 + R2$	Contents of $R1$ plus $R2$ transferred to $R0$
$R2 \leftarrow \overline{R2}$	Complement of the contents of $R2$ (1's complement)
$R2 \leftarrow \overline{R2} + 1$	2's complement of the contents of $R2$
$R0 \leftarrow R1 + \overline{R2} + 1$	$R1$ plus 2's complement of $R2$ transferred to $R0$ (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of $R1$ (count up)
$R1 \leftarrow R1 - 1$	Decrement the contents of $R1$ (count down)





$$\textcircled{1} R_0 \leftarrow R_1 + R_2$$

$$\overline{T/C} \quad L_0: R_0 \leftarrow R_1 + R_2$$

$$\textcircled{2} R_0 \leftarrow \overline{R_2}$$

$$L1: R_1 \leftarrow FFH(-1)$$

$$L0 \quad T/C: R_0 \leftarrow R_1 + \overline{R_2} + 1$$

$$\overline{R_0} \leftarrow \overline{R_1} \quad R_0 \leftarrow -R_1 \quad R_0 \leftarrow R_2 - R_1;$$

- > bu yapı bunları gösterilemiyor

$$\textcircled{3} R_0 \leftarrow \overline{R_2} + 1$$

$$L1: R \leftarrow 0$$

$$L0 \quad T/C: R_0 \leftarrow \overline{R_2} + 1 + R_1$$

$$\textcircled{4} R_0 \leftarrow R_1 - 1$$

$$L2: R_2 \leftarrow 1$$

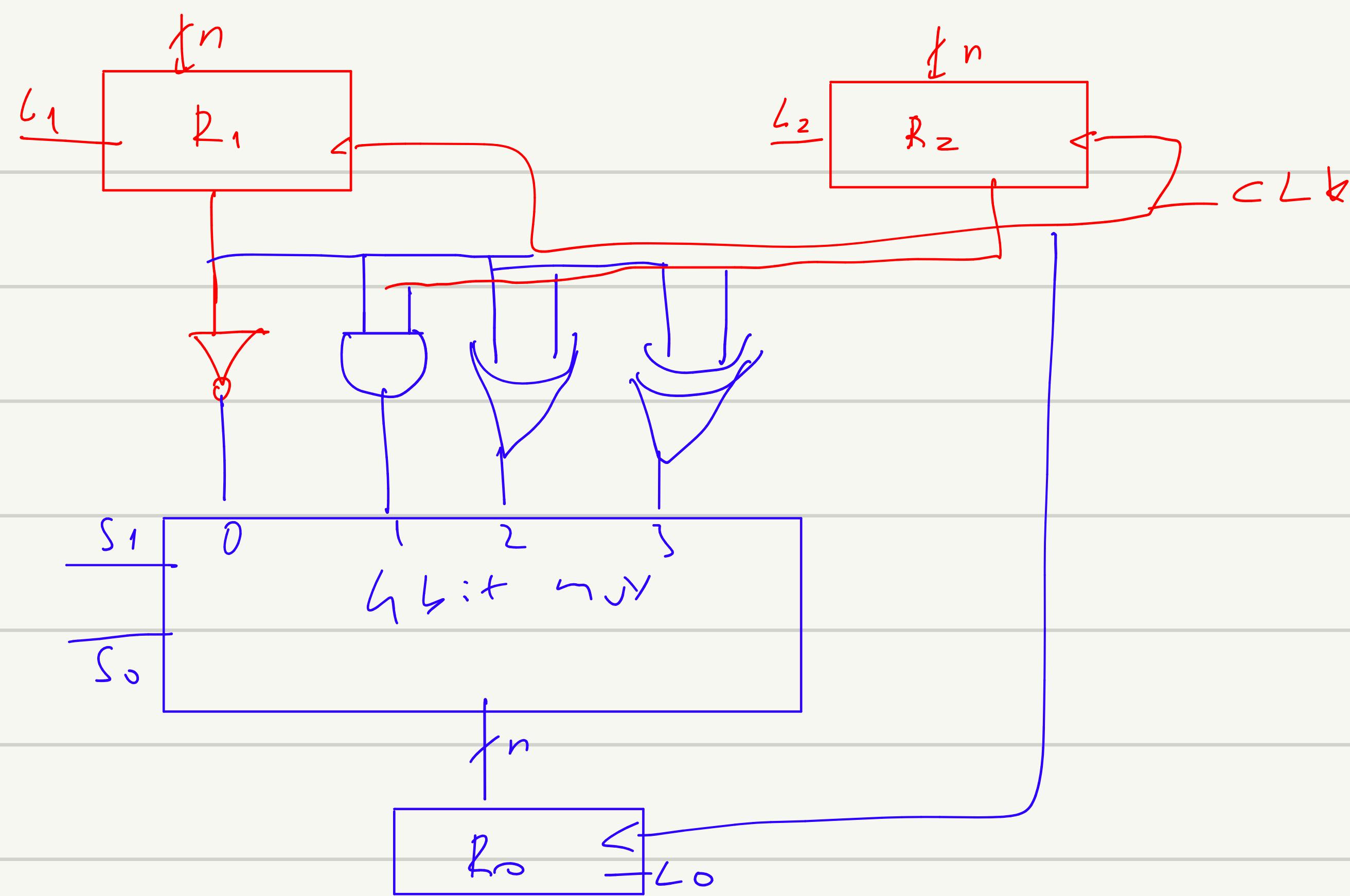
$$L0 \quad T/C: R_0 \leftarrow R_1 - R_2$$

$$L1: R_1 \leftarrow 0$$

$$L0 \quad T/C: R_0 \leftarrow R_1 - R_2$$

□ TABLE 7-4
Logic Microoperations

Symbolic designation	Description
$R0 \leftarrow \overline{R1}$	Logical bitwise NOT (1's complement)
$R0 \leftarrow R1 \wedge R2$	Logical bitwise AND (clears bits)
$R0 \leftarrow R1 \vee R2$	Logical bitwise OR (sets bits)
$R0 \leftarrow R1 \oplus R2$	Logical bitwise XOR (complements bits)



① $R_0 \leftarrow \overline{R_1}$

$L_0 \quad \overline{S_1 S_0} :$
 $R_0 \leftarrow R_1$

② $R_0 \leftarrow R_1 \wedge R_2$

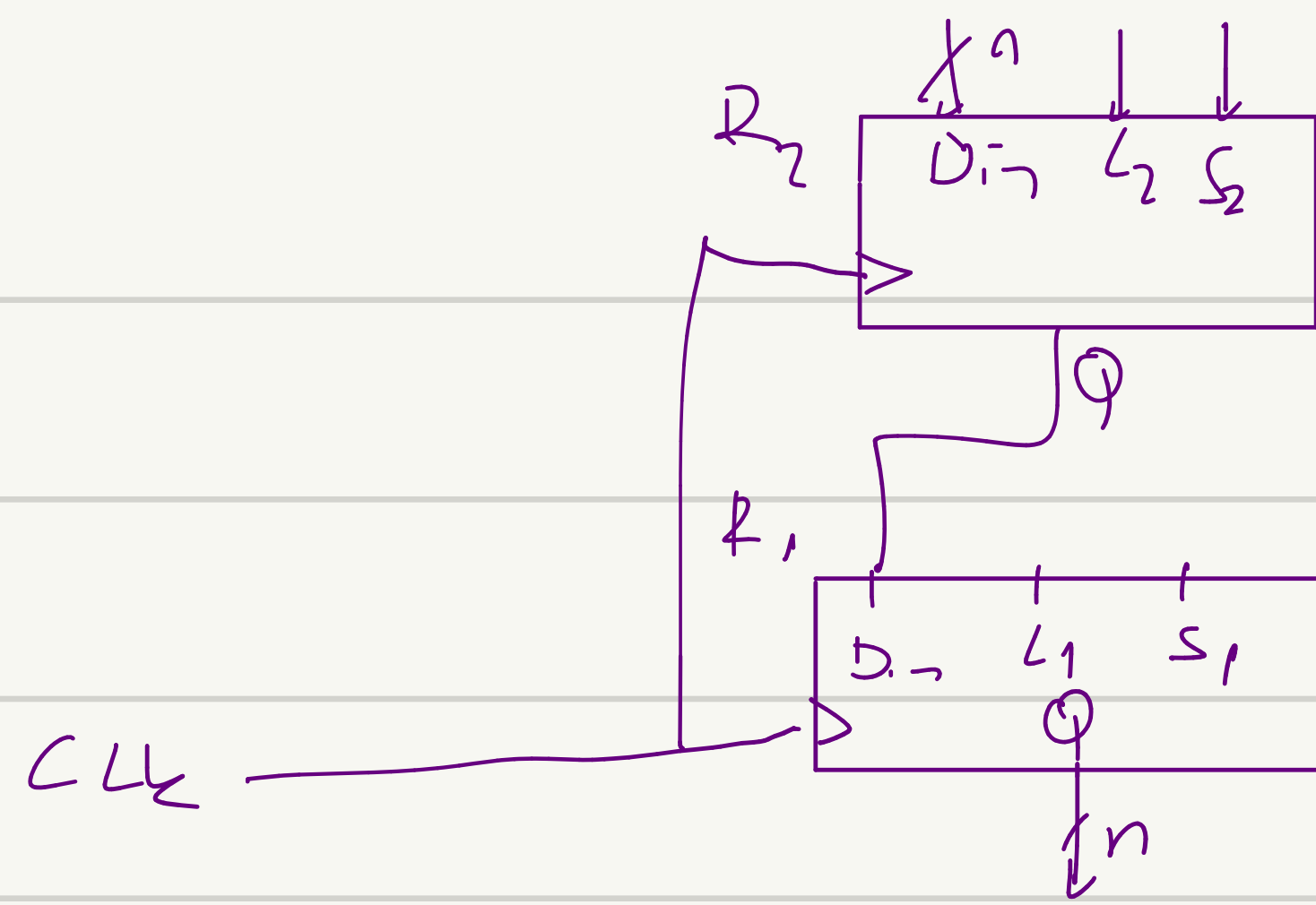
$L_0 \quad \overline{S_1 S_0} :$
 $R_0 \leftarrow R_1 \wedge R_2$

③ $R_0 \leftarrow R_1 \oplus R_2$

$L_0 \quad S_1 S_2 :$
 $R_0 \leftarrow R_1 \oplus R_2$

□ TABLE 7-5
Examples of Shifts

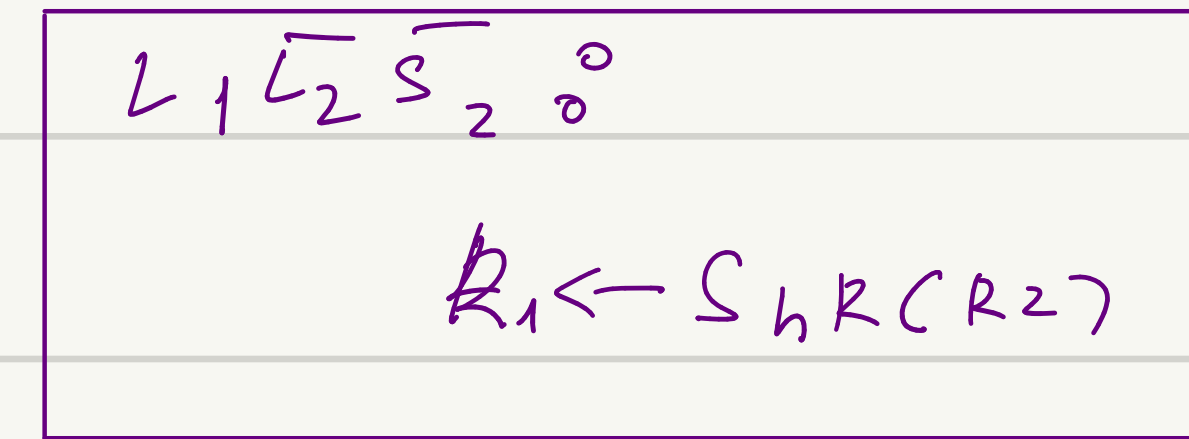
Type	Symbolic designation	Eight-bit examples	
		Source <i>R2</i>	After shift: Destination <i>R1</i>
shift left	$R1 \leftarrow sl\ R2$	10011110	00111100
shift right	$R1 \leftarrow sr\ R2$	11100101	01110010



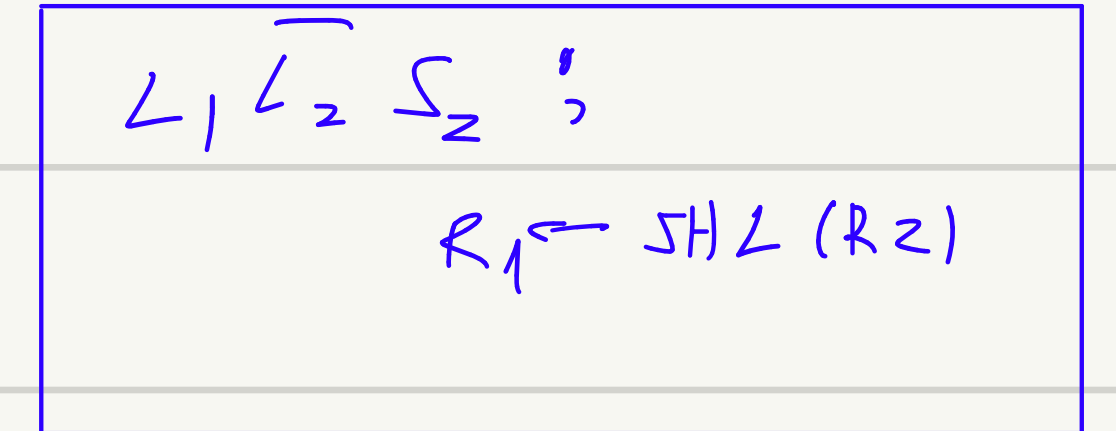
$L=1$ Paralel gökleme
 $L=0$ $S=0$ Sağa shift
 $L=0$ $S=1$ Left shift

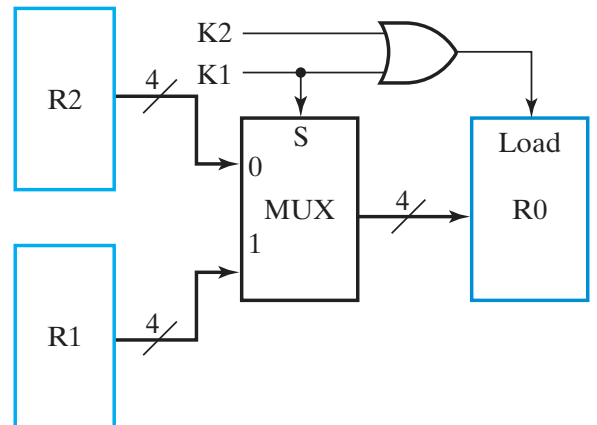
Sayı öteleme

① $R_1 \leftarrow SHR(R_2)$

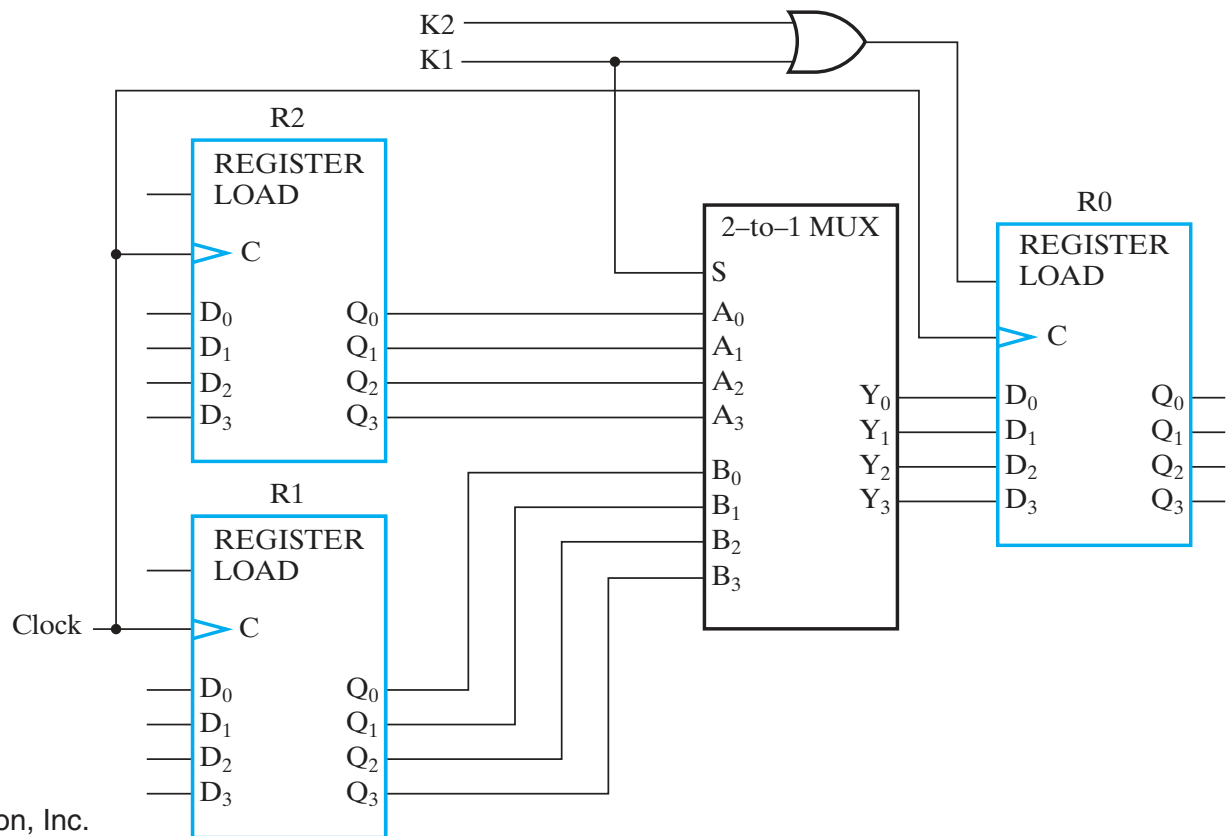


② $R_1 \leftarrow SHL(R_2)$

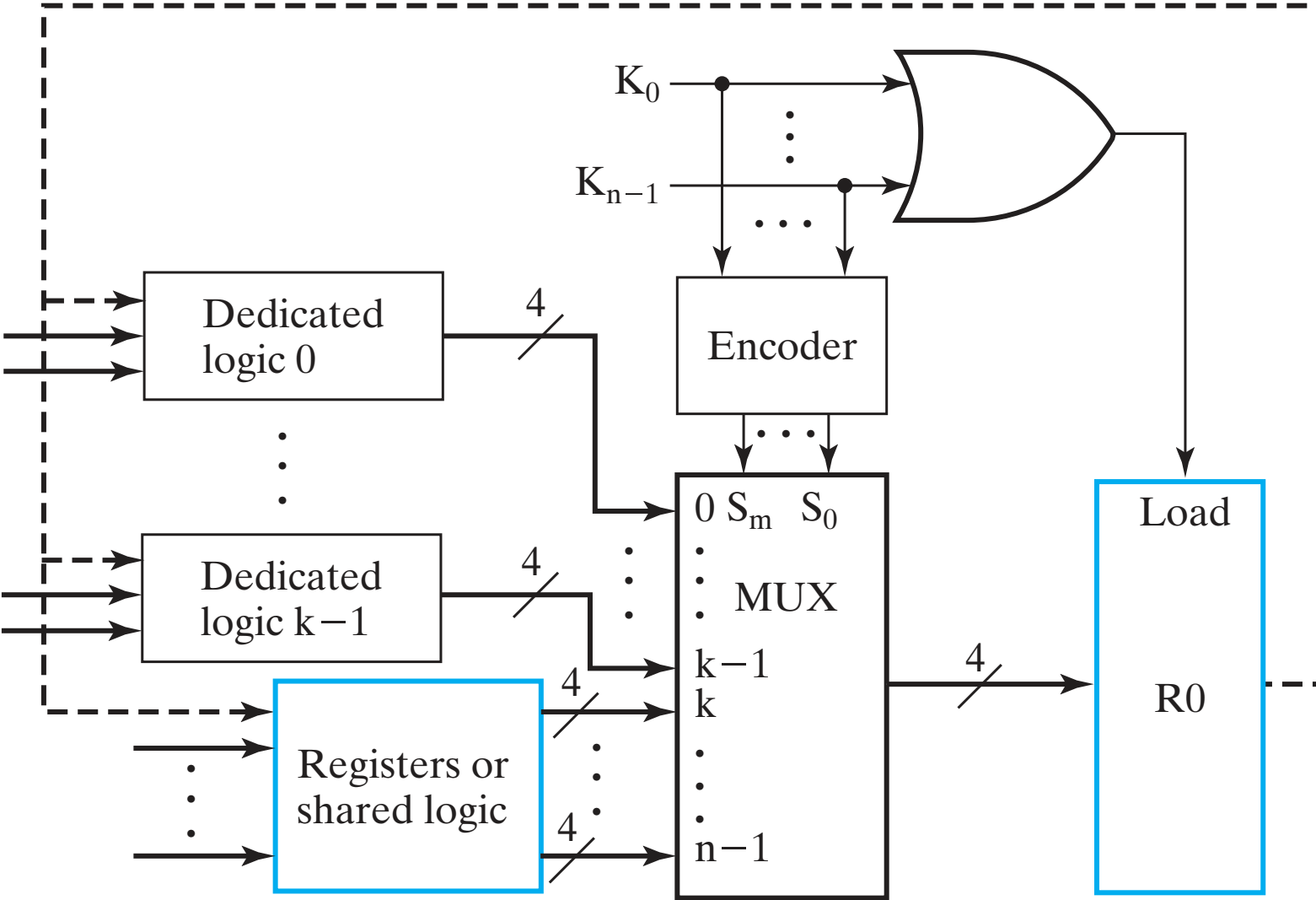


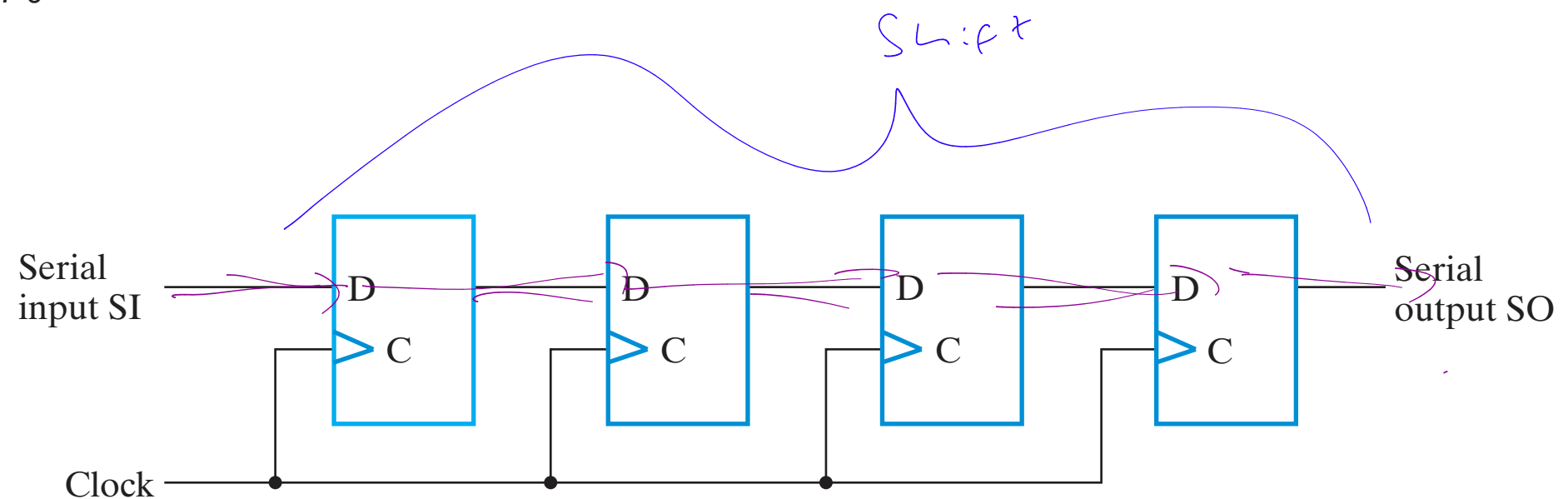


(a) Block diagram

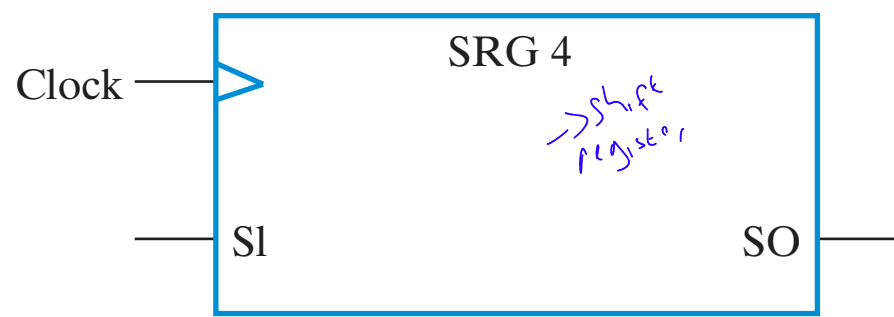


(b) Detailed logic

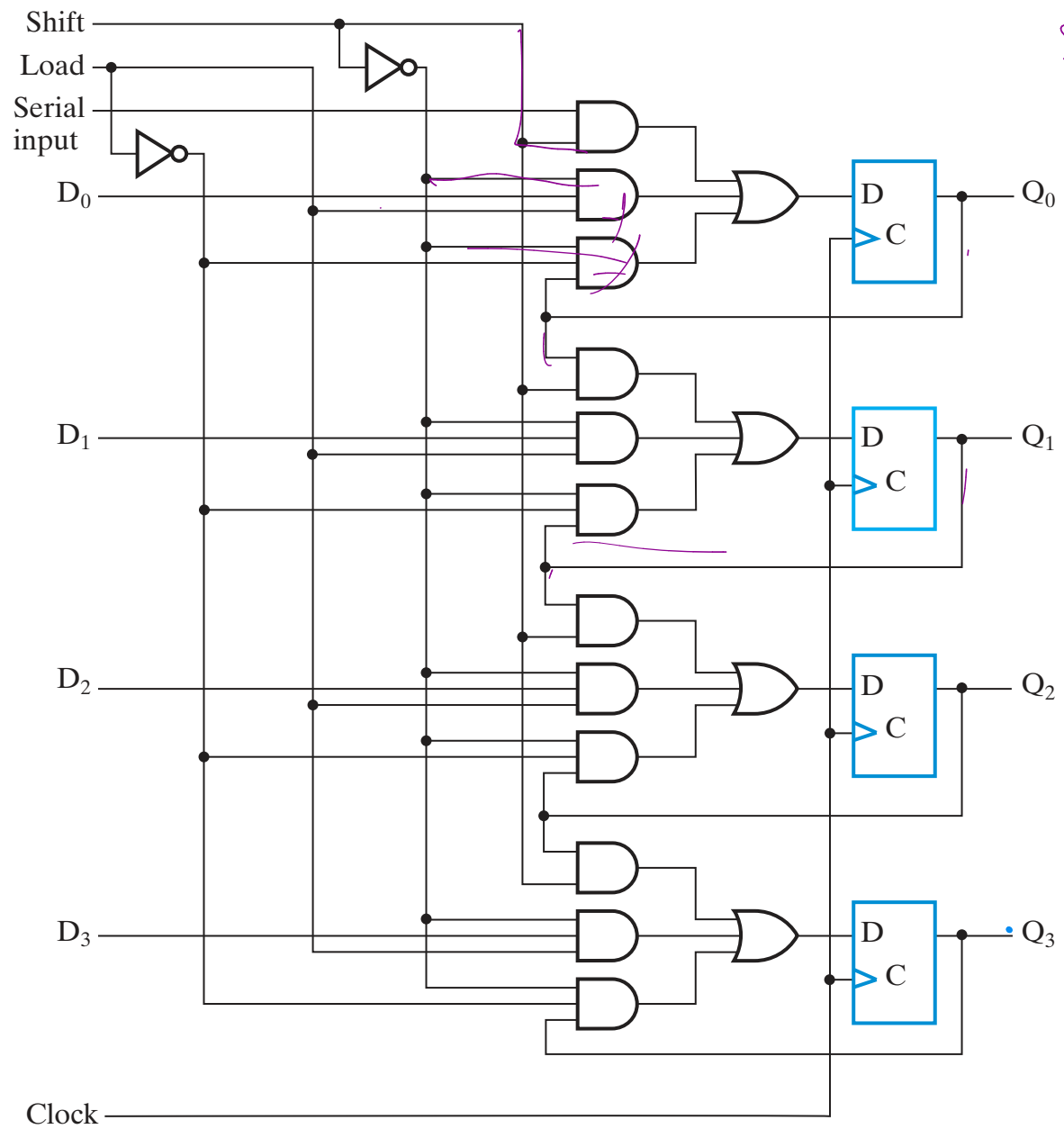




(a) Logic diagram

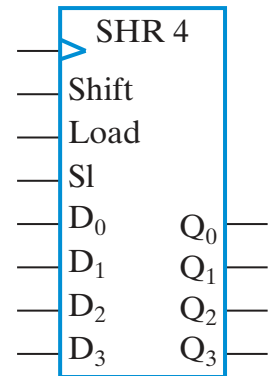


(b) Symbol



SL
00
01
10
11

Dr
pu
SHL
SHL

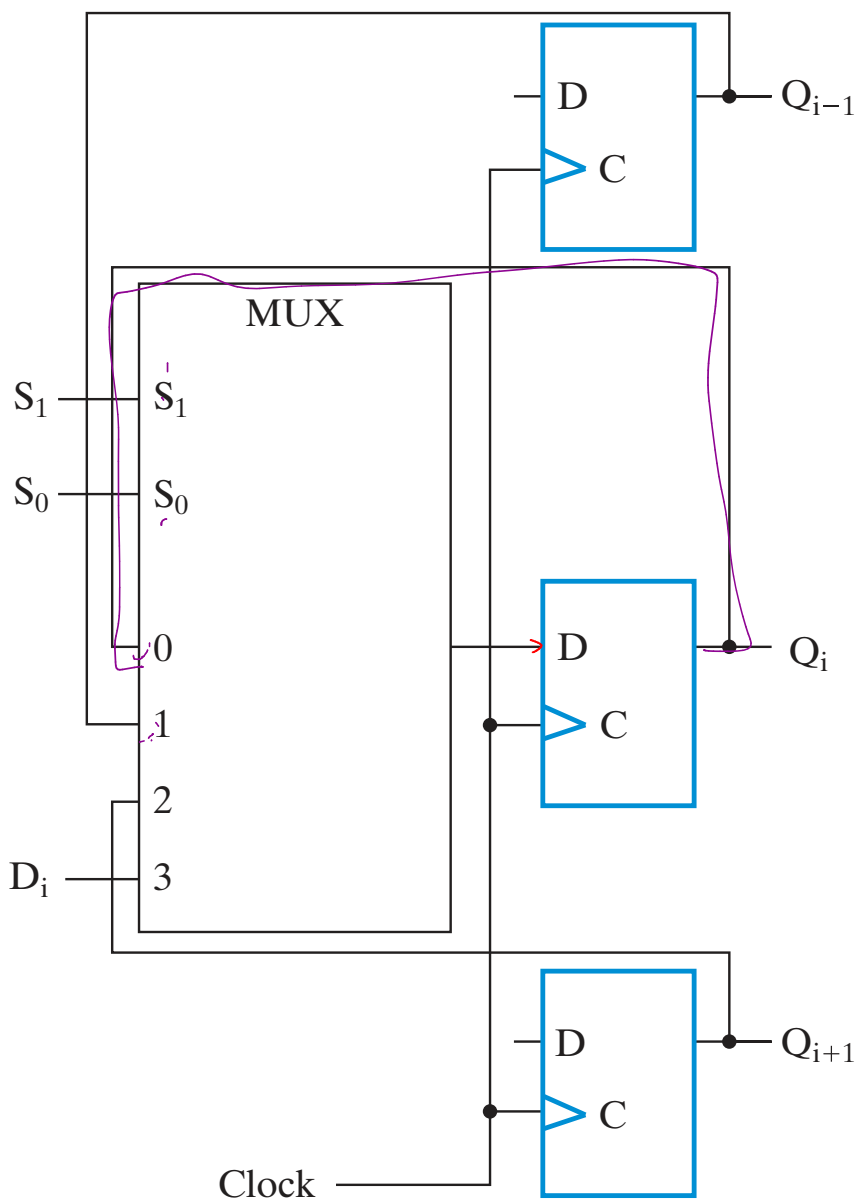


(b) Symbol

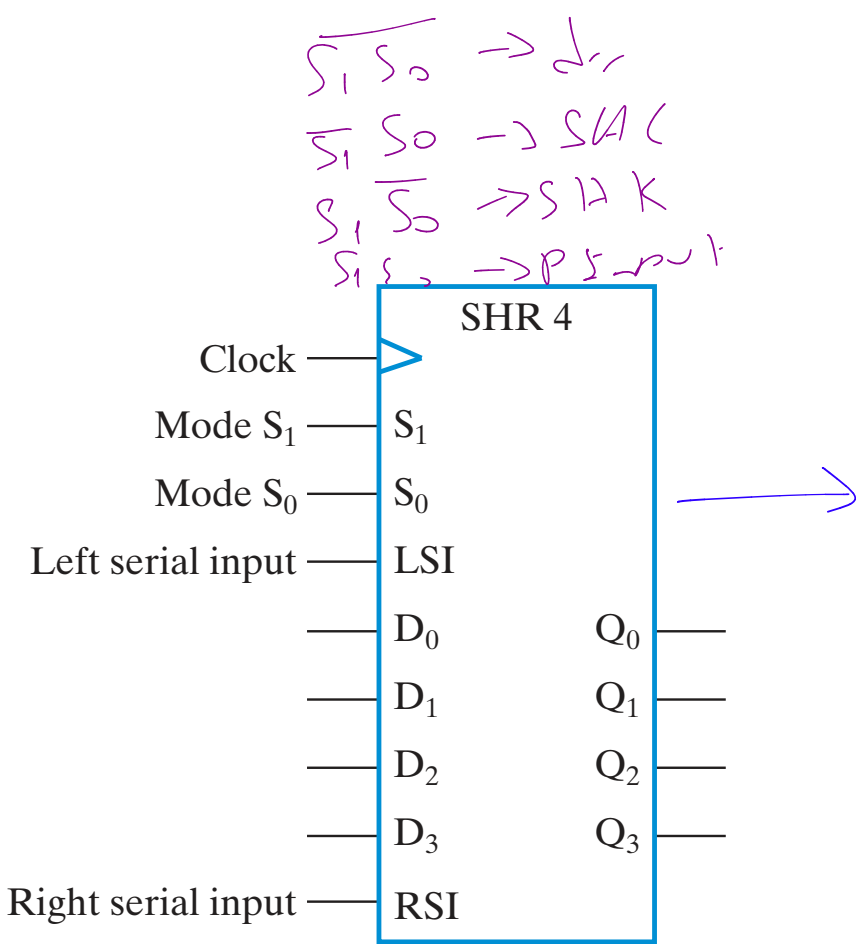
Q₃ ← Q₂ ← Q₁ ← Q₀

□ TABLE 7-6
Function Table for the Register of Figure 7-10

Shift	Load	Operation
0	0	No change
0	1	Load parallel data
1	X	Shift down from Q_0 to Q_3



(a) Logic diagram of one typical stage

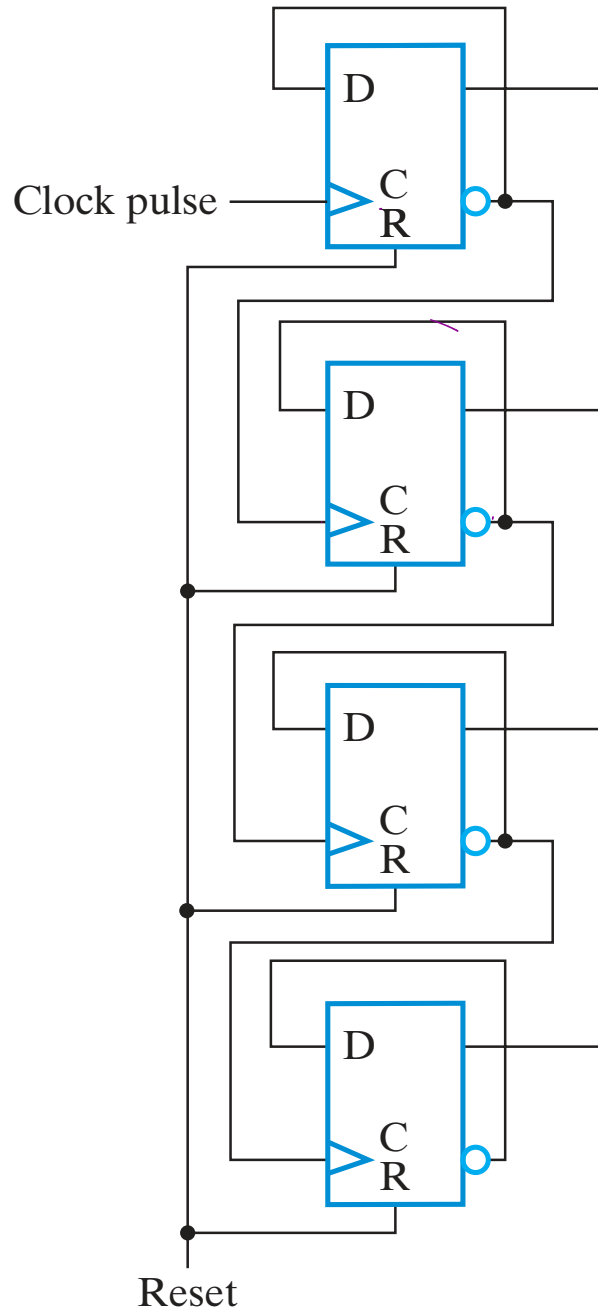


(b) Symbol

□ TABLE 7-7
Function Table for the Register of Figure 7-7

Mode control		Register Operation
S_1	S_0	
0	0	No change
0	1	Shift down
1	0	Shift up
1	1	Parallel load

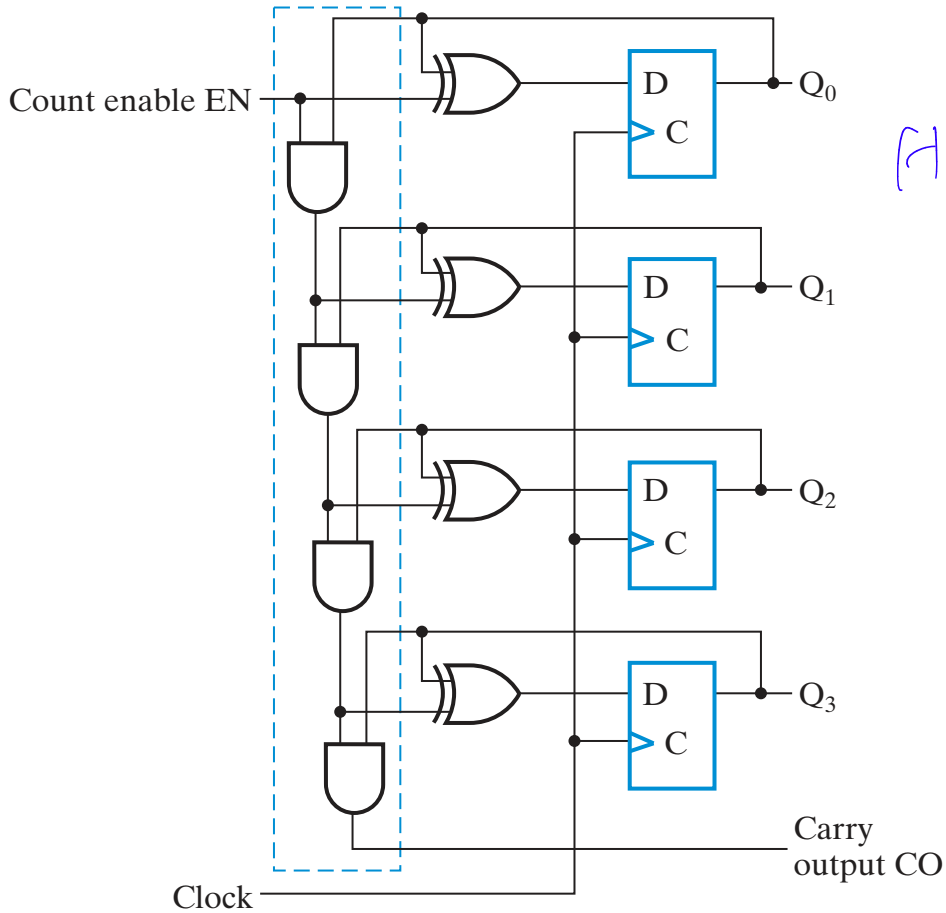
0 0 0 0
 1 0 0 0
 0 1 0 0



Ripple counter

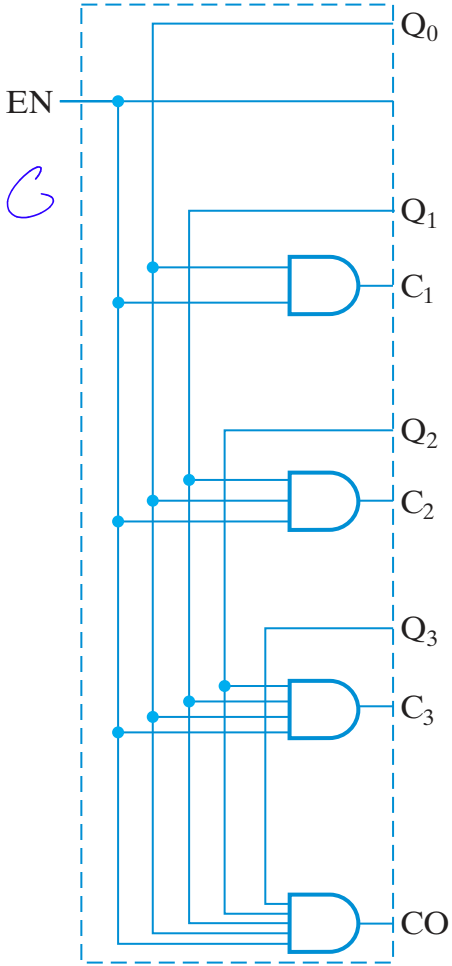
TABLE 7-8
Counting Sequence of Binary Counter

Upward Counting Sequence				Downward Counting Sequence			
Q ₃	Q ₂	Q ₁	Q ₀	Q ₃	Q ₂	Q ₁	Q ₀
0	0	0	0	1	1	1	1
0	0	0	1	1	1	1	0
0	0	1	0	1	1	0	1
0	0	1	1	1	1	0	0
0	1	0	0	1	0	1	1
0	1	0	1	1	0	1	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	0	0
1	0	0	0	0	1	1	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	0	1
1	0	1	1	0	1	0	0
1	1	0	0	0	0	1	1
1	1	0	1	0	0	1	0
1	1	1	0	0	0	0	1
1	1	1	1	0	0	0	0

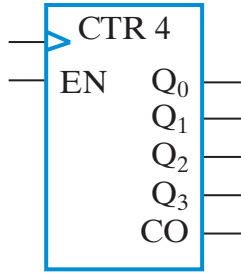


(a) Logic diagram-serial gating

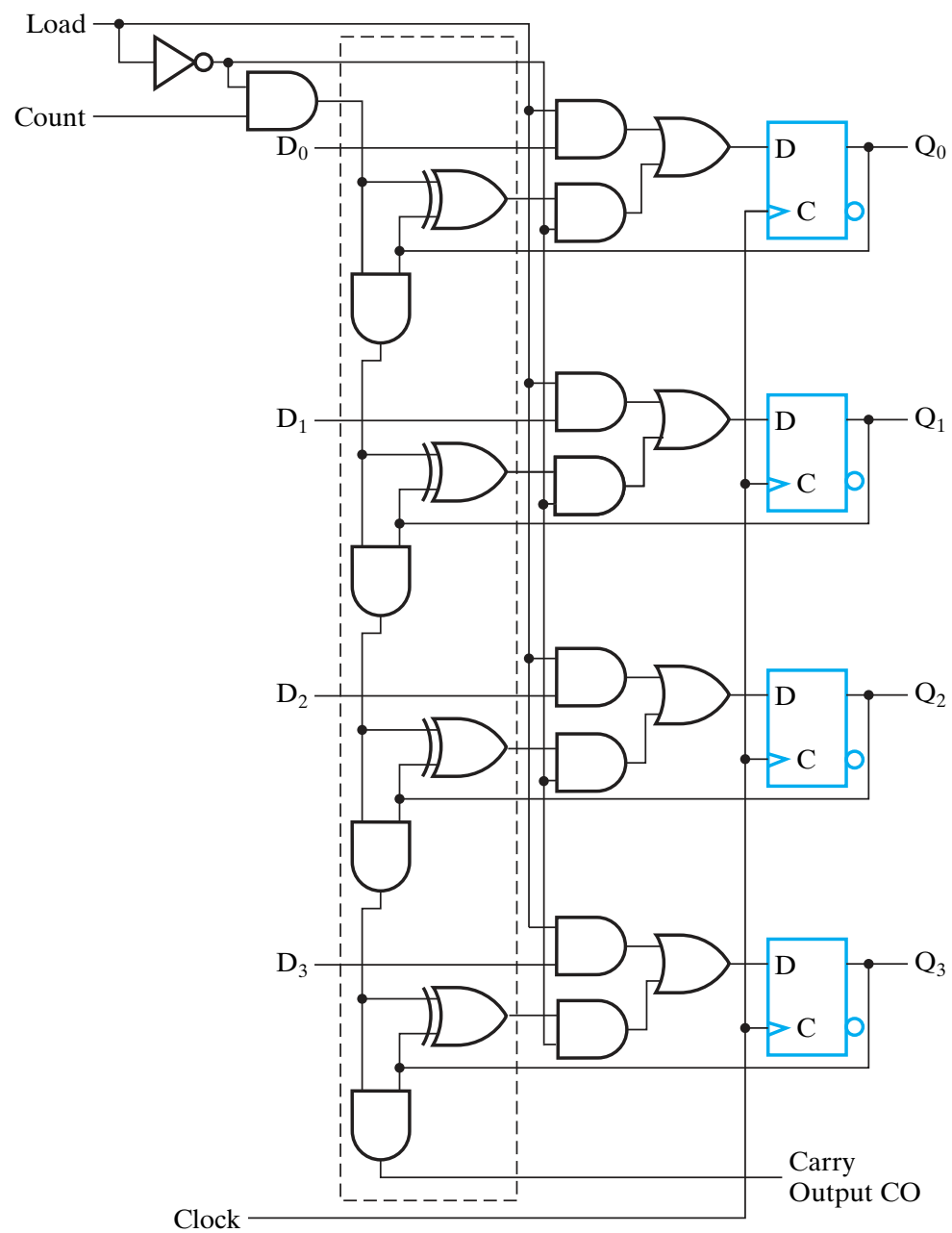
H.C



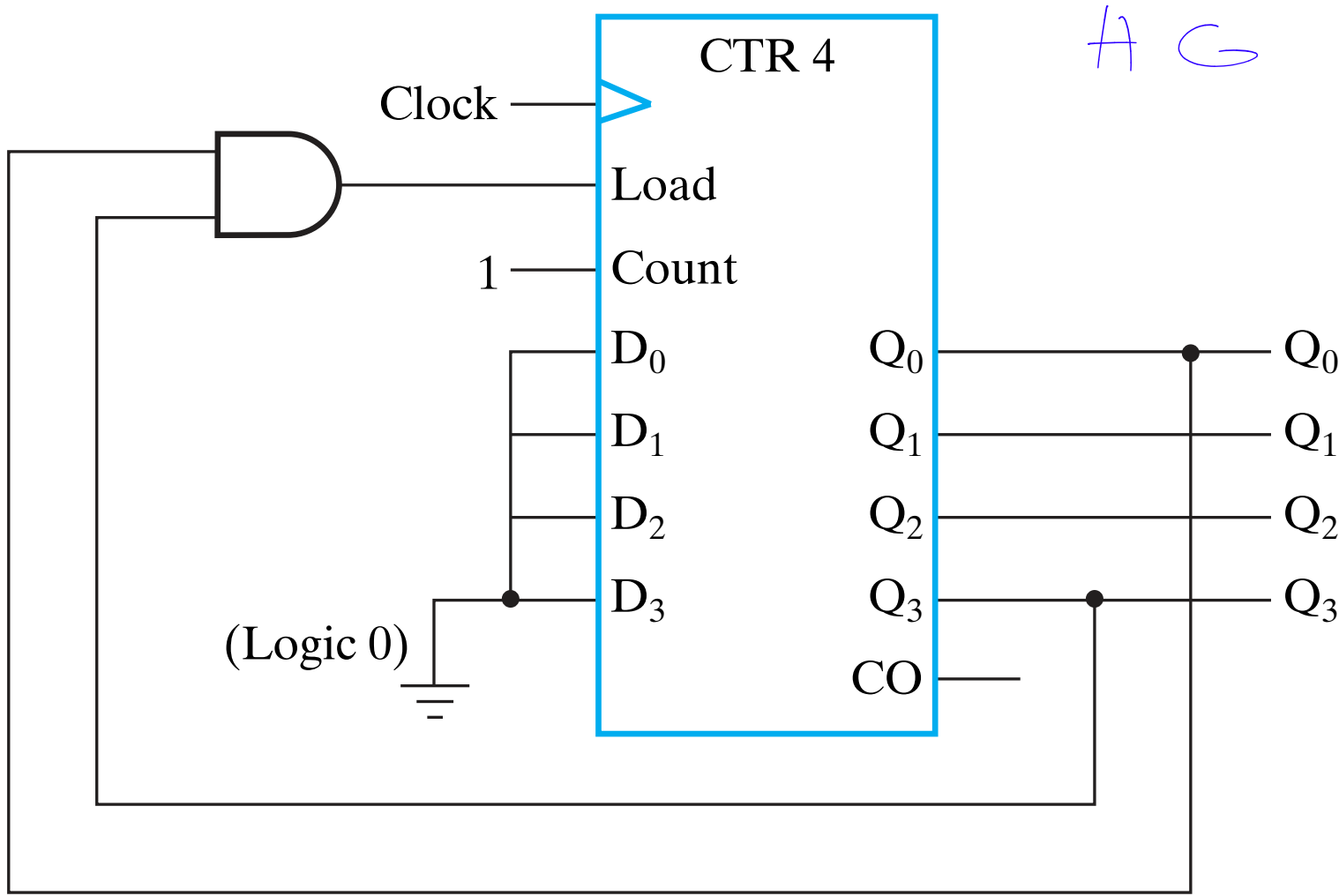
(b) Logic diagram-parallel gating



(c) Symbol



2) e



H C

TABLE 7-9
State Table and Flip-Flop Inputs for BCD Counter

Present State				Next State				Output
Q ₈	Q ₄	Q ₂	Q ₁	D ₈ =	D ₄ =	D ₂ =	D ₁ =	Y
				Q ₈ (t+1)	Q ₄ (t+1)	Q ₂ (t+1)	Q ₁ (t+1)	
0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	1	0	0
0	0	1	0	0	0	1	1	0
0	0	1	1	0	1	0	0	0
0	1	0	0	0	1	0	1	0
0	1	0	1	0	1	1	0	0
0	1	1	0	0	1	1	1	0
0	1	1	1	1	0	0	0	0
1	0	0	0	1	0	0	1	0
1	0	0	1	0	0	0	0	1

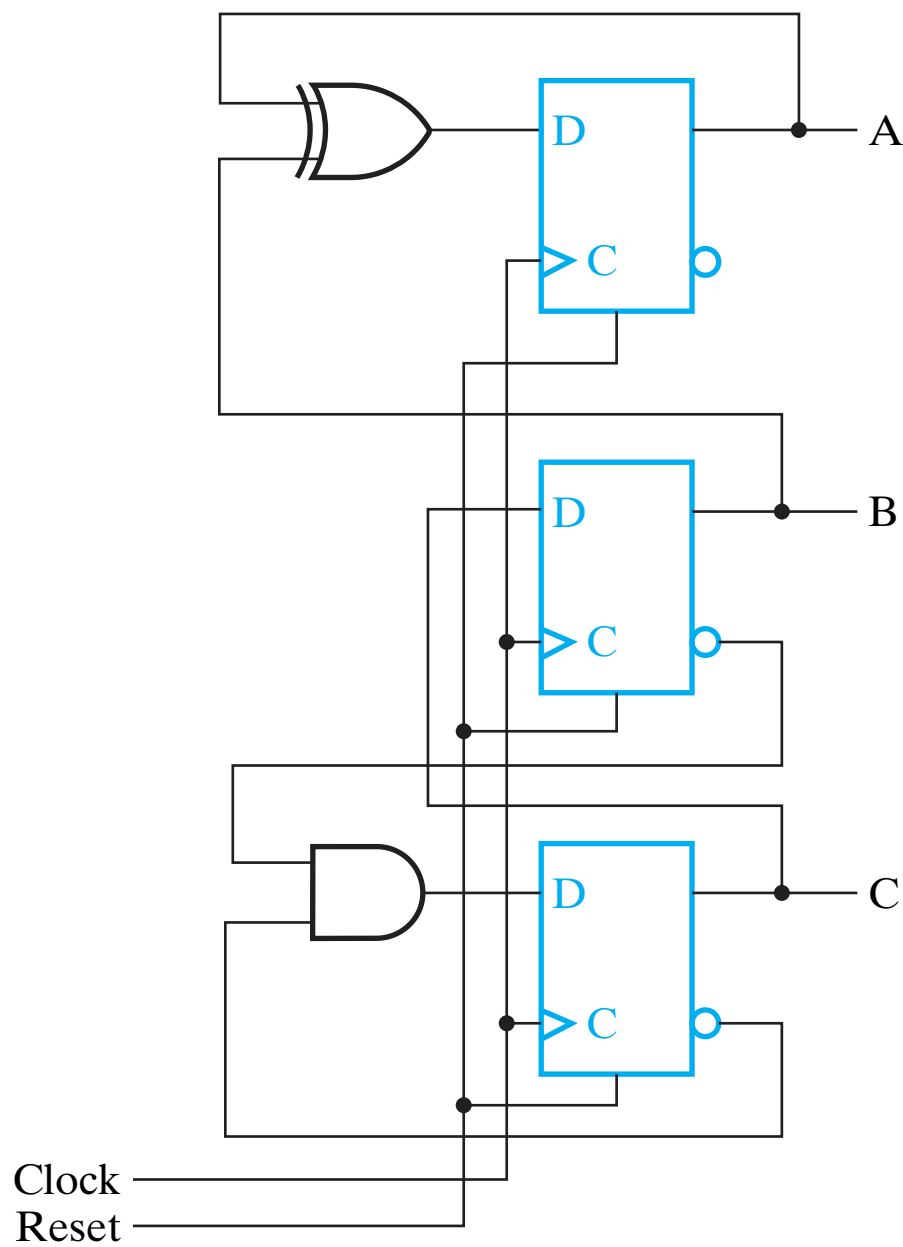
❑

TABLE 7-10

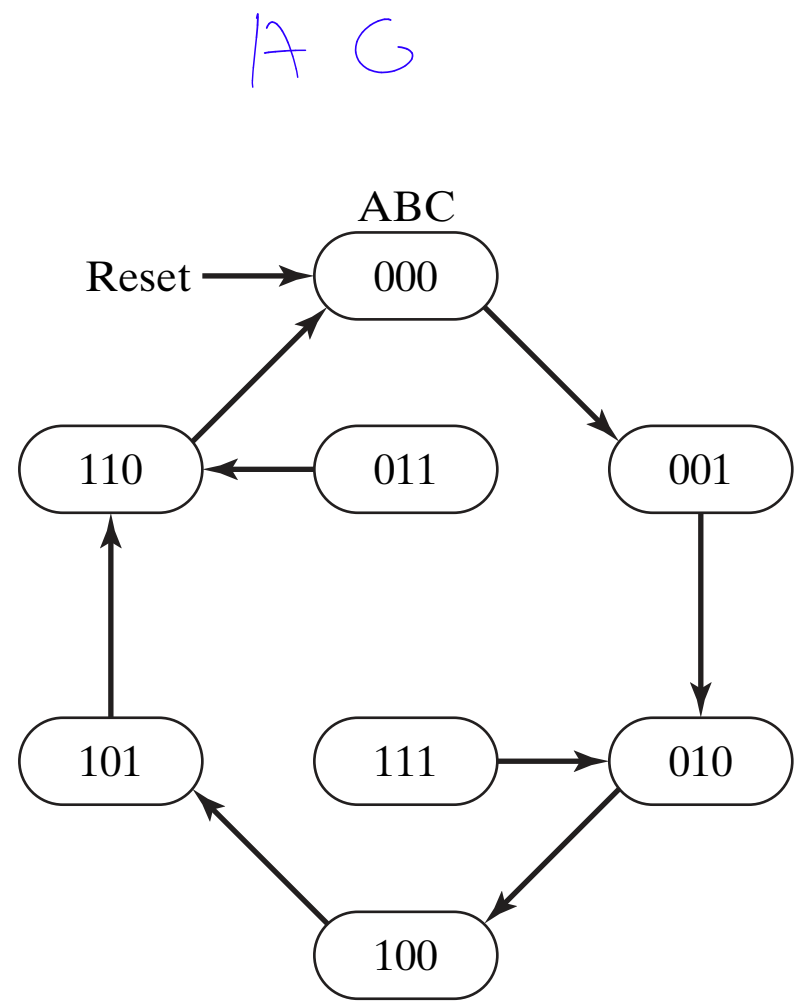
State Table and Flip-Flop Inputs for Counter

Present State			Next State		
A	B	C	DA = DB = DC = A(t+1)B(t+1)C(t+1)		
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	0	0	0

A C



(a)

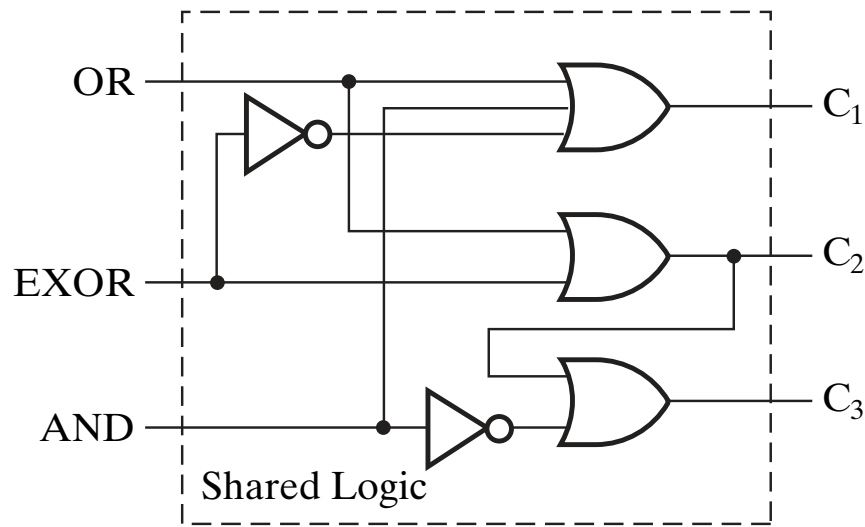


(b)

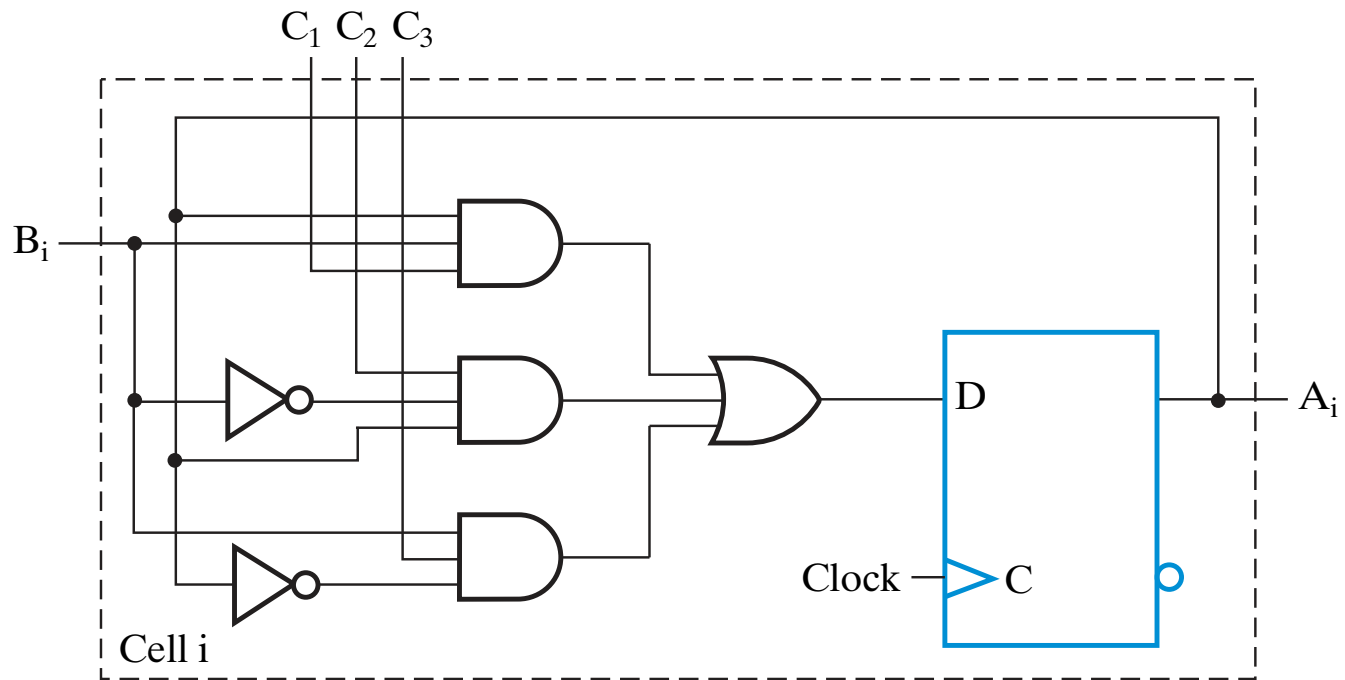
A C

TABLE 7-11
State Table and Flip-Flop Inputs for Example 7-1

Present State A	Next State A(t + 1)						
	(AND = 0) ·(EXOR = 0) ·(OR = 0)	(OR = 1) ·(B = 0)	(OR = 1) ·(B = 1)	(EXOR = 1) ·(B = 0)	(EXOR = 1) ·(B = 1)	(AND = 1) ·(B = 0)	(AND = 1) ·(B = 1)
0	0	0	1	0	1	0	0
1	1	1	1	1	0	0	1



AG

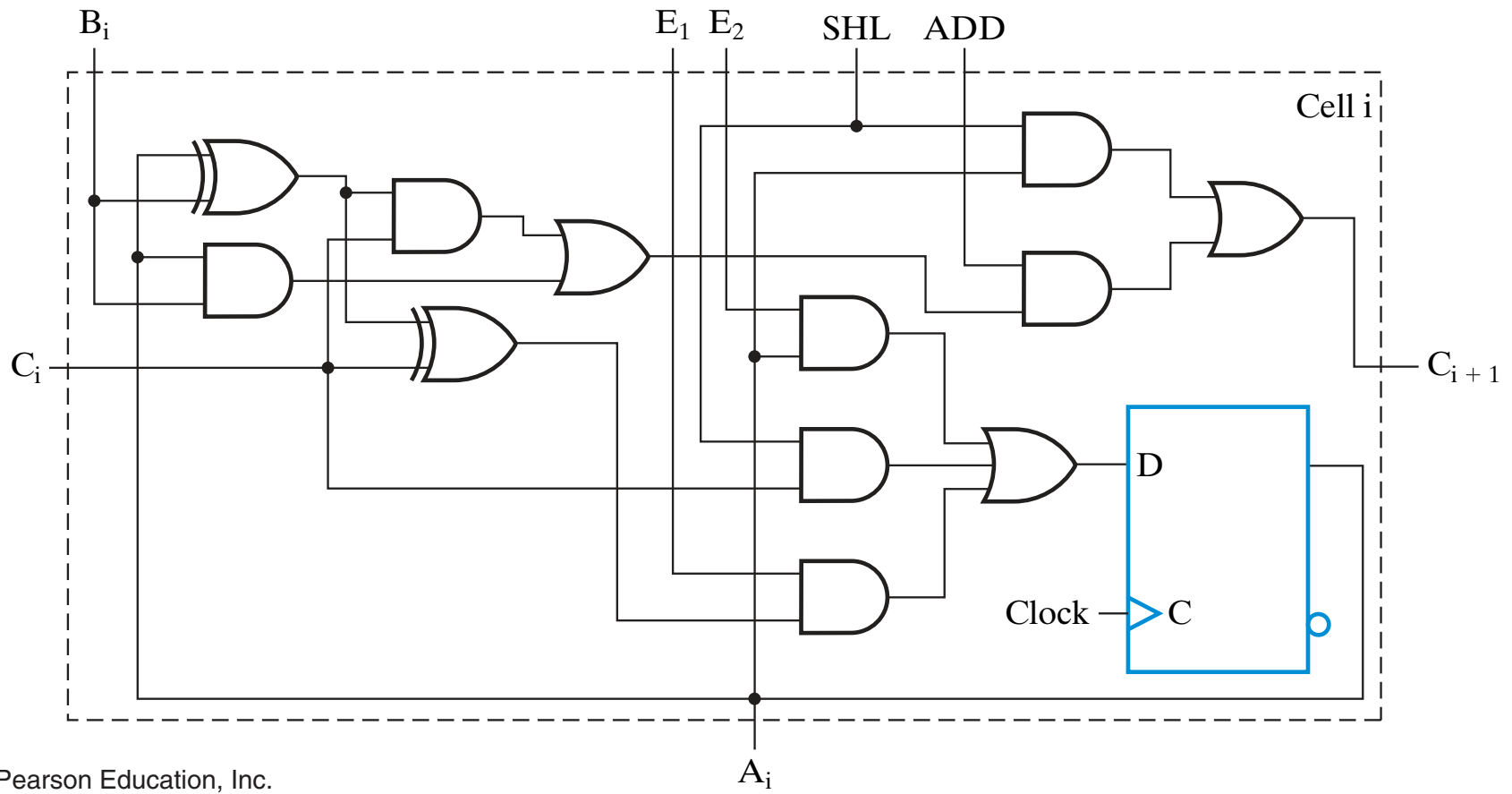
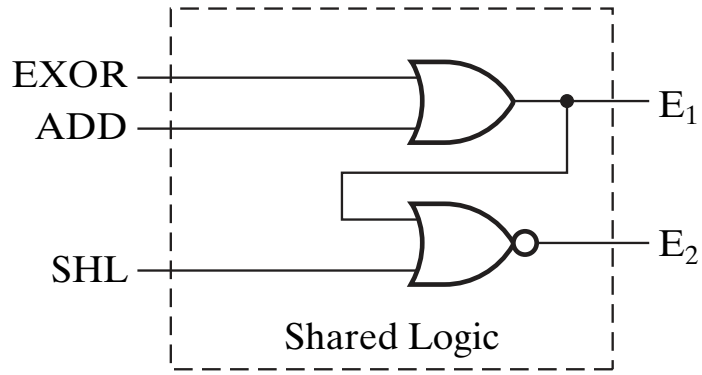


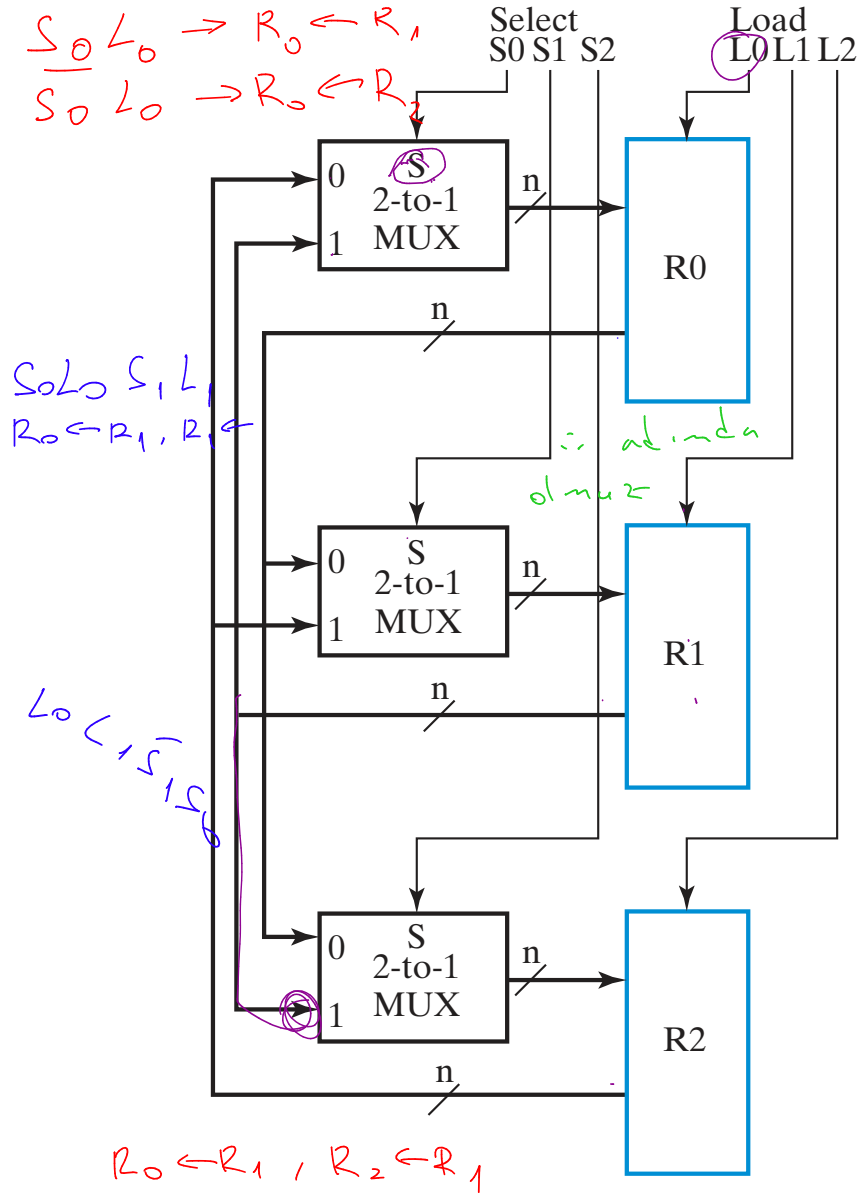
A C

TABLE 7-12
State Table and Flip-Flop Inputs for Register-Cell Design in Example 7-2

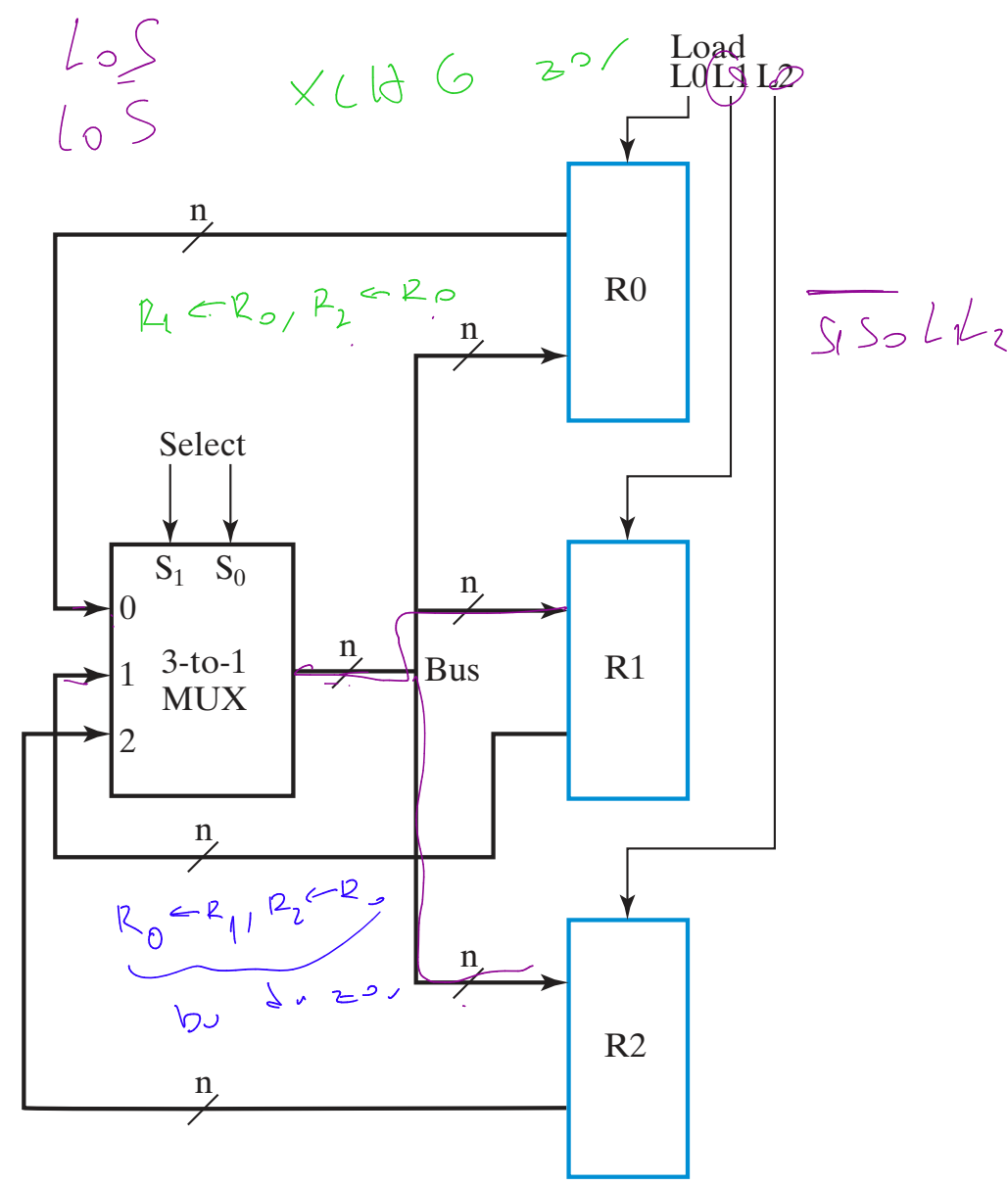
Present State A_i	Inputs												Next State $A_i(t + 1)$ /Output C_{i+1}											
	SHL = 0				SHL = 1				EXOR = 1				ADD = 1											
	EXOR = 0				$B_i = 0$				$B_i = 0$				$B_i = 0$											
	ADD = 0				$C_i = 0$								$C_i = 0$											
0	0/X				0/0 1/0 0/0 1/0				0/X 1/X				0/0 1/0 1/0 0/1											
1	1/X				0/1 1/1 0/1 1/1				1/X 0/X				1/0 0/1 0/1 1/1											

7 6





(a) Dedicated multiplexers

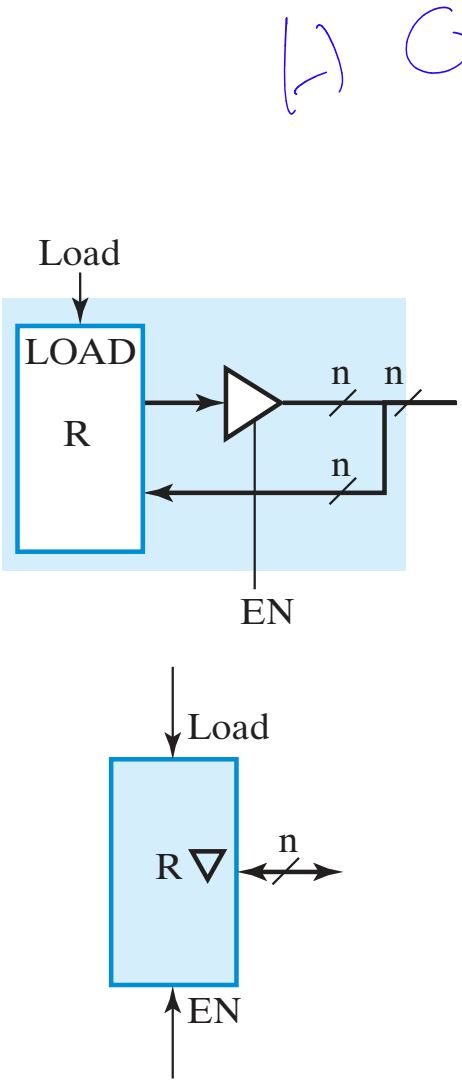


(b) Single bus

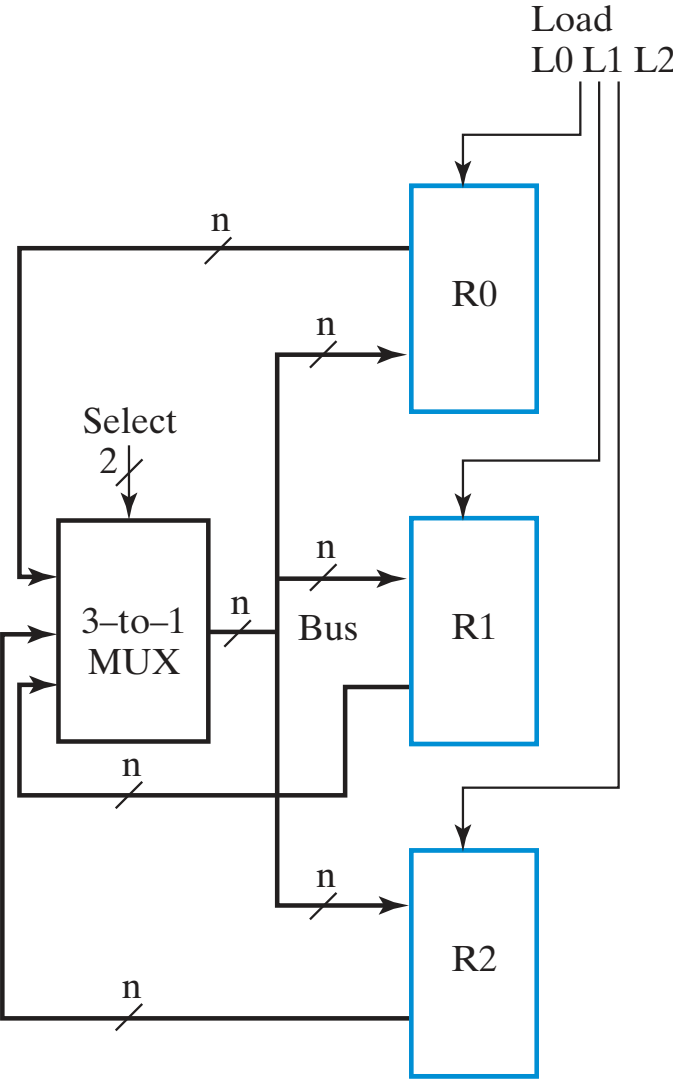
TABLE 7-13
Examples of Register Transfers Using the Single Bus
in Figure 7-19(b)

Register Transfer	Select		Load		
	S1	S0	L2	L1	L0
$R0 \leftarrow R2$	1	0	0	0	1
$R0 \leftarrow R1, R2 \leftarrow R1$	0	1	1	0	1
$R0 \leftarrow R1, R1 \leftarrow R0$	Impossible				

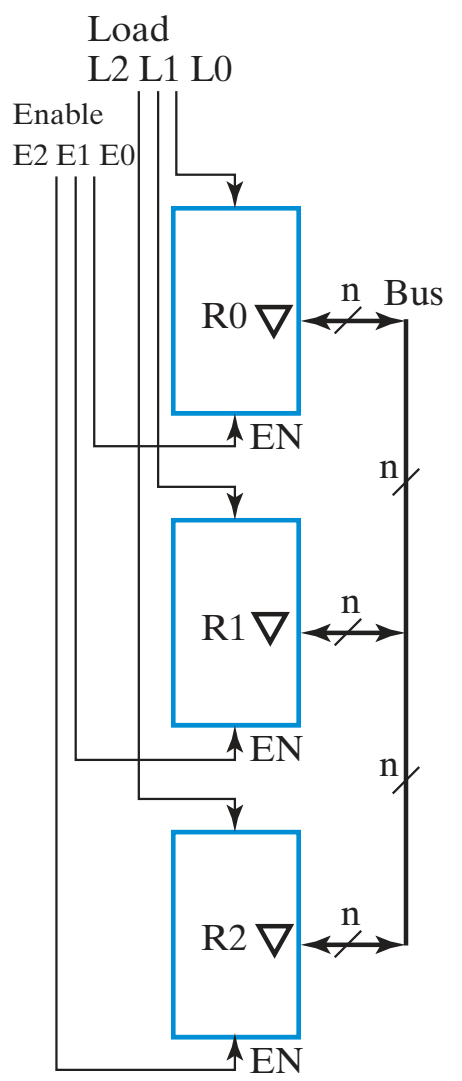
1) G



(a) Register with bidirectional input-output lines and symbol

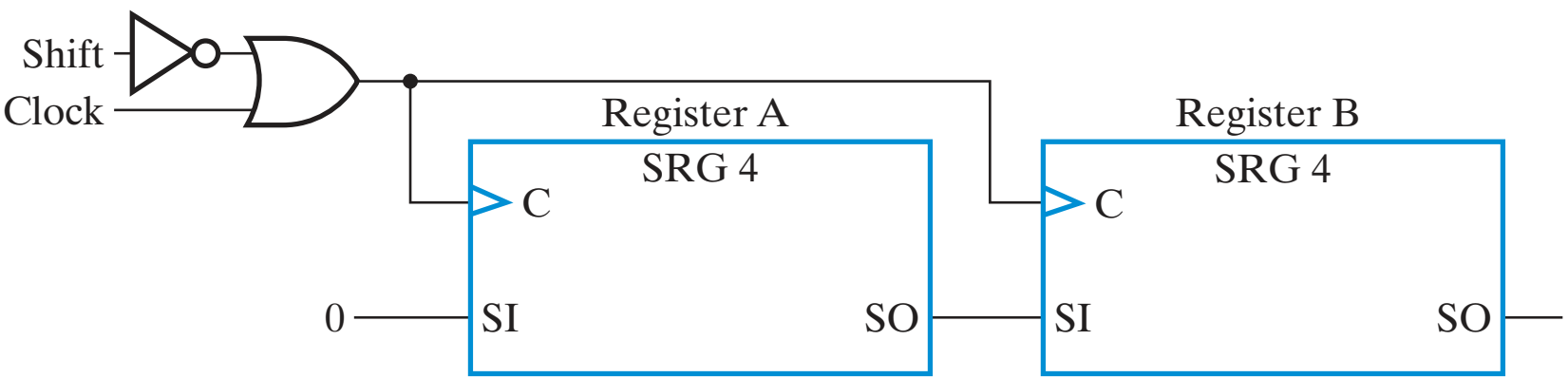


(b) Multiplexer bus

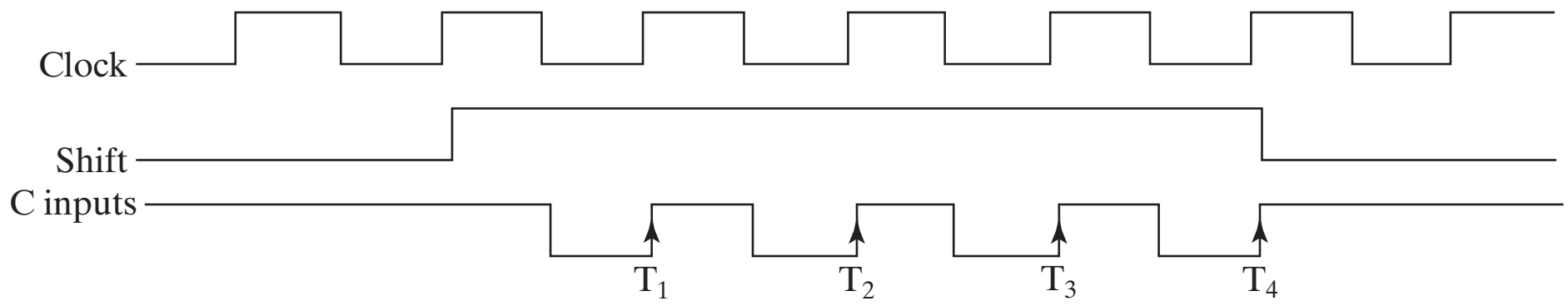


(c) Three-state bus using registers with bidirectional lines

17 C



(a) Block diagram



(b) Timing diagram

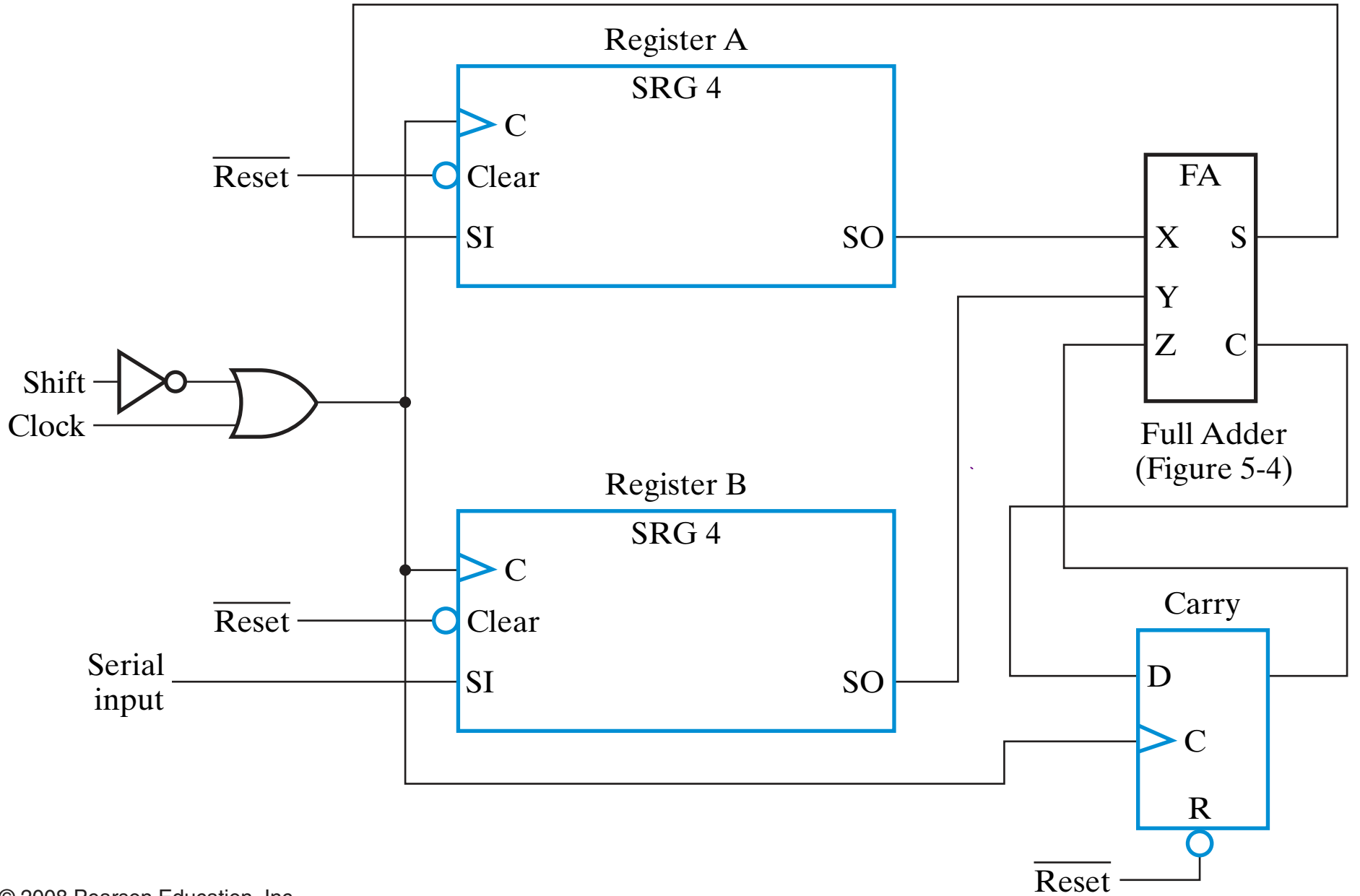
H C

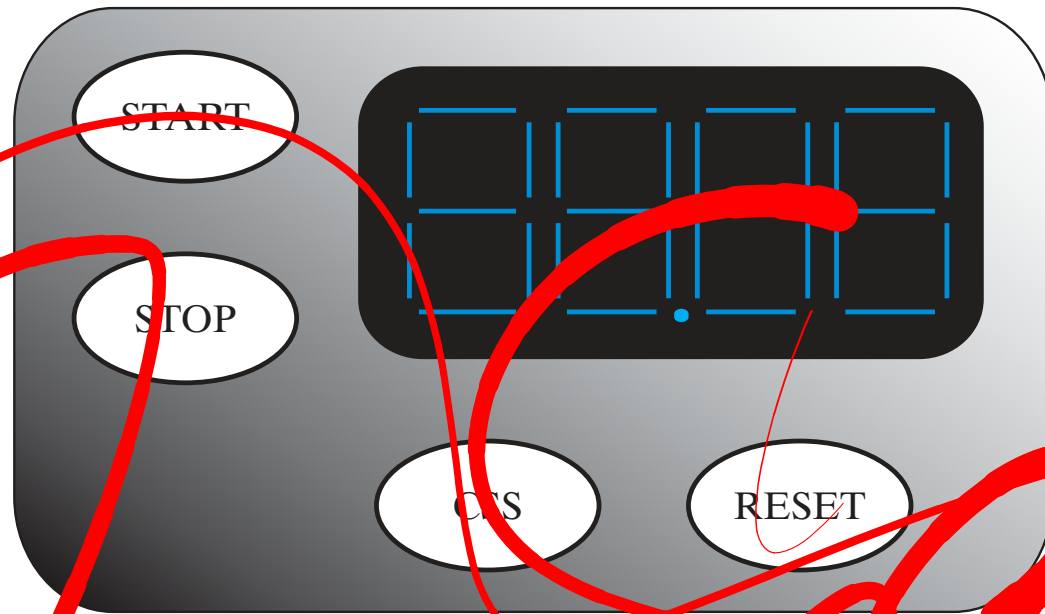
TABLE 7-14

Example of Serial Transfer

Timing pulse	Shift Register A				Shift Register B			
Initial value	1	0	1	1	0	0	1	0
After T_1	0	1	0	1	1	0	0	1
After T_2	0	0	1	0	1	1	0	0
After T_3	0	0	0	1	0	1	1	0
After T_4	0	0	0	0	1	0	1	1

Serial Coplagic y.b:



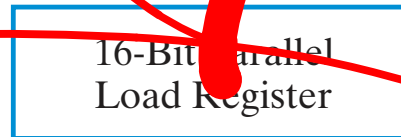


(a)

TM



SD



(b)

TABLE 7-15
Inputs, Outputs, and Registers of the DashWatch

Symbol	Function	Type
START	Initialize timer to 0 and start timer	Control input
STOP	Stop timer and display timer	Control input
CSS	Compare, store and display shortest dash time	Control input
RESET	Set shortest value to 10011001	Control input
B ₁	Digit 1 data vector a, b, c, d, e, f, g to display	Data output vector
B ₀	Digit 0 data vector a, b, c, d, e, f, g to display	Data output vector
DP	Decimal point to display (= 1)	Data output
B ₋₁	Digit -1 data vector a, b, c, d, e, f, g to display	Data output vector
B ₋₂	Digit -2 data vector a, b, c, d, e, f, g to display	Data output vector
B	The 29-bit display input vector (B ₁ , B ₀ , DP, B ₋₁ , B ₋₂)	Data output vector
TM	4-Digit BCD counter	16-Bit register
SP	Parallel load register	16-Bit register

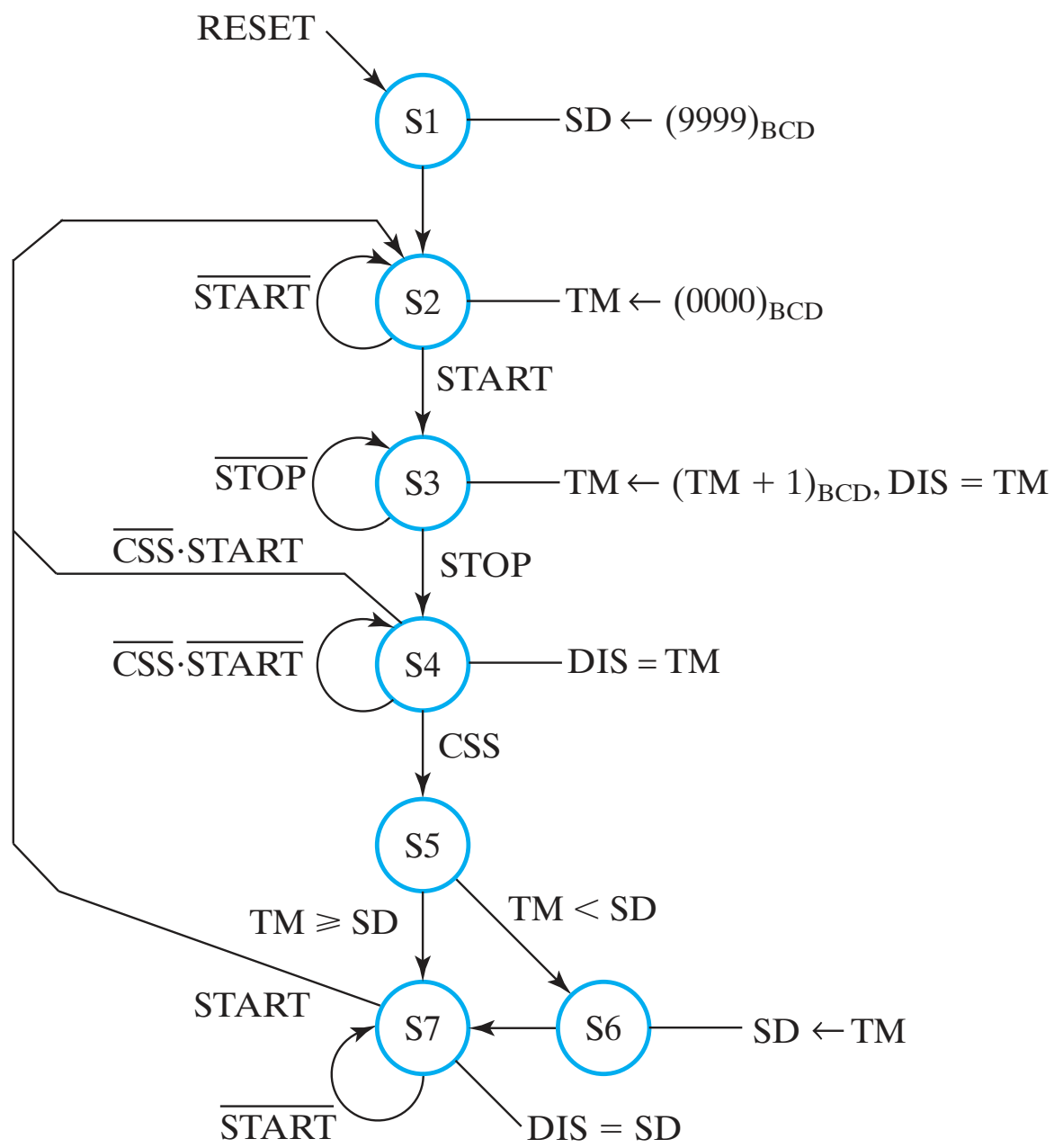
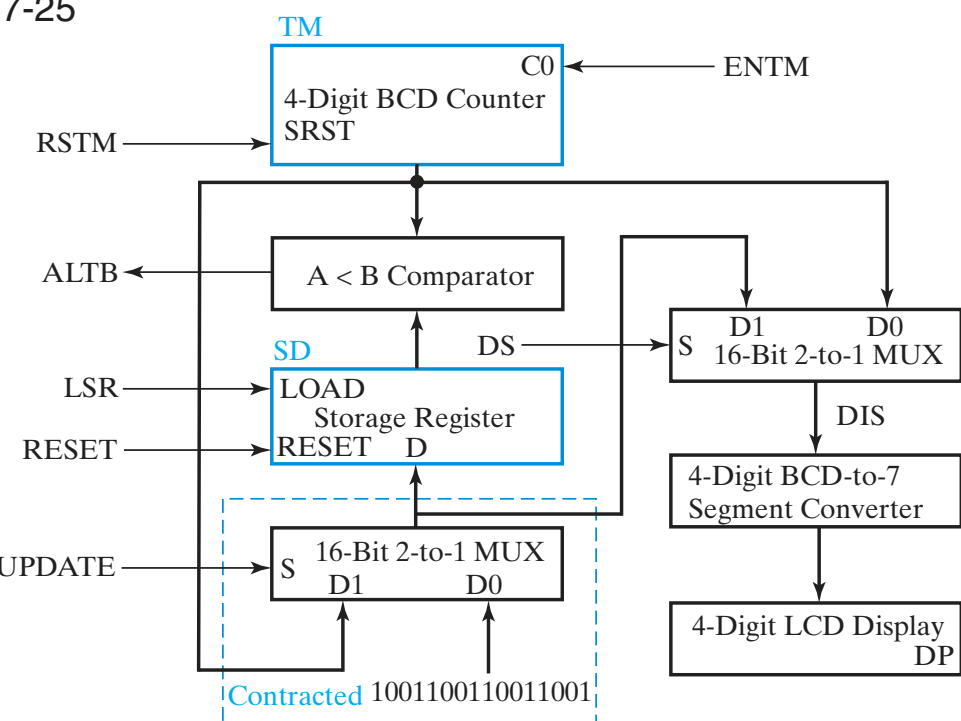
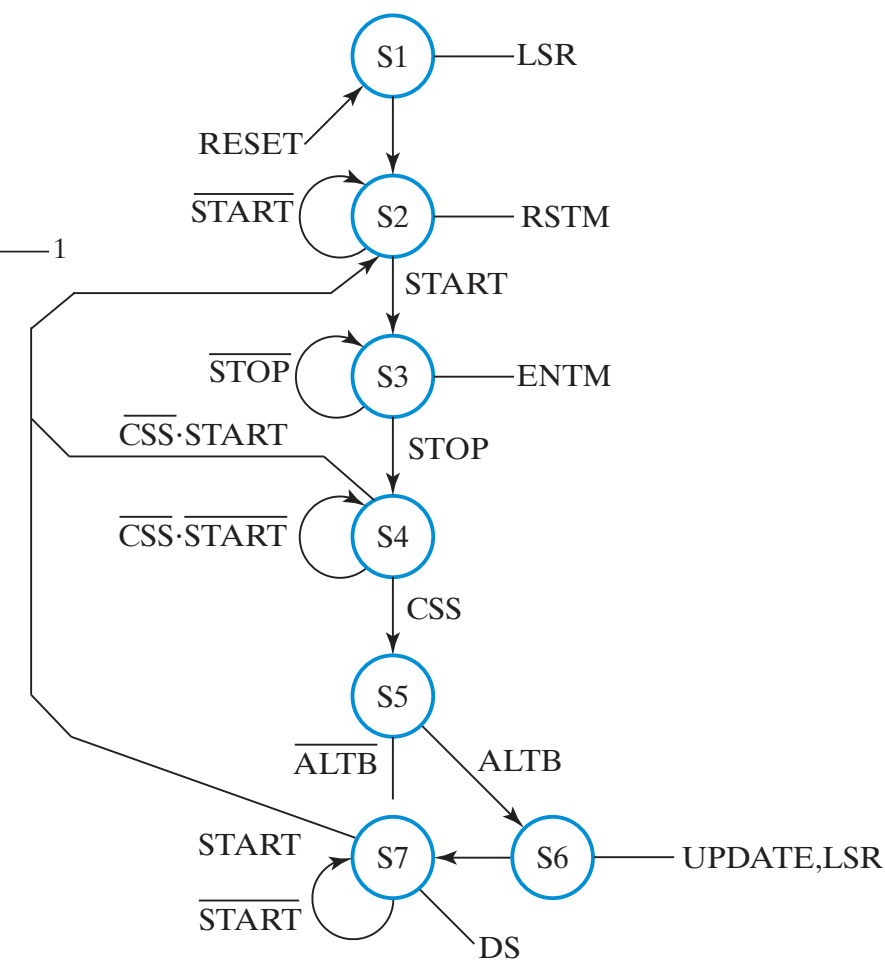


TABLE 7-16
Datapath Output Actions and Status Generation with Control and Status Signals

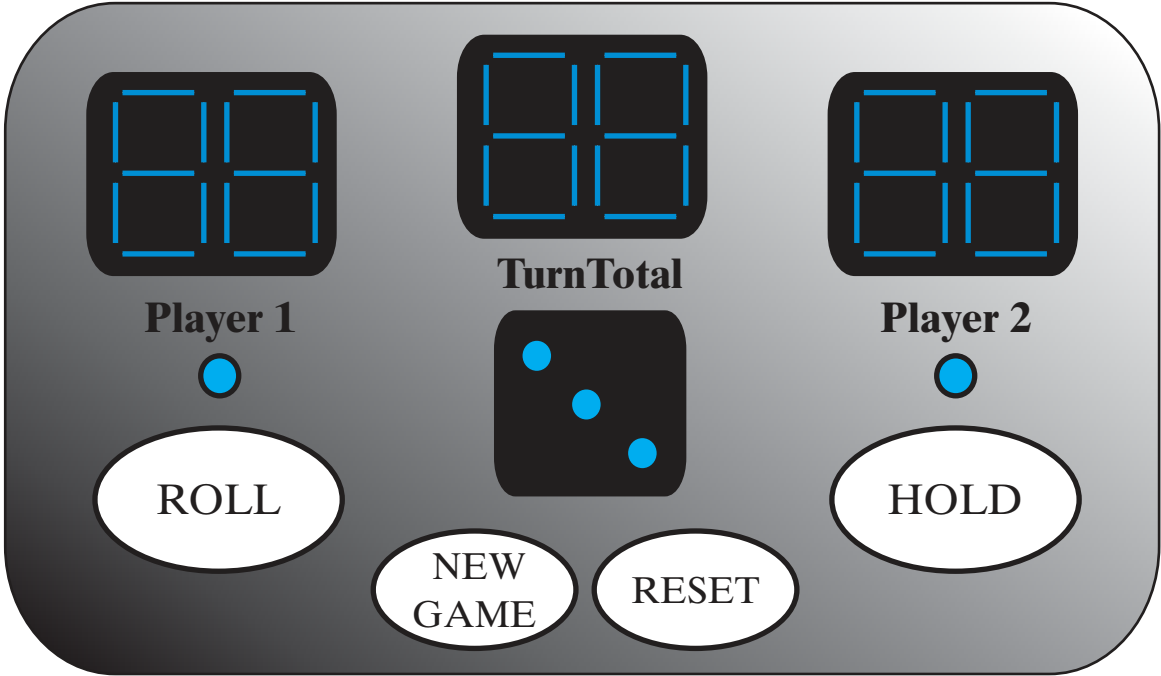
Action or Status	Control or Status Signals	Meaning for Values 1 and 0
$TM \leftarrow (0000)_{BCD}$	RSTM	1: Reset TM to 0 (synchronous reset) 0: No reset of TM
$TM \leftarrow (TM + 1)_{BCD}$	ENTM	1: BCD count up TM by 1, 0: hold TM value
$SD \leftarrow (9999)_{BCD}$	UPDATE LSR	0: Select 1001100110011001 for loading SD 1: Enable load SD, 0: disable load SD
$SD \leftarrow TM$	UPDATE LSR	1: Select TM for loading SD Same as above
DIS = TM DIS = SD	DS	0: Select TM for DIS 1: Select SD for DIS
TM < SD TM ≥ SD	ALTB	1: TM less than SD 0: TM greater than or equal to SD



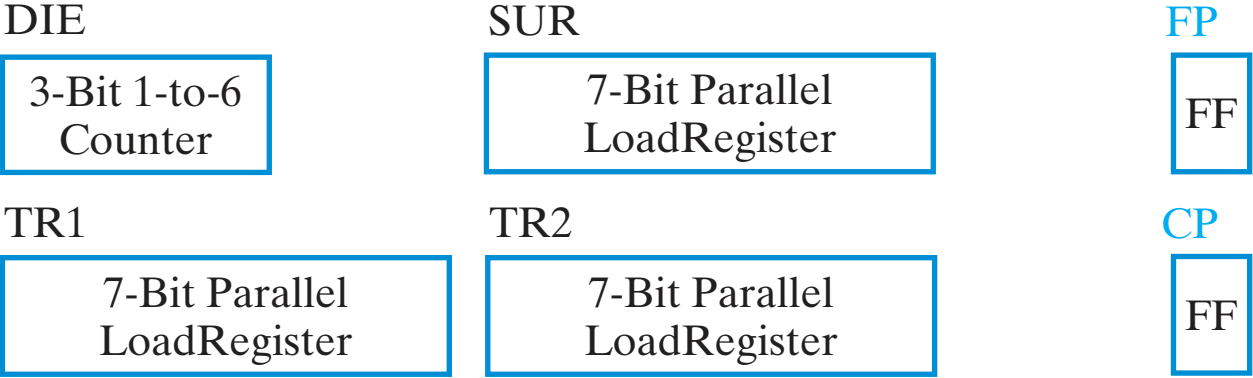
(a)



(b)



(a)



Datapath Registers

Control Registers

(b)

TABLE 7-17
Inputs, Outputs, and Registers of PIG

Symbol	Name/Function	Type
ROLL	1: Starts die rolling, 0: Stops die rolling	Control input
HOLD	1: Ends player turn, 0: Continues player turn.	Control input
NEW_GAME	1: Starts new game, 0: Continues current game	Control input
RESET	1: Resets game to INIT state, 0: No action	Control input
DDIS	7-bit LED die display array	Data output vector
SUB	14-bit 7-segment pair (a, b, c, d, e, f, g) to Turn Total display	Data output vector
TP1	14-bit 7-segment pair (a, b, c, d, e, f, g) to Player 1 display	Data output vector
TP2	14-bit 7-segment pair (a, b, c, d, e, f, g) to Player 2 display	Data output vector
P1	1: Player 1 LED on, 0: Player 1 LED off	Data output
P2	1: Player 2 LED on, 0: Player 2 LED off	Data output
DIE	Die value—Specialized counter to count 1,...,6,1,...	3-bit data register
SUR	Subtotal for active player—parallel load register	7-bit data register
TR1	Total for Player 1—parallel load register	7-bit data register
TR2	Total for Player 2—parallel load register	7-bit data register
FP	First player—flip-flop 0: Player 1, 1: Player 2	1-bit control register
CP	Current player—flip-flop 0: Player 1, 1: Player 2	1-bit control register

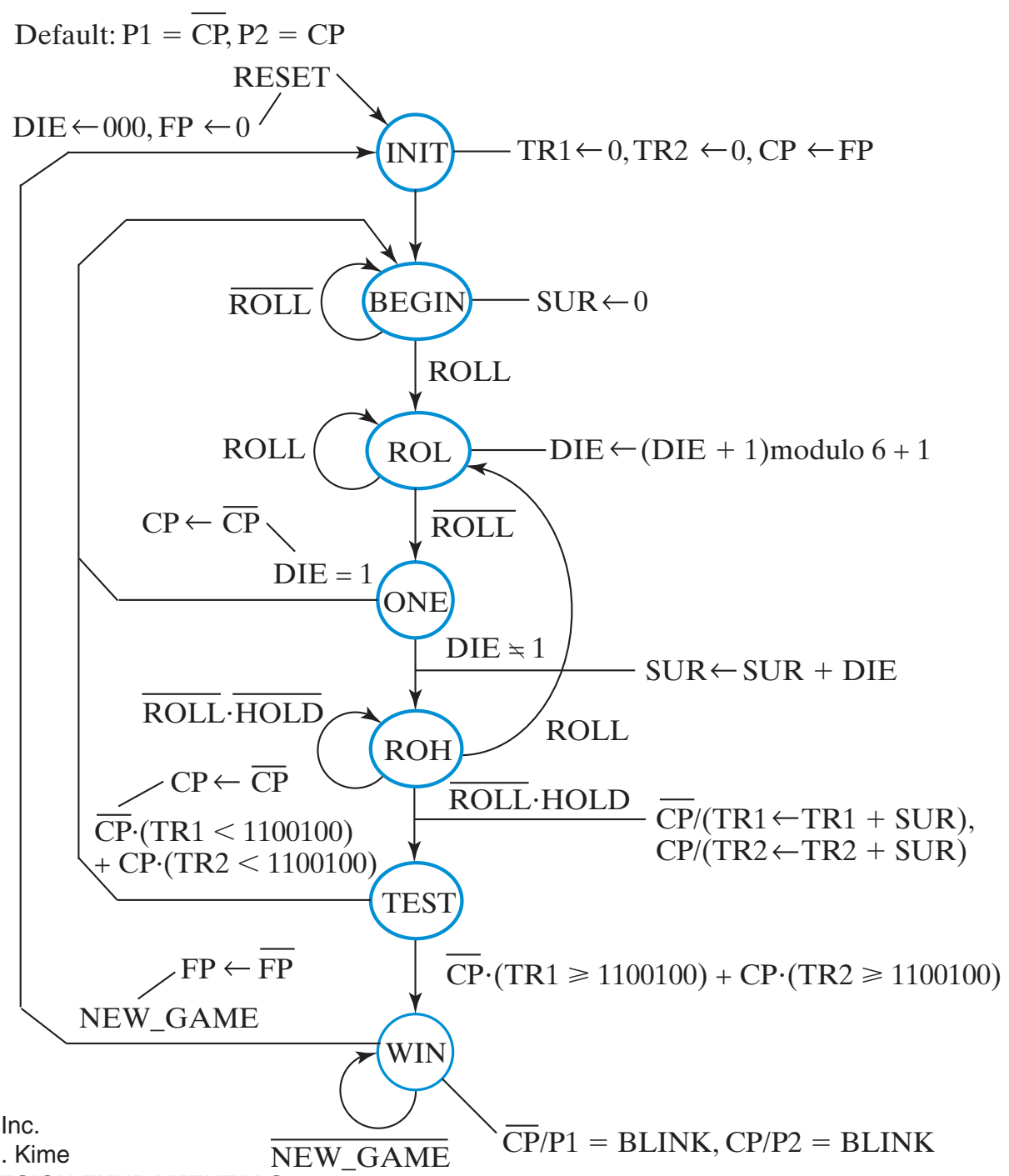
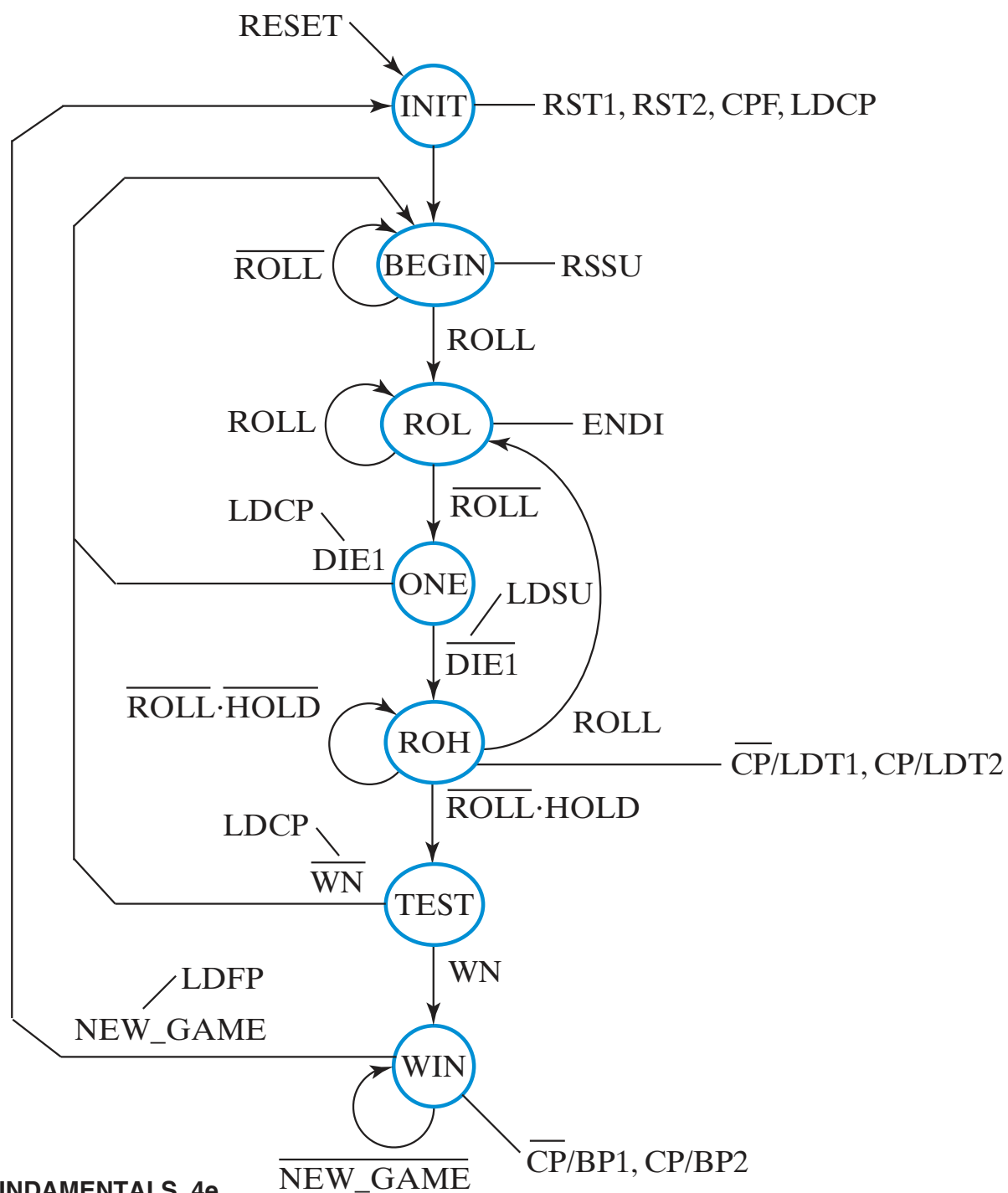
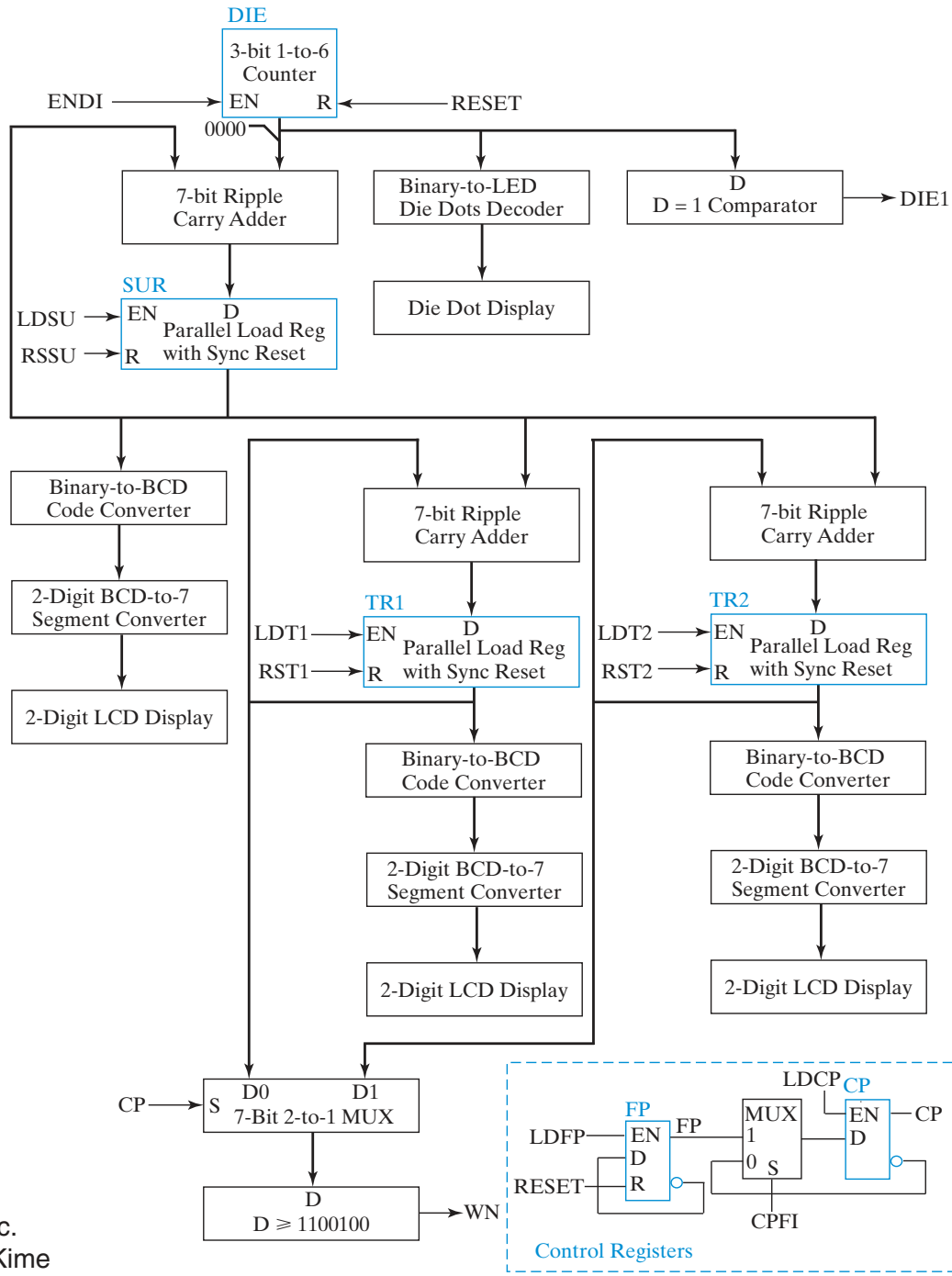


TABLE 7-18
Datapath Output Actions and Control and Status Signals for PIG

Action or Status	Control or Status Signals	Meaning for Values 1 and 0
TR1 ← 0 TR1 ← TR1 + SUR	RST1 LDT1	1: Reset TR1 (synchronous reset), 0: No action 1: Add SUR to TR1, 0: No action
TR2 ← 0 TR2 ← TR2 + SUR	RST2 LDT2	1: Reset TR2 (synchronous reset), 0: No action 1: Add SUR to TR2, 0: No action
SUR ← 0 SUR ← SUR + DIE	RSSU LDSU	1: Reset SUR (synchronous reset), 0: No action 1: Add DIE to SUR, 0: No action
DIE ← 000 if (DIE = 110) DIE ← 001 else DIE ← DIE + 1	RESET ENDI	1: Reset DIE to 000 (asynchronous reset) 1: Enable DIE to increment, 0: Hold DIE value
P1 = BLINK	BP1	1: Connect P1 to BLINK, 0: Connect P1 to 1
P2 = BLINK	BP2	1: Connect P2 to BLINK, 0: Connect P2 to 1
CP ← FP CP ← $\overline{CP}$	CPFI LDCP CPFI LDCP	1: Select FP for CP 1: Load CP, 0: No action 0: Select $\overline{CP}$ for CP 1: Load CP, 0: No action
FP ← 0 FP ← $\overline{FP}$	RESET FPI	Asynchronous reset 1: Invert FP, 0: Hold FP
DIE = 1 DIE ≠ 1	DIE1	1: DIE equal to 1 0: DIE not equal to 1
TR1 ≥ 1100100	CP WN	0: Select TR1 for ≥ 1100100 1: The selected TRi ≥ 1100100 0: The selected TRi < 1100100
TR2 ≥ 1100100	CP WN	1: Select TR2 for ≥ 1100100 1: The selected TRi ≥ 1100100 0: The selected TRi < 1100100






```
-- 4-bit Left Shift Register with Reset
```

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity srg_4_r is
```

```
    port (CLK, RESET, SI : in std_logic;
```

```
          Q : out std_logic_vector(3 downto 0);
```

```
          SO : out std_logic);
```

```
end srg_4_r;
```

```
architecture behavioral of srg_4_r is
```

```
    signal shift : std_logic_vector(3 downto 0);
```

```
begin
```

```
    process (RESET, CLK)
```

```
    begin
```

```
        if (RESET = '1') then
```

```
            shift <= "0000";
```

```
        elsif (CLK'event and (CLK = '1')) then
```

```
            shift <= shift(2 downto 0) & SI;
```

```
        end if;
```

```
    end process;
```

```
        Q <= shift;
```

```
        SO <= shift(3);
```

```
    end behavioral;
```

-- 4-bit Binary Counter with Reset

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity count_4_r is
    port (CLK, RESET, EN : in std_logic;
          Q : out std_logic_vector(3 downto 0);
          CO : out std_logic);
end count_4_r;

architecture behavioral of count_4_r is
    signal count : std_logic_vector(3 downto 0);
begin
    process (RESET, CLK)
    begin
        if (RESET = '1') then
            count <= "0000";
        elsif (CLK'event and (CLK = '1') and (EN = '1')) then
            count <= count + "0001";
        end if;
    end process;
    Q <= count;
    CO <= '1' when count = "1111" and EN = '1' else '0';
end behavioral;
```

```
// 4-bit Left Shift Register with Reset
```

```
module srg_4_r_v (CLK, RESET, SI, Q,SO);  
    input CLK, RESET, SI;  
    output [3:0] Q;  
    output SO;
```

```
reg [3:0] Q;
```

```
assign SO = Q[3];
```

```
always@(posedge CLK or posedge RESET)  
begin
```

```
    if (RESET)  
        Q <= 4'b0000;
```

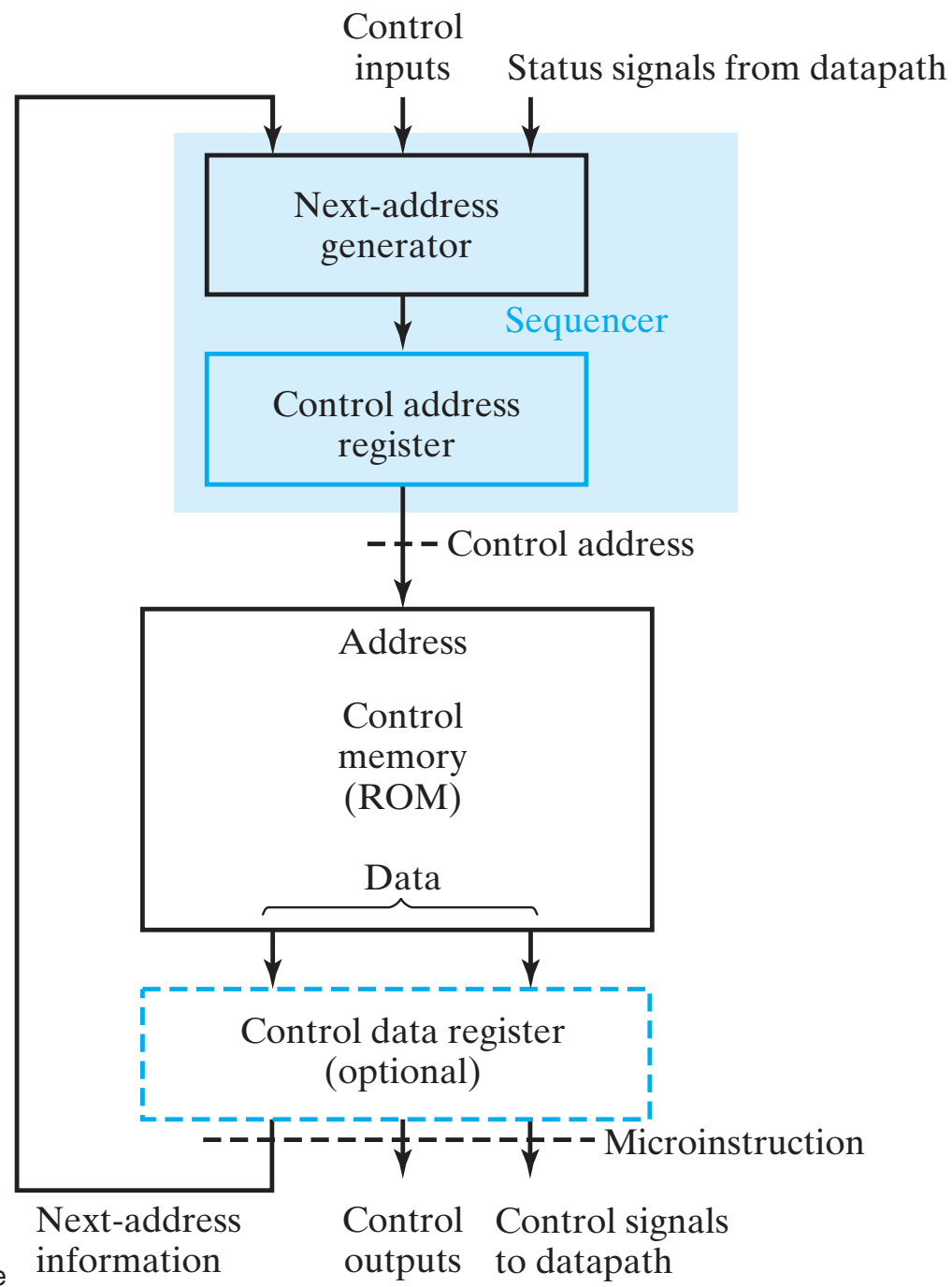
```
    else  
        Q <= {Q[2:0], SI};
```

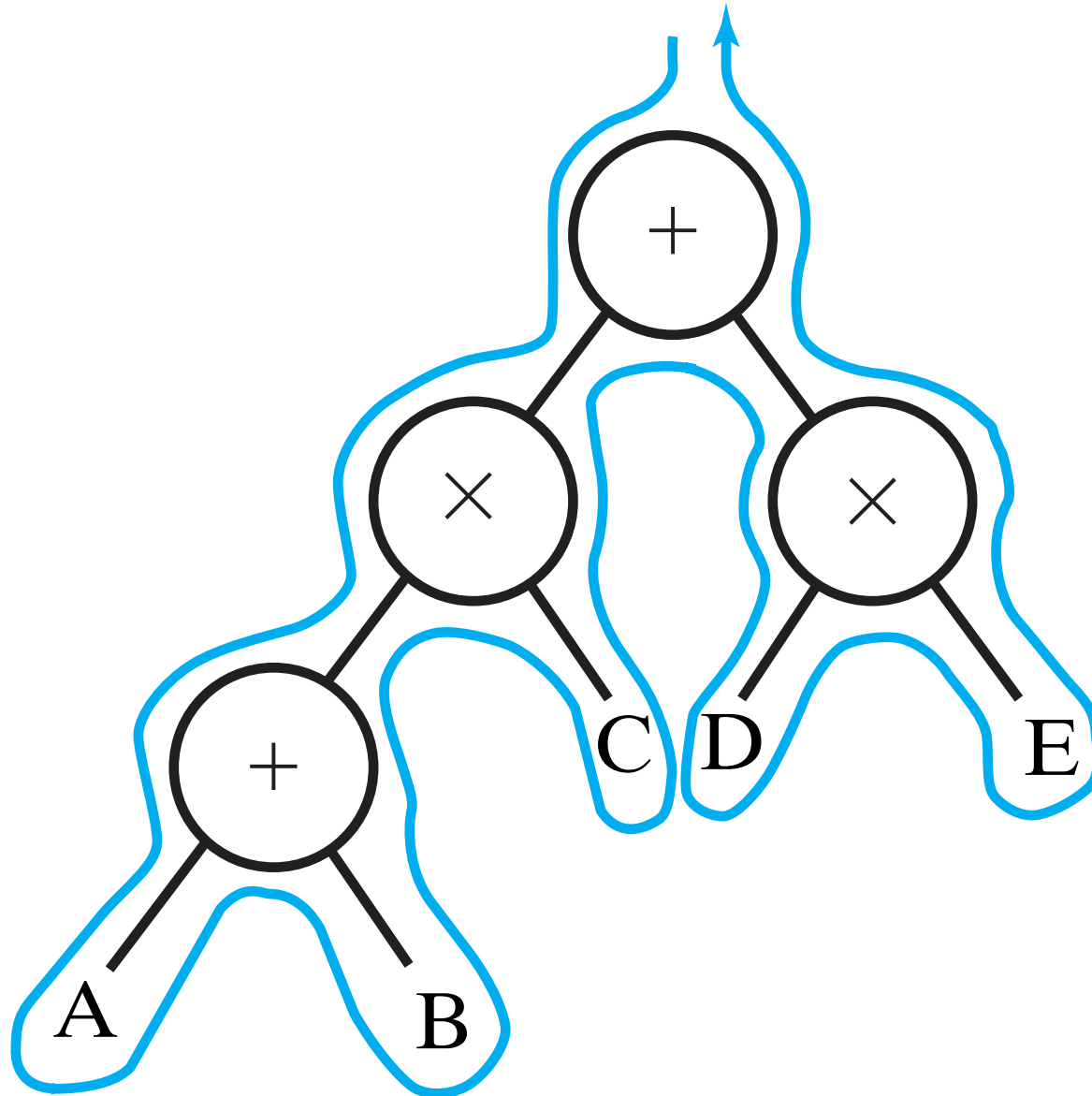
```
end
```

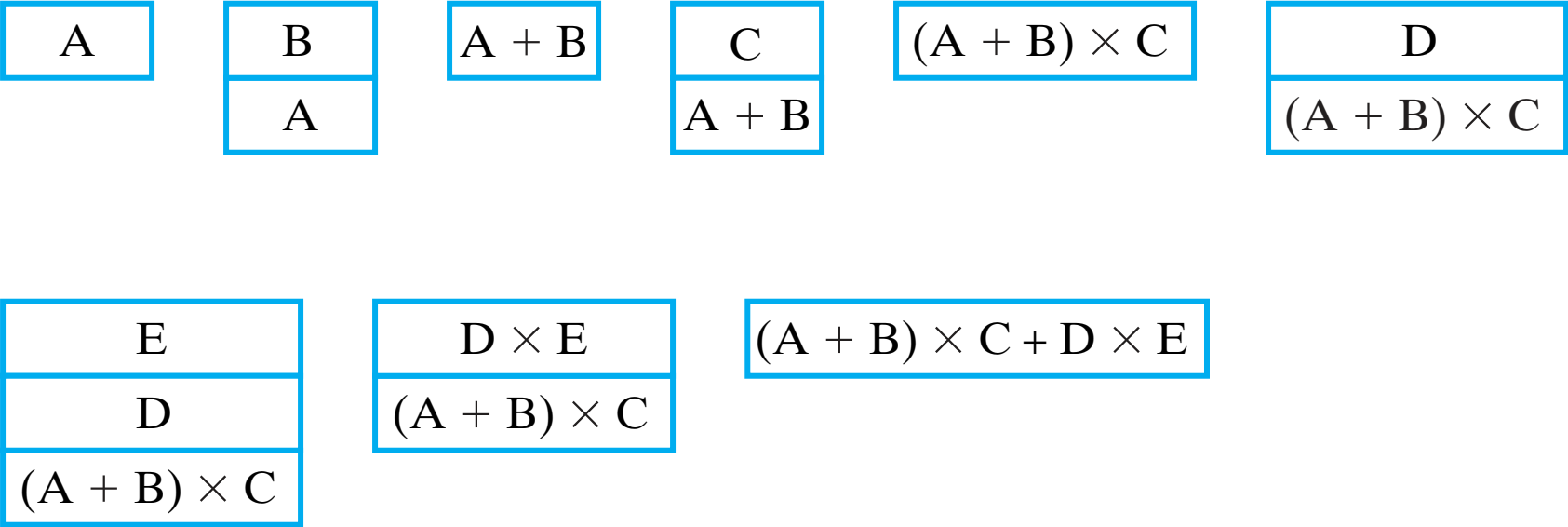
```
endmodule
```

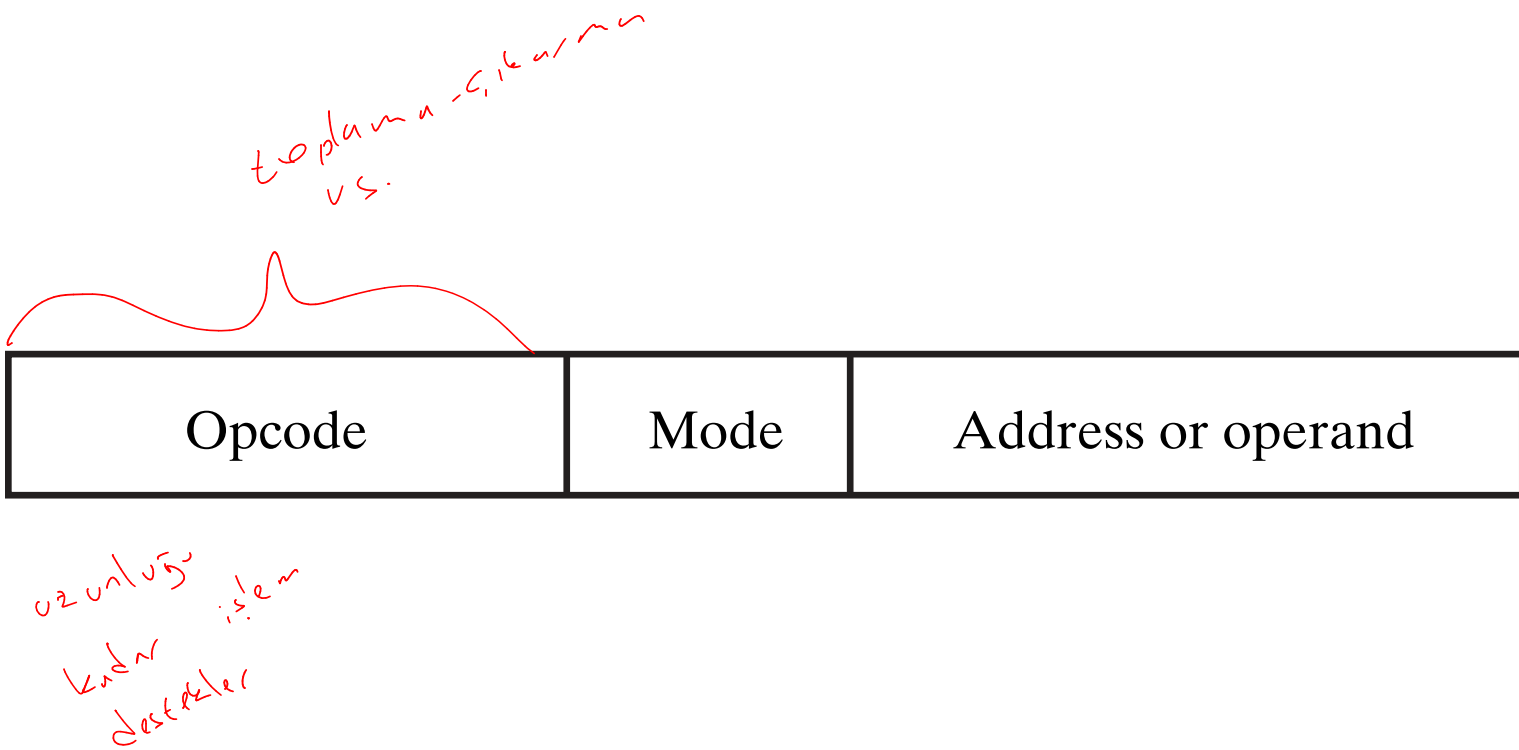
7-33 // 4-bit Binary Counter with Reset

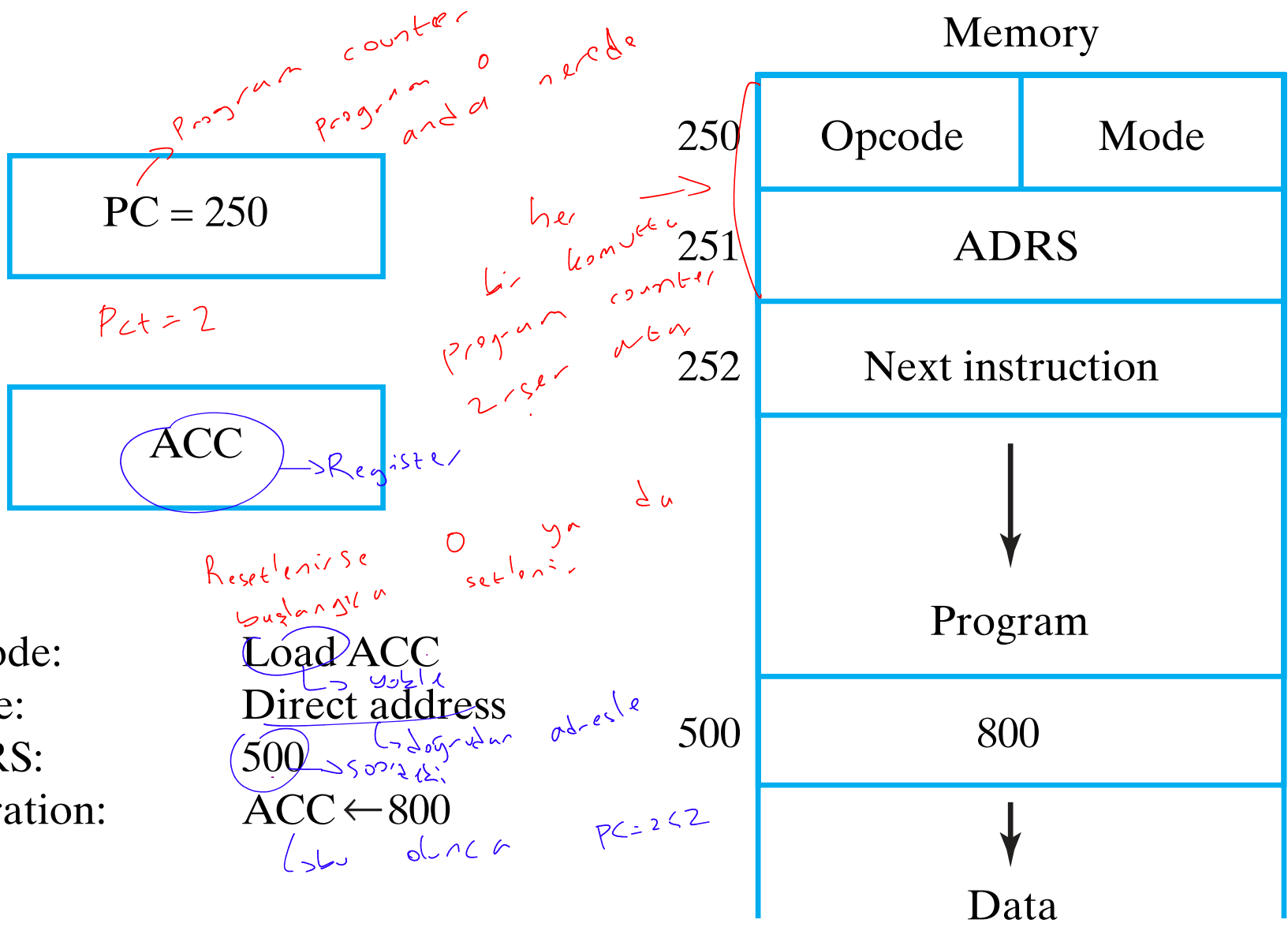
```
module count_4_r_v (CLK, RESET, EN, Q, CO);  
    input CLK, RESET, EN;  
    output [3:0] Q;  
    output CO;  
  
    reg [3:0] Q;  
  
    assign CO = (count == 4'b1111 && EN == 1'b1) ? 1 : 0;  
    always@(posedge CLK or posedge RESET)  
        begin  
            if (RESET)  
                Q <= 4'b0000;  
            else if (EN)  
                Q <= Q + 4'b0001;  
            end  
        endmodule
```











PC = 300

ACC

Opcode: Branch if ACC = 0

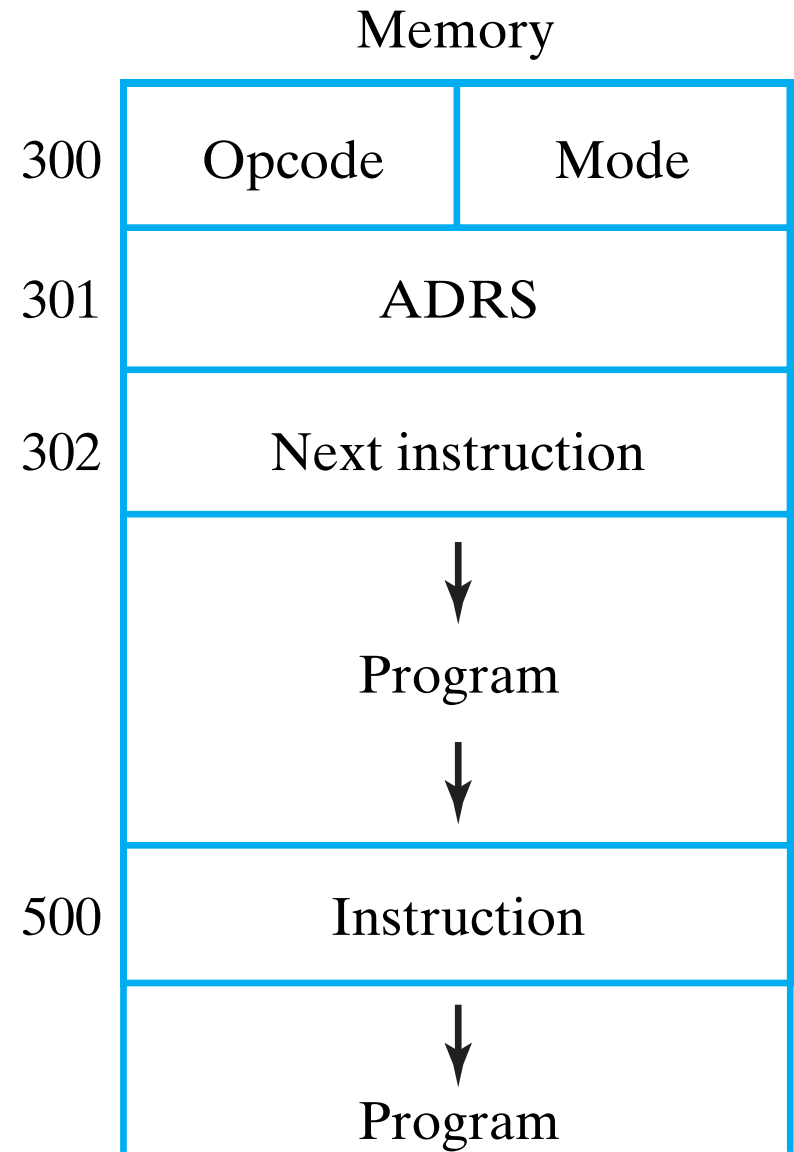
Mode: Direct address

ADRS: 500

Operation: $PC \leftarrow 500$ if $ACC = 0$
 $PC \leftarrow 302$ if $ACC \neq 0$

if (ACC == 0)

↓



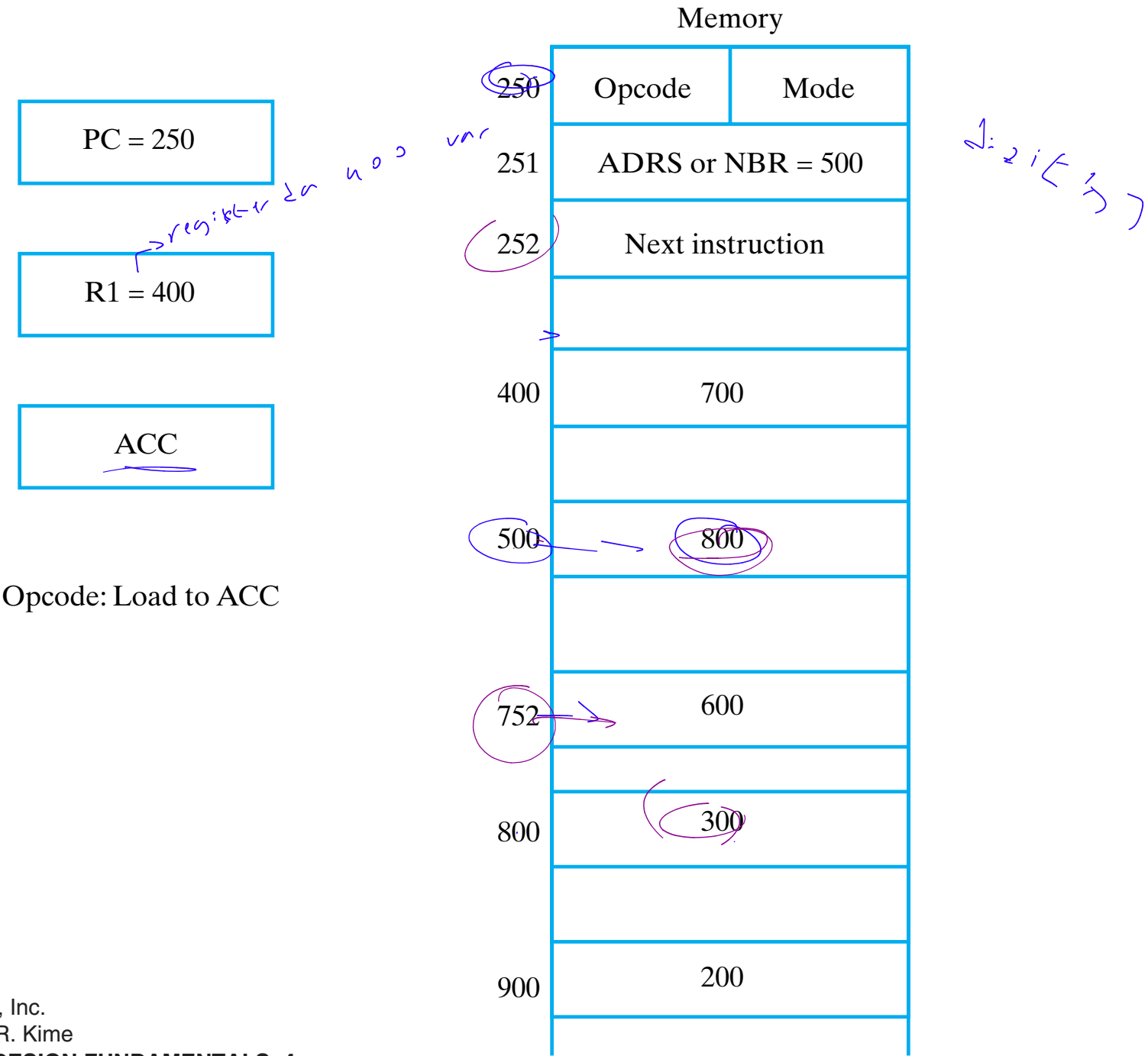


TABLE 10-1
Symbolic Convention for Addressing Modes

			Refers to Figure 10-6	
Addressing Mode	Symbolic Convention	Register Transfer	Effective Address	Contents of ACC
Direct	LDA ADRS	$ACC \leftarrow M[ADRS]$	500	800
Immediate	LDA #NBR	$ACC \leftarrow NBR$	251	500
Indirect	LDA [ADRS]	$ACC \leftarrow M[M[ADRS]]$	800	300
Relative	LDA \$ADRS	$ACC \leftarrow M[ADRS + PC]$	752	600
Index	LDA ADRS (R1)	$ACC \leftarrow M[ADRS + R1]$	900	200
Register	LDA R1	$ACC \leftarrow R1$	—	400
Register-indirect	LDA (R1)	$ACC \leftarrow M[R1]$	400	700

□ TABLE 10-2

Typical Data Transfer Instructions

Name	Mnemonic
Load	LD
Store	ST
Move	MOVE
Exchange	XCH
Push	PUSH
Pop	POP
Input	IN
Output	OUT

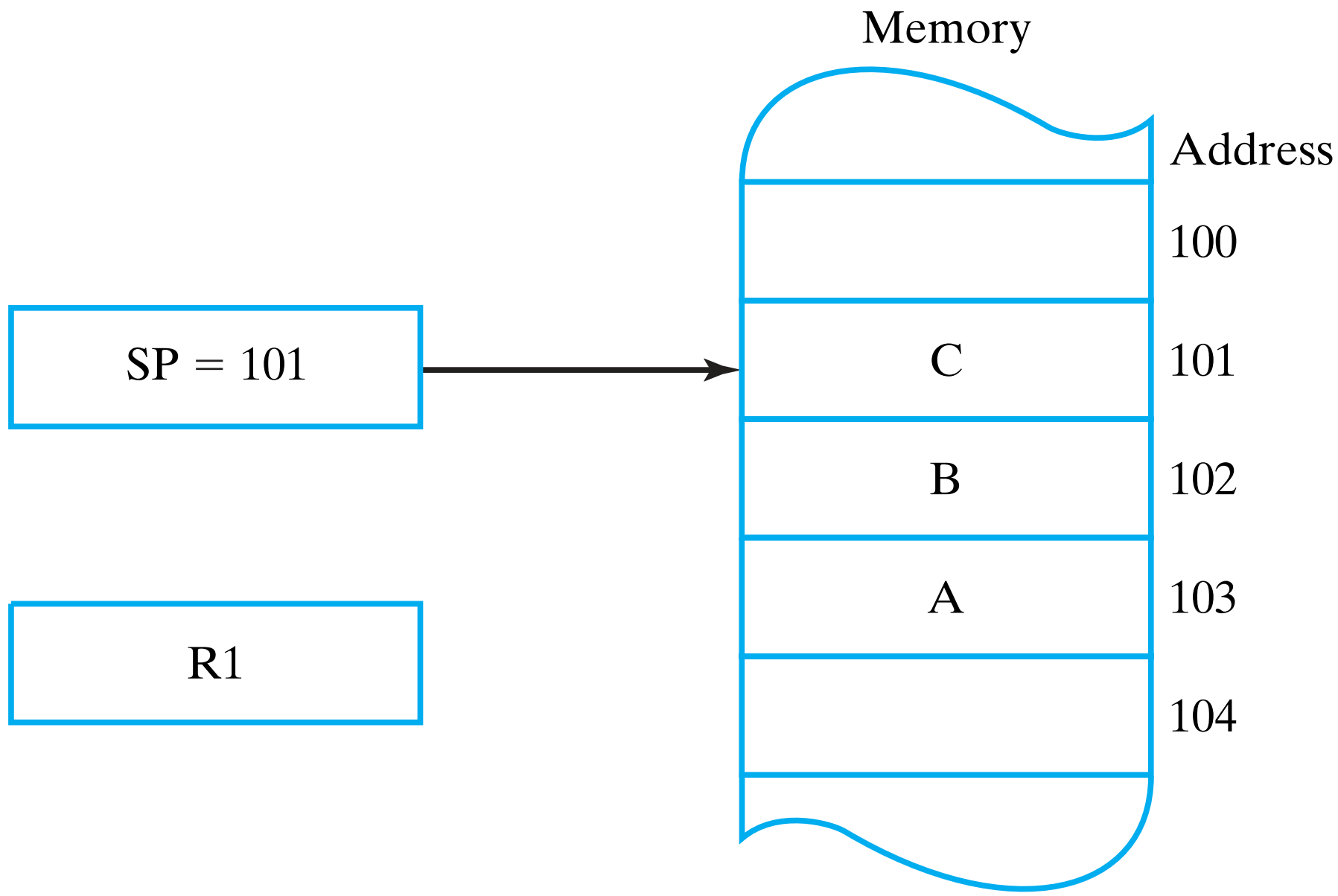


TABLE 10-3 **Typical Arithmetic Instructions**

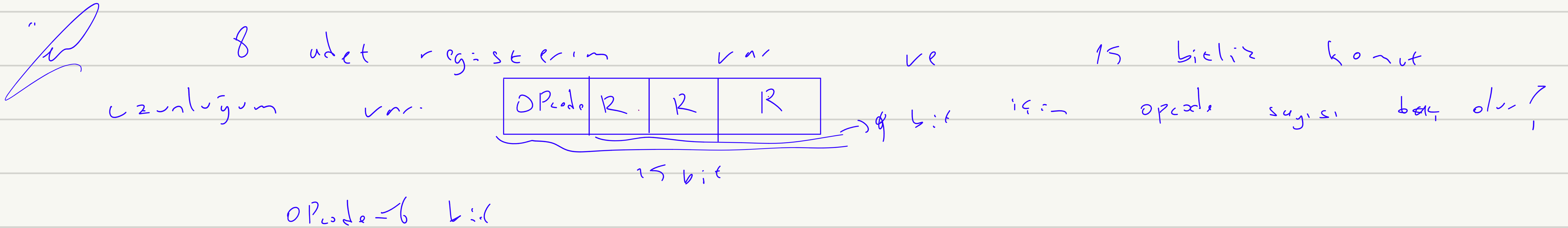
ADD R_1, R_2, R_3

$R_1 \leftarrow R_2 + R_3$

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Subtract reverse	SUBR
Negate	NEG

$A - B$
 $B - A$

$(A - B)$ $(B - A)$
 dis. $(A - B)$



R₁(4) bitinde 1 var mı? R_n = 7 ise 00000111
 and 00010000
 olamaz

AND R₁, 10H

R₁ değeri 11e'ye düşürülür

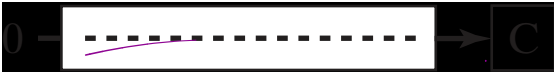
□ TABLE 10-4

Typical Logical and Bit-Manipulation Instructions

Name	Mnemonic
Clear	CLR
Set	SET
Complement	NOT
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC

→ Register of 10

TABLE 10-5
Typical Shift Instructions

Name	Mnemonic	Diagram
Logical shift right	SHR	
Logical shift left	SHL	
Arithmetic shift right	SHRA	
Arithmetic shift left	SHLA	
Rotate right	ROR	
Rotate left	ROL	
Rotate right with carry	RORC	
Rotate left with carry	ROLC	



□ TABLE 10-6

Evaluating Biased Exponents

Exponent E in decimal	Biased exponent $e = E + 127$	
	Decimal	Binary
-126	$-126 + 127 = 1$	00000001
-001	$-001 + 127 = 126$	01111110
000	$000 + 127 = 127$	01111111
+001	$001 + 127 = 128$	10000000
+126	$126 + 127 = 253$	11111101
+127	$127 + 127 = 254$	11111110

TABLE 10-7

Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Call procedure	CALL
Return from procedure	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TEST

Dallam

→ koşulu var CMP ile
salış

→ koşulu ok

→ fonksiyon çağırış

□ TABLE 10-8

Conditional Branch Instructions Relating to Status Bits in the PSR

Branch Condition	Mnemonic	Test Condition
Branch if zero	BZ	$Z = 1$
Branch if not zero	BNZ	$Z = 0$
Branch if carry	BC	$C = 1$
Branch if no carry	BNC	$C = 0$
Branch if minus	BN	$N = 1$
Branch if plus	BNN	$N = 0$
Branch if overflow <i>→ Disruption signal generated</i>	BV	$V = 1$
Branch if no overflow	BNV	$V = 0$

$A - B \geq 0$

$Z = 0$
 $C = 0$

TABLE 10-9
Conditional Branch Instructions for Unsigned Numbers

Branch Condition	Mnemonic	Condition	Status Bits*
Branch if above	BA	$A > B$	$C + Z = 0$
Branch if above or equal	BAE	$A \geq B$	$C = 0$
Branch if below	BB	$A < B$	$C = 1$
Branch if below or equal	BBE	$A \leq B$	$C + Z = 1$
Branch if equal	BE	$A = B$	$Z = 1$
Branch if not equal	BNE	$A \neq B$	$Z = 0$

*Note that C here is a borrow bit.

$A = 1100 \quad 0010$
 $B = 0011 \quad 1011$

$CMP \ A, B \rightarrow$

1100	0010
- 0011	1011
1000	0111

$\rightarrow$ Single gap in 2's complement

$CMP \ A, B \rightarrow$

1100	0010
+ 1100	0101
1000	0111

$$V = 1 \oplus 1 = 0$$

$$N = 1$$

$$Z = 0$$

$(1011 \quad 0100) \rightarrow SHLA$

$$C = 1$$

0110 1000

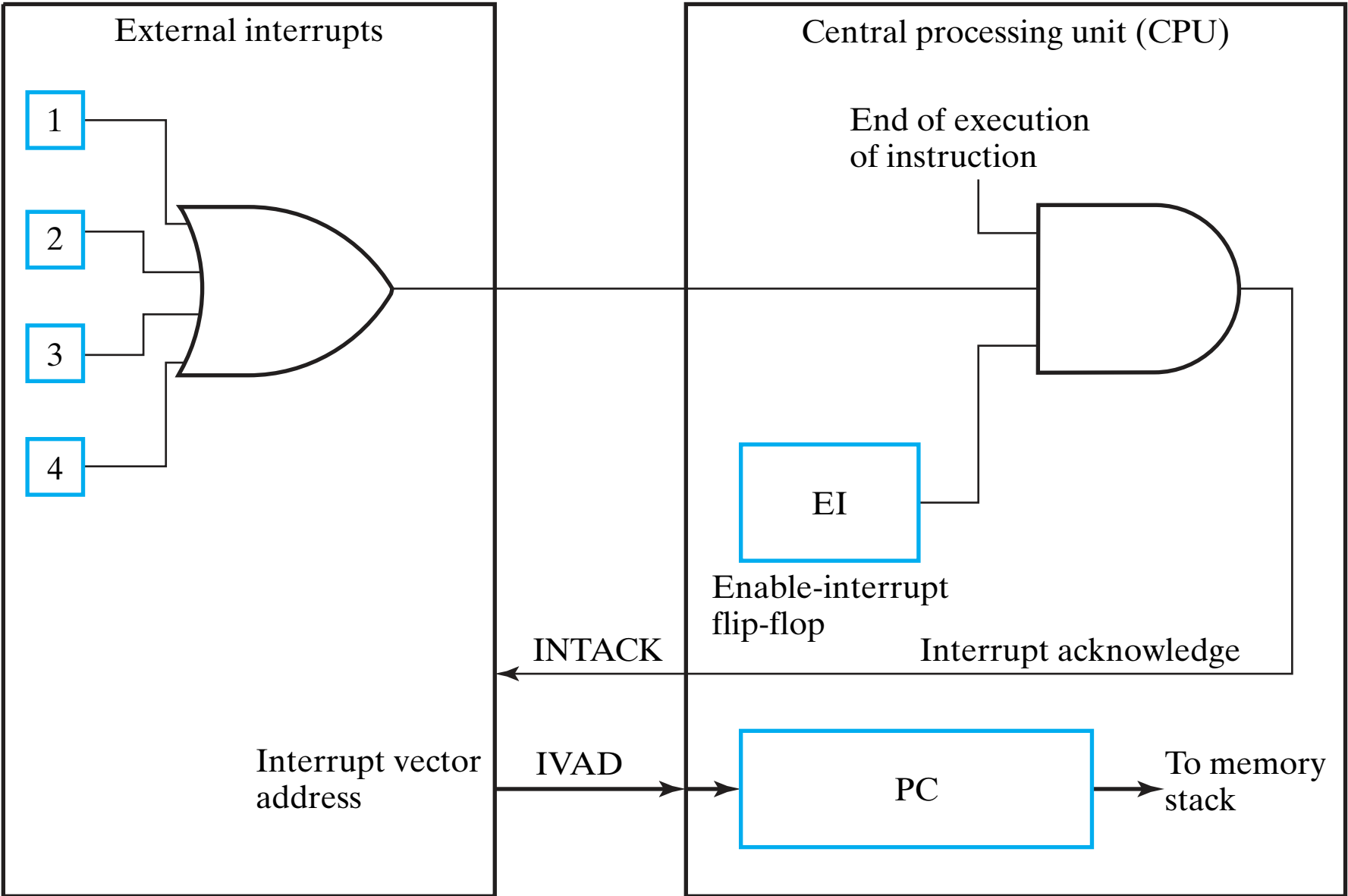
$$Z = 0$$

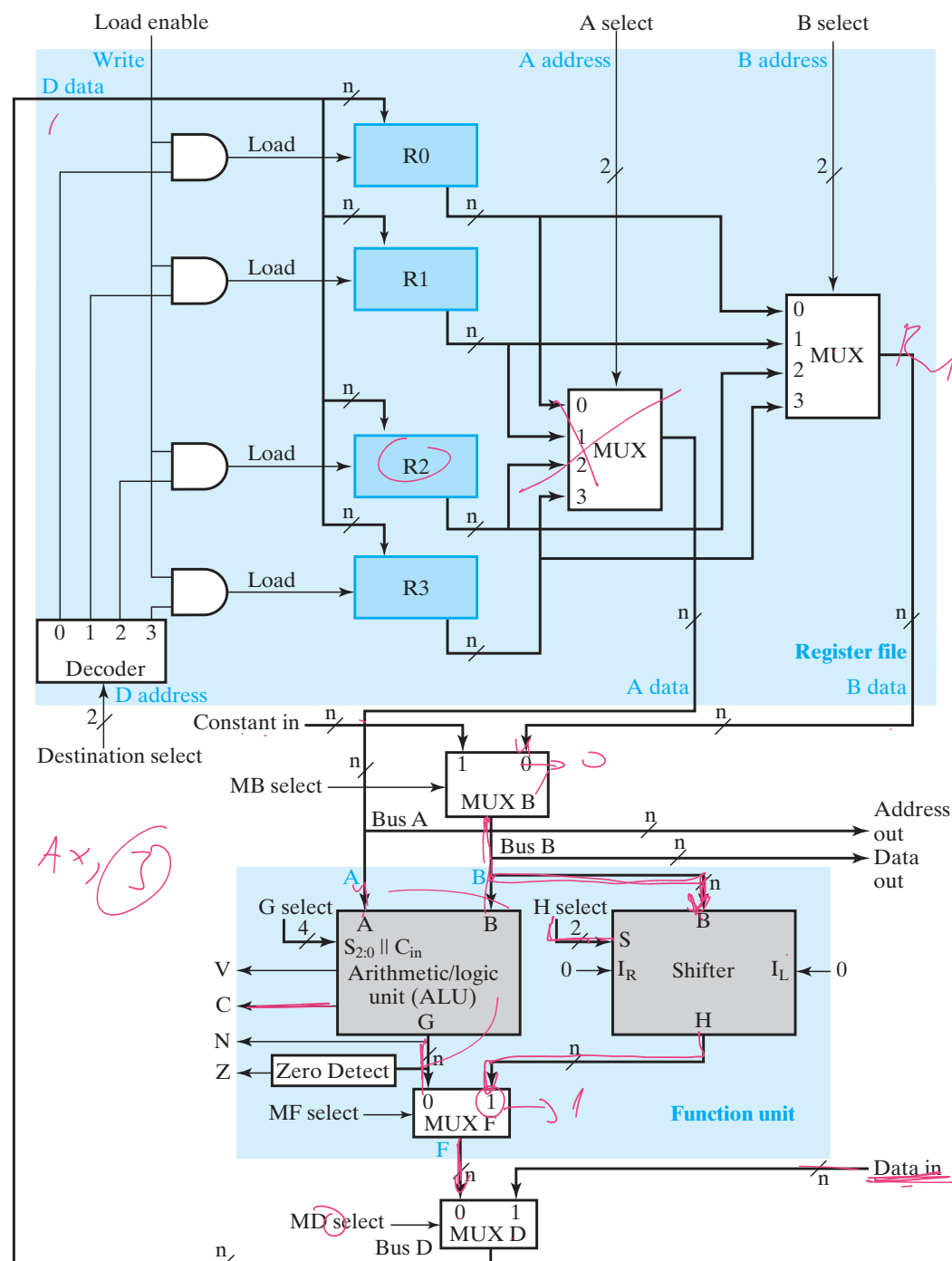
$$N = 0$$

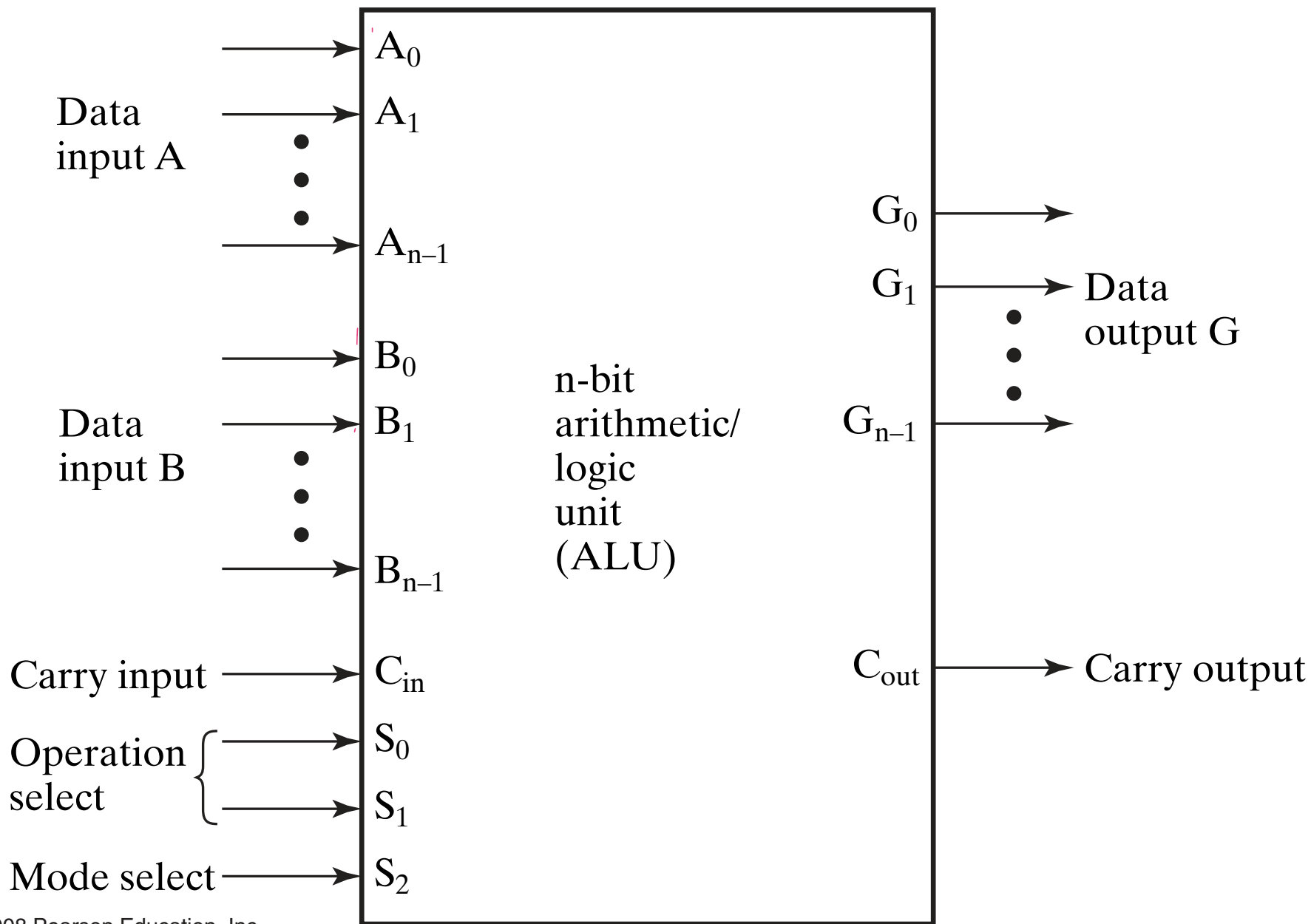
$V = 1 \rightarrow$ sign positive odd

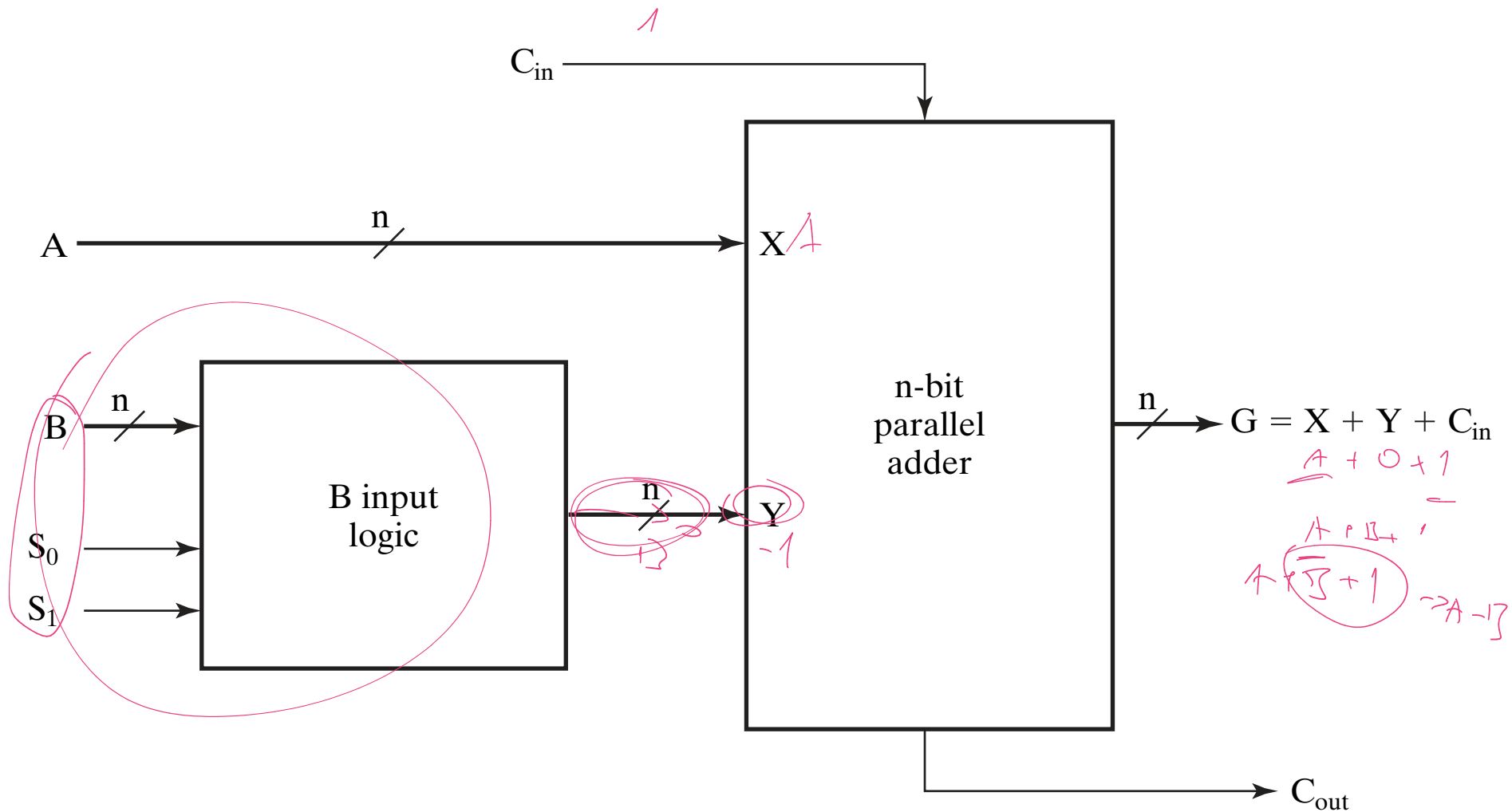
TABLE 10-10
Conditional Branch Instructions for Signed Numbers

Branch condition	Mnemonic	Condition	Status Bits
Branch if greater	BG	$A > B$	$(N \oplus V) + Z = 0$
Branch if greater or equal	BGE	$A \geq B$	$N \oplus V = 0$
Branch if less	BL	$A < B$	$N \oplus V = 1$
Branch if less or equal	BLE	$A \leq B$	$(N \oplus V) + Z = 1$
Branch if equal	BE	$A = B$	$Z = 1$
Branch if not equal	BNE	$A \neq B$	$Z = 0$







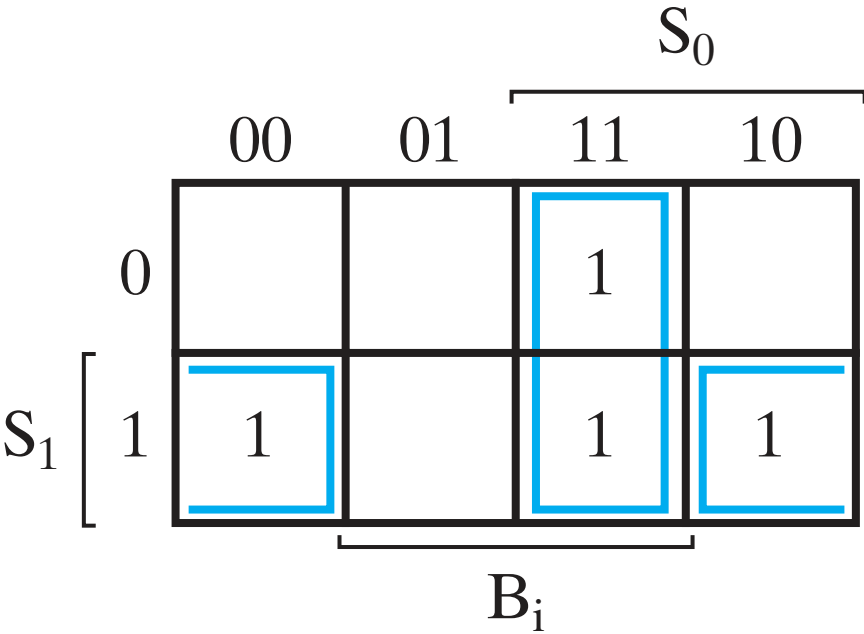


□ **TABLE 9-1**
Function Table for Arithmetic Circuit

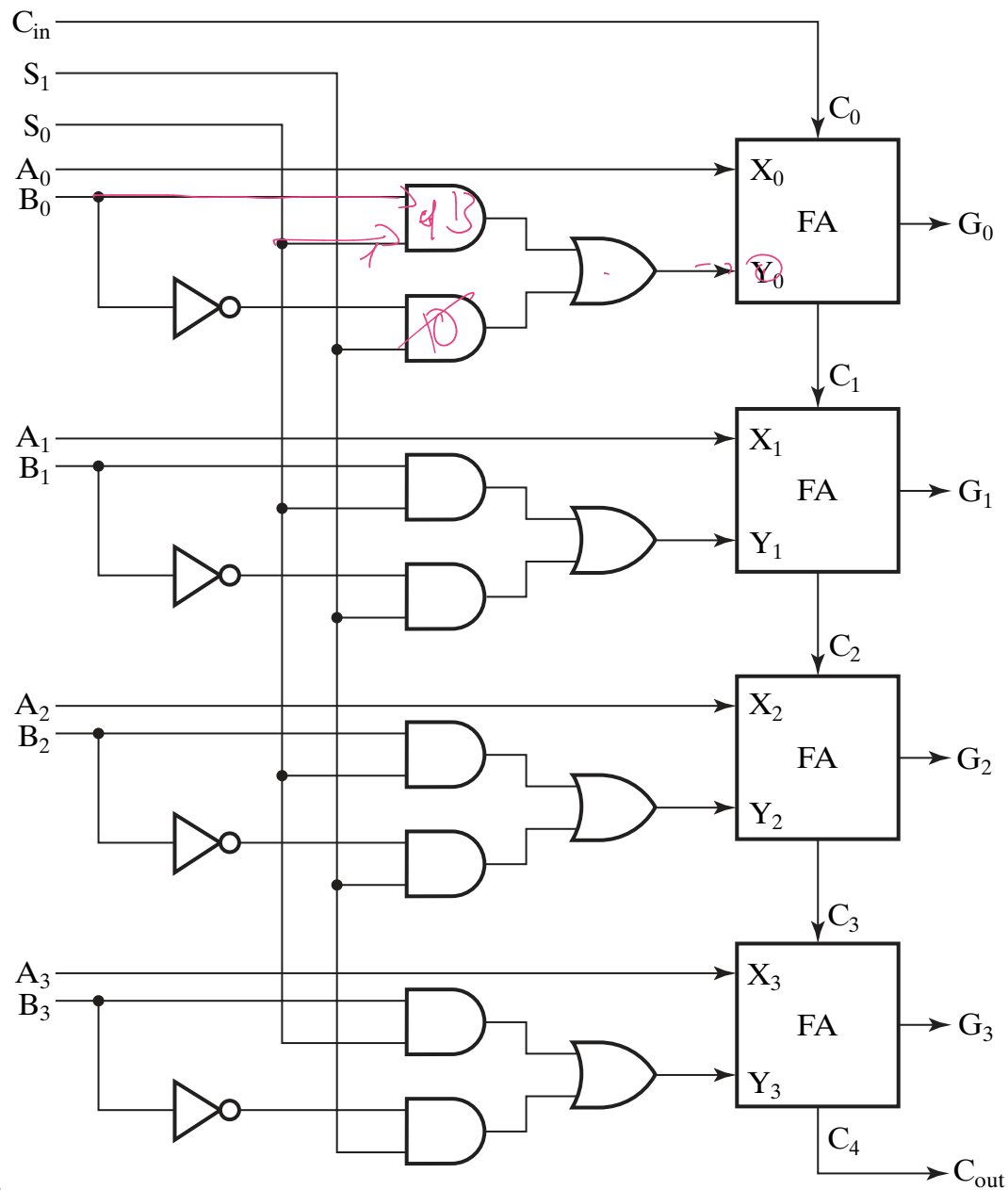
Select		Input	$G = (A \vee Y \vee C_{in})$	
S_1	S_0	Y	$C_{in} = 0$	$C_{in} = 1$
0	0	all 0s	$G = A$ (transfer)	$G = A + 1$ (increment)
0	1	B	$G = A + B$ (add)	$G = A + B + 1$
1	0	$\overline{B}$	$G = A + \overline{B}$	$G = A + \overline{B} + 1$ (subtract)
1	1	all 1s	$G = A - 1$ (decrement)	$G = A$ (transfer)

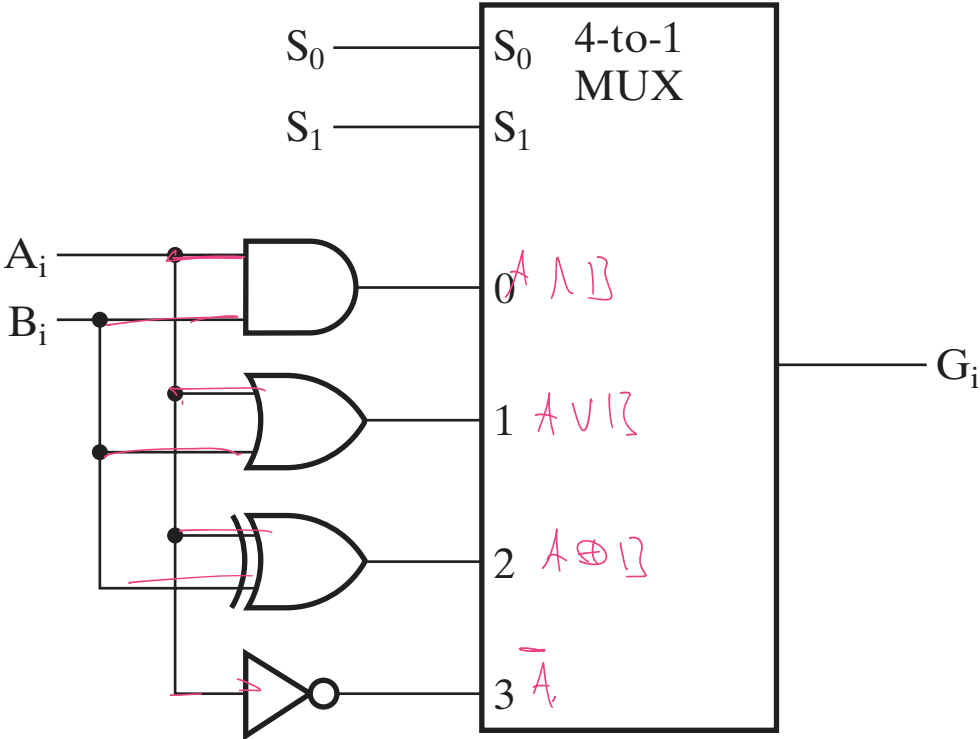
Inputs			Output
S_1	S_0	B_i	Y_i
0	0	0	0 $Y_i = 0$
0	0	1	0
0	1	0	0 $Y_i = B_i$
0	1	1	1
1	0	0	1 $Y_i = \overline{B_i}$
1	0	1	0
1	1	0	1 $Y_i = 1$
1	1	1	1

(a) Truth table



(b) Map simplification:
 $Y_i = B_i S_0 + \overline{B_i} S_1$





(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$G = A \wedge B$	AND
0	1	$G = A \vee B$	OR
1	0	$G = A \oplus B$	XOR
1	1	$G = \bar{A}$	NOT

(b) Function table

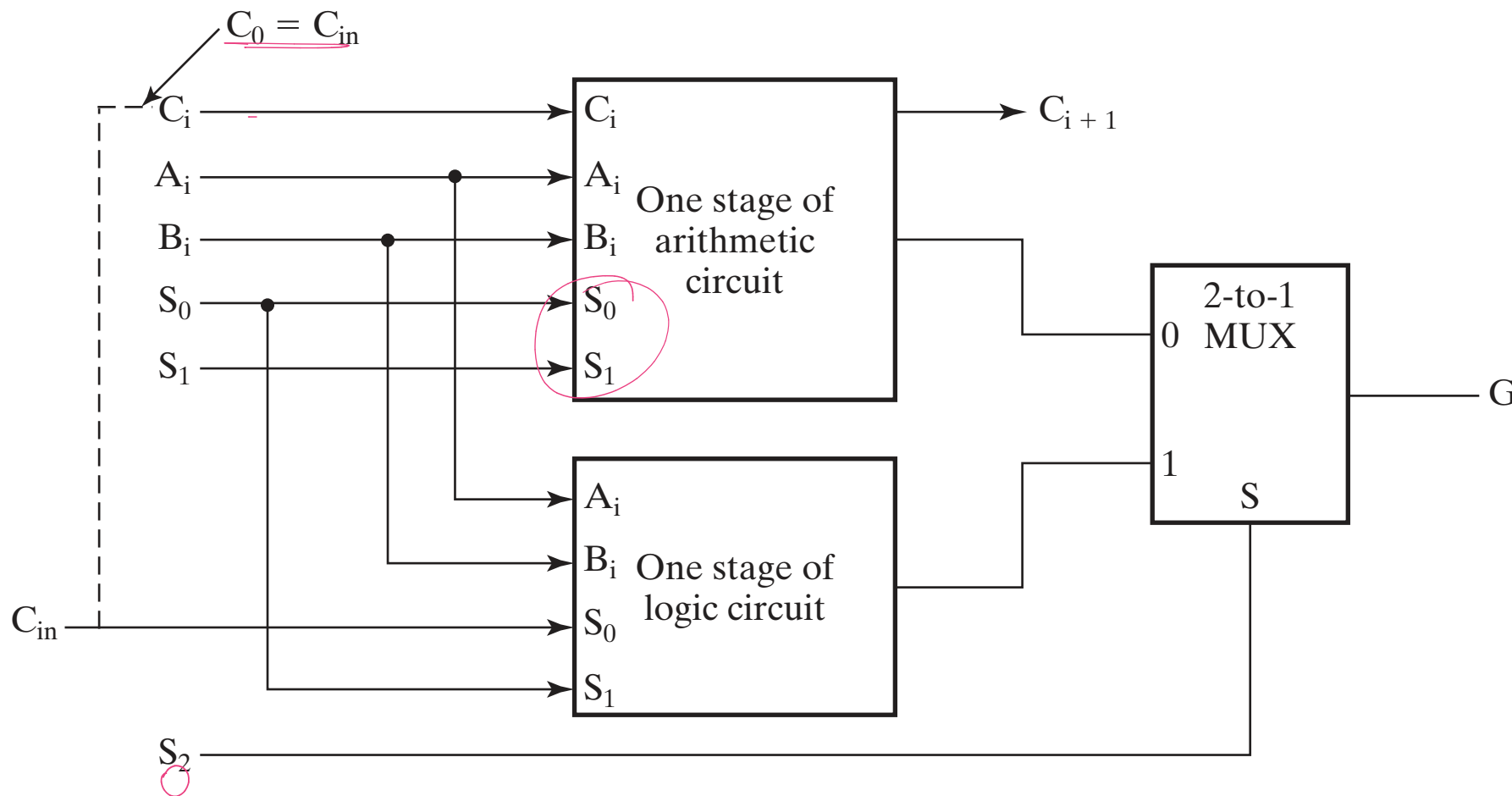


TABLE 9-2
Function Table for ALU

Operation Select				Operation	Function
S_2	S_1	S_0	C_{in}		
0	0	0	0	$G = A$	Transfer A
0	0	0	1	$G = A + 1$	Increment A
0	0	1	0	$G = A + B$	Addition
0	0	1	1	$G = A + B + 1$	Add with carry input of 1
0	1	0	0	$G = A + \overline{B}$	A plus 1s complement of B
0	1	0	1	$G = A + \overline{B} + 1$	Subtraction
0	1	1	0	$G = A - 1$	Decrement A
0	1	1	1	$G = A$	Transfer A
1	X	0	0	$G = A \wedge B$	AND
1	X	0	1	$G = A \vee B$	OR
1	X	1	0	$G = A \oplus B$	XOR
1	X	1	1	$G = \overline{A}$	NOT (1s complement)

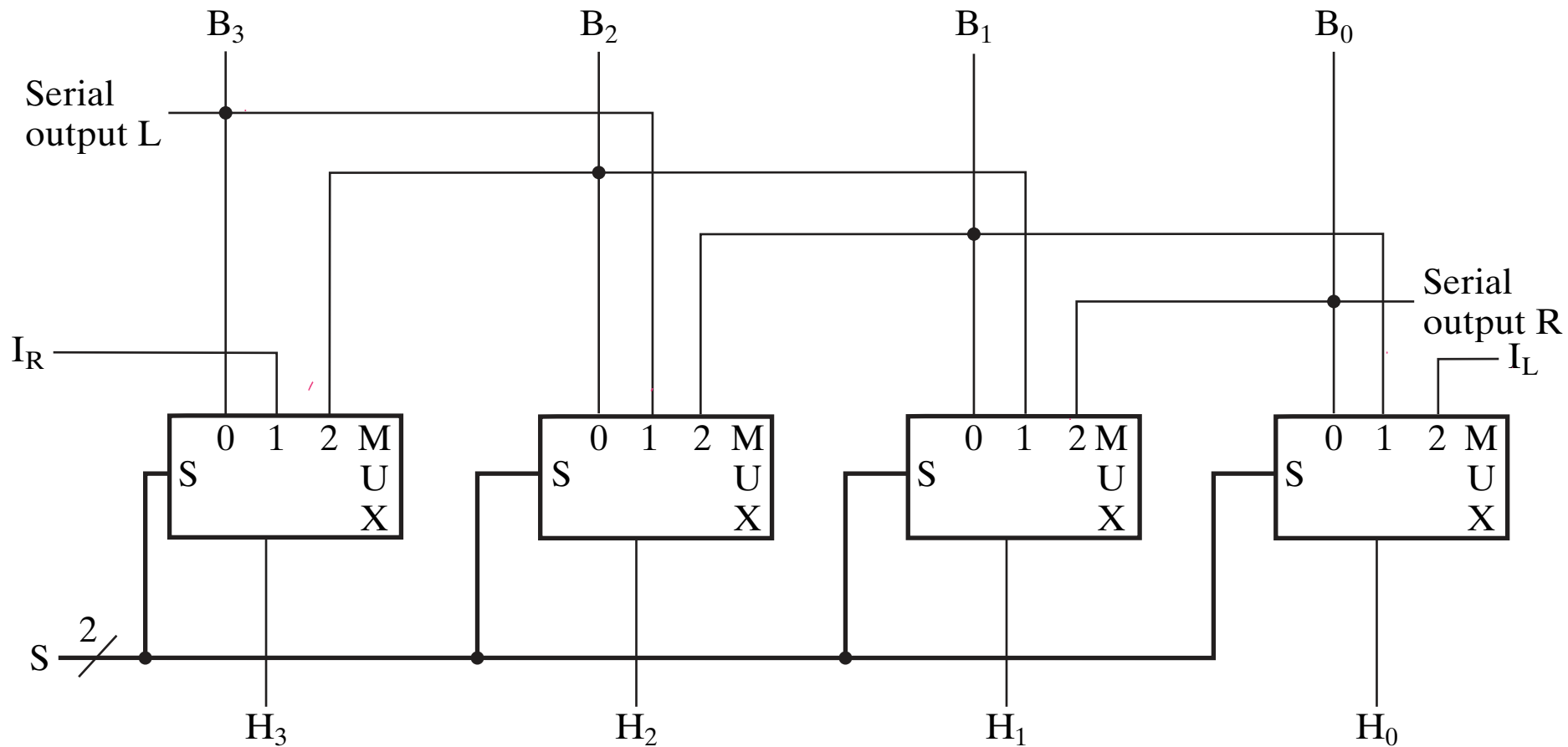
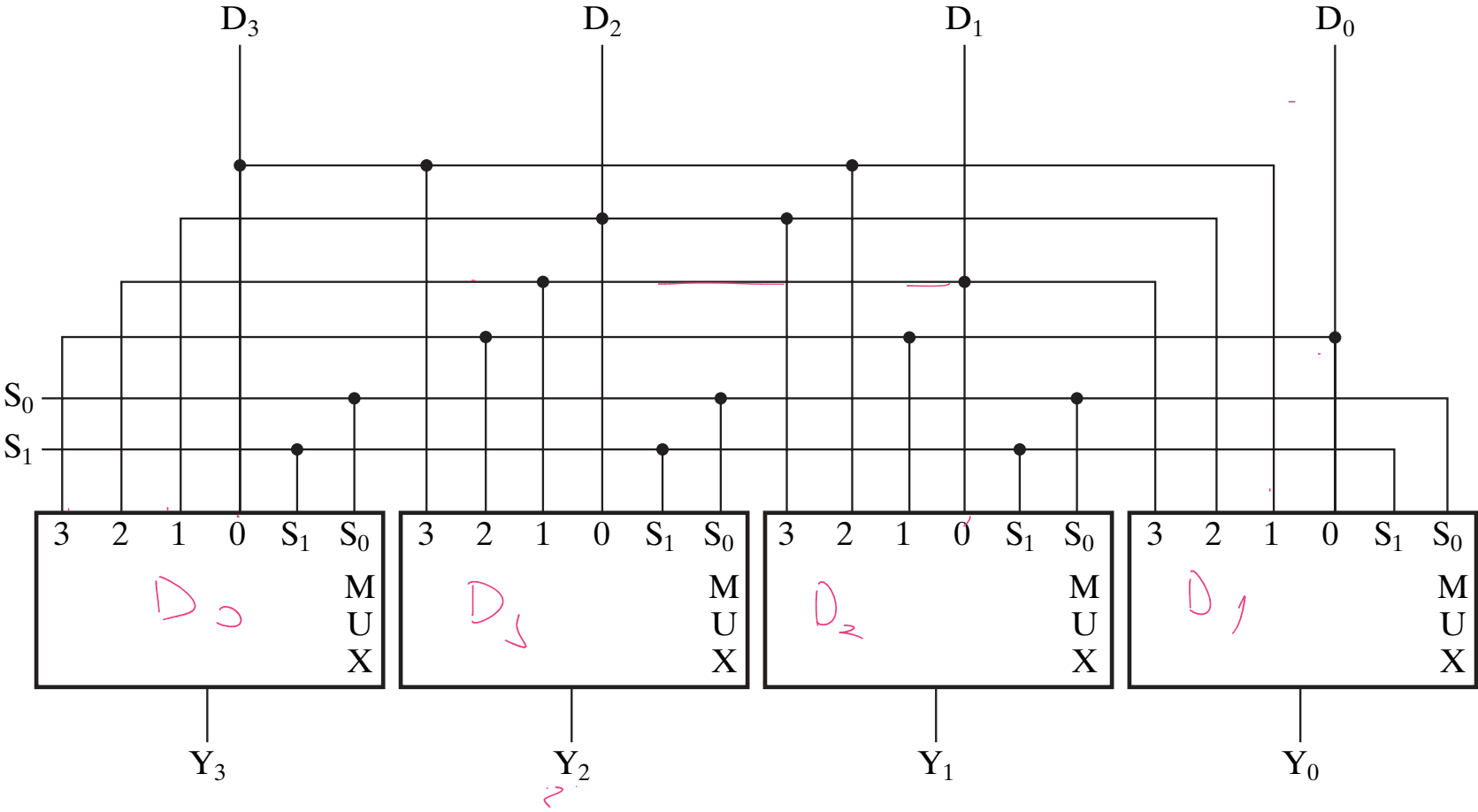
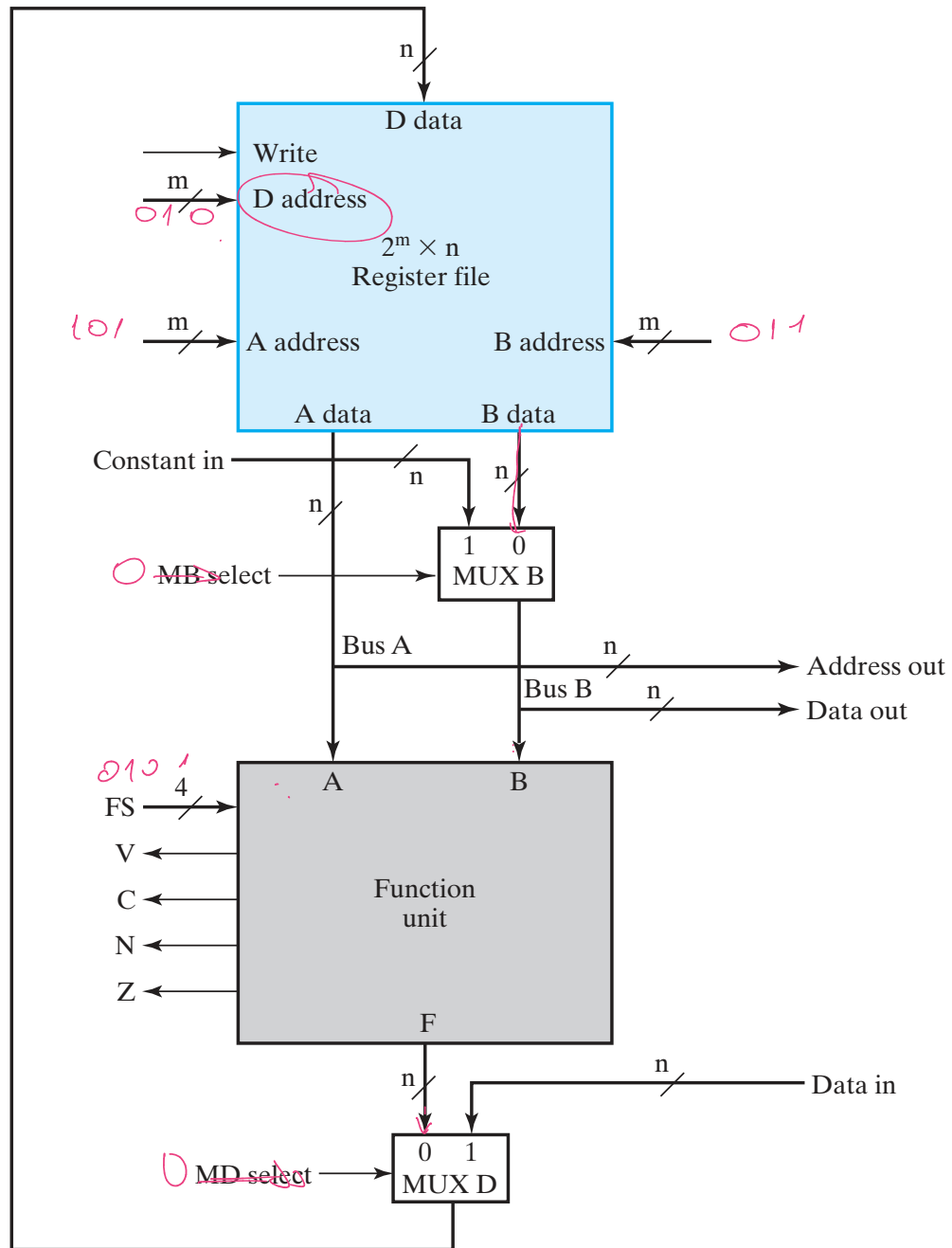


TABLE 9-3
Function Table for 4-Bit Barrel Shifter

Select		Output				Operation
S_1	S_0	Y_3	Y_2	Y_1	Y_0	
0	0	D_3	D_2	D_1	D_0	No rotation
0	1	D_2	D_1	D_0	D_3	Rotate one position
1	0	D_1	D_0	D_3	D_2	Rotate two positions
1	1	D_0	D_3	D_2	D_1	Rotate three positions



$$R_2 \leftarrow R_3 - R_3$$



□ **TABLE 9-4**

***G* Select, *H* Select, and *MF* Select Codes Defined
in Terms of *FS* Codes**

Function Select

<i>FS</i> (3:0)	<i>MF</i> Select	<i>G</i> Select(3:0)	<i>H</i> Select(3:0)	Microoperation
0000	0	0000	XX	$F = A$
0001	0	0001	XX	$F = A + 1$
0010	0	0010	XX	$F = A + B$
0011	0	0011	XX	$F = A + B + 1$
0100	0	0100	XX	$F = A + \overline{B}$
→ 0101	0	0101	XX	$F = A + \overline{B} + 1$ → $A - B$
0110	0	0110	XX	$F = A - 1$
0111	0	0111	XX	$F = A$
1000	0	1X00	XX	$F = A \wedge B$
1001	0	1X01	XX	$F = A \vee B$
1010	0	1X10	XX	$F = A \oplus B$
1011	0	1X11	XX	$F = \overline{A}$
1100	1	XXXX	00	$F = B$
1101	1	XXXX	01	$F = \text{sr } B$
1110	1	XXXX	10	$F = \text{sl } B$

Brand
R ← *201*

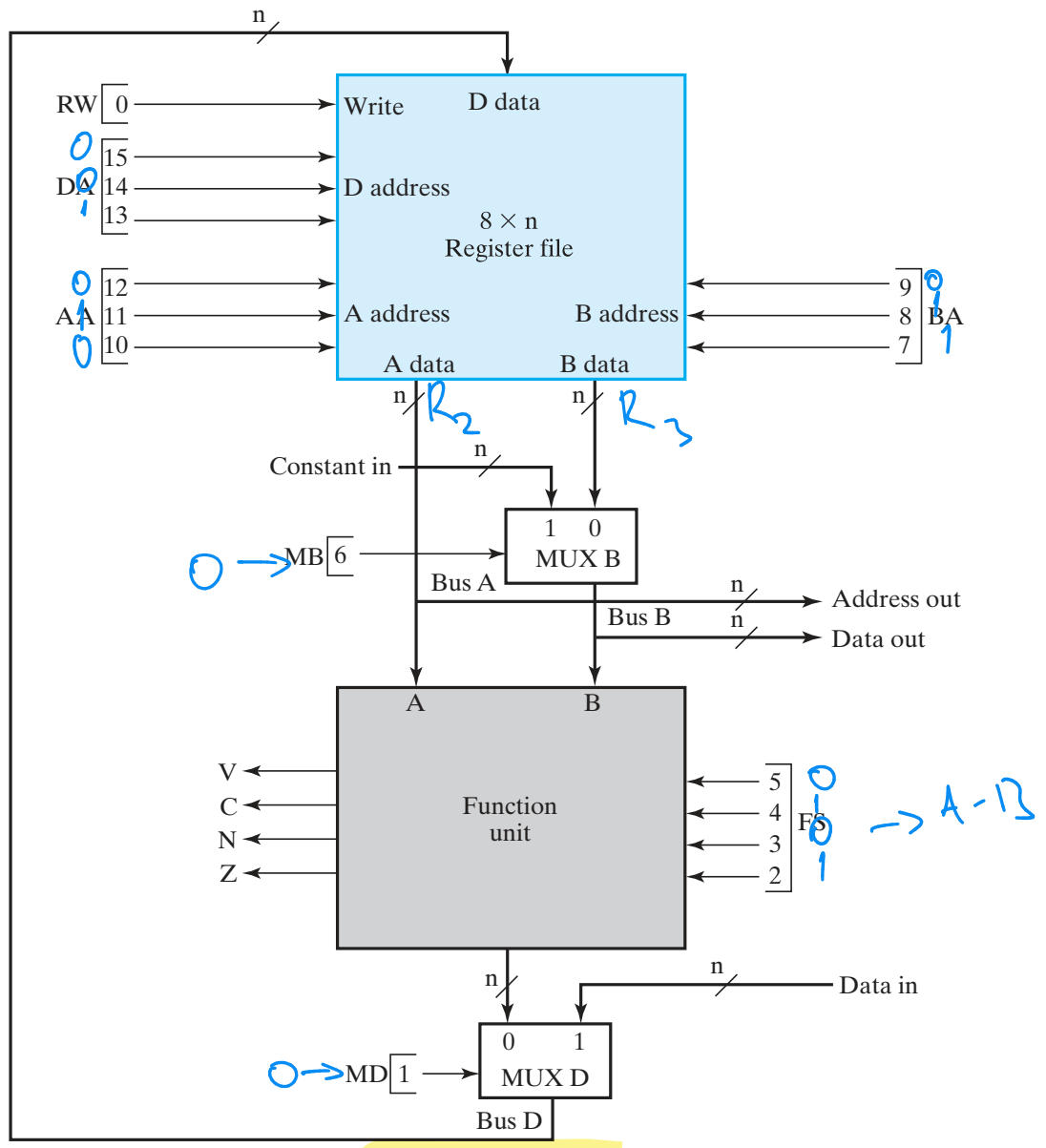
Handwritten notes:

$\overline{DA} \rightarrow R_1$

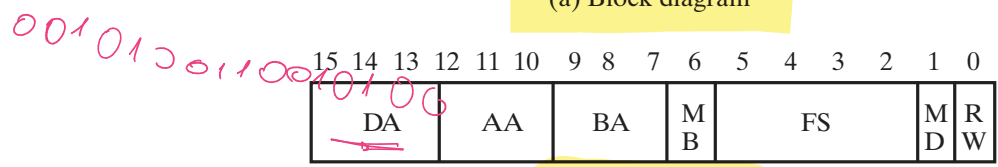
$\overline{AA} \rightarrow R_2$

$\overline{BA} \rightarrow R_3$

$R_1 \leftarrow R_2 - R_3$

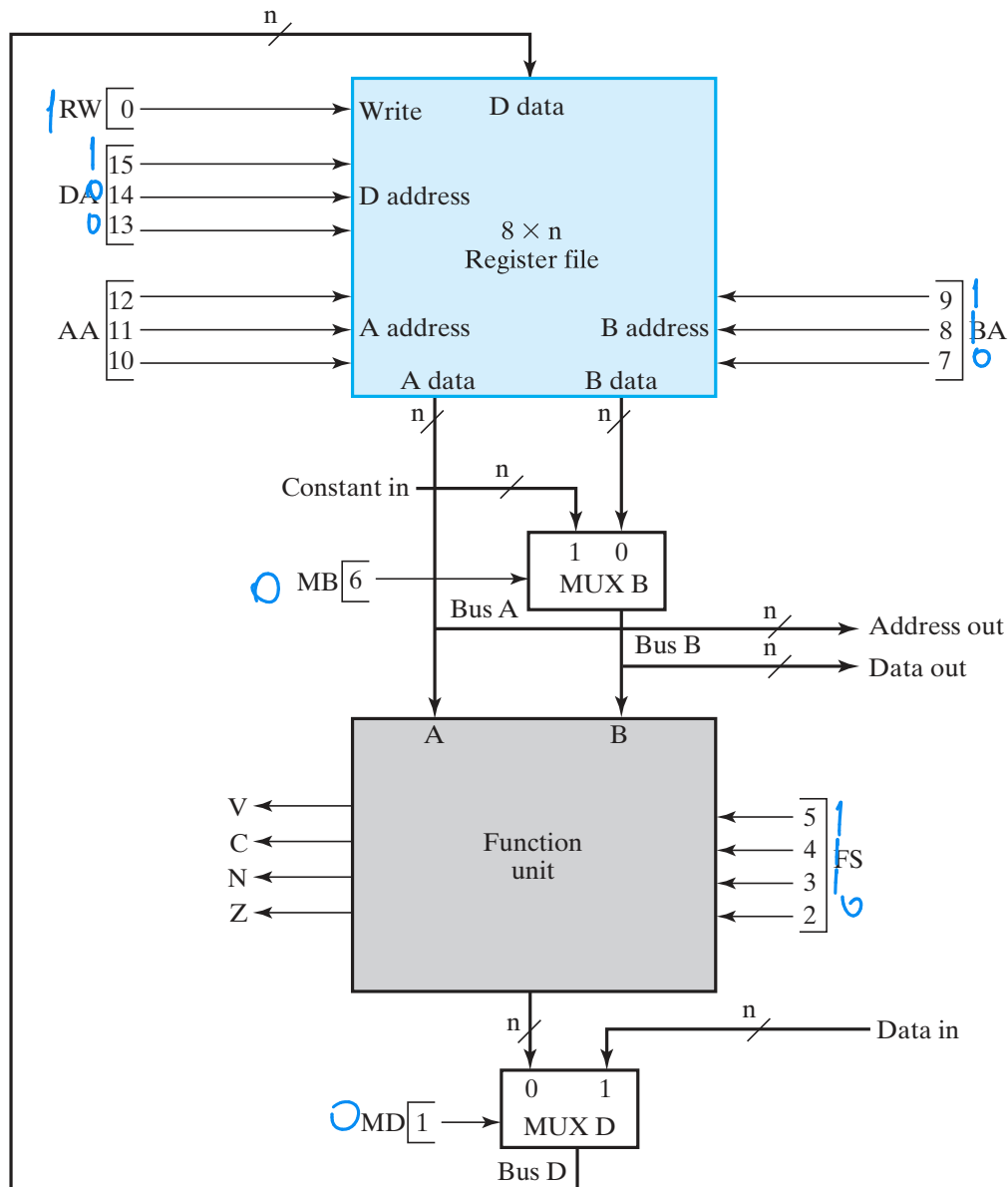


(a) Block diagram

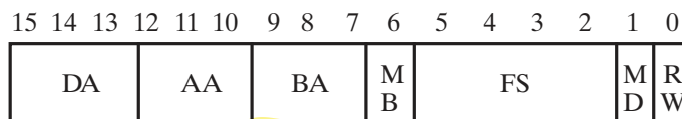


(b) Control word

$R_n \leftarrow s/r\ 6$



(a) Block diagram



(b) Control word

TABLE 9-5
Encoding of Control Word for the Datapath

DA, AA, BA		MB		FS		MD		RW	
Function	Code	Function	Code	Function	Code	Function	Code	Function	Code
$R0 \rightarrow$	000	Register	0	$F = A$	0000	Function	0	No Write	0
$R1 \rightarrow$	001	Constant	1	$F = A + 1$	0001	Data in	1	Write	1
$R2 \rightarrow$	010			$F = A + B$	0010				
$R3 \rightarrow$	011			$F = A + \overline{B} + 1$	0011				
$R4 \rightarrow$	100			$F = A + \overline{B}$	0100				
$R5 \rightarrow$	101			$F = A + \overline{B} + 1$	0101				
$R6 \rightarrow$	110			$F = A - 1$	0110				
$R7 \rightarrow$	111			$F = A$	0111				
				$F = A \wedge B$	1000				
				$F = A \vee B$	1001				
				$F = A \oplus B$	1010				
				$F = \overline{A}$	1011				
				$F = B$	1100				
				$F = \text{sr } B$	1101				
				$F = \text{sl } B$	1110				

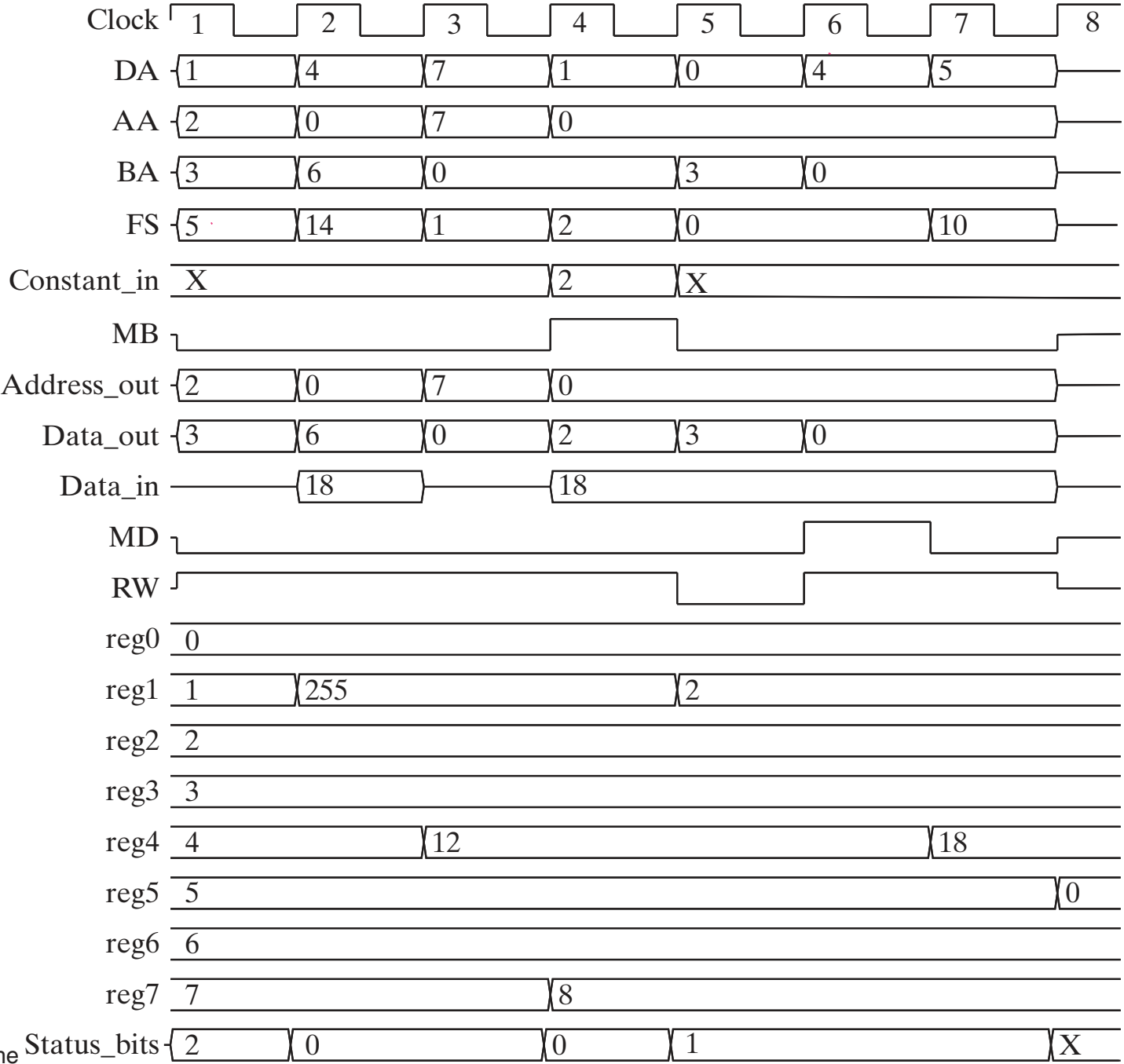
TABLE 9-6
Examples of Microoperations for the Datapath, Using Symbolic Notation

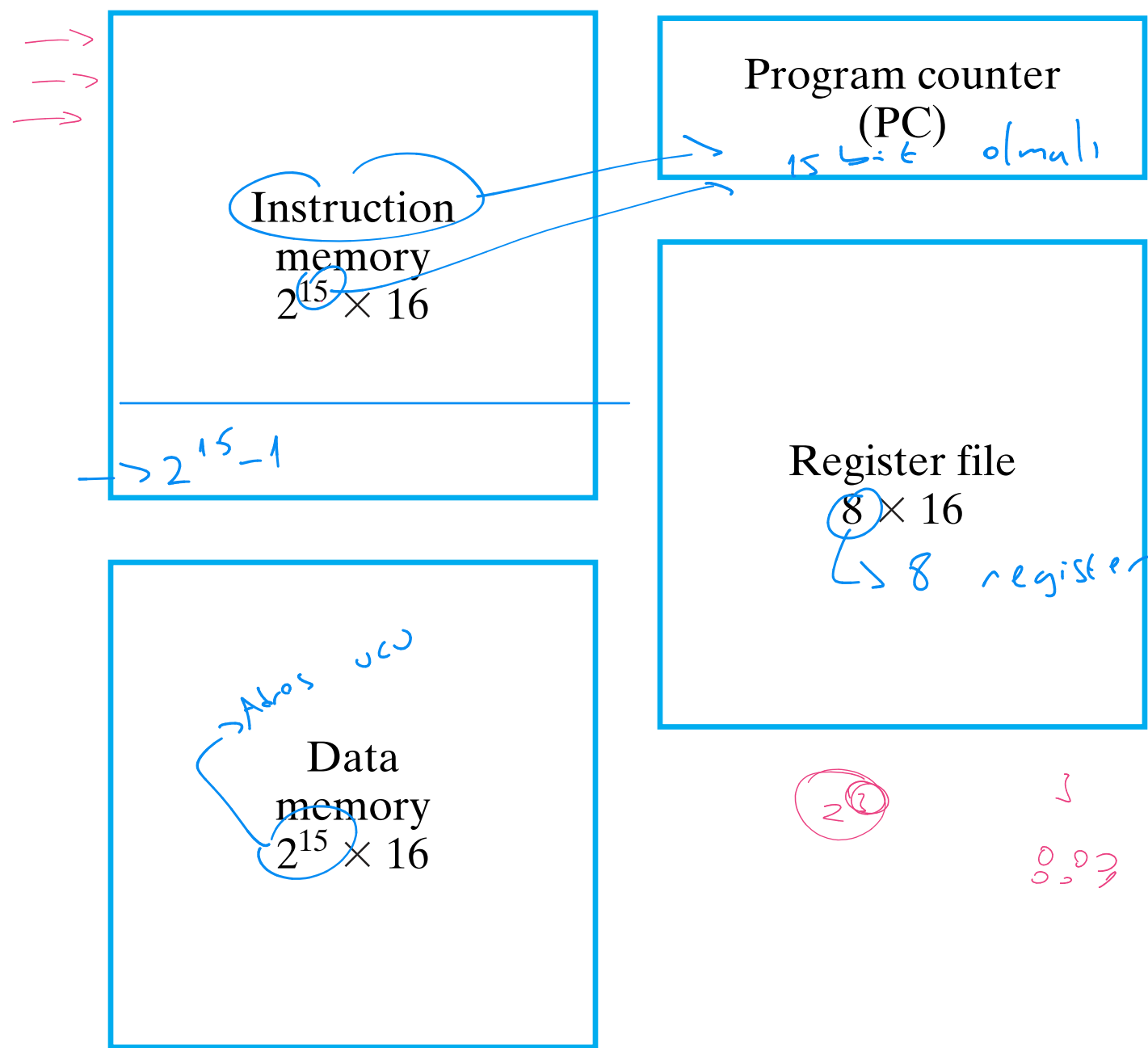
Micro-operation	DA	AA	BA	MB	FS	MD	RW
$R1 \leftarrow R2 - R3$	$R1$	$R2$	$R3$	Register	$F = A + \overline{B} + 1$	Function	Write
$R4 \leftarrow \text{sl } R6$	$R4$	—	$R6$	Register	$F = \text{sl } B$	Function	Write
$R7 \leftarrow R7 + 1$	$R7$	$R7$	—	—	$F = A + 1$	Function	Write
$R1 \leftarrow R0 + 2$	$R1$	$R0$	—	Constant	$F = A + B$	Function	Write
$\text{Data out} \leftarrow R3$	—	—	$R3$	Register	—	—	No Write
$R4 \leftarrow \text{Data in}$	$R4$	—	—	—	—	Data in	Write
$R5 \leftarrow 0$	$R5$	$R0$	$R0$	Register	$F = A \oplus B$	Function	Write

TABLE 9-7

Examples of Microoperations from Table 9-6, Using Binary Control Words

Micro-operation	DA	AA	BA	MB	FS	MD	RW
$R1 \leftarrow R2 - R3$	001	010	011	0	0101	0	1
$R4 \leftarrow s1 R6$	100	XXX	110	0	1110	0	1
$R7 \leftarrow R7 + 1$	111	111	XXX	X	0001	0	1
$R1 \leftarrow R0 + 2$	001	000	XXX	1	0010	0	1
$\text{Data out} \leftarrow R3$	XXX	XXX	011	0	XXXX	X	0
$R4 \leftarrow \text{Data in}$	100	XXX	XXX	X	XXXX	1	1
$R5 \leftarrow 0$	101	000	000	0	1010	0	1





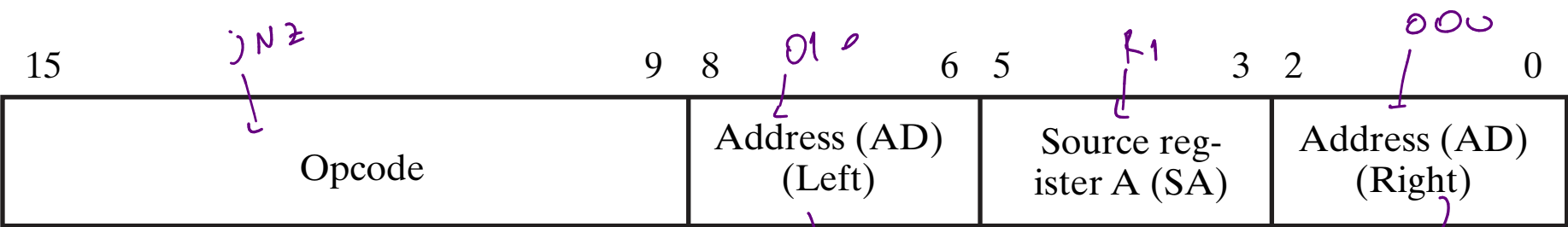
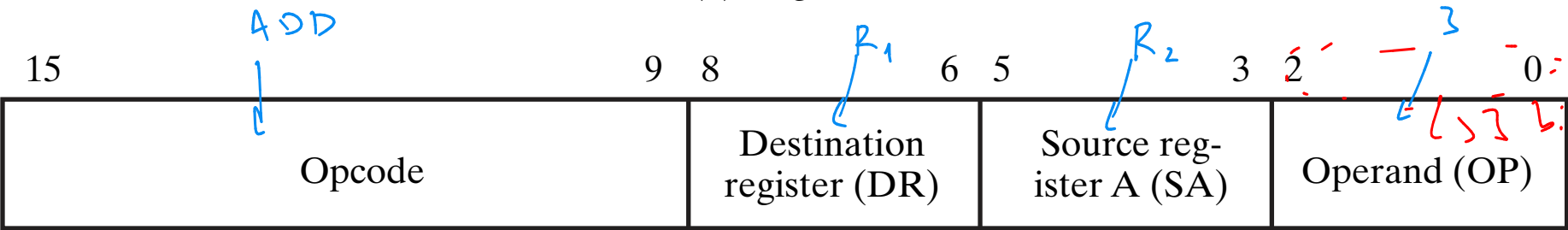
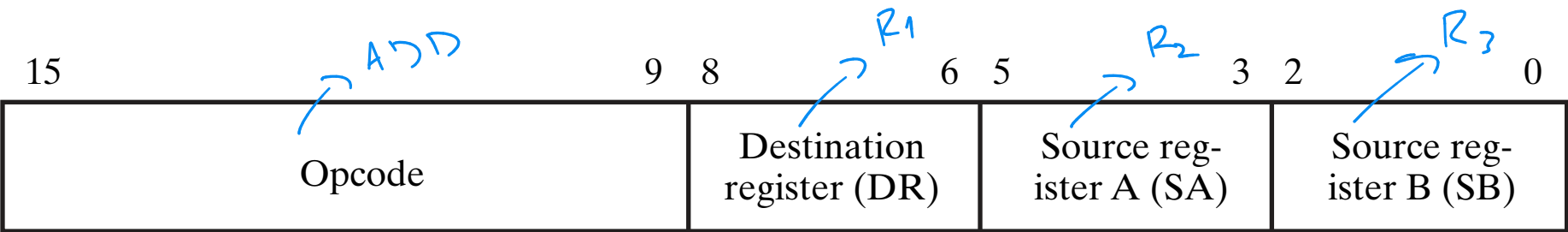


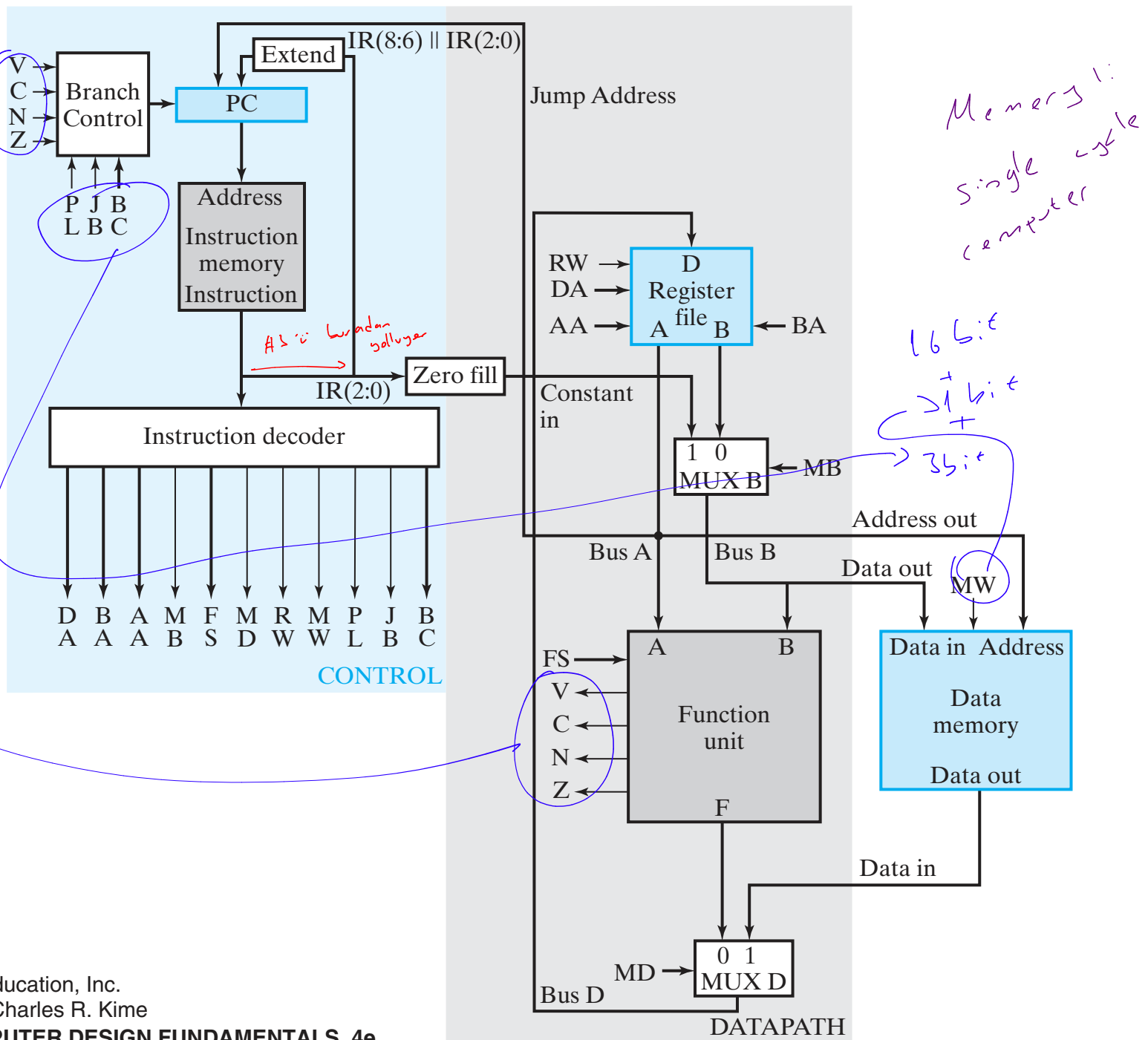
TABLE 9-8
Instruction Specifications for the Simple Computer

Instruction	Opcode	Mne- monic	Format	Description	Status Bits
Move A	<u>0000000</u>	MOVA	RD, RA	$R[DR] \leftarrow R[SA]^*$	N, Z
Increment	0000001	INC	RD, RA	$R[DR] \leftarrow R[SA] + 1^*$	N, Z
Add	0000010	ADD	RD, RA, RB	$R[DR] \leftarrow R[SA] + R[SB]^*$	N, Z
Subtract	<u>0000101</u>	SUB	RD, RA, RB	$R[DR] \leftarrow R[SA] - R[SB]^*$	N, Z
Decrement	0000110	DEC	RD, RA	$R[DR] \leftarrow R[SA] - 1^*$	N, Z
AND	0001000	AND	RD, RA, RB	$R[DR] \leftarrow R[SA] \wedge R[SB]^*$	N, Z
OR	0001001	OR	RD, RA, RB	$R[DR] \leftarrow R[SA] \vee R[SB]^*$	N, Z
Exclusive OR	0001010	XOR	RD, RA, RB	$R[DR] \leftarrow R[SA] \oplus R[SB]^*$	N, Z
NOT	0001011	NOT	RD, RA	$R[DR] \leftarrow \overline{R[SA]}^*$	N, Z
Move B	0001100	MOVB	RD, RB	$R[DR] \leftarrow R[SB]^*$	
Shift Right	0001101	SHR	RD, RB	$R[DR] \leftarrow sr R[SB]^*$	
Shift Left	<u>0001110</u>	SHL	RD, RB	$R[DR] \leftarrow sl R[SB]^*$	
Load Immediate	<u>1001100</u>	LDI	RD, OP	$R[DR] \leftarrow zf OP^*$	
Add Immediate	<u>1000010</u>	ADI	RD, RA, OP	$R[DR] \leftarrow R[SA] + zf OP^*$	N, Z
Load	<u>0010000</u>	LD	RD, RA	$R[DR] \leftarrow M[SA]^*$	
Store	<u>0100000</u>	ST	RA, RB	$M[SA] \leftarrow R[SB]^*$	
Branch on Zero	<u>1100000</u>	BRZ	RA, AD	if $(R[SA] = 0)$ $PC \leftarrow PC + se AD$, N, Z if $(R[SA] \neq 0)$ $PC \leftarrow PC + 1$	
Branch on Negative	<u>1100001</u>	BRN	RA, AD	if $(R[SA] < 0)$ $PC \leftarrow PC + se AD$, N, Z if $(R[SA] \geq 0)$ $PC \leftarrow PC + 1$	
Jump	<u>1110000</u>	JMP	RA	$PC \leftarrow R[SA]$	

* For all of these instructions, $PC \leftarrow PC + 1$ is also executed to prepare for the next cycle.

TABLE 9-9
Memory Representation of Instructions and Data

Decimal Address	Memory Contents	Decimal Opcode	Other Fields	Operation
25	0000101 001 010 011	5 (Subtract)	DR:1, SA:2, SB:3	$R1 \leftarrow R2 - R3$
35	0100000 000 100 101	32 (Store)	SA:4, SB:5	$M[R4] \leftarrow R5$
45	1000010 010 111 011 <i>$R_2 \leftarrow R_7 + 3$</i>	66 (Add Immediate)	DR:2, SA:7, OP:3	$R2 \leftarrow R7 + 3$
55	1100000 101 110 100 <i>$\rightarrow R_6$ $101100 \rightarrow -20$</i>	96 (Branch on Zero)	AD: 44, SA:6	If $R6 = 0$, $PC \leftarrow PC - 20$
70	00000000011000000 <i>yanlıs</i>	Data = 192. After execution of instruction in 35, Data = 80.		



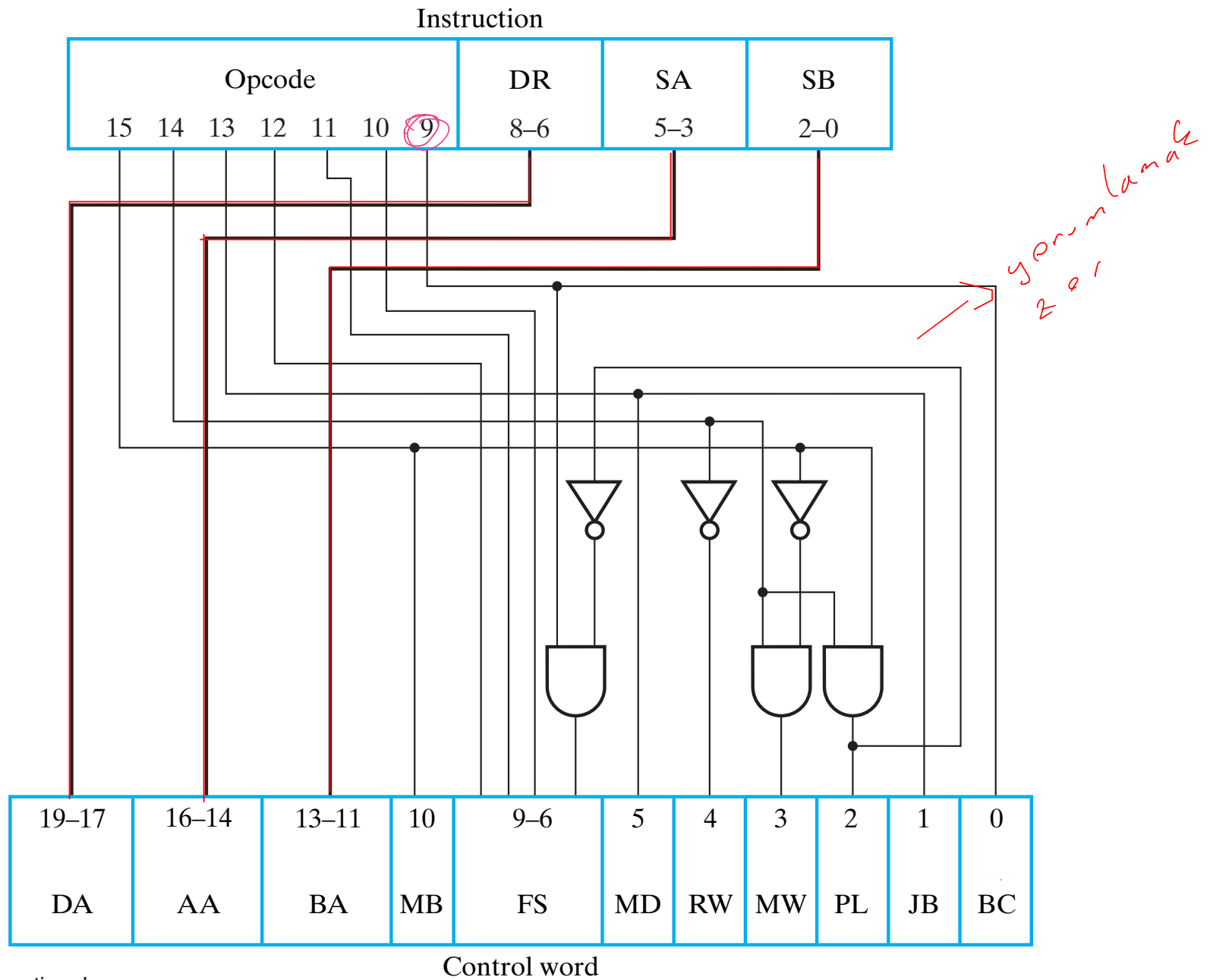


TABLE 9-10
Truth Table for Instruction Decoder Logic

4 B way
yeru-lamak
belas

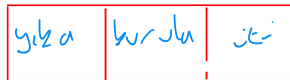
Instruction Function Type	Instruction Bits				Control Word Bits						
	15	14	13	9	MB	MD	RW	MW	PL	JB	BC
Function-unit operations using registers	0	0	0	X	0	0	1	0	0	X	X
Memory read	0	0	1	X	0	1	1	0	0	X	X
Memory write	0	1	0	X	0	X	0	1	0	X	X
Function-unit operations using register and constant	1	0	0	X	1	0	1	0	0	X	X
Conditional branch on zero (Z)	1	1	0	0	X	X	0	0	1	0	0
Conditional branch on negative (N)	1	1	0	1	X	X	0	0	1	0	1
Unconditional jump	1	1	1	X	X	X	0	0	1	1	X

TABLE 9-11
Six Instructions for the Single-Cycle Computer

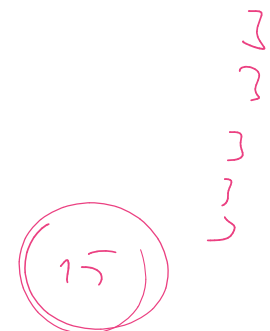
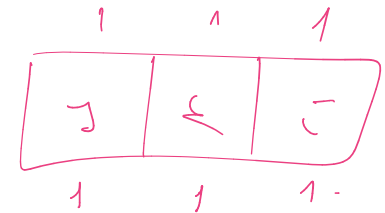
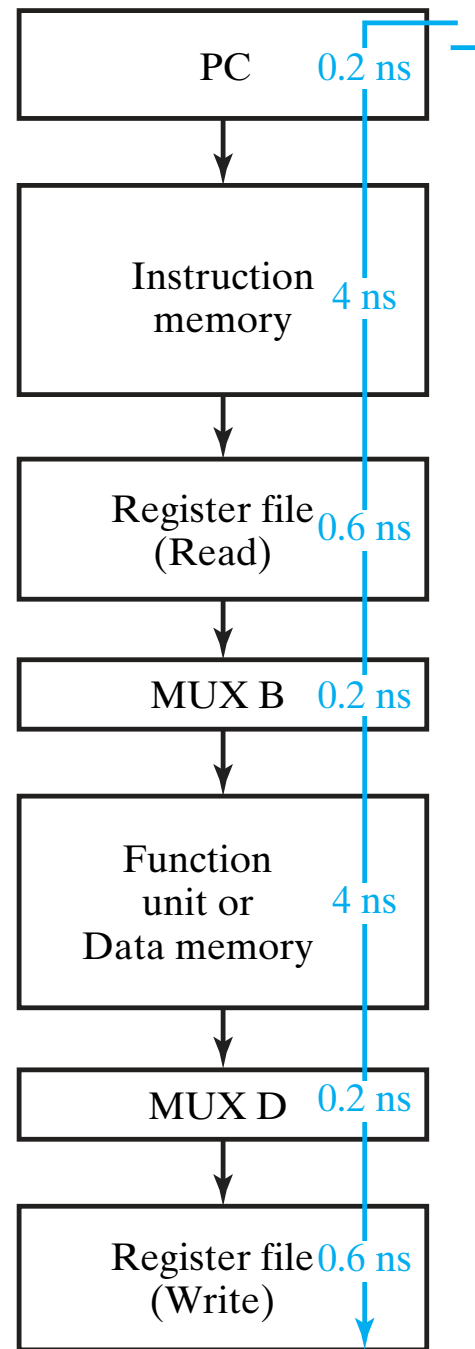
Operation Code	Symbolic Name	Format	Description	Function	MB	MD	RW	MW	PL	JB	BC
1000010	ADI	Immediate	Add immediate operand	$R[DR] \leftarrow R[SA] + zf\ I(2:0)$	1	0	1	0	0	0	0
0010000	LD	Register	Load memory content into register	$R[DR] \leftarrow M[R[SA]]$	0	1	1	0	0	1	0
0100000	ST	Register	Store register content in memory	$M[R[SA]] \leftarrow R[SB]$	0	1	0	1	0	0	0
0001110	SL	Register	Shift left	$R[DR] \leftarrow sl\ R[SB]$	0	0	1	0	0	1	0
0001011	NOT	Register	Complement register	$R[DR] \leftarrow \overline{R[SA]}$	0	0	1	0	0	0	1
1100000	BRZ	Jump/Branch	If $R[SA] = 0$, branch to $PC + se\ AD$	If $R[SA] = 0$, $PC \leftarrow PC + se\ AD$ If $R[SA] \neq 0$, $PC \leftarrow PC + 1$	1	0	0	0	1	0	0

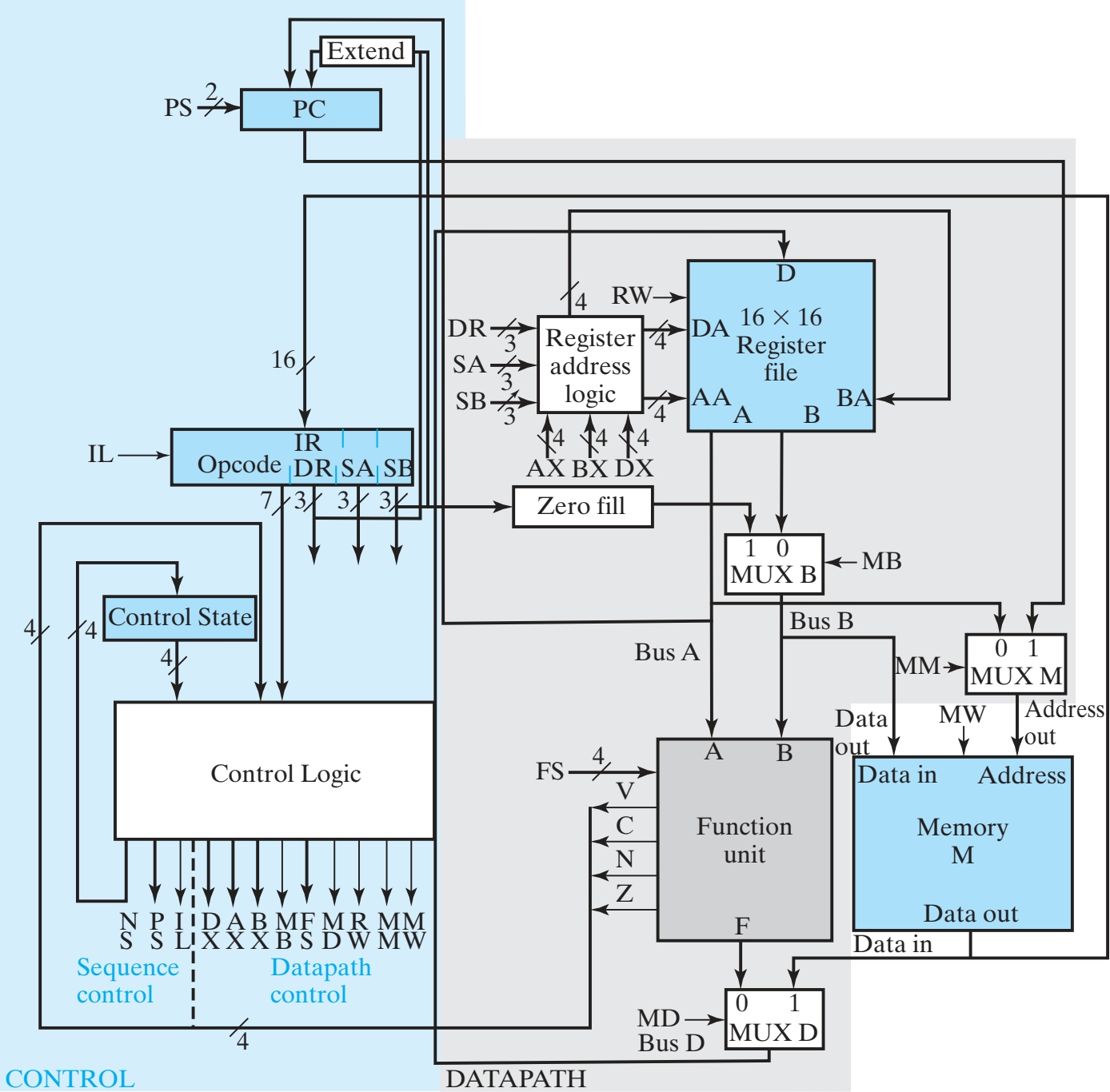
SSC $\rightarrow$ Single
MSC $\rightarrow$ Multi

↓
Pipeline
hızlı olması için
işlem bölüşme olur



(yika bitince
diğerini yikamaya
başla
→ Disadvantaji
R₂ kullanmaya
R₂ için gencelleme
çalıştırma süresi





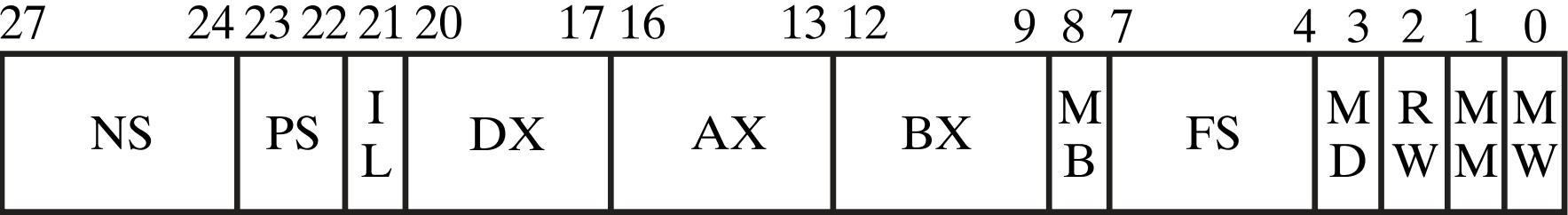


TABLE 9-12
Control-Word Information for Datapath

DX	AX	BX	Code	MB	Code	FS	Code	MD	RW	MM	MW	Code
$R[DR]$	$R[SA]$	$R[SB]$	0XXX	Register	0	$F = A$	0000	FnUt	No Write	Address out	No Write	0
$R8$	$R8$	$R8$	1000	Constant	1	$F = A + 1$	0001	Data in	Write	PC	Write	1
$R9$	$R9$	$R9$	1001			$F = A + B$	0010					
$R10$	$R10$	$R10$	1010			Unused	0011					
$R11$	$R11$	$R11$	1011			Unused	0100					
$R12$	$R12$	$R12$	1100			$F = A + \overline{B} + 1$	0101					
$R13$	$R13$	$R13$	1101			$F = A - 1$	0110					
$R14$	$R14$	$R14$	1110			Unused	0111					
$R15$	$R15$	$R15$	1111			$F = A \wedge B$	1000					
						$F = A \vee B$	1001					
						$F = A \oplus B$	1010					
						$F = \overline{A}$	1011					
						$F = B$	1100					
						$F = \text{sr } B$	1101					
						$F = \text{sl } B$	1110					
						Unused	1111					

TABLE 9-13
Control Information for Sequence Control

NS		PS		IL	
Next State		Action	Code	Action	Code
Gives next state of control state register		Hold PC	00	No load	0
		Inc PC	01	Load IR	1
		Branch	10		
		Jump	11		

$PC \leftarrow PC + 1$
 Σ transition conditions on merged arcs

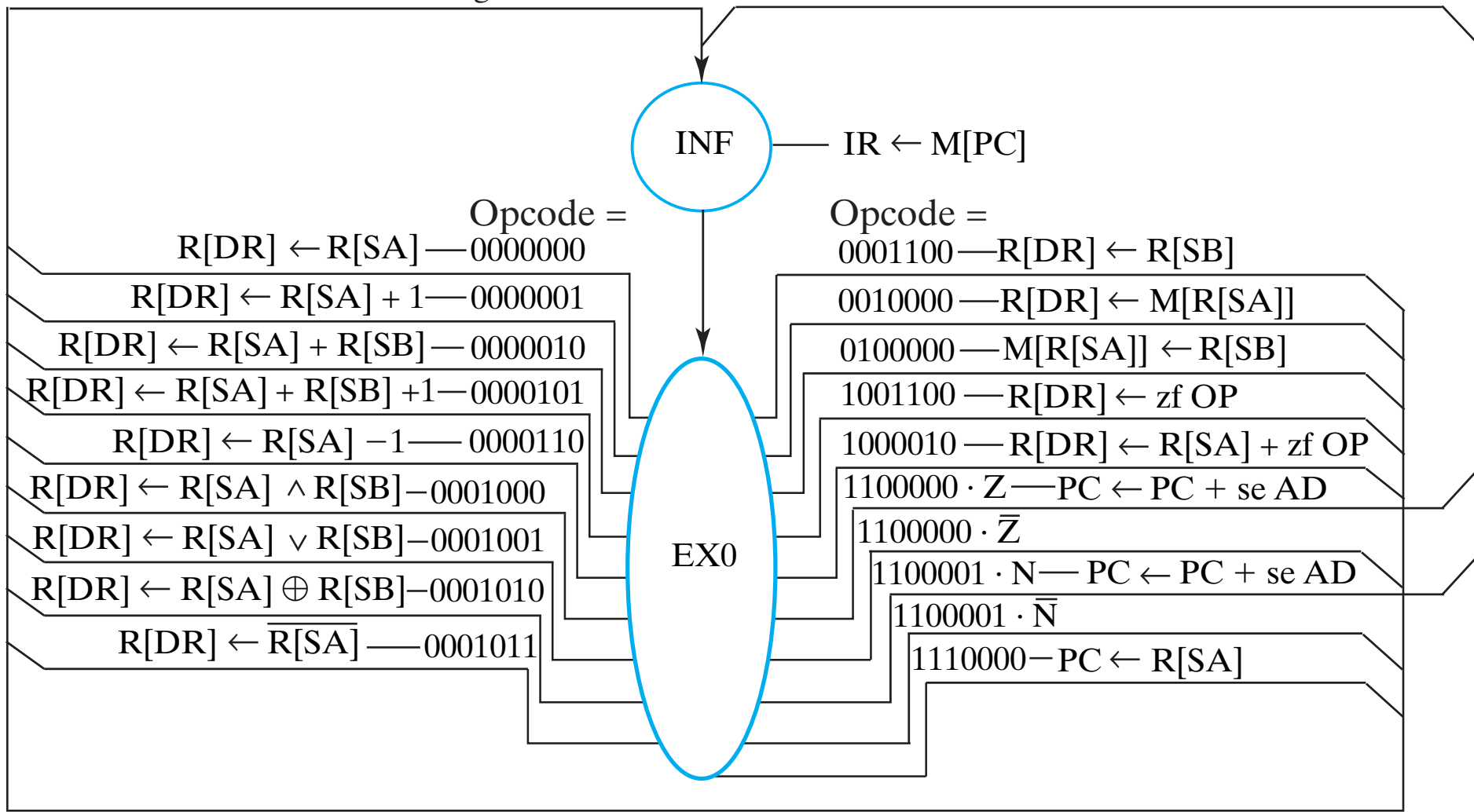
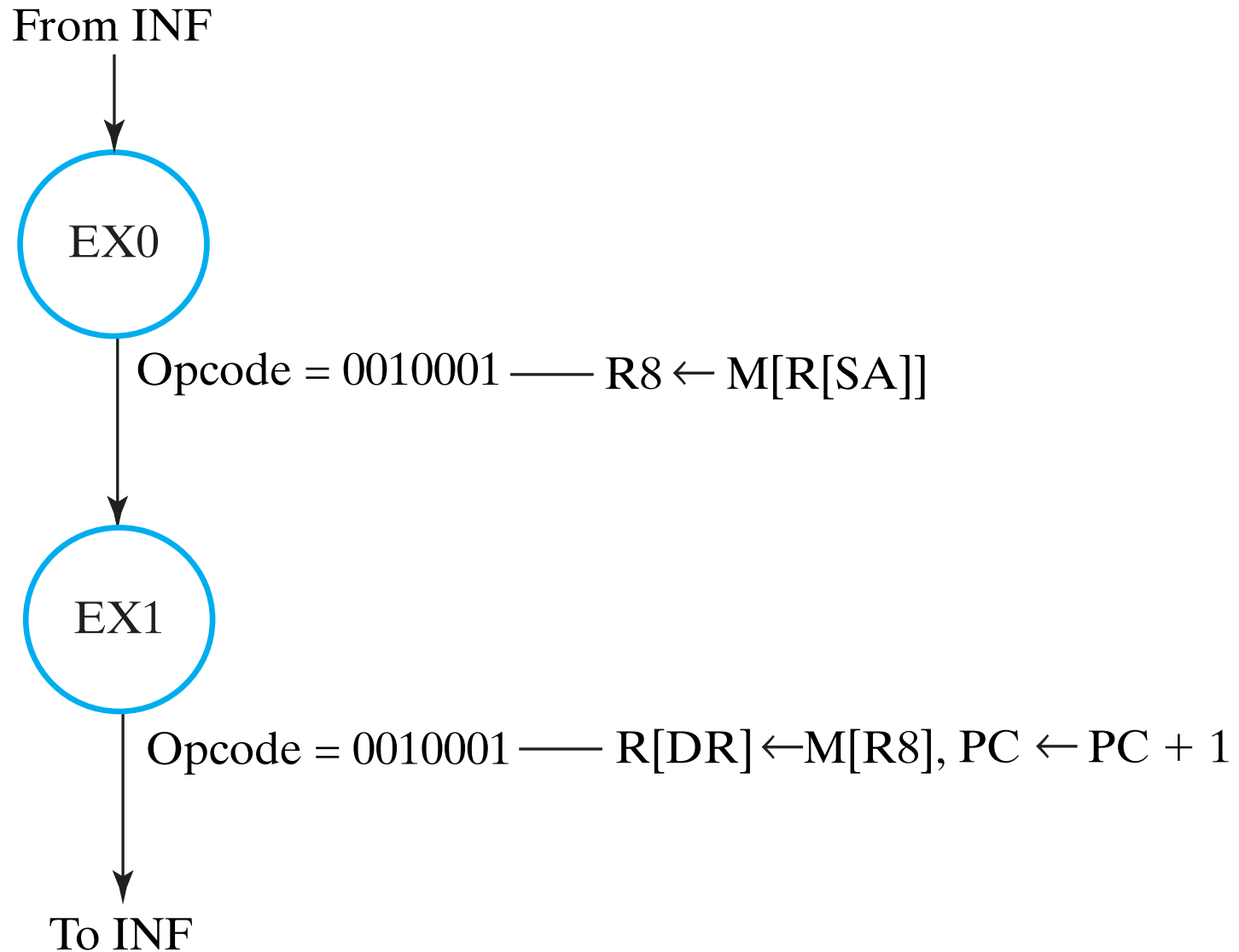


TABLE 9-14
 State Table for Two-Cycle Instructions

State	Inputs		Next State	Outputs											Comments
	Opcode	VCNZ		I L	P S	DX	AX	BX	M B	FS	M D	R W	M M	M W	
INF	XXXXXXX	XXXX	EX0	1	00	XXXX	XXXX	XXXX	X	XXXX	X	0	1	0	$IR \leftarrow M[PC]$
EX0	0000000	XXXX	INF	0	01	0XXX	0XXX	XXXX	X	0000	0	1	X	0	MOVA $R[DR] \leftarrow R[SA]^*$
EX0	0000001	XXXX	INF	0	01	0XXX	0XXX	XXXX	X	0001	0	1	X	0	INC $R[DR] \leftarrow R[SA] + 1^*$
EX0	0000010	XXXX	INF	0	01	0XXX	0XXX	0XXX	0	0010	0	1	X	0	ADD $R[DR] \leftarrow R[SA] + R[SB]^*$
EX0	0000101	XXXX	INF	0	01	0XXX	0XXX	0XXX	0	0101	0	1	X	0	SUB $R[DR] \leftarrow R[SA] + \overline{R[SB]} + 1^*$
EX0	0000110	XXXX	INF	0	01	0XXX	0XXX	XXXX	X	0110	0	1	X	0	DEC $R[DR] \leftarrow R[SA] + (-1)^*$
EX0	0001000	XXXX	INF	0	01	0XXX	0XXX	0XXX	0	1000	0	1	X	0	AND $R[DR] \leftarrow R[SA] \wedge R[SB]^*$
EX0	0001001	XXXX	INF	0	01	0XXX	0XXX	0XXX	0	1001	0	1	X	0	OR $R[DR] \leftarrow R[SA] \vee R[SB]^*$
EX0	0001010	XXXX	INF	0	01	0XXX	0XXX	0XXX	0	1010	0	1	X	0	XOR $R[DR] \leftarrow R[SA] \oplus R[SB]^*$
EX0	0001011	XXXX	INF	0	01	0XXX	0XXX	XXXX	X	1011	0	1	X	0	NOT $R[DR] \leftarrow \overline{R[SA]}^*$
EX0	0001100	XXXX	INF	0	01	0XXX	XXXX	0XXX	0	1100	0	1	X	0	MOVB $R[DR] \leftarrow R[SB]^*$
EX0	0010000	XXXX	INF	0	01	0XXX	0XXX	XXXX	X	XXXX	1	1	0	0	LD $R[DR] \leftarrow M[R[SA]]^*$
EX0	0100000	XXXX	INF	0	01	XXXX	0XXX	0XXX	0	XXXX	X	0	0	1	ST $M[R[SA]] \leftarrow R[SB]^*$
EX0	1001100	XXXX	INF	0	01	0XXX	XXXX	XXXX	1	1100	0	1	0	0	LDI $R[DR] \leftarrow zf\ OP^*$
EX0	1000010	XXXX	INF	0	01	0XXX	0XXX	XXXX	1	0010	0	1	0	0	ADI $R[DR] \leftarrow R[SA] + zf\ OP^*$
EX0	1100000	XXX1	INF	0	10	XXXX	0XXX	XXXX	X	0000	X	0	0	0	BRZ $PC \leftarrow PC + se\ AD$
EX0	1100000	XXX0	INF	0	01	XXXX	0XXX	XXXX	X	0000	X	0	0	0	BRZ $PC \leftarrow PC + 1$
EX0	1100001	XX1X	INF	0	10	XXXX	0XXX	XXXX	X	0000	X	0	0	0	BRN $PC \leftarrow PC + se\ AD$
EX0	1100001	XX0X	INF	0	01	XXXX	0XXX	XXXX	X	0000	X	0	0	0	BRN $PC \leftarrow PC + 1$
EX0	1110000	XXXX	INF	0	11	XXXX	0XXX	XXXX	X	0000	X	0	0	0	JMP $PC \leftarrow R[SA]$

* For this state and input combination, $PC \leftarrow PC + 1$ also occurs.



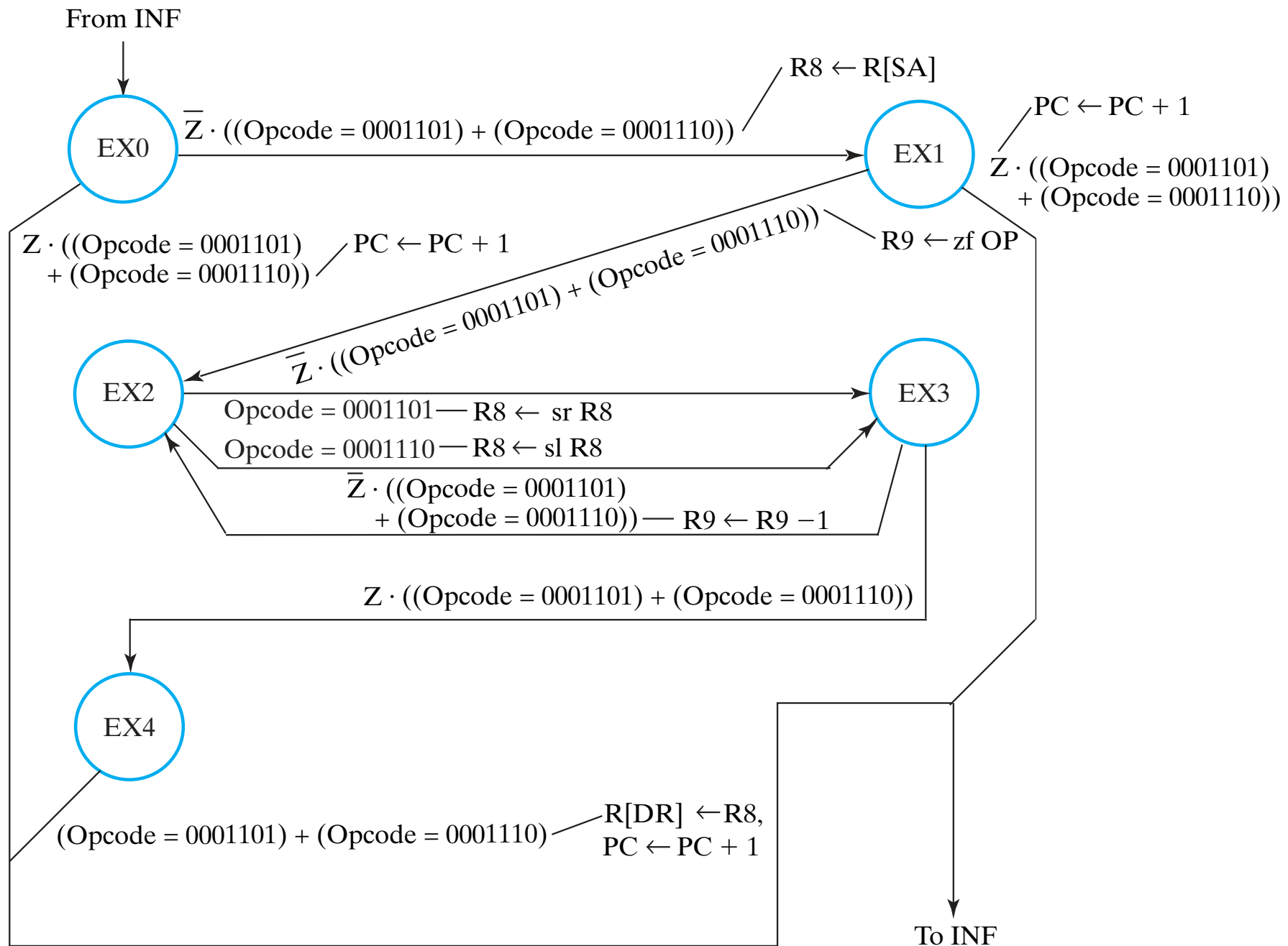


TABLE 9-15
State Table for Illustration of Instructions Having Three or More Cycles

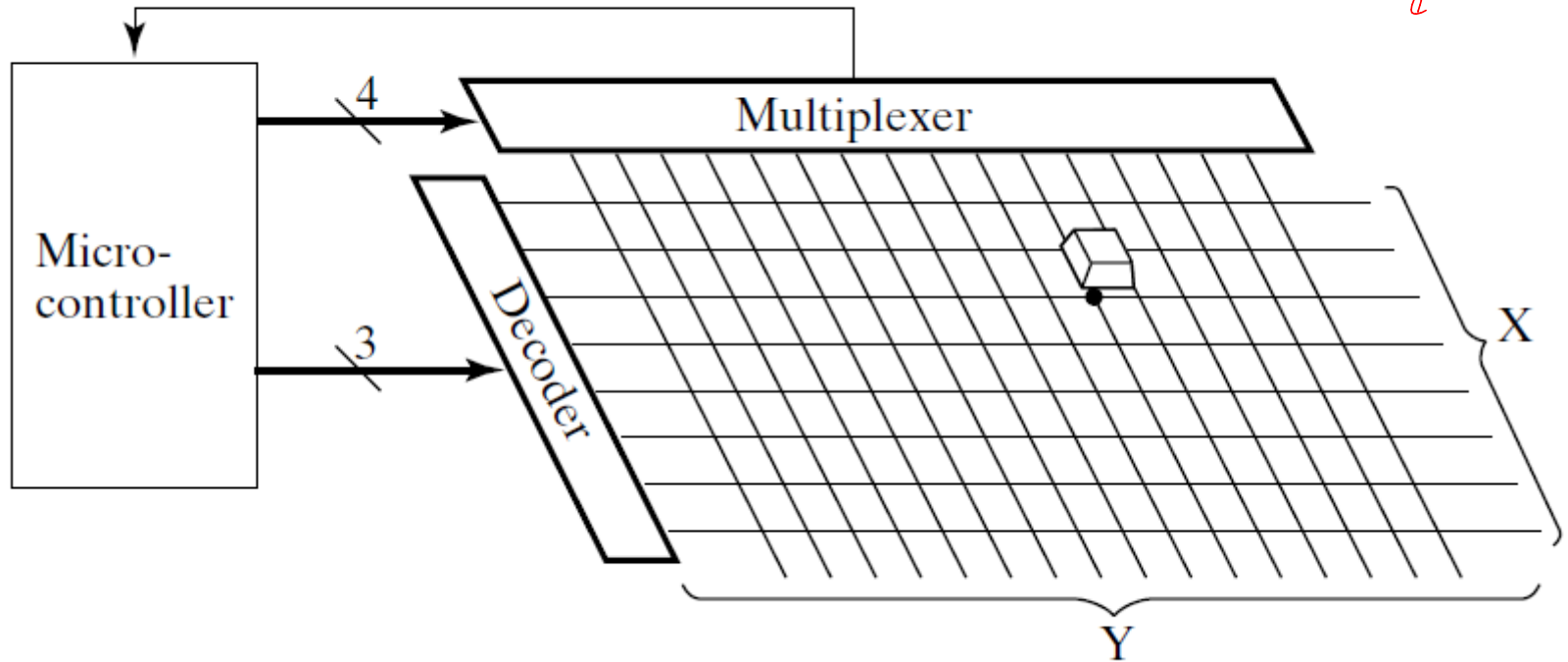
State	Inputs		Next state	Outputs											Comments	
	Opcode	VCNZ		I									M			
					L	PS	DX	AX	BX	MB	FS	MD		RW		MM
EX0	0010001	XXXX	EX1	0	00	1000	0XXX	XXXX	X	0000	1	1	X	0	LRI	$R8 \leftarrow M[R[SA]], \rightarrow EX1$
EX1	0010001	XXXX	INF	0	01	0XXX	1000	XXXX	X	0000	1	1	X	0	LRI	$R[DR] \leftarrow M[R8], \rightarrow INF^*$
EX0	0001101	XXX0	EX1	0	00	1000	0XXX	XXXX	X	0000	0	1	X	0	SRM	$R8 \leftarrow R[SA], \overline{Z}: \rightarrow EX1$
EX0	0001101	XXX1	INF	0	01	1000	0XXX	XXXX	X	0000	0	1	X	0	SRM	$R8 \leftarrow R[SA], Z: \rightarrow INF^*$
EX1	0001101	XXX0	EX2	0	00	1001	XXXX	XXXX	1	1100	0	1	X	0	SRM	$R9 \leftarrow zf\ OP, \overline{Z}: \rightarrow EX2$
EX1	0001101	XXX1	INF	0	01	1001	XXXX	XXXX	1	1100	0	1	X	0	SRM	$R9 \leftarrow zf\ OP, Z: \rightarrow INF^*$
EX2	0001101	XXXX	EX3	0	00	1000	XXXX	1000	0	1101	0	1	X	0	SRM	$R8 \leftarrow sr\ R8, \rightarrow EX3$
EX3	0001101	XXX0	EX2	0	00	1001	1001	XXXX	X	0110	0	1	X	0	SRM	$R9 \leftarrow R9 - 1, \overline{Z}: \rightarrow EX2$
EX3	0001101	XXX1	EX4	0	00	1001	1001	XXXX	X	0110	0	1	X	0	SRM	$R9 \leftarrow R9 - 1, Z: \rightarrow EX4$
EX4	0001101	XXXX	INF	0	01	0XXX	1000	XXXX	X	0000	0	1	X	0	SRM	$R[DR] \leftarrow R8, \rightarrow INF^*$
EX0	0001110	XXX0	EX1	0	00	1000	0XXX	XXXX	X	0000	0	1	X	0	SLM	$R8 \leftarrow R[SA], \overline{Z}: \rightarrow EX1$
EX0	0001110	XXX1	INF	0	01	1000	0XXX	XXXX	X	0000	0	1	X	0	SLM	$R8 \leftarrow R[SA], Z: \rightarrow INF^*$
EX1	0001110	XXX0	EX2	0	00	1001	XXXX	XXXX	1	1100	0	1	X	0	SLM	$R9 \leftarrow zf\ OP, \overline{Z}: \rightarrow EX2$
EX1	0001110	XXX1	INF	0	01	1001	XXXX	XXXX	1	1100	0	1	X	0	SLM	$R9 \leftarrow zf\ OP, Z: \rightarrow INF^*$
EX2	0001110	XXXX	EX3	0	00	1000	XXXX	1000	0	1110	0	1	X	0	SLM	$R8 \leftarrow sl\ R8, \rightarrow EX3$
EX3	0001110	XXX0	EX2	0	00	1001	1001	XXXX	X	0110	0	1	X	0	SLM	$R9 \leftarrow R9 - 1, \overline{Z}: \rightarrow EX2$
EX3	0001110	XXX1	EX4	0	00	1001	1001	XXXX	X	0110	0	1	X	0	SLM	$R9 \leftarrow R9 - 1, Z: \rightarrow EX4$
EX4	0001110	XXXX	INF	0	01	0XXX	1000	XXXX	X	0000	0	1	X	0	SLM	$R[DR] \leftarrow R8, \rightarrow IF^*$

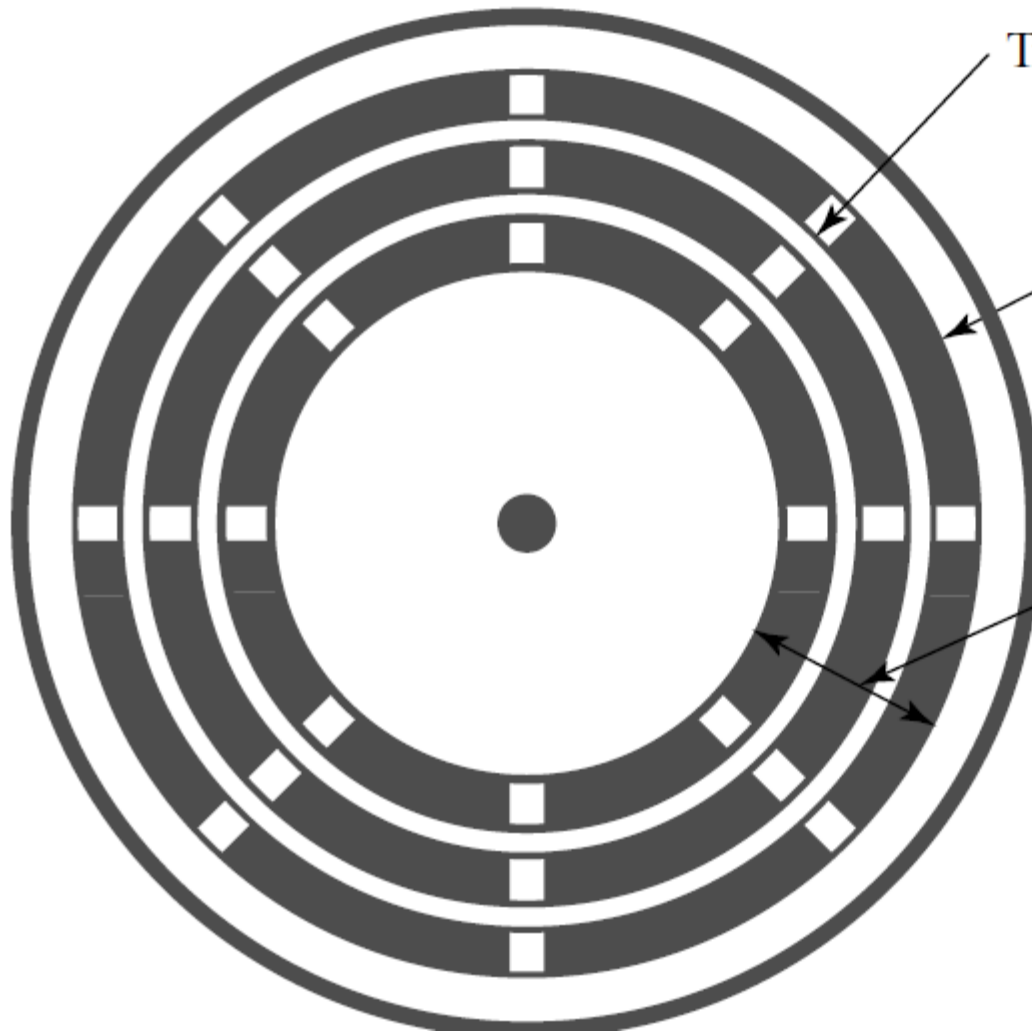
*For this state and input combination, $PC \leftarrow PC + 1$ also occurs.

Input- Output

Klasse

elektr. Kessel





Track

Sector

Head positioning

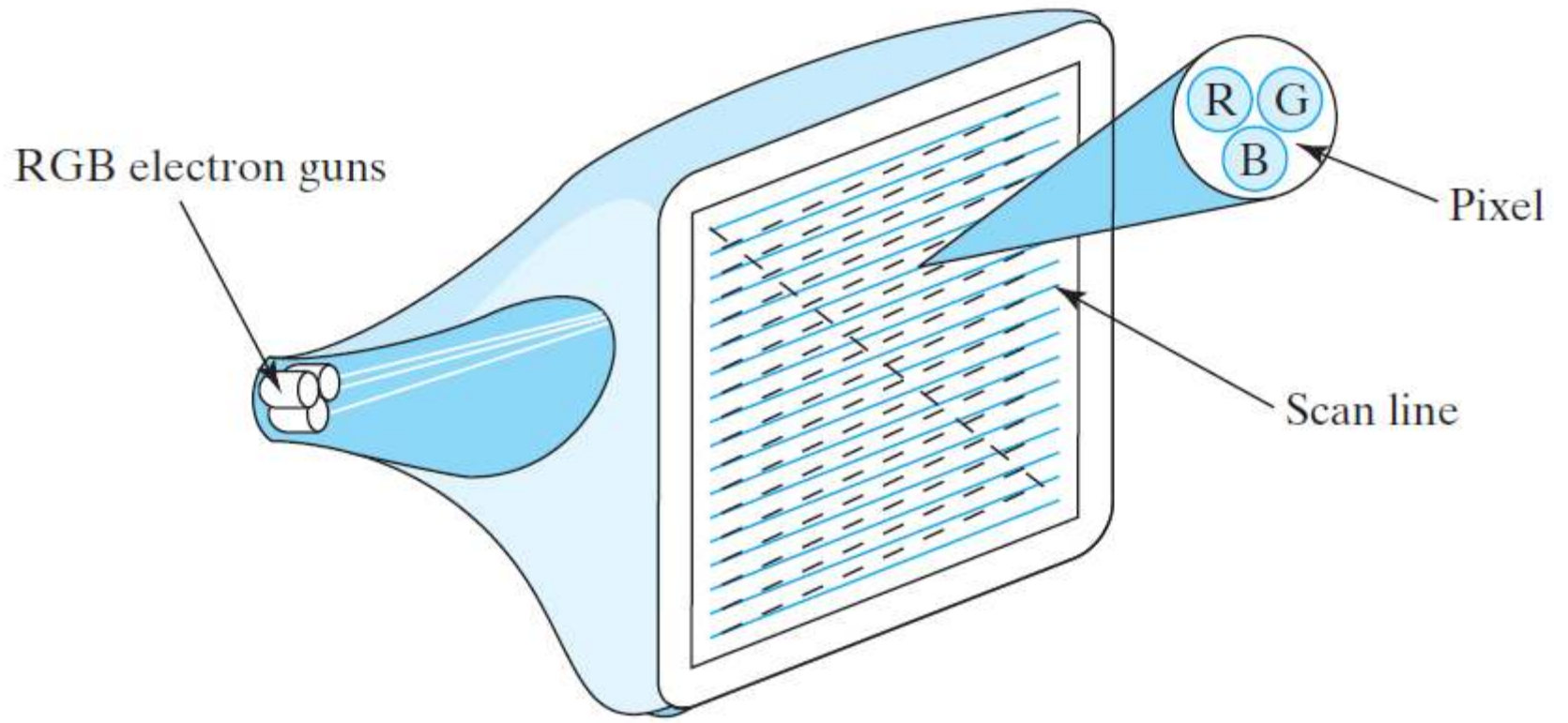
HDD

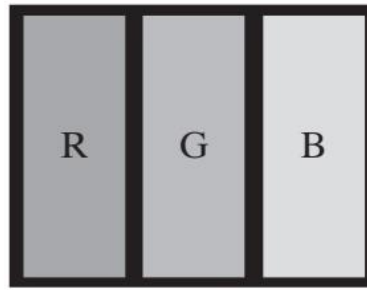
önemli
değiştir

işlet

elektronik

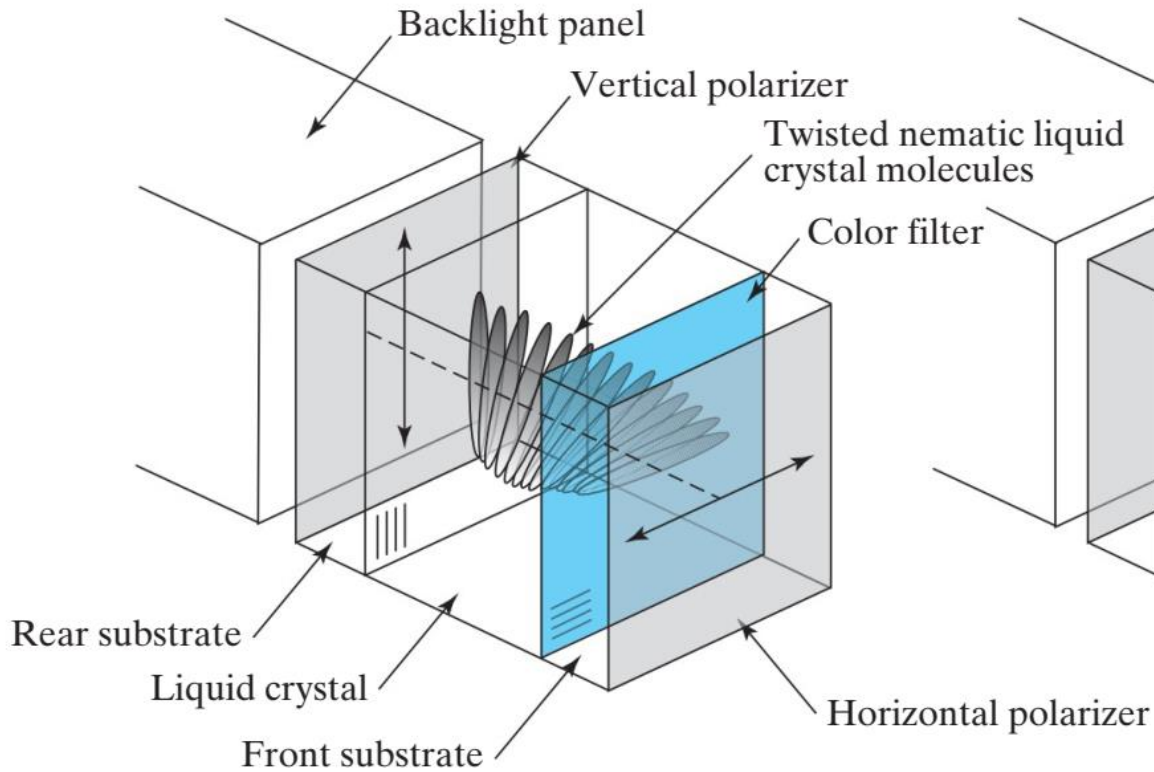
AG



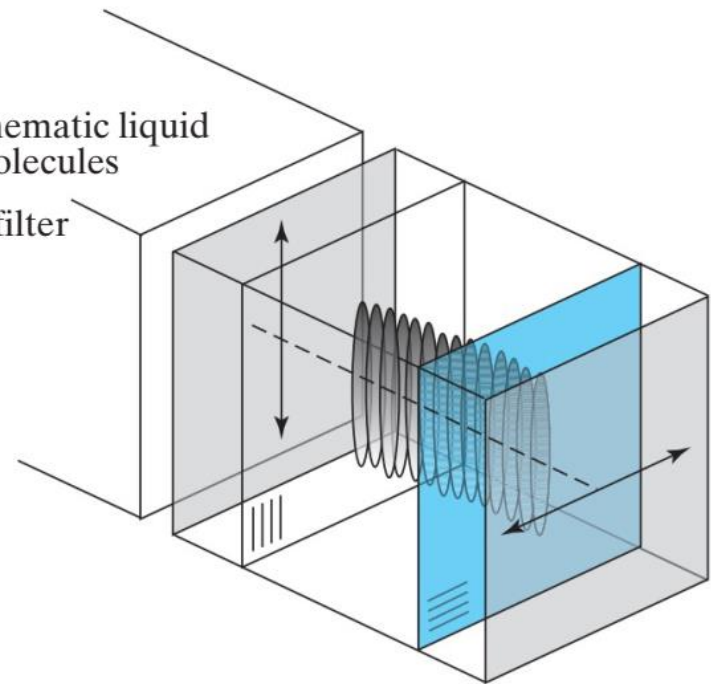


(a) LCD screen pixel

H G

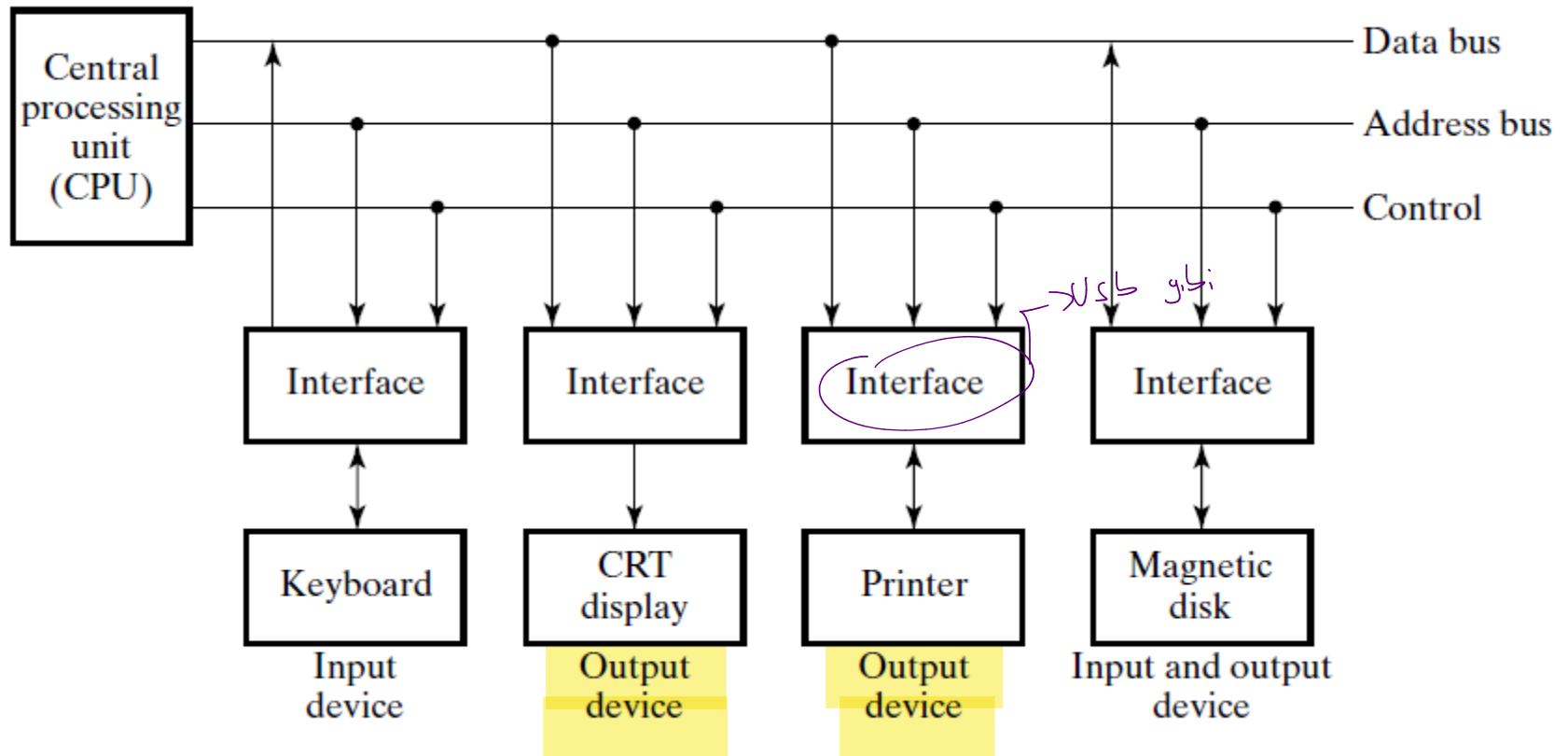


(b) Twisted nematic LCD technology

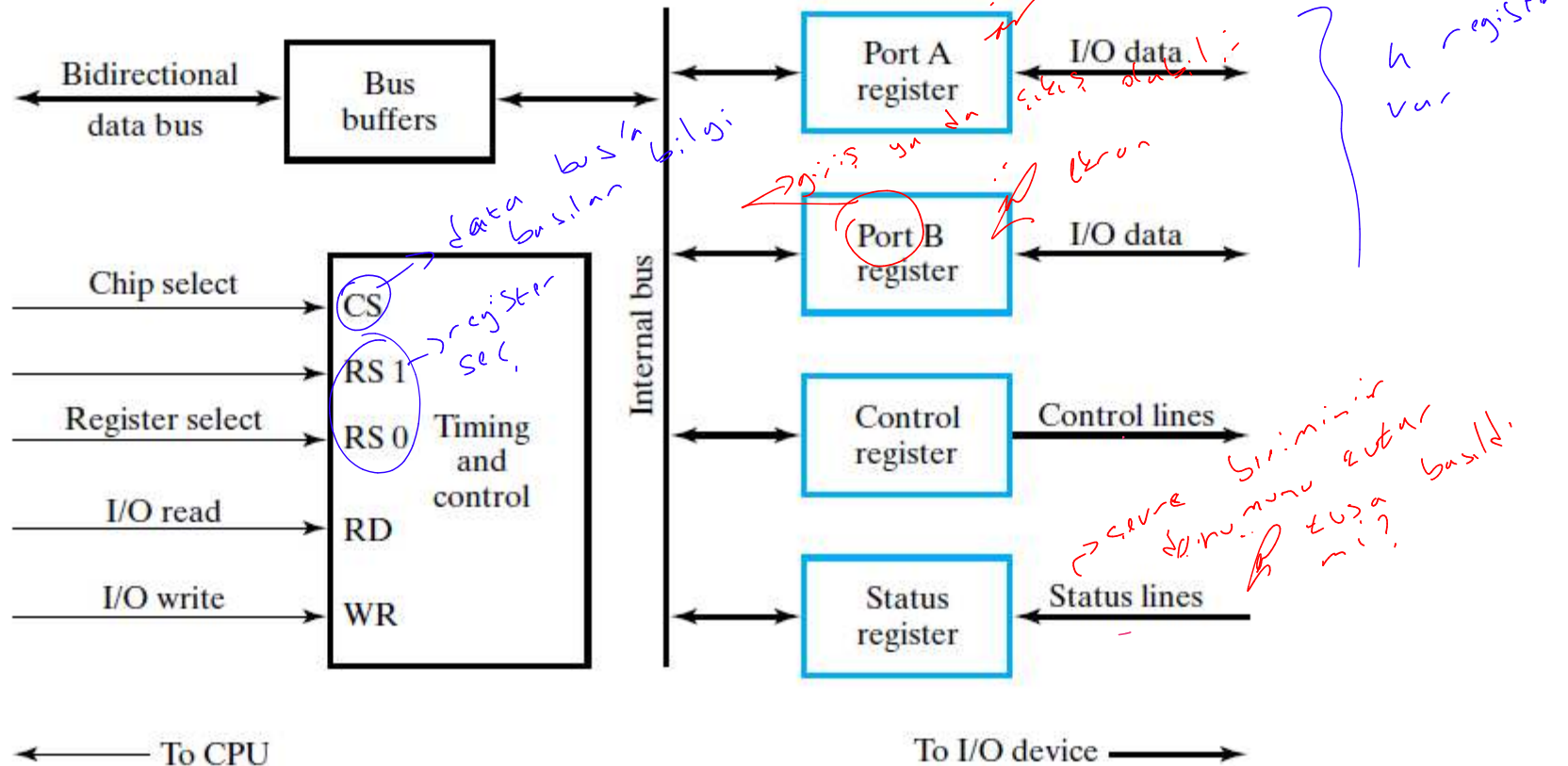


(c) Twisted nematic LCD with maximum voltage applied

Her birinde interface
olmasinin sebebi
her cihaz farklı yapıda



Burasi Interfacin iqi



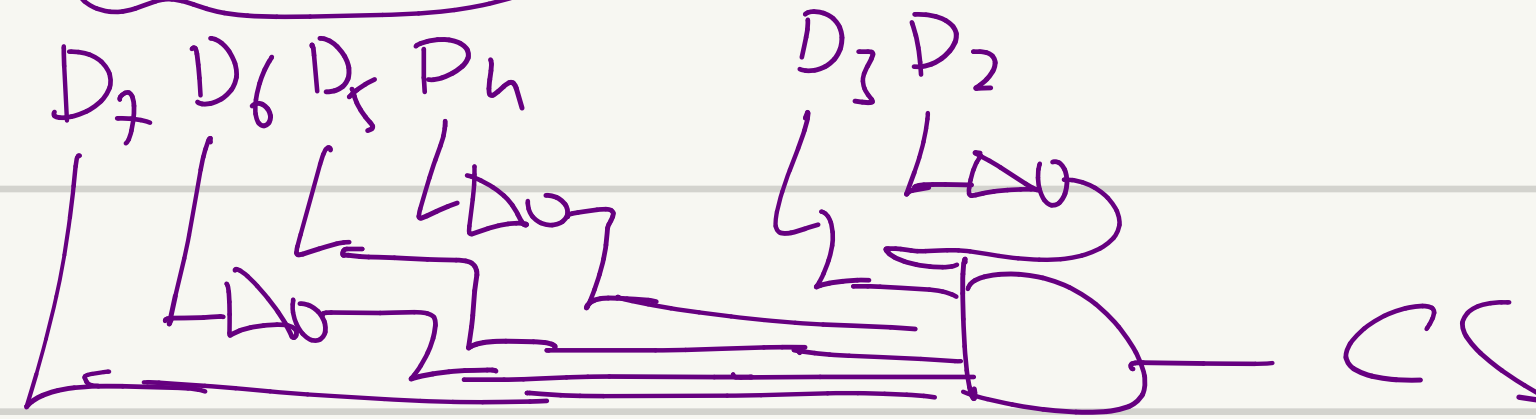
CS	RS1	RS0	Register selected
0	x	x	None: data bus in high-impedance state
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	1	1	Status register

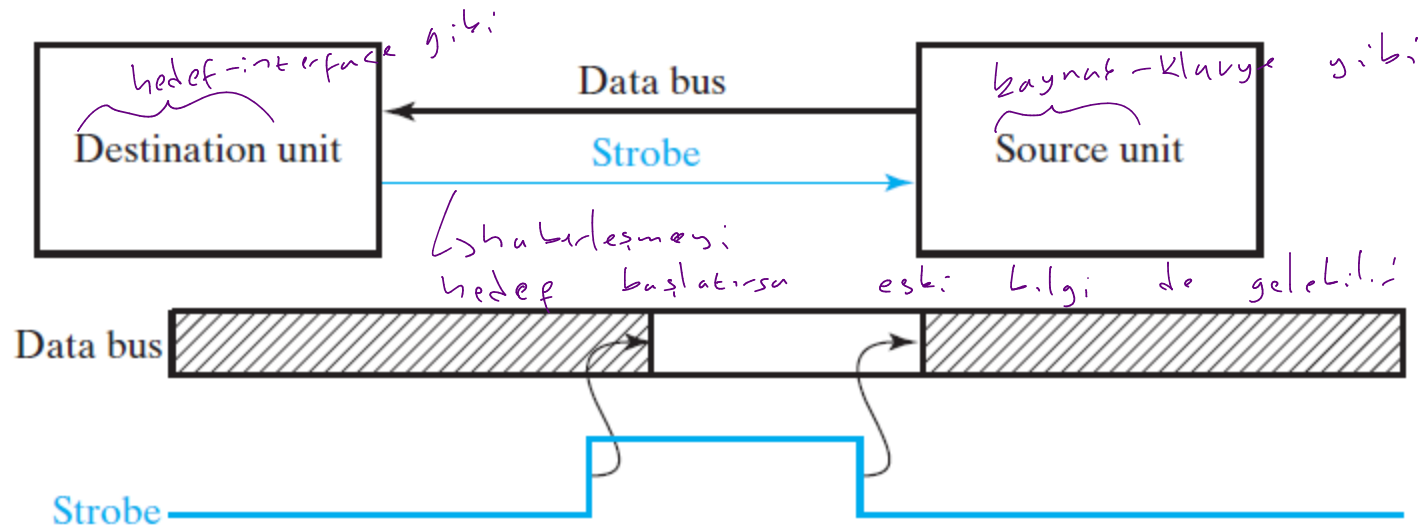
Klavye $\rightarrow$ Port A $\rightarrow$ A8H $\rightarrow$ 1010, 1000

□ $\rightarrow$ A9H

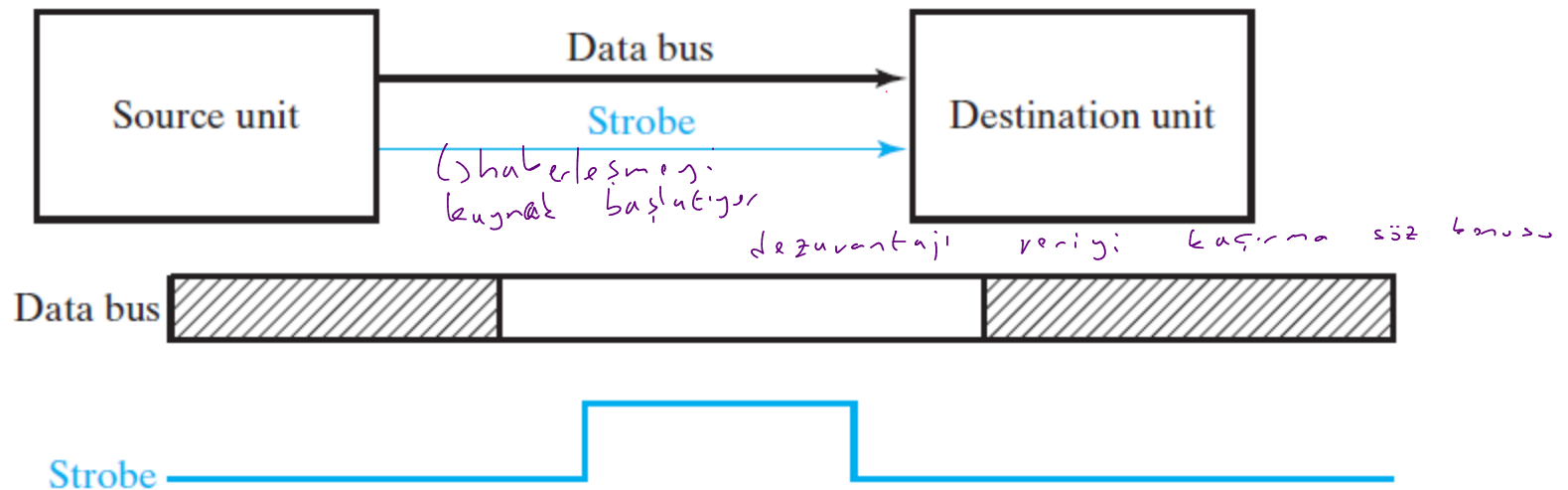
CR $\rightarrow$ AAH

SR $\rightarrow$ ADH



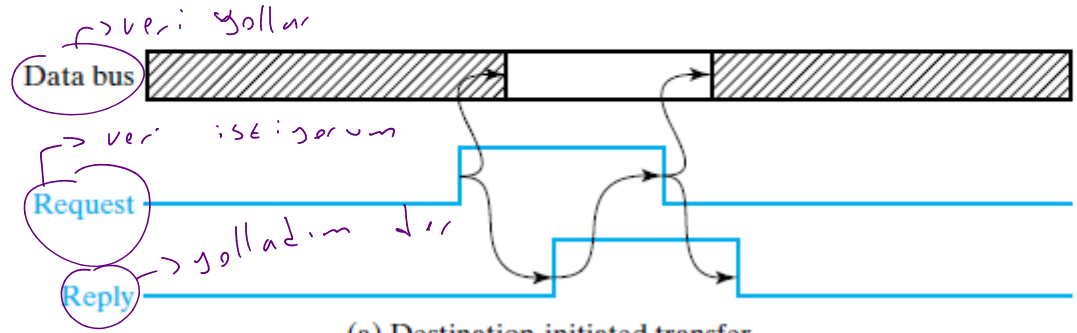
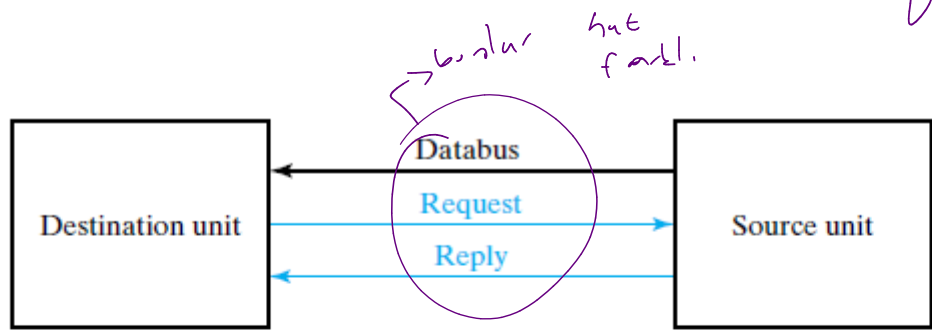


(a) Destination-initiated transfer

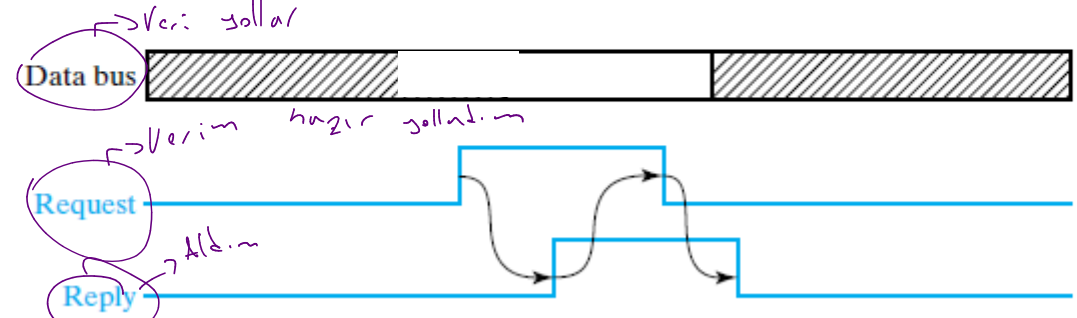
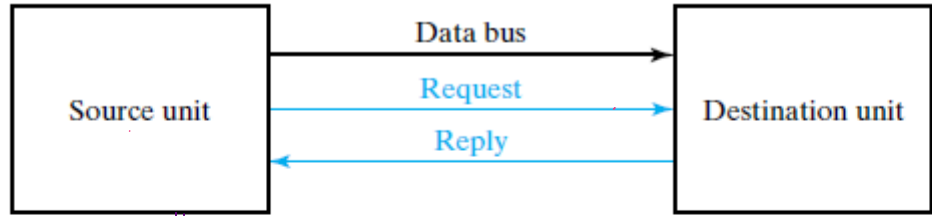


(b) Source-initiated transfer

Veri iletim: gili



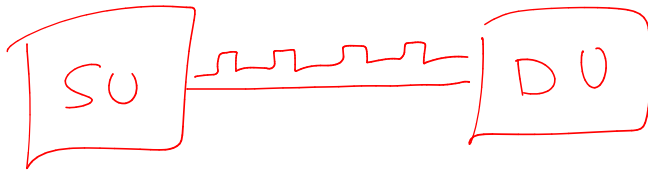
(a) Destination-initiated transfer



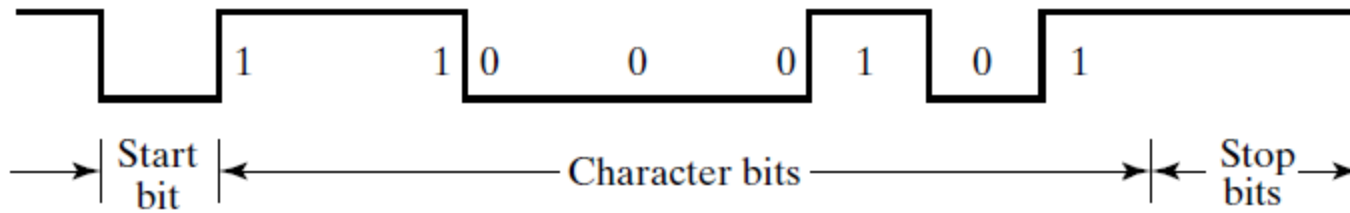
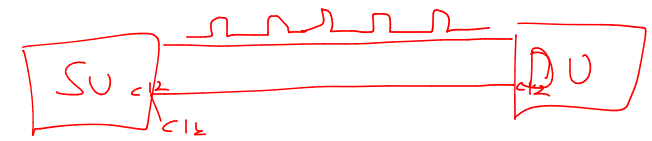
(b) Source-initiated transfer

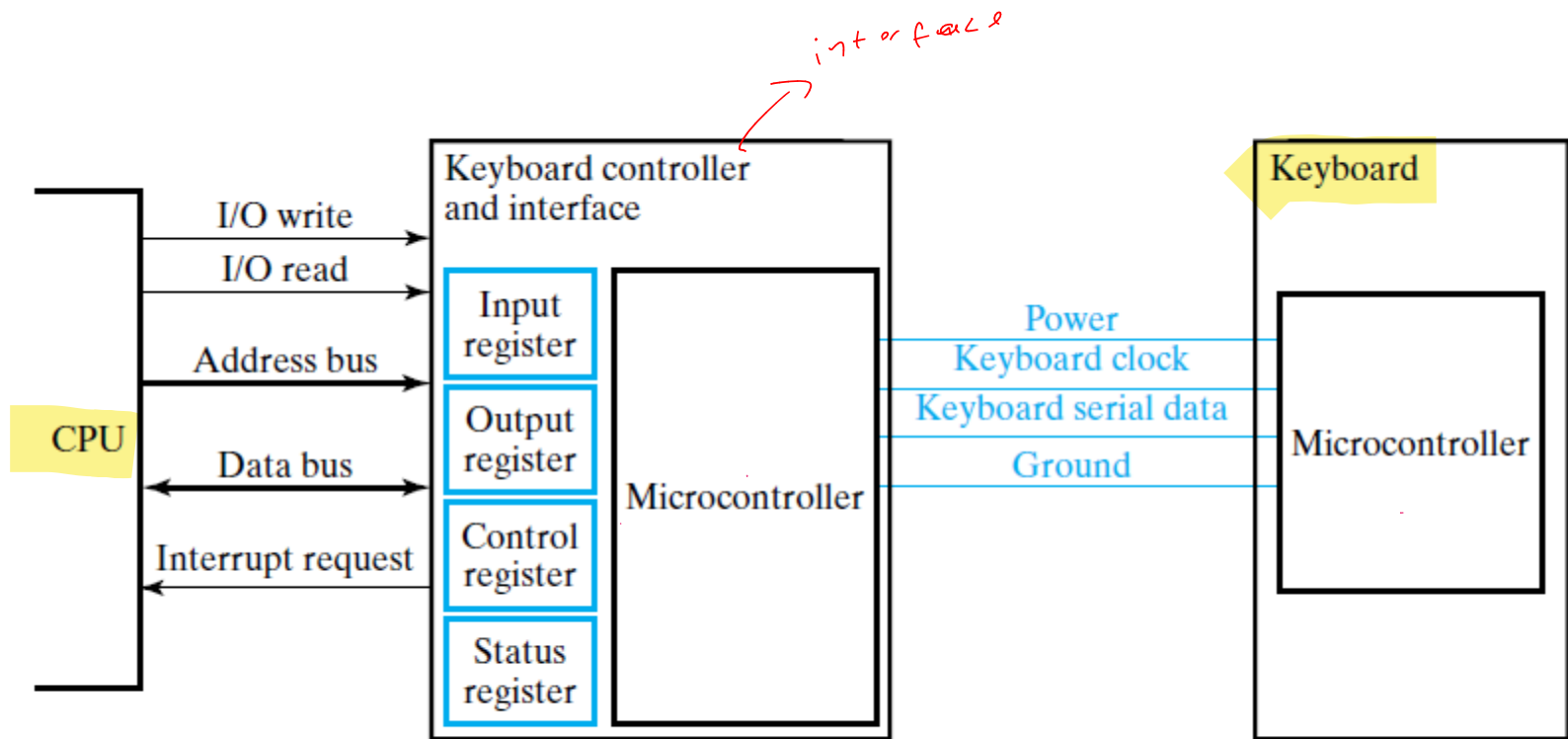
Seri: Alabores, me

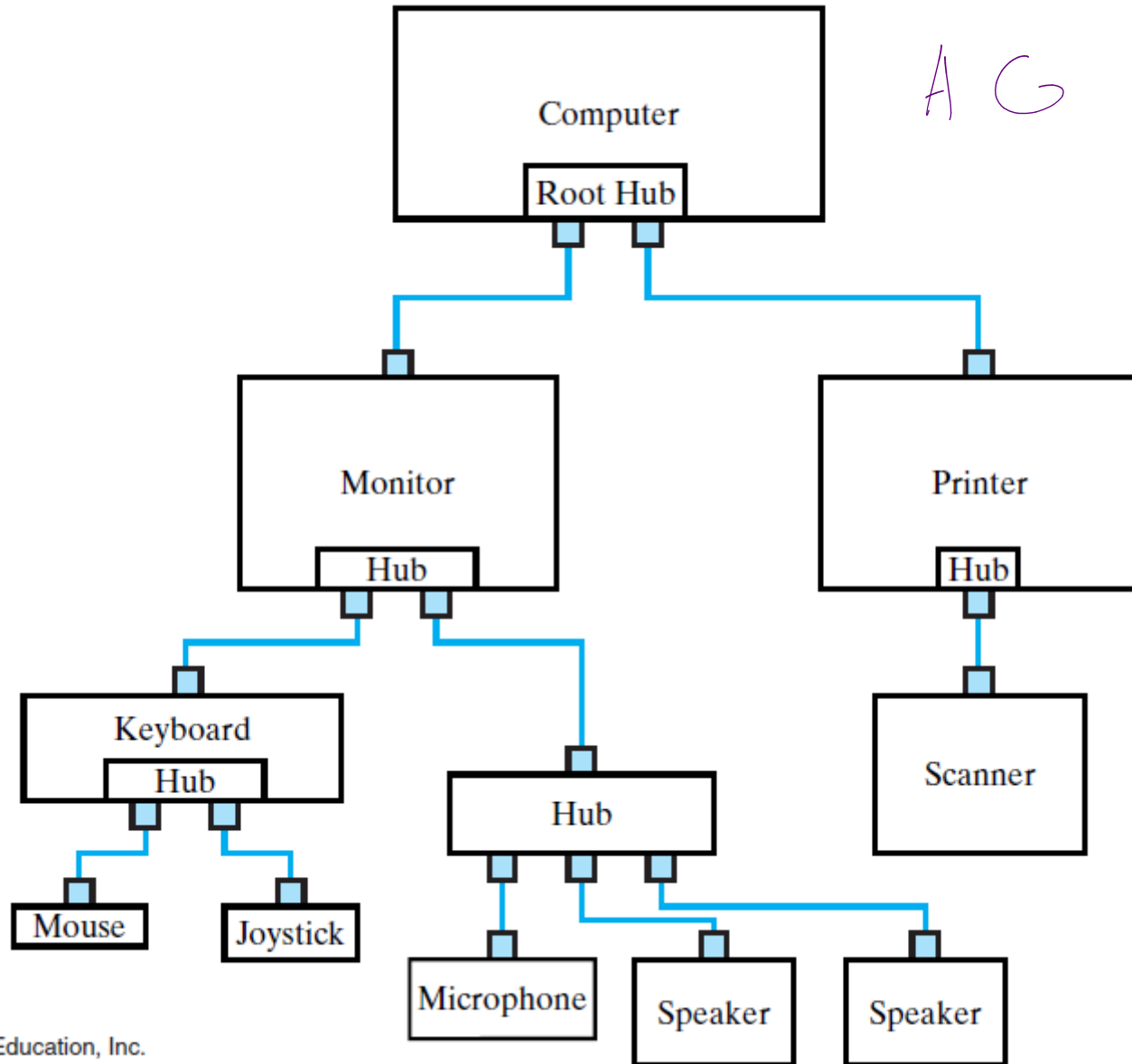
Asenkron



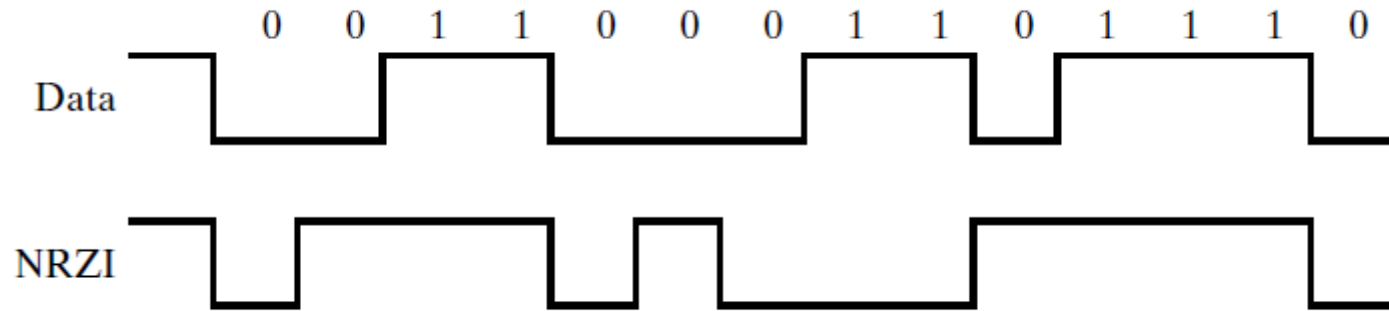
Sinkron







A C



AG

SYNC	PID	Packet Specific Data	CRC	EOP
------	-----	----------------------	-----	-----

(a) General packet format

SYNC 8 bits	Type 4 bits 1001	Check 4 bits 0110	Device Address 7 bits	Endpoint Address 4 bits	CRC	EOP
----------------	------------------------	-------------------------	-----------------------------	-------------------------------	-----	-----

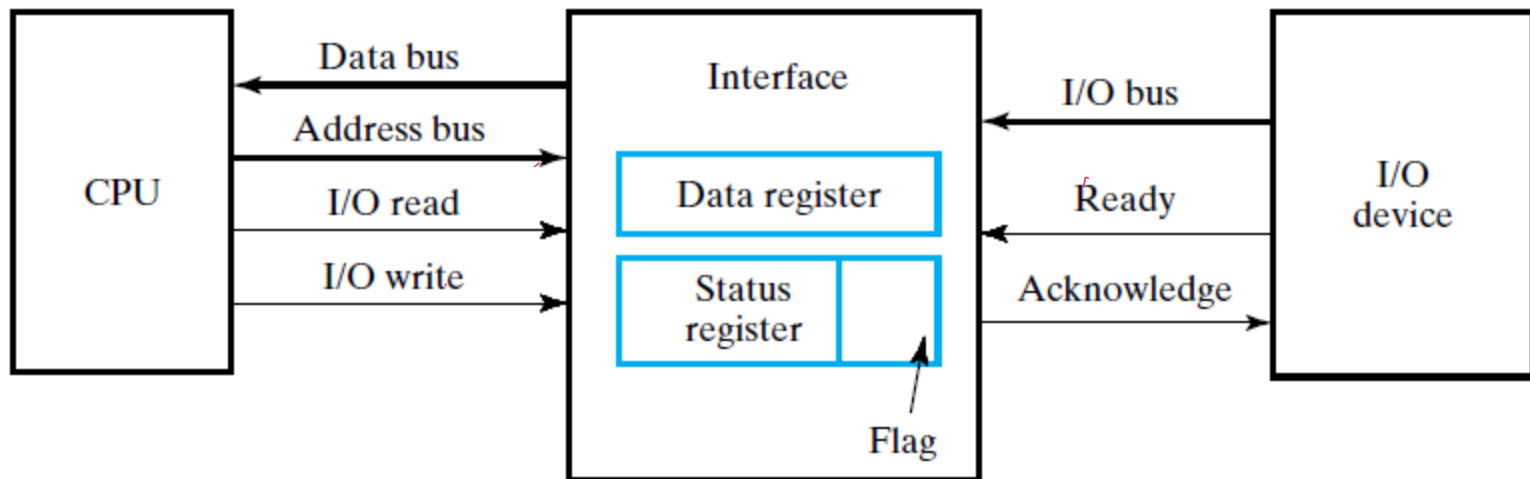
(b) Output packet

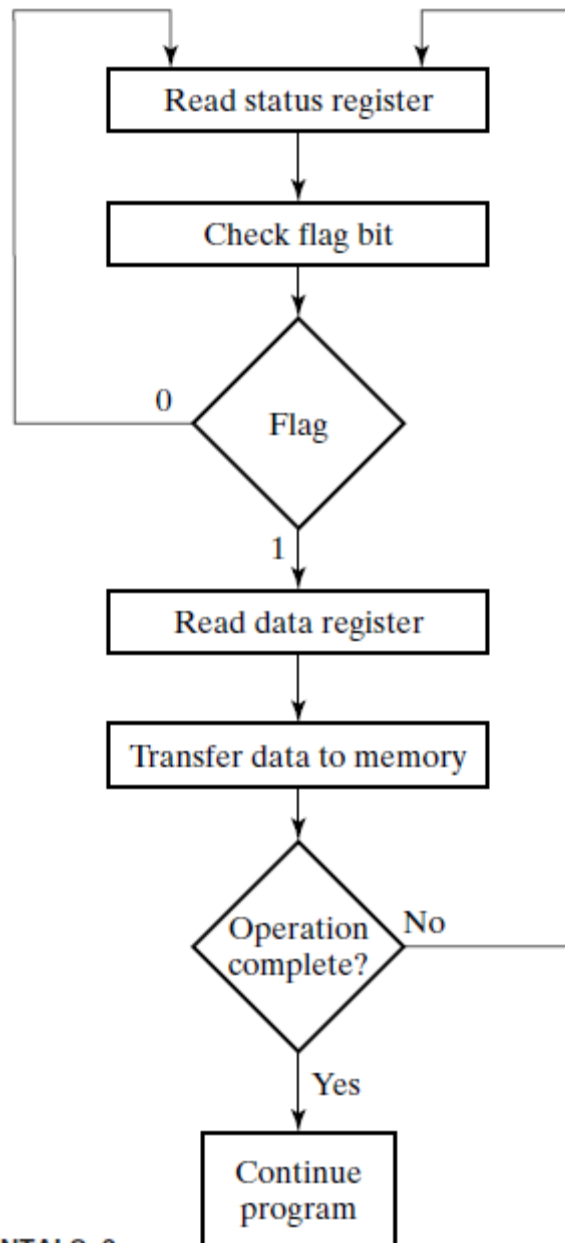
SYNC 8 bits	Type 4 bits 1100	Check 4 bits 0011	Data (Up to 1024 bytes)	CRC	EOP
----------------	------------------------	-------------------------	----------------------------	-----	-----

(c) Data packet (Data0 type)

SYNC 8 bits	Type 4 bits 0100	Check 4 bits 1011	EOP
----------------	------------------------	-------------------------	-----

(d) Handshake packet (Acknowledge type)





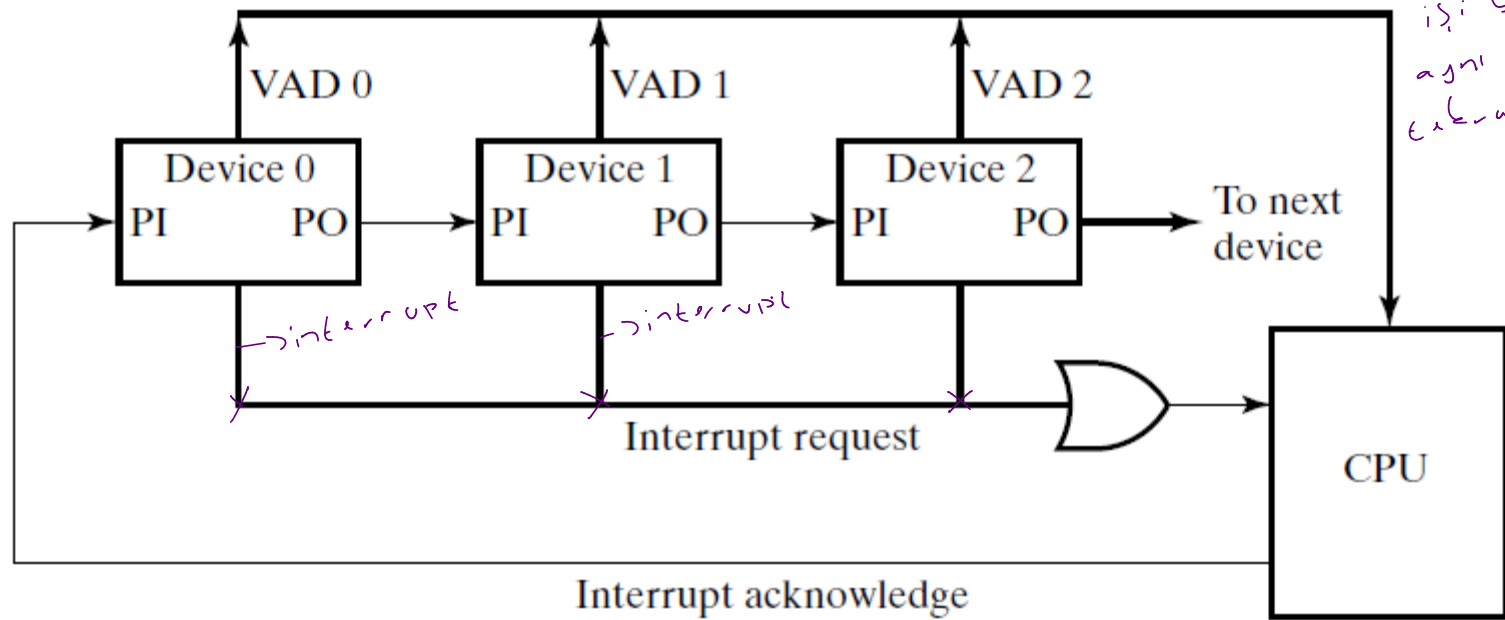
Pooling
mantığı
for içinde kontrol
kullanılıyor

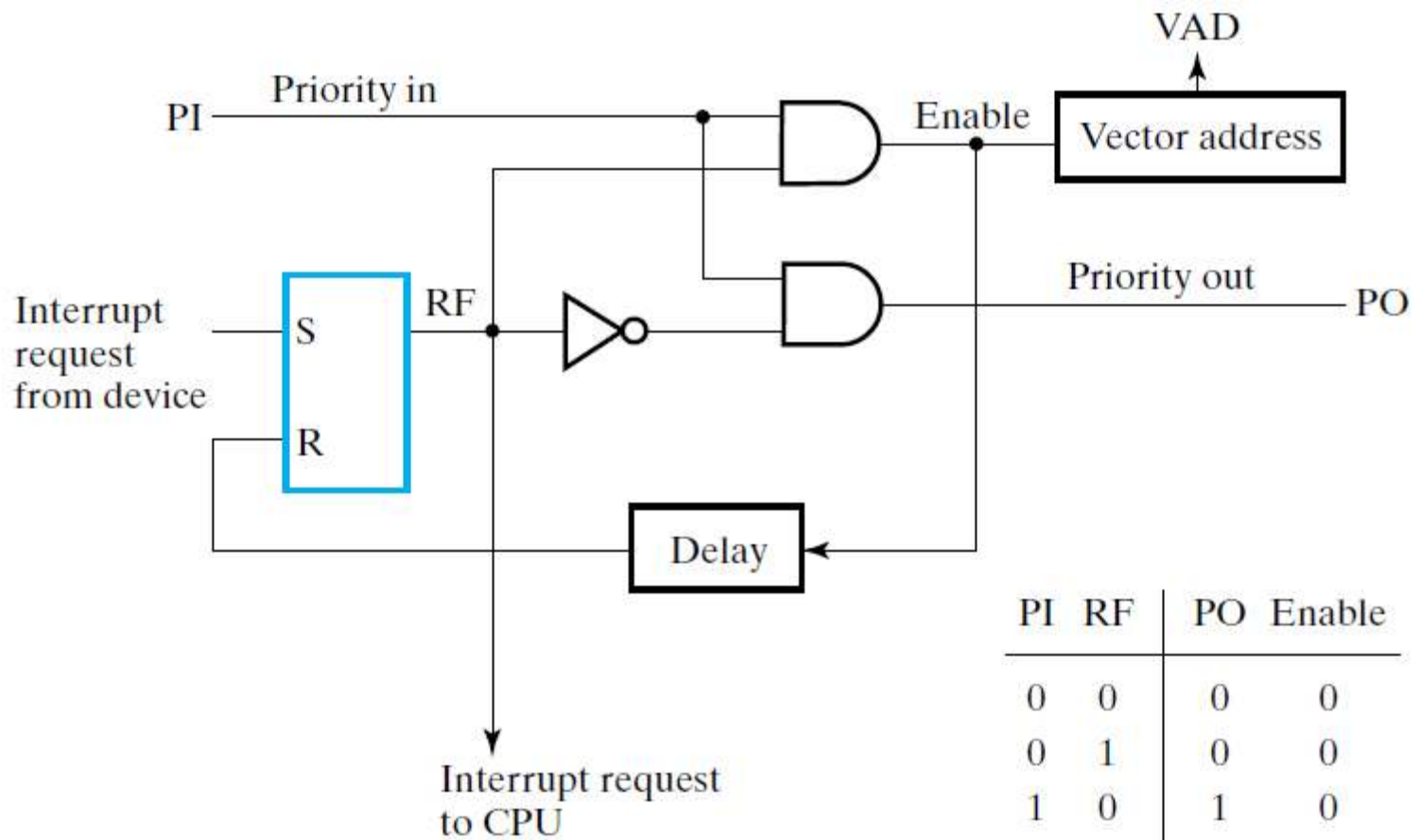
Daisy chain interrupt önce sonra gider, gider, bile

ack gitti o'na göre VAD'daki adres CPU'ya D1-D2 diye bak ar değiştirir. En önceliklisi yapar 2 üncüsü olsa yapmaz. Sonra

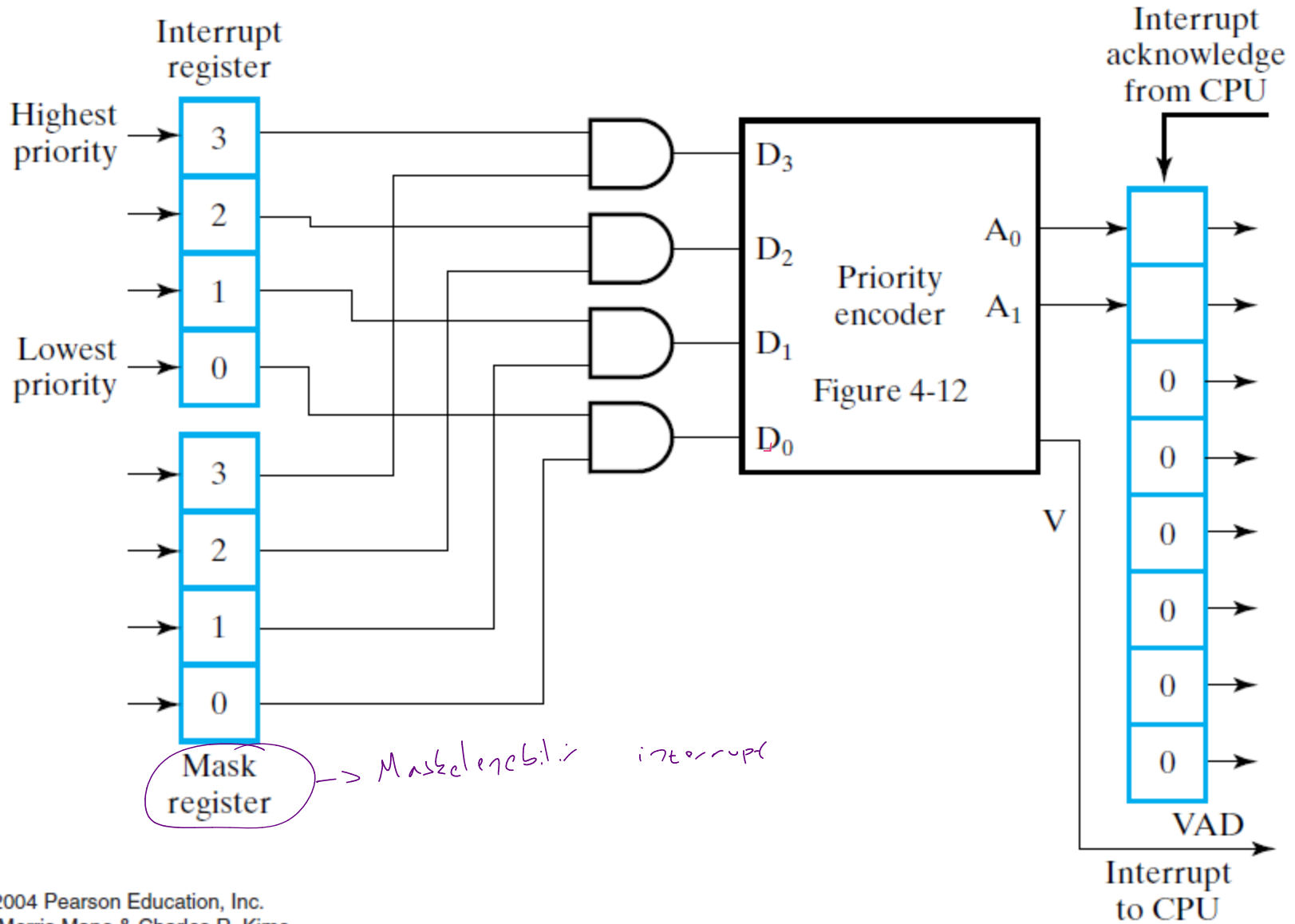
interrupt mantığı

interrupt çıkışı olduğu işin CPU işi bitirip aynı işleneni eklenir.

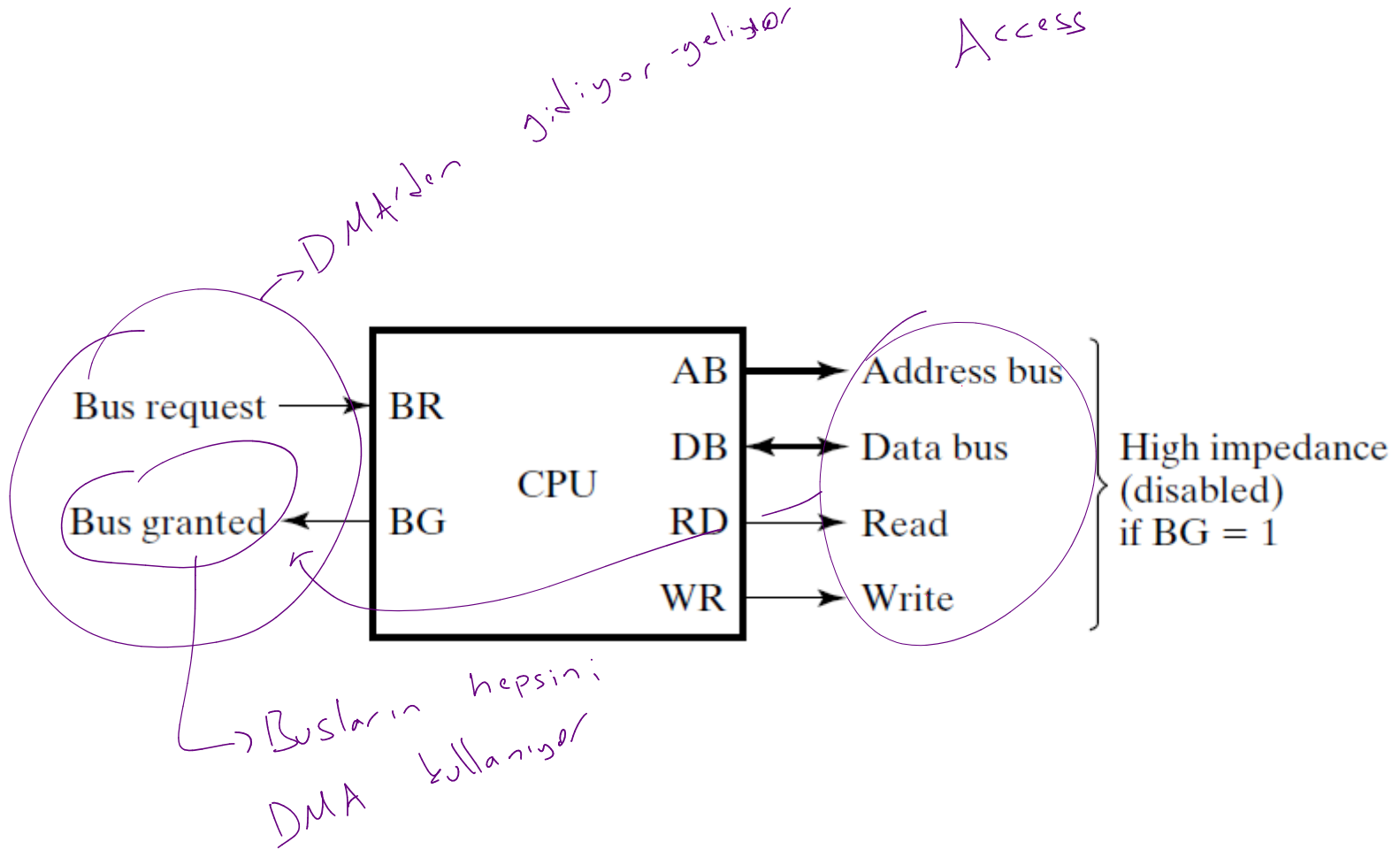




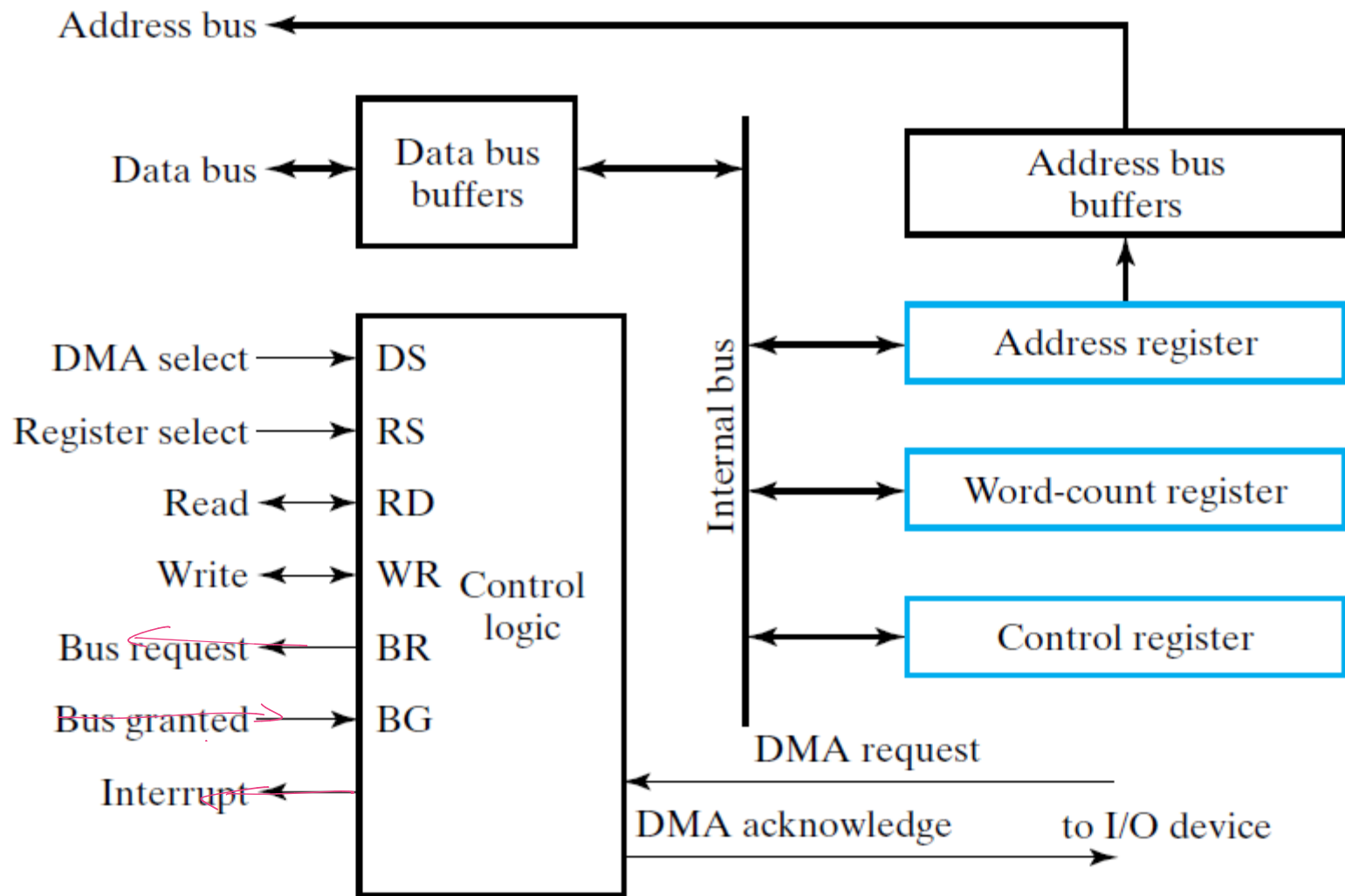
2. Last in first out



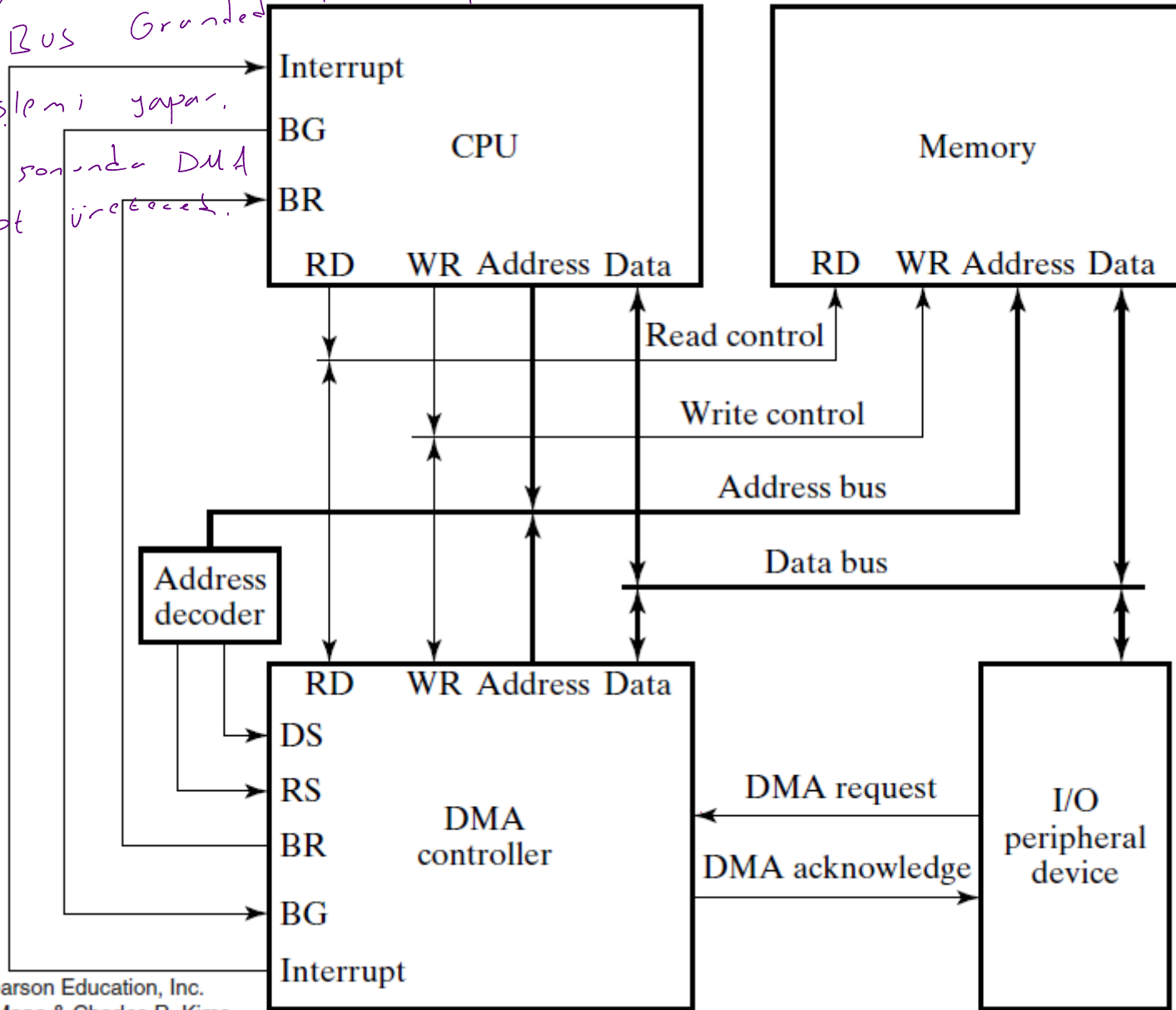
Direct Memory Access



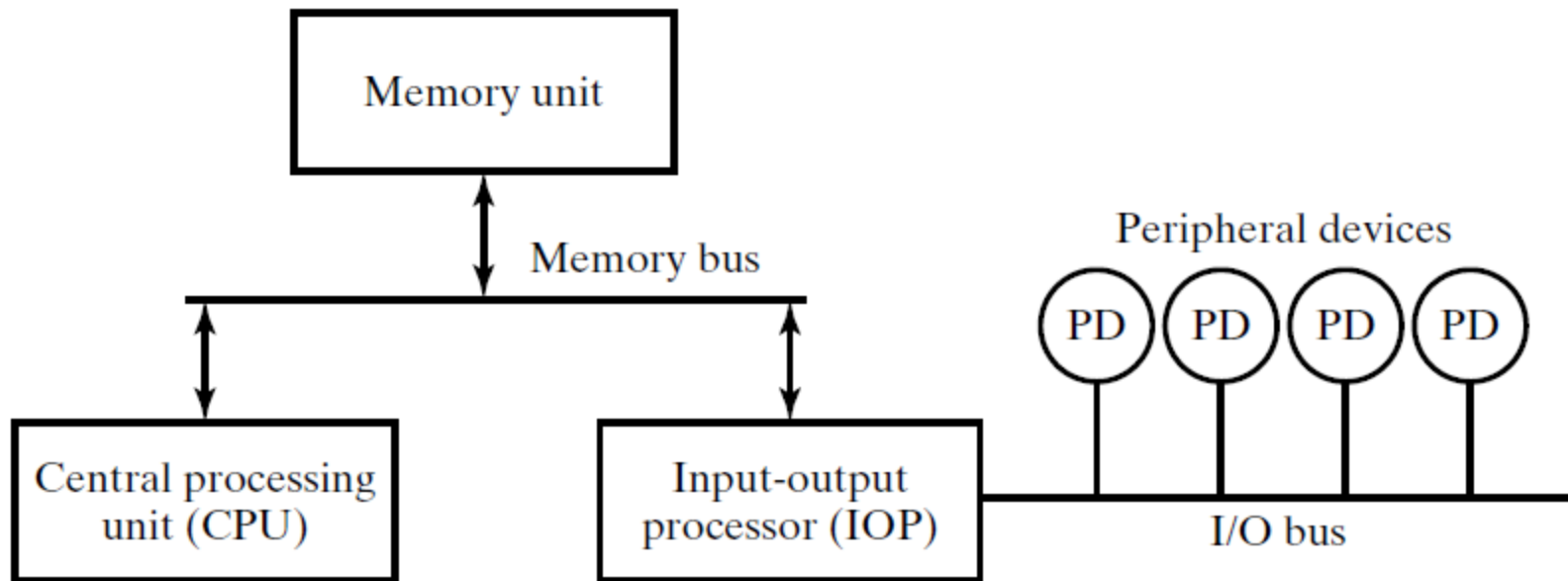
DMA is
yarp's

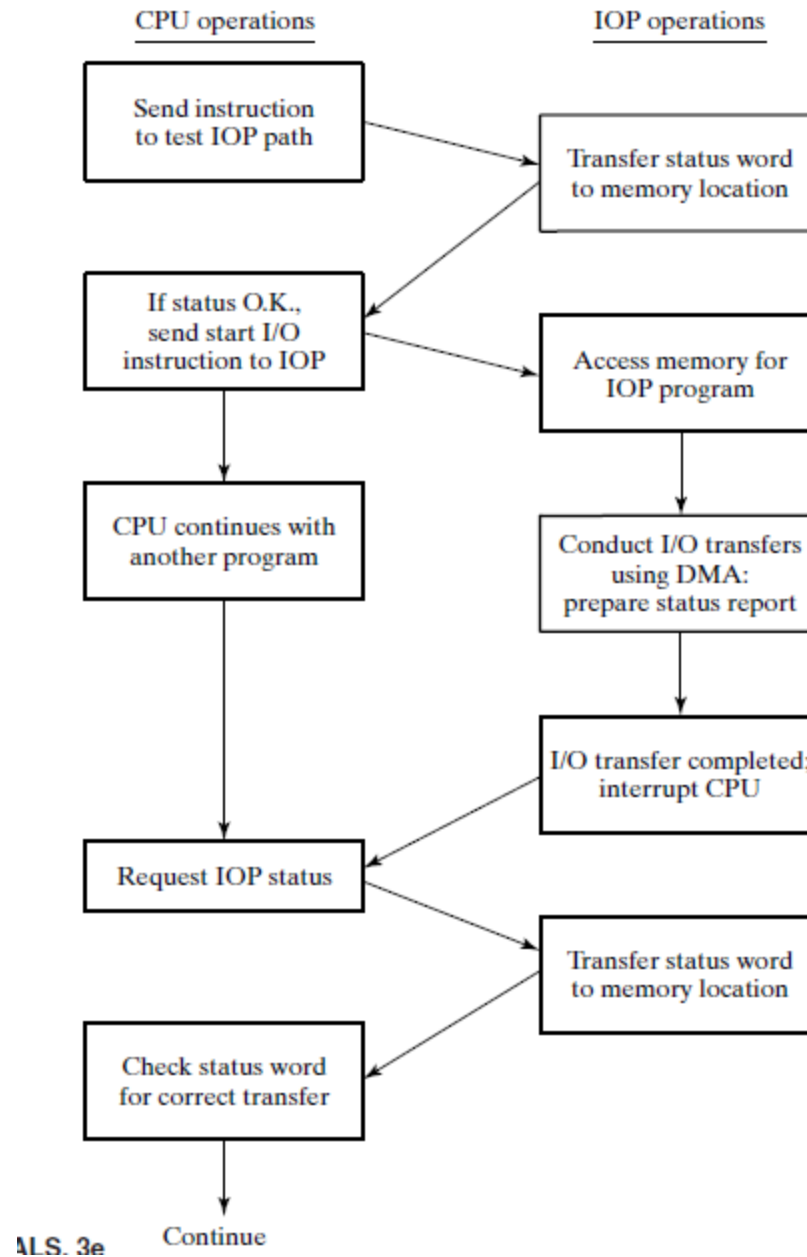


- 1-) CPU, DMA'i programlayacak. (Memory başlangıç adresi, IO b. d., veri miktarı vs.)
- 2-) DMA, Bus Request talebiyle yolları ile erişim hakkı veriyor.
- 3-) CPU, Bus Granted
- 4-) DMA işlemi yapar.
- 5-) işlem sonunda DMA interrupt verecek.



tlc





A G



Cache Memory

Outline

Hafiza
Higazi

- Memory Hierarchy
- Direct-Mapped Cache
- Write-Through, Write-Back
- Cache replacement policy
- Examples

Principal of Locality

- Cache work on a principal known as **locality of reference**. This principal states asserts that programs continually use and re-use the same locations.

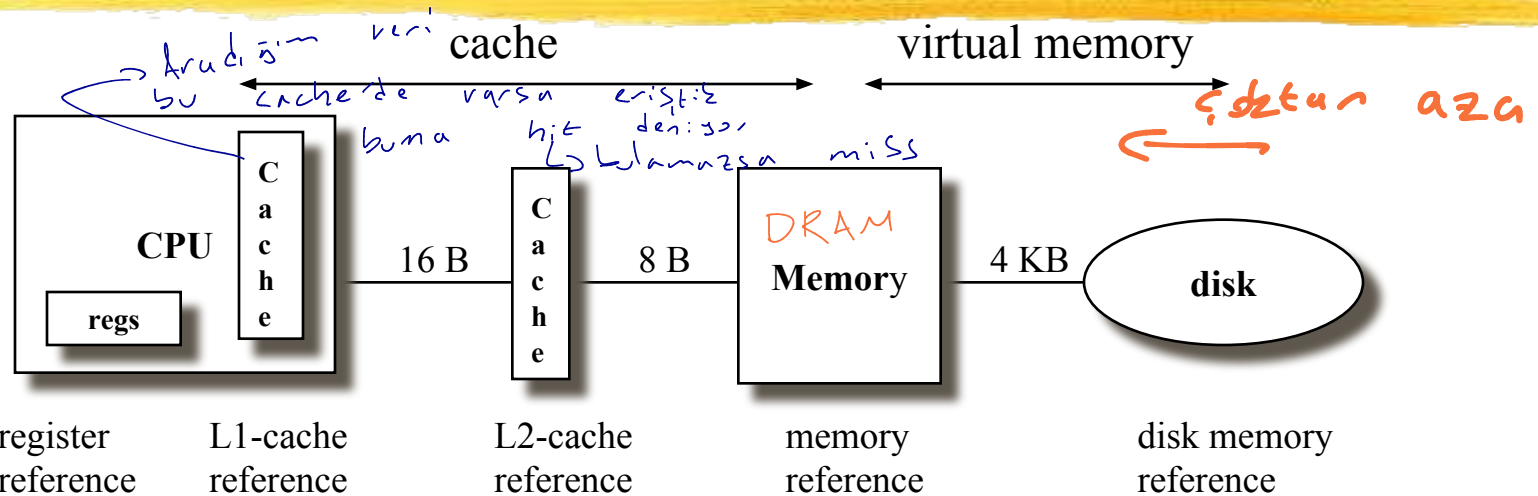
- Instructions: Loops, common subroutines
- Data: Look-up Tables, data sets(arrays)

→ Aynı hafızadaki bilgiye arada bir erişim.

- **Temporal Locality (locality in time)**: Same location will be referenced again soon
- **Spatial Locality (locality in space)**: Nearby locations will be referenced soon

→ Hafızanın ve çevresindeki bilgilerin cache alınması
→ Dizinin elemanları

The Tradeoff



size:	608 B	128k B	512kB -- 4MB	128 MB	27GB
speed:	1.4 ns	4.2 ns	16.8 ns	112 ns	9 ms
\$/Mbyte:			\$90/MB	\$2-6/MB	\$0.01/MB
block size:	4 B	4 B	16 B	4-8 KB	

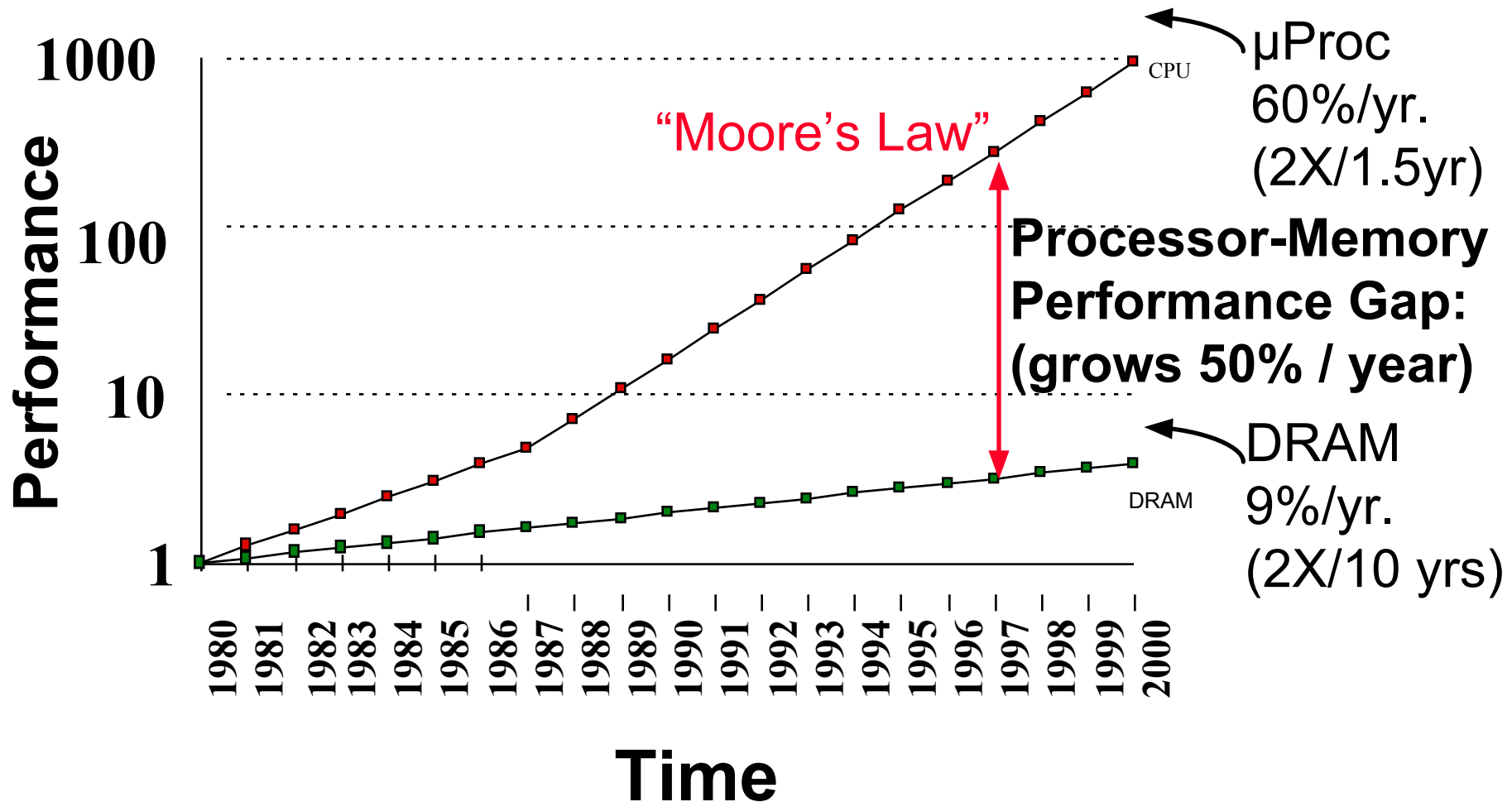
larger, slower, cheaper



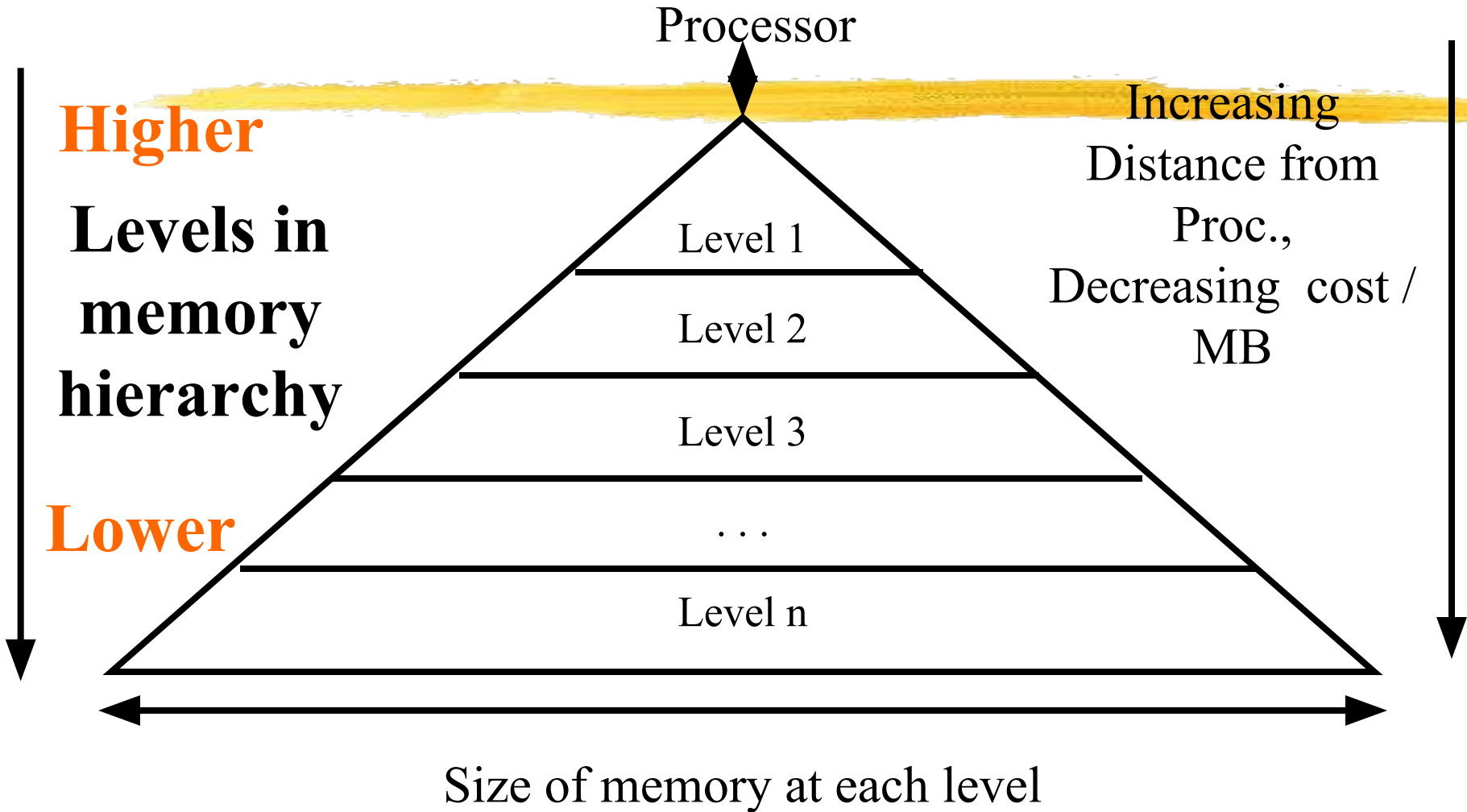
(Numbers are for a DEC Alpha 21264 at 700MHz)

MOORE's LAW

Processor-DRAM Memory Gap (latency)



Memory Hierarchy



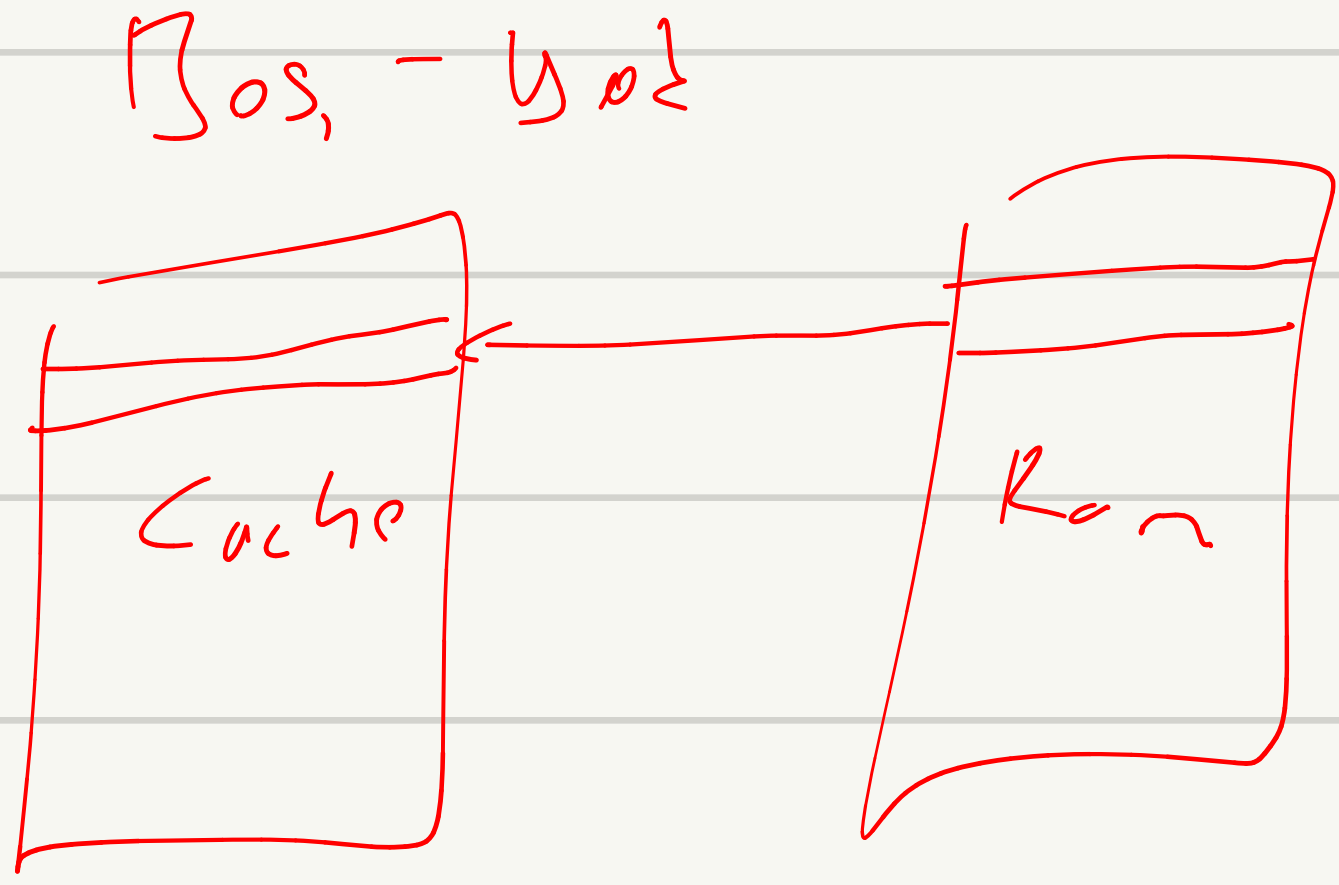
Cache Design

- How do we organize cache?
- Where does each memory address map to? (Remember that cache is subset of memory, so multiple memory addresses map to the same cache location.)
→ Anıfızadaki adres blegünü nereye koyacağız / dolussa nereye koyacağız
- How do we know which elements are in cache?
→ Aradığım veri: Cache'de var mı? Erişim hızını ne kadar olursun
- How do we quickly locate them?

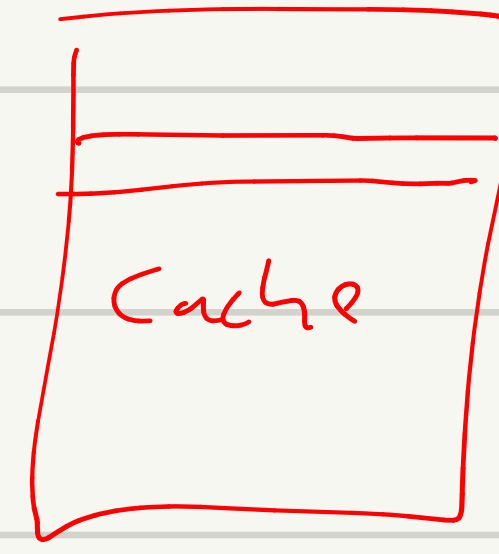
Cache mappings

- There are three types of methods for mapping between the main memory addresses and the cache addresses.
→ Adresler üzerindeki bilgiyi cache'e hangi metodlarla çekeceğiz?
- Fully associative cache *→ Her yer olabilir*
- Direct mapped cache *→ En basit; hangisi satır nereye gidecek belli, ramdaki yer belli*
- Set associative cache *→ Satırlar belli*

Direct map Cache



Dolu - Yok

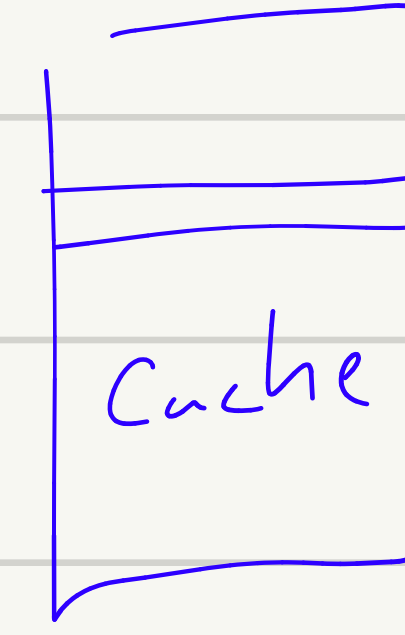


üstüne yaz

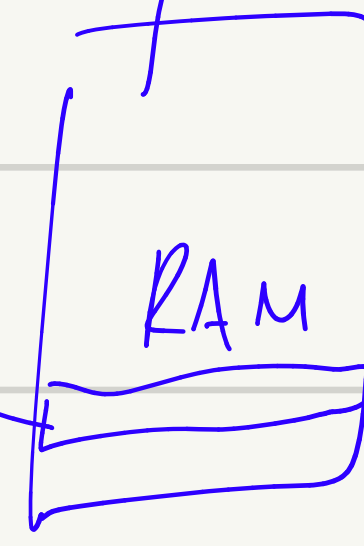


Fully Associative cache

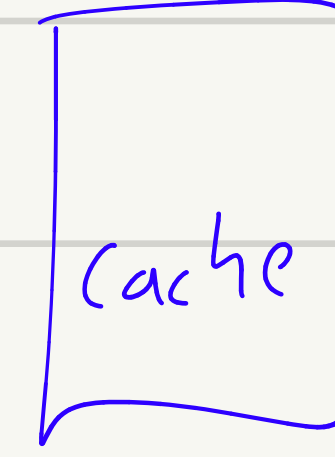
Bos - Yok



Giden veri herhangi
bir adrese yazılır



Dolu - Yok



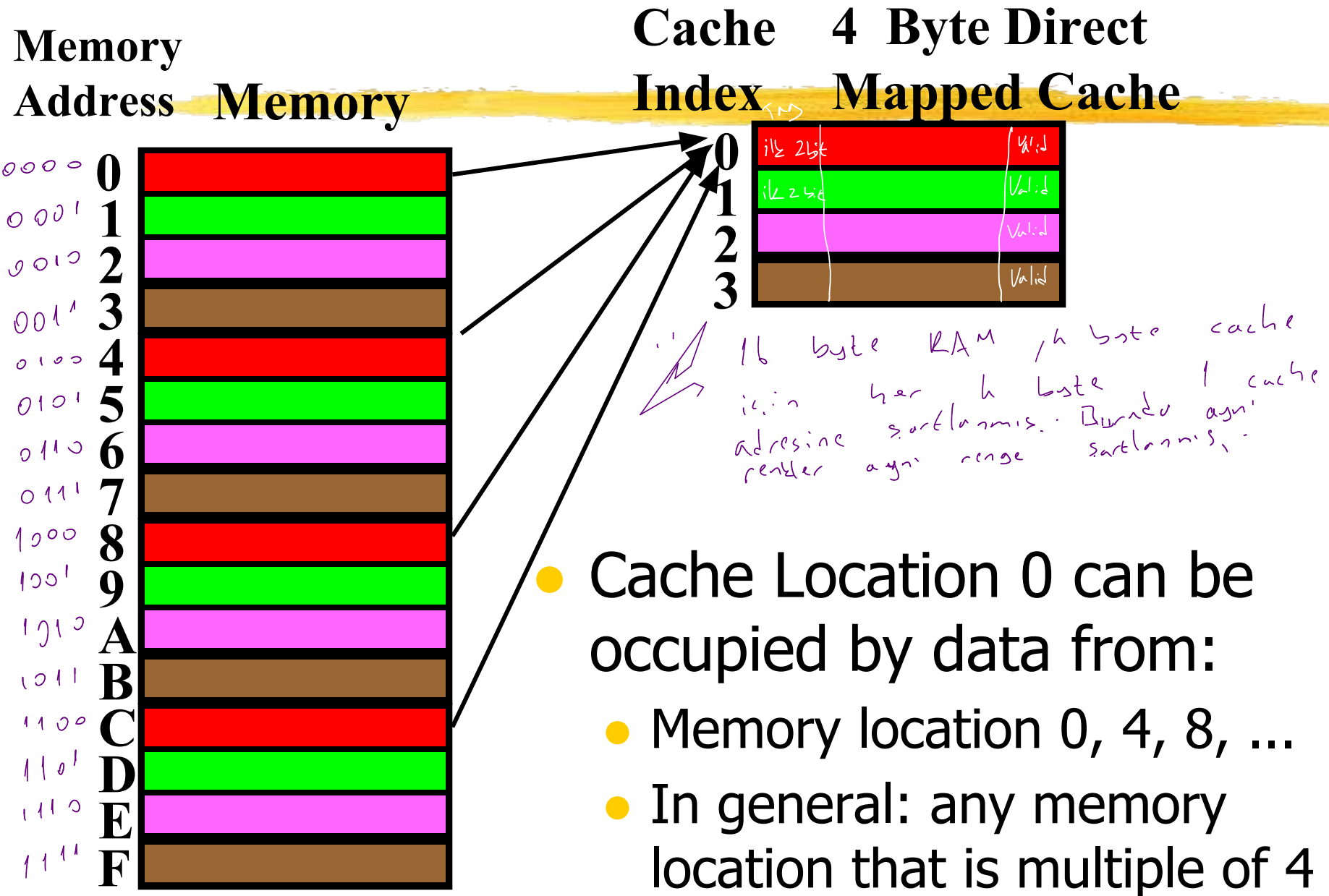
Giden veri: LIFO, LRU
v.b. kriterlerinden biri
birine göre yazılır

Direct-Mapped Cache



- In a direct-mapped cache, each memory address is associated with one possible block within the cache
 - Therefore, we only need to look in a single location in the cache for the data if it exists in the cache
 - Block is the unit of transfer between cache and memory

Direct-Mapped Cache

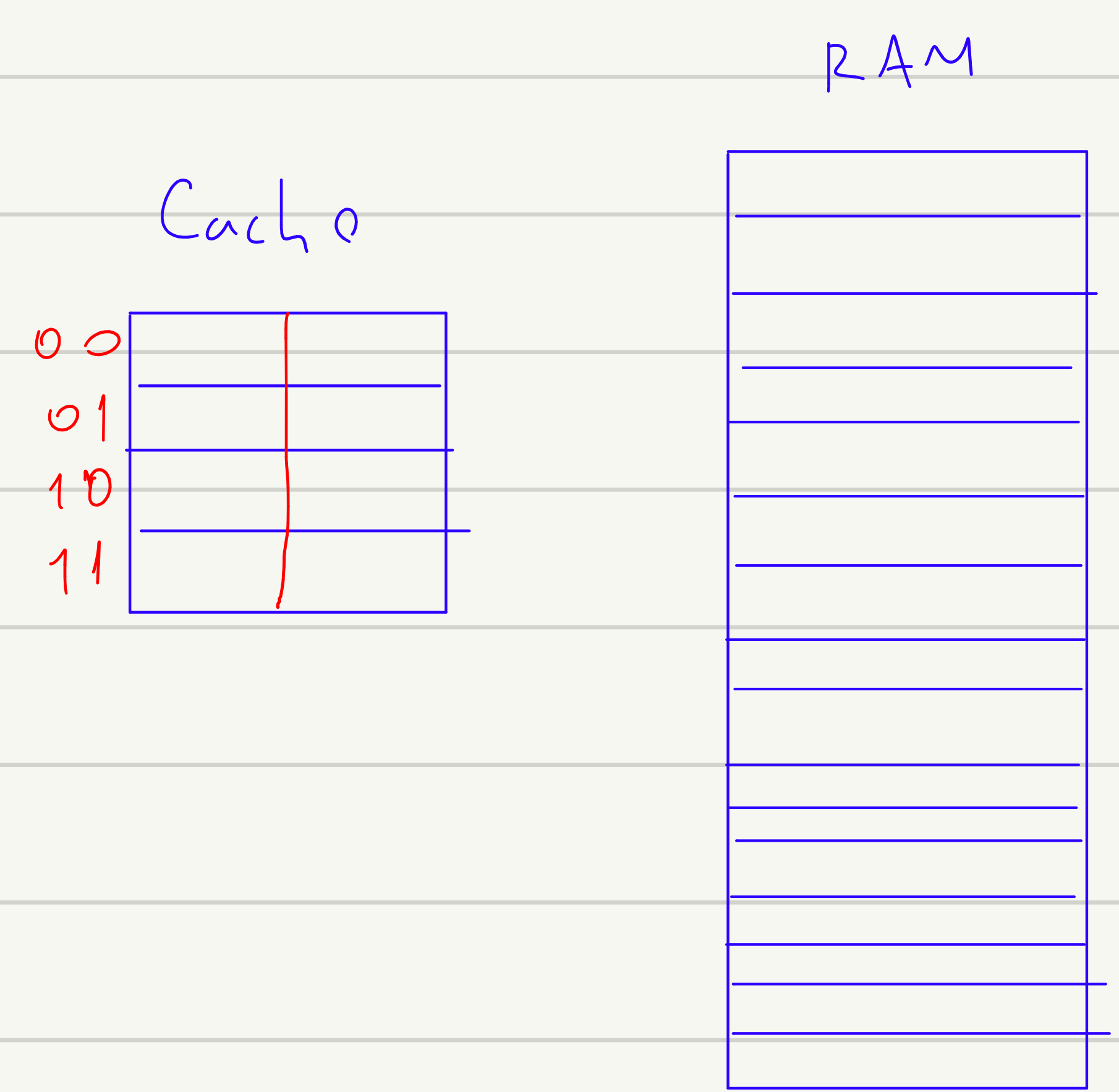


Issues with Direct-Mapped

- Since multiple memory addresses map to same cache index, how do we tell which one is in there?
- What if we have a block size > 1 byte?
- Result: divide memory address into three fields

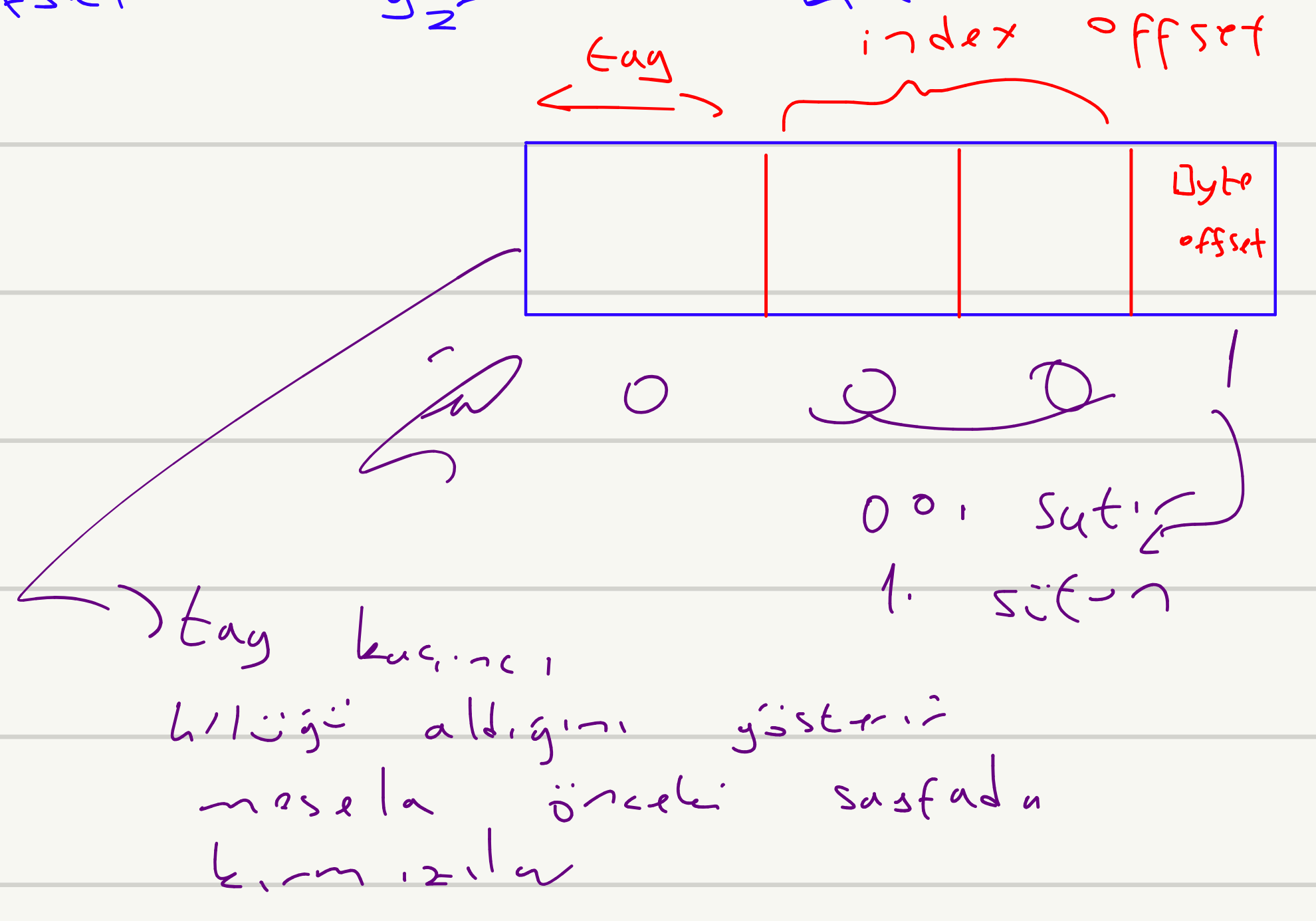


Toplam kapasitesi: 8 byte olan, blok genişliği: 2 byte olan bir direct map cache yapısı için 16 byte'lık hafıza kullanılıyor. Buna göre cache'in tag genişliği nedir?



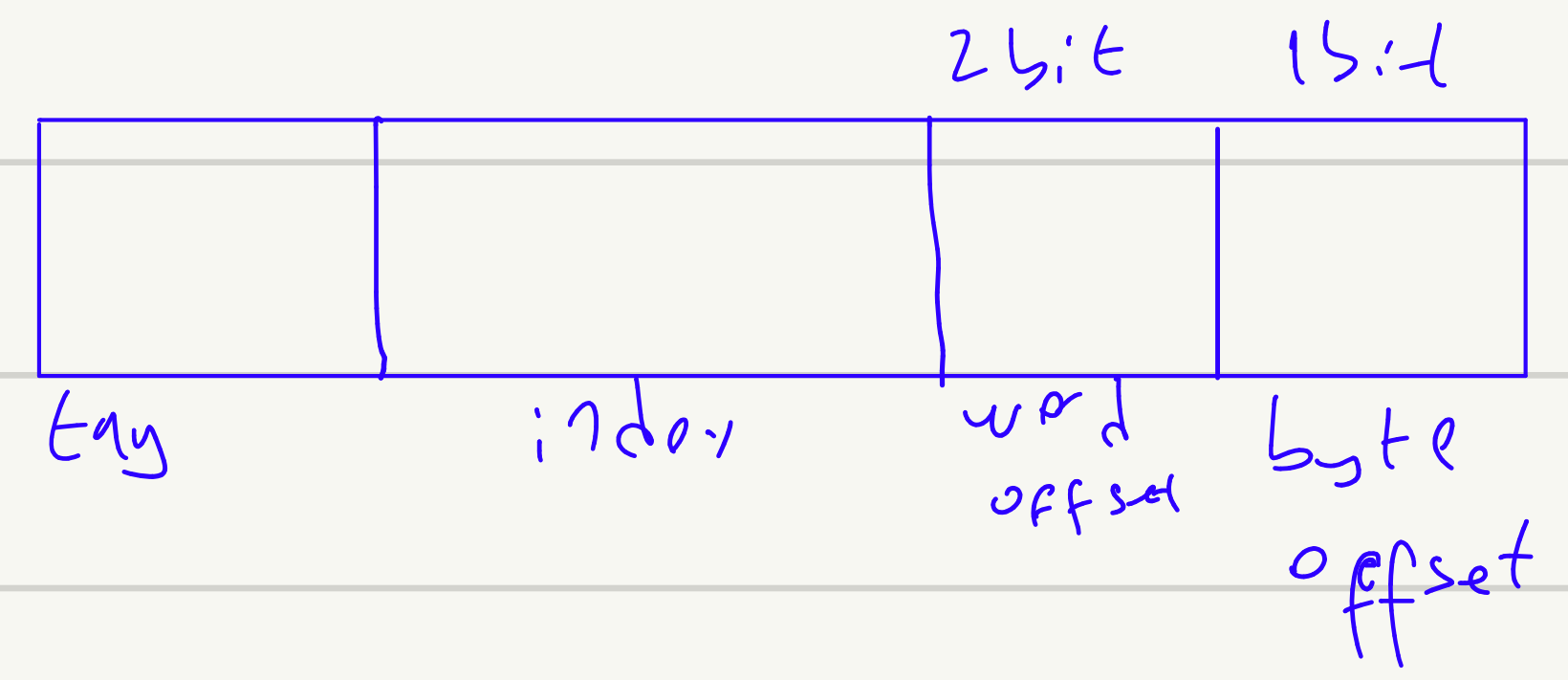
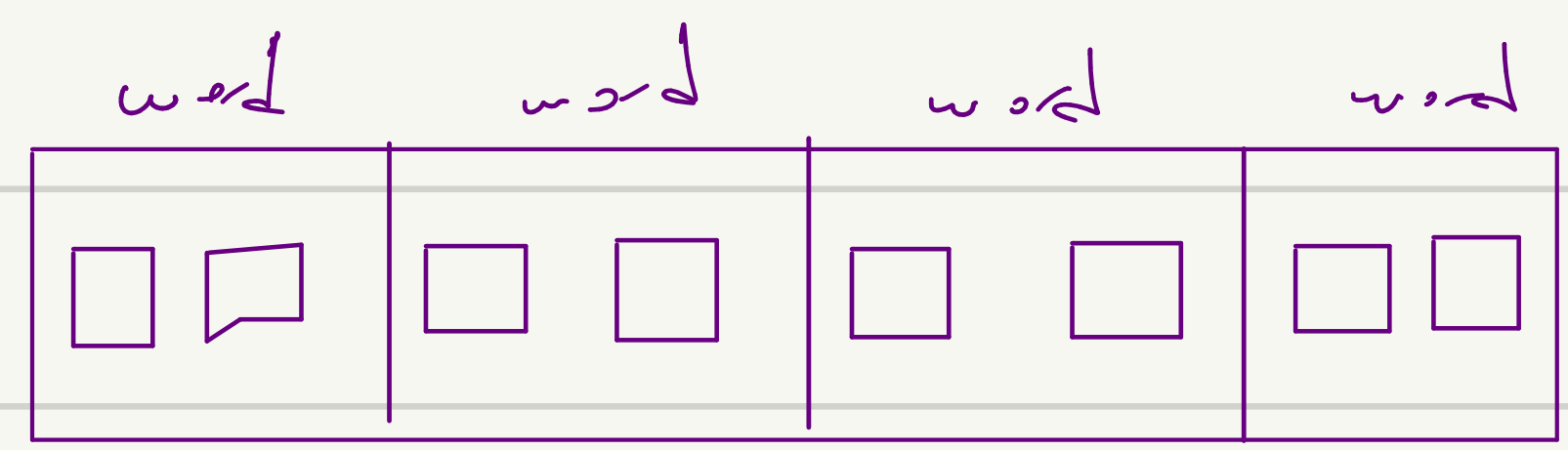
① $H\ cache = \frac{8}{2} = 4$ $\log_2 4 = 2$ bit

② Byte offset $\log_2 2 = 1$ bit



* 8086 $\rightarrow$ 16 bit 1 word $\rightarrow$ 2 byte

cache blok genişliği 4 word

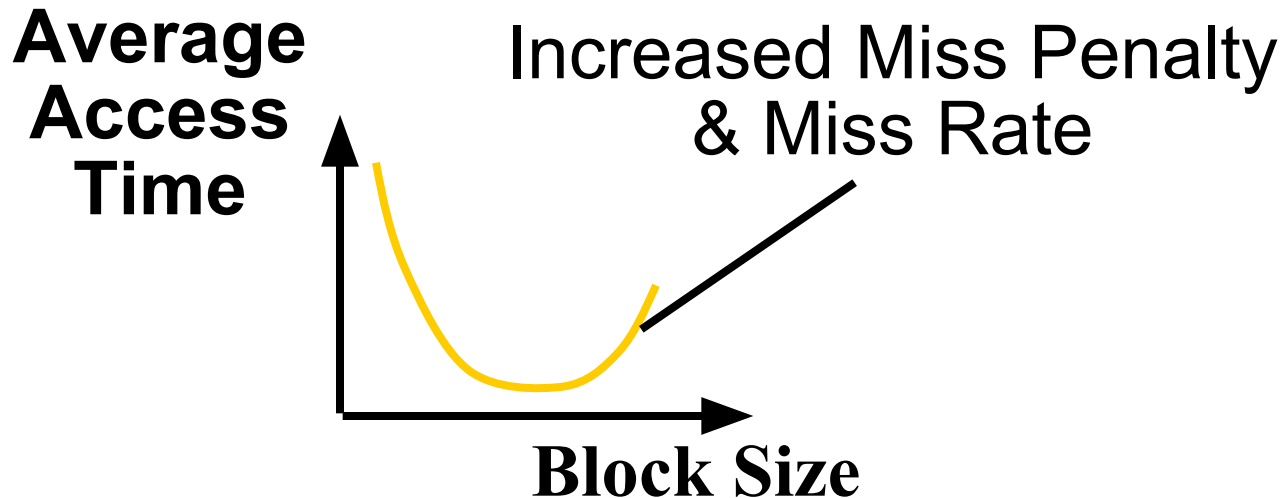
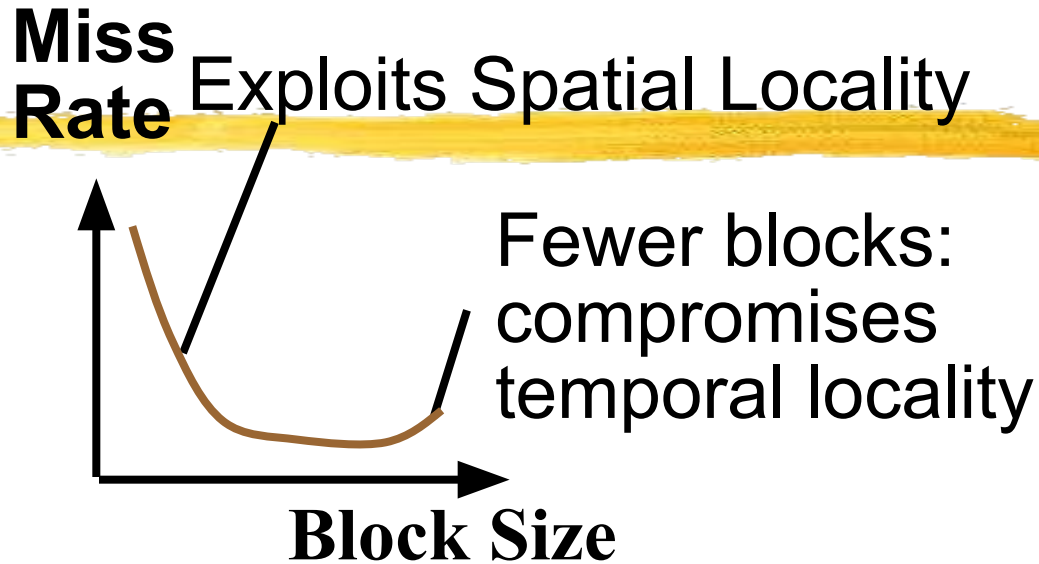


Direct-Mapped Cache Terminology



- All fields are read as unsigned integers.
- Index: specifies the cache index (which “row” of the cache we should look in)
- Offset: once we’ve found correct block, specifies which byte within the block we want
- Tag: the remaining bits after offset and index are determined; these are used to distinguish between all the memory addresses that map to the same location

Block Size Tradeoff Conclusions



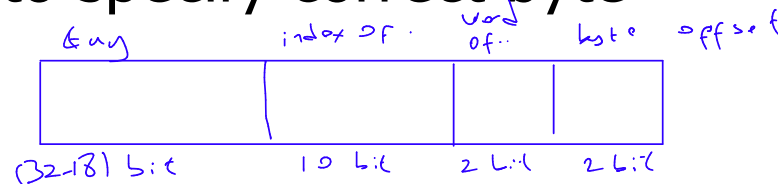
Direct-Mapped Cache Example

$$2^{14}$$

- Suppose we have a 16KB of data in a direct-mapped cache with 4 word blocks
- Determine the size of the tag, index and offset fields if we're using a 32-bit architecture
- Offset
 - need to specify correct byte within a block
 - block contains 4 words = 16 bytes = 2^4 bytes
 - need 4 bits to specify correct byte

$$\frac{2^{14}}{2^4} = 2^{10} \text{ byte}$$

1 word 4 byte



Direct-Mapped Cache Example

- Index: (~index into an "array of blocks")
 - need to specify correct row in cache
 - cache contains 16 KB = 2^{14} bytes
 - block contains 2^4 bytes (4 words)

rows/cache = # blocks/cache (since there's one block/row)

= bytes/cache bytes/row

= 2^{14} bytes/cache 2^4 bytes/row

= 2^{10} rows/cache

need 10 bits to specify this many rows

Direct-Mapped Cache Example

- Tag: use remaining bits as tag
 - tag length = mem addr length - offset - index
$$= 32 - 4 - 10 \text{ bits}$$
$$= 18 \text{ bits}$$
 - so tag is leftmost 18 bits of memory address
- Why not full 32 bit address as tag?
 - All bytes within block need same address (-4b)
 - Index must be same for every address within a block, so its redundant in tag check, thus can leave off to save memory (- 10 bits in this example)

Accessing data in a direct mapped cache

Memory

- Ex.: 16KB of data, direct-mapped, 4 word blocks
- Read 4 addresses
 - 0x00000014, 0x0000001C, 0x00000034, 0x00008014
- Memory values on right:
 - only cache/memory level of hierarchy

Address (hex)	Value of Word
...	...
00000010	a
<u>00000014</u>	b
00000018	c
<u>0000001C</u>	d
...	...
00000030	e
<u>00000034</u>	f
00000038	g
0000003C	h
...	...
00008010	i
<u>00008014</u>	j
00008018	k
0000801C	l
...	...

Accessing data in a direct mapped cache

- 4 Addresses:
 - 0x00000014, 0x0000001C, 0x00000034, 0x00008014
- 4 Addresses divided (for convenience) into Tag, Index, Byte Offset fields

000000000000000000000000	00000000001	0100
000000000000000000000000	00000000001	1100
000000000000000000000000	00000000011	0100
000000000000000000000010	00000000001	0100
Tag	Index	Offset

Accessing data in a direct mapped cache

- So let's go through accessing some data in this cache
 - 16KB data, direct-mapped, 4 word blocks
- Will see 4 types of events:
- Cache loading: Before any words and tags have been loaded into the cache, all locations contain invalid information. As lines are fetched from main memory into the cache, cache entries become valid. To represent this structure a **valid bit** added to each cache entry.
- cache miss: nothing in cache in appropriate block, so fetch from memory
- cache hit: cache block is valid and contains proper address, so read desired word
- cache miss, block replacement: wrong data is in cache at appropriate block, so discard it and fetch desired data from memory

Cache Loading



- Before any words and tags have been loaded into the cache, all locations contains invalid information. As lines are fetched from main memory into the cache, cache entries become valid. To represent this structure a **valid bit** added to each cache entry. This valid bit indicates that the associated cache line is valid(1) or invalid(0).
- If the valid bit is 0, then a cache miss occurs even if the tag matches the address from CPU, required addressed word to be taken from main memory.

16 KB Direct Mapped Cache, 16B blocks

- Valid bit: determines whether anything is stored in that row (when computer initially turned on, all entries are invalid)

Valid

Example Block

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	0				
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Read 0x00000014 = 0...00 0..001 0100

● 000000000000000000000000 000000000001 0100
Tag field Index field Offset

Valid						
Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f	
0	0					
1	0					
2	0					
3	0					
4	0					
5	0					
6	0					
7	0					
...		...				
1022	0					
1023	0					

So we read block 1 (0000000001)

● 000000000000000000000000 0000000001 0100
Valid Tag field Index field Offset

Index	Valid	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0					
<u>1</u>	0					
2	0					
3	0					
4	0					
5	0					
6	0					
7	0					
...						
1022	0					
1023	0					

No valid data

● 000000000000000000000000 000000000001 0100
Tag field Index field Offset

Valid

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
<u>1</u>	0				
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				
...					
1022	0				
1023	0				

So load that data into cache, setting tag, valid

 00000000000000000000 00000000001 0100

Valid	Tag field	Index field	Offset
ex	Tag	0x0-3	0x4-7
		0x8-b	0xc-f

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	0	a	b	c	d
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				

● ● ●

• • •

1022	0					
1023	0					

Read from cache at offset, return word b

● 000000000000000000000000 000000000001 0100
Tag field Index field Offset

Valid

Index	Tag	0x0-3	<u>0x4-7</u>	0x8-b	0xc-f
0	0				
<u>1</u>	0	a	b	c	d
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Read 0x0000001C = 0...00 0..001 1100

● 000000000000000000000000 000000000001 1100
Tag field Index field Offset

Valid

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	1	a	b	c	d
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Data valid, tag OK, so read offset return word d

● 000000000000000000000000 000000000001 1100

Valid

Index	Tag	0x0-3	0x4-7	0x8-b	<u>0xc-f</u>
0	0				
<u>1</u>	<u>1</u> <u>0</u>	a	b	c	d
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Read 0x00000034 = 0...00 0..011 0100

● 000000000000000000000000 0000000011 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	1	a	b	c	d
2	0				
3	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

So read block 3

● 000000000000000000000000 00000000011 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	0	a	b	c	d
2	0				
<u>3</u>	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

No valid data

● 000000000000000000000000 00000000011 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	0	a	b	c	d
2	0				
<u>3</u>	0				
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Load that cache block, return word f

● 000000000000000000000000 000000000011 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	<u>0x4-7</u>	0x8-b	0xc-f
0	0				
1	0	a	b	c	d
2	0				
<u>3</u>	<u>0</u>	e	<u>f</u>	g	h
4	0				
5	0				
6	0				
7	0				

...

...

1022	0				
1023	0				

Read 0x00008014 = 0...10 0..001 0100

- 0000000000000000000010 000000000001 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	1	a	b	c	d
2	0				
3	1	e	f	g	h
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

So read Cache Block 1, Data is Valid

● 0000000000000000000010 000000000001 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
<u>1</u>	<u>1</u>	a	b	c	d
2	0				
3	1	e	f	g	h
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Cache Block 1 Tag does not match (0 != 2)

- 0000000000000000000010 000000000001 0100

Valid

Tag field

Index field

Offset

Index	Valid	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0					
<u>1</u>	1	<u>0</u>	a	b	c	d
2	0					
3	1	0	e	f	g	h
4	0					
5	0					
6	0					
7	0					
...						
1022	0					
1023	0					

Miss, so replace block 1 with new data & tag

● 0000000000000000000010 000000000001 0100
Valid Tag field Index field Offset

Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0				
1	1	i	j	k	l
2	0				
3	1	e	f	g	h
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

And return word j

● 0000000000000000000010 000000000001 0100
Valid Tag field Index field Offset

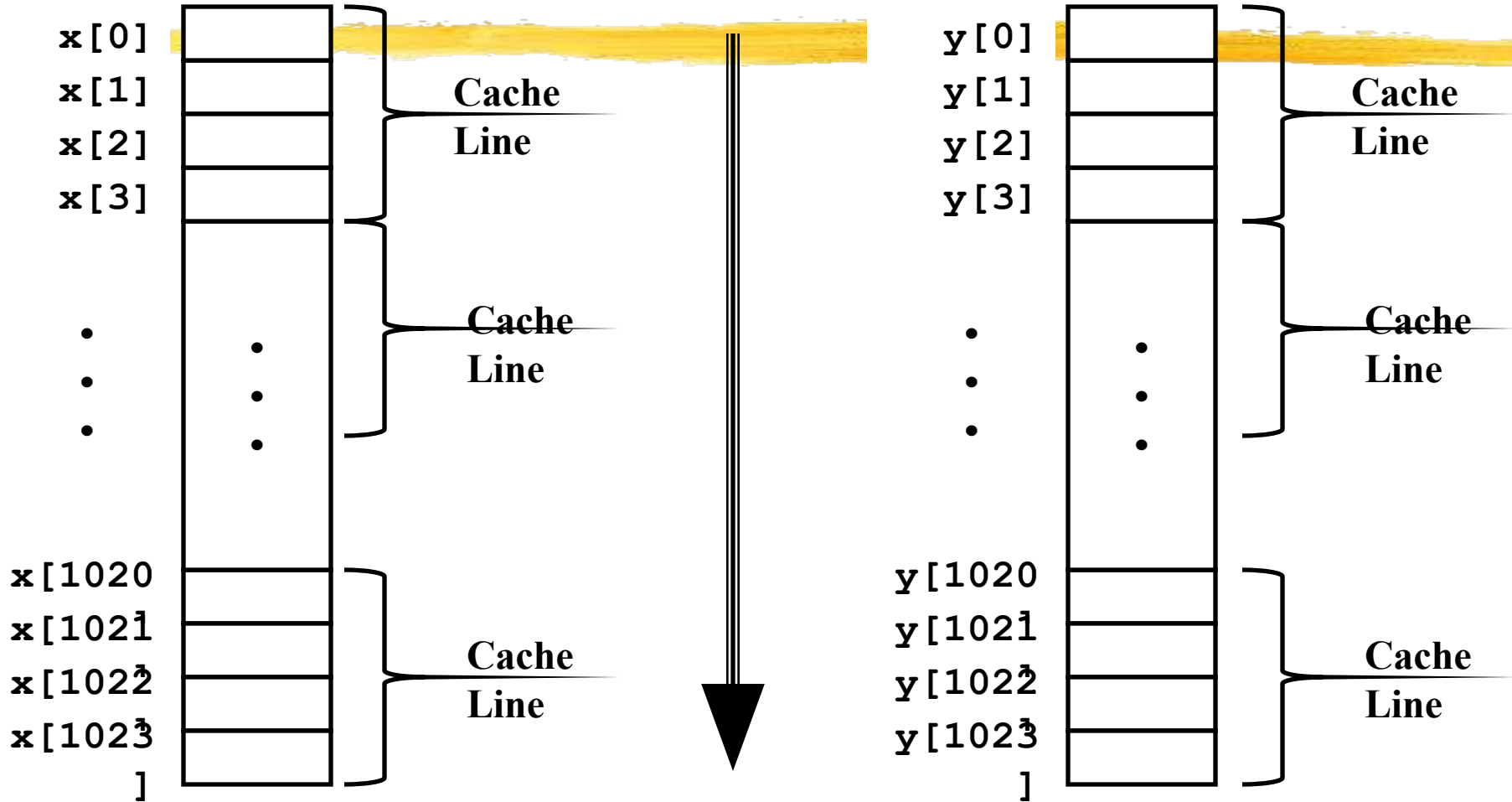
Index	Tag	0x0-3	<u>0x4-7</u>	0x8-b	0xc-f
0	0				
1	1	2	i	j	k
2	0				
3	1	0	e	f	g
4	0				
5	0				
6	0				
7	0				
...			...		
1022	0				
1023	0				

Vector Product Example

```
float dot_prod(float x[1024], y[1024])
{
    float sum = 0.0;
    int i;
    for (i = 0; i < 1024; i++)
        sum += x[i]*y[i];
    return sum;
}
```

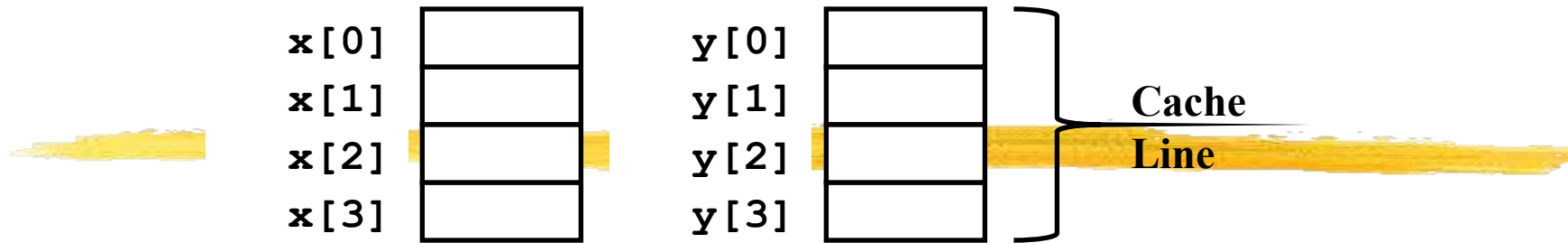
- Machine
 - DEC Station 5000
 - MIPS Processor with 64KB direct-mapped cache, 16 B line size
- Performance
 - Good case: 24 cycles / element
 - Bad case: 66 cycles / element

Thrashing Example



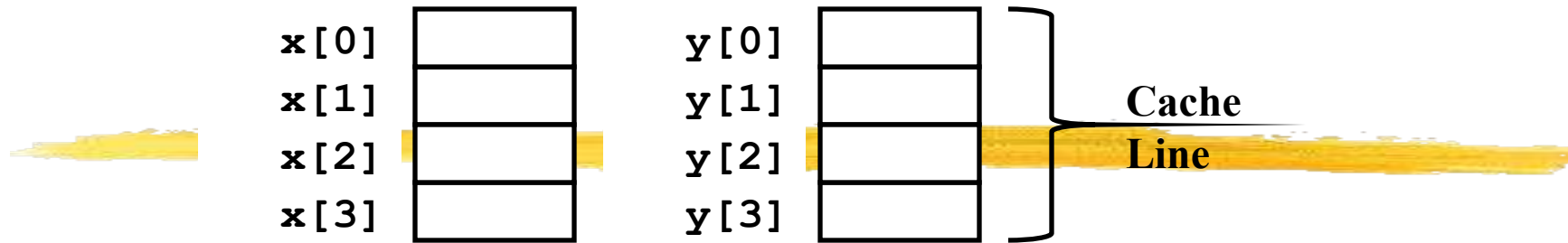
- Access one element from each array per iteration

Thrashing Example: Good Case



- Access Sequence
 - Read $x[0]$
 - $x[0], x[1], x[2], x[3]$ loaded
 - Read $y[0]$
 - $y[0], y[1], y[2], y[3]$ loaded
 - Read $x[1]$
 - Hit
 - Read $y[1]$
 - Hit
 - • • •
 - 2 misses / 8 reads
- Analysis
 - $x[i]$ and $y[i]$ map to different cache lines
 - Miss rate = 25%
 - Two memory accesses / iteration
 - On every 4th iteration have two misses

Thrashing Example: Bad Case



- Access Pattern
 - Read $x[0]$
 - $x[0]$, $x[1]$, $x[2]$, $x[3]$ loaded
 - Read $y[0]$
 - $y[0]$, $y[1]$, $y[2]$, $y[3]$ loaded
 - Read $x[1]$
 - $x[0]$, $x[1]$, $x[2]$, $x[3]$ loaded
 - Read $y[1]$
 - $y[0]$, $y[1]$, $y[2]$, $y[3]$ loaded
 - 8 misses / 8 reads
- Analysis
 - $x[i]$ and $y[i]$ map to same cache lines
 - Miss rate = 100%
 - Two memory accesses / iteration
 - On *every* iteration have two misses

- **Average access time**
- Average access time, t_a , given by:
- $t_a = t_c + (1 - h)t_m$
- assuming again that the first access must be to the cache before an access is made to the main memory. Only read requests are consider so far.
- **Example**
- If hit ratio is 0.85 (a typical value), main memory access time is 50 ns and cache access time is 5 ns, then average access time is $5 + 0.15 * 50 = 12.5$ ns.
- **THROUGHOUT t_c is the time to access the cache, get (or write) the data if a hit or recognize a miss. In practice, these times could be different.**

Example-1

- Assume
 - Hit Time = 1 cycle
 - Miss rate = 5%
 - Miss penalty = 20 cycles
 - Average memory access time = $t_c + m * t_m$
(m:miss rate, h:hit rate,
 t_m : main memory access time, t_c : Cache access time)
Average memory access time = $1 + 0.05 \times 20 = 2$ cycle
- or
- $= t_c * h + (t_m + t_c) * m$
 $1 \times 0.95 + 21 \times 0.05 = 2$ cycle

Example-2

- Assume
 - L1 Hit Time = 1 cycle
 - L1 Miss rate = 5%
 - L2 Hit Time = 5 cycles
 - L2 Miss rate = 15% (% L1 misses that miss)
 - L2 Miss Penalty = 100 cycles
- L1 miss penalty = $5 + 0.15 * 100 = 20$
- Average memory access time = $1 + 0.05 * 20$
= 2 cycle

Replacement Algorithms

- *When a block is fetched, which block in the target set should be replaced?*
- Optimal algorithm:
 - replace the block that will not be used for the longest time (must know the future)
- Usage based algorithms:
 - Least recently used (LRU)
 - replace the block that has been referenced least recently
 - hard to implement
- Non-usage based algorithms:
 - First-in First-out (FIFO)
 - treat the set as a circular queue, replace head of queue.
 - easy to implement
 - Random (RAND)
 - replace a random block in the set
 - even easier to implement

Implementing RAND and FIFO

- FIFO:
 - maintain a modulo E counter for each set.
 - counter in each set points to next block for replacement.
 - increment counter with each replacement.
- RAND:
 - maintain a single modulo E counter.
 - counter points to next block for replacement in any set.
 - increment counter according to some schedule:
 - each clock cycle, each mem reference, or each replacement.
- LRU
 - Need state machine for each set
 - Encodes usage ordering of each element in set
 - $E!$ possibilities $\Rightarrow \sim E \log E$ bits of state

What Happens on a Write?



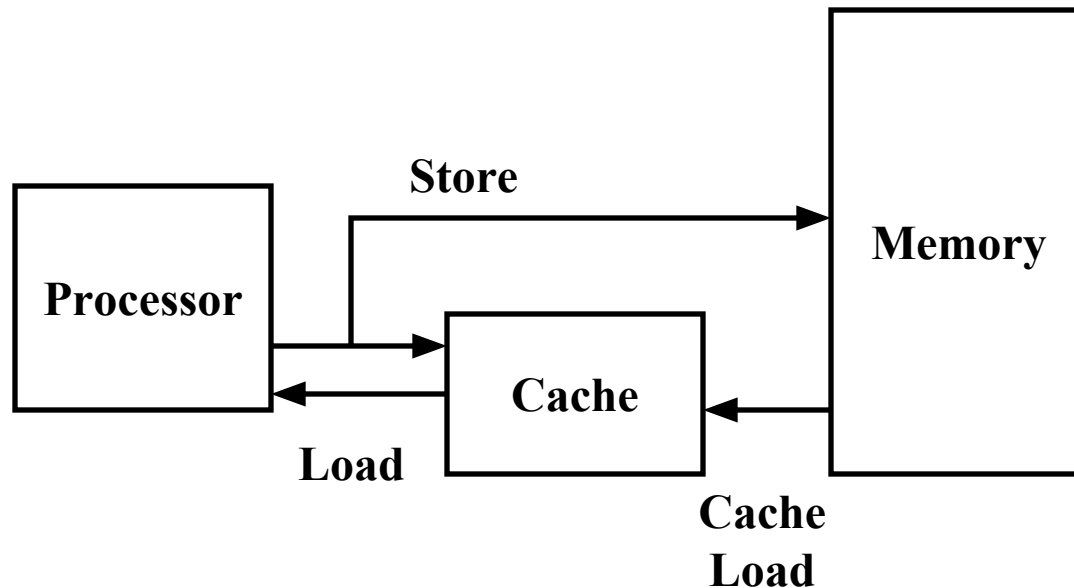
- Need to keep cache consistent with the main memory
 - Reads are easy - require no modification
 - Writes- when does the update occur
- Write-Through
 - On cache write- always update main memory as well
- Write-Back
 - On cache write- remember that block is modified (dirty)
 - Update main memory when dirty block is replaced
 - Sometimes need to flush cache (I/O, multiprocessing)

What to do on a write hit?

- Write-through
 - update the word in cache block and corresponding word in memory
- Write-back
 - update word in cache block
 - allow memory word to be "stale"
 - => add 'dirty' bit to each line indicating that memory needs to be updated when block is replaced
 - => OS flushes cache before I/O !!!
- Performance trade-offs?

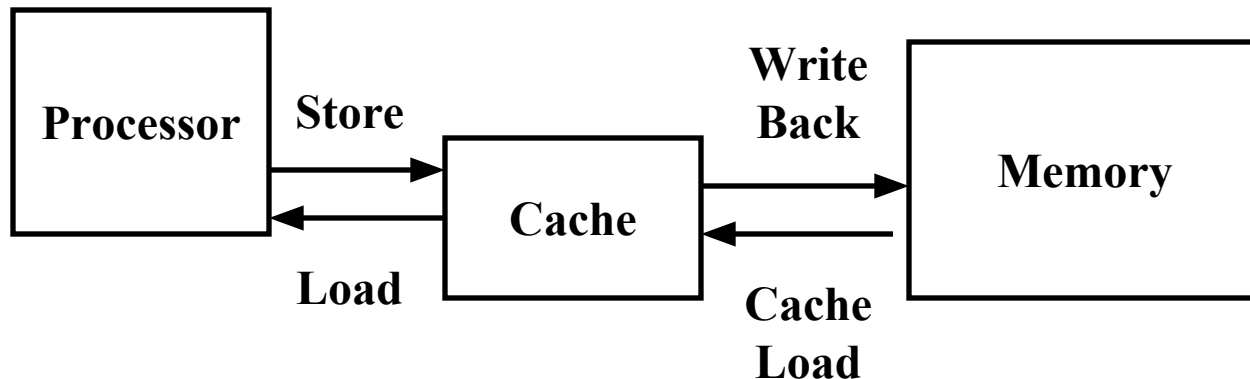
Write Through

- Store by processor updates cache *and* memory
- Memory always consistent with cache
- ~2X more loads than stores
- WT always combined with write buffers so that don't wait for lower level memory



Write Back

- Store by processor only updates cache line
- Modified line written to memory only when it is evicted
 - Requires “dirty bit” for each line
 - Set when line in cache is modified
 - Indicates that line in memory is stale
- Memory not always consistent with cache
- No writes of repeated writes

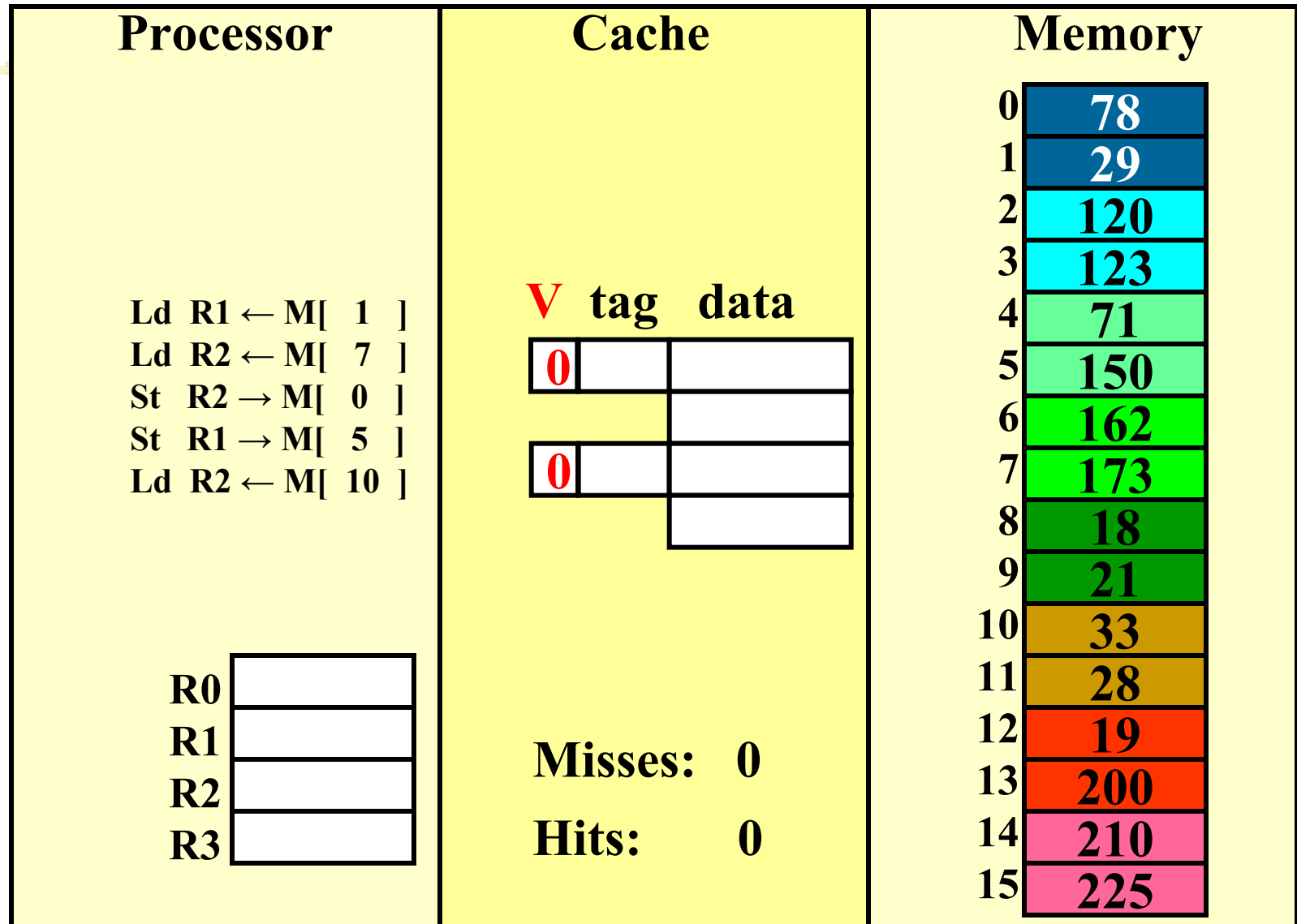


Store Miss?

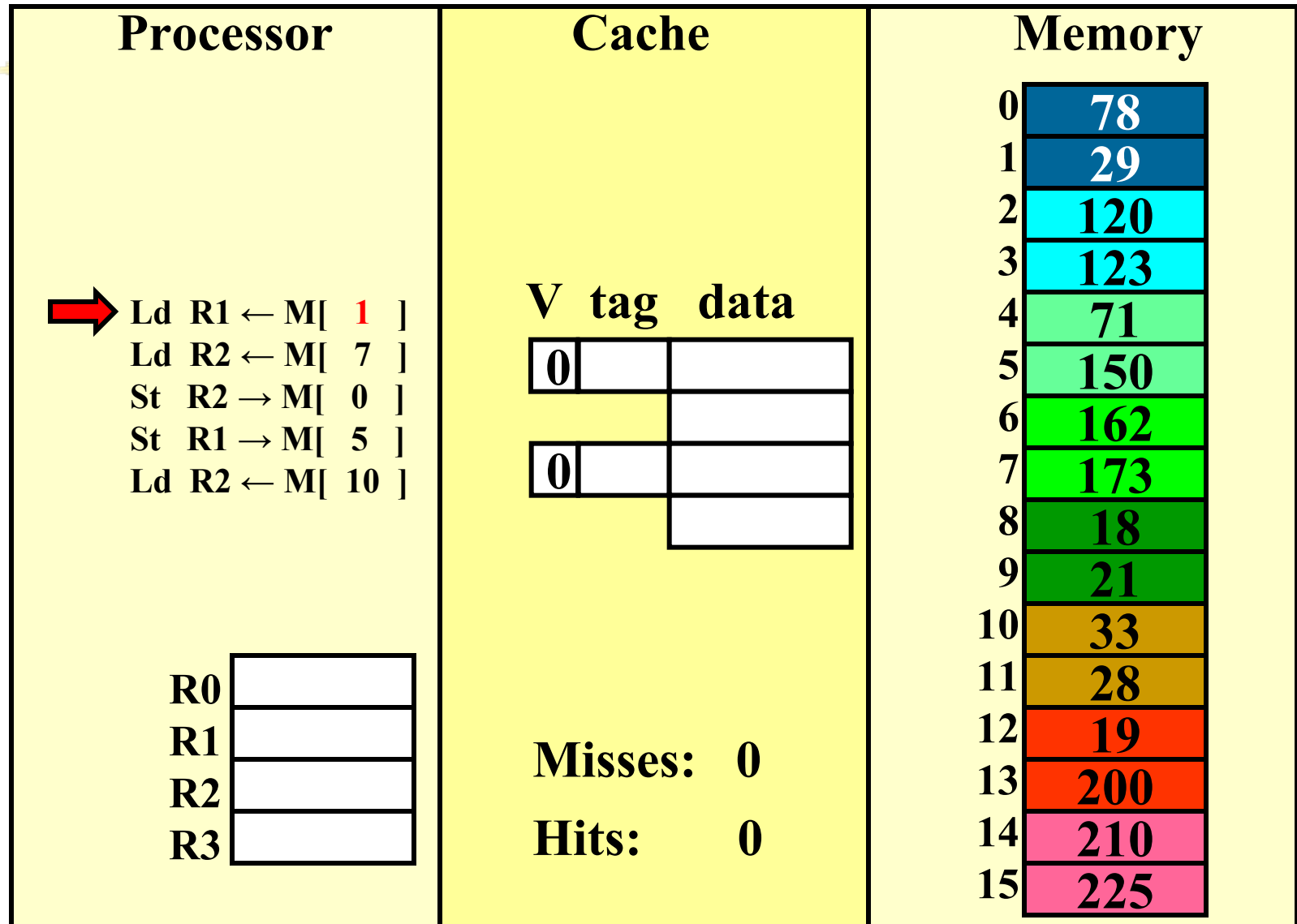


- Write-Allocate
 - Bring written block into cache
 - Update word in block
 - Anticipate further use of block
- No-write Allocate
 - Main memory is updated
 - Cache contents unmodified

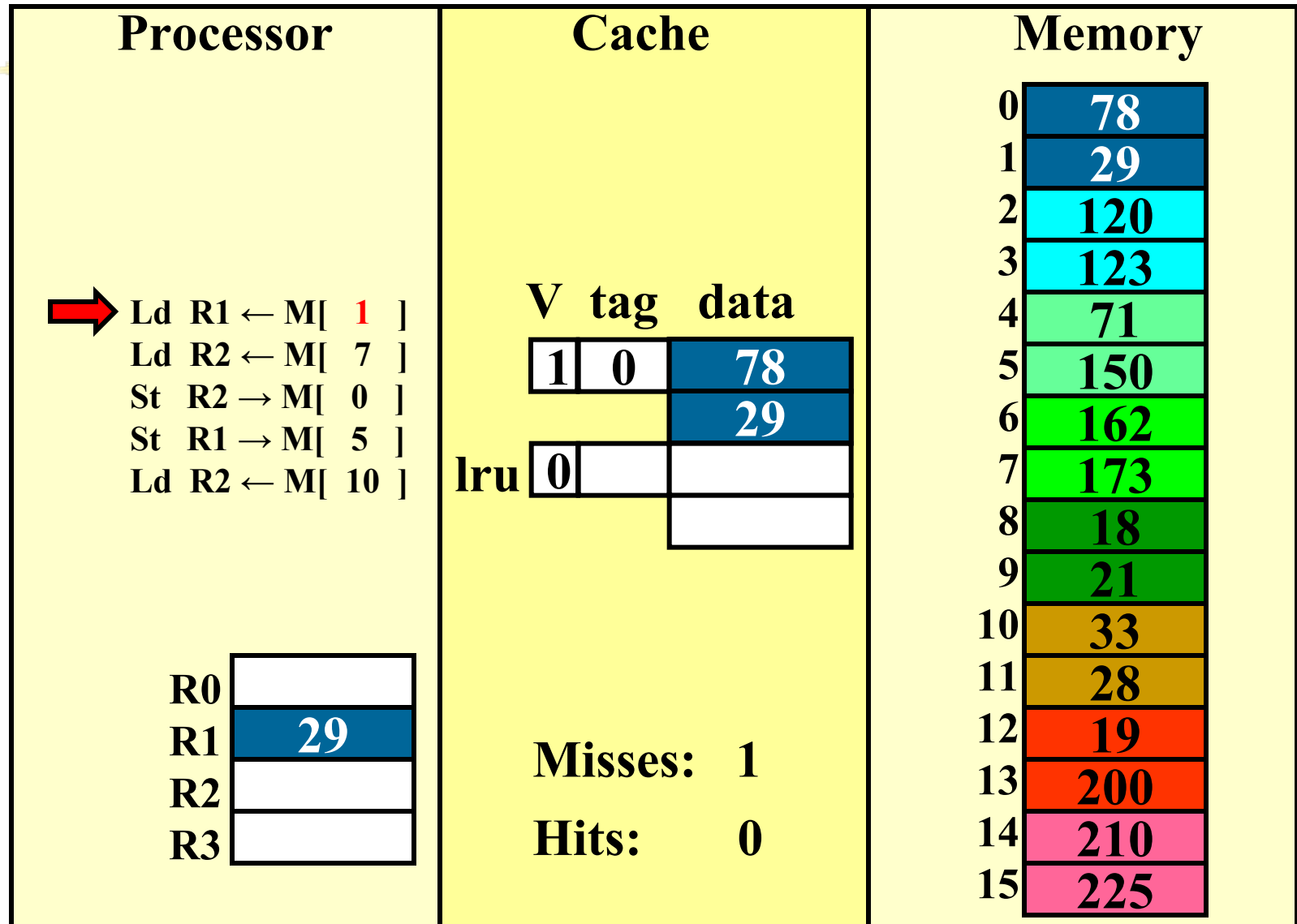
Handling stores (write-through)



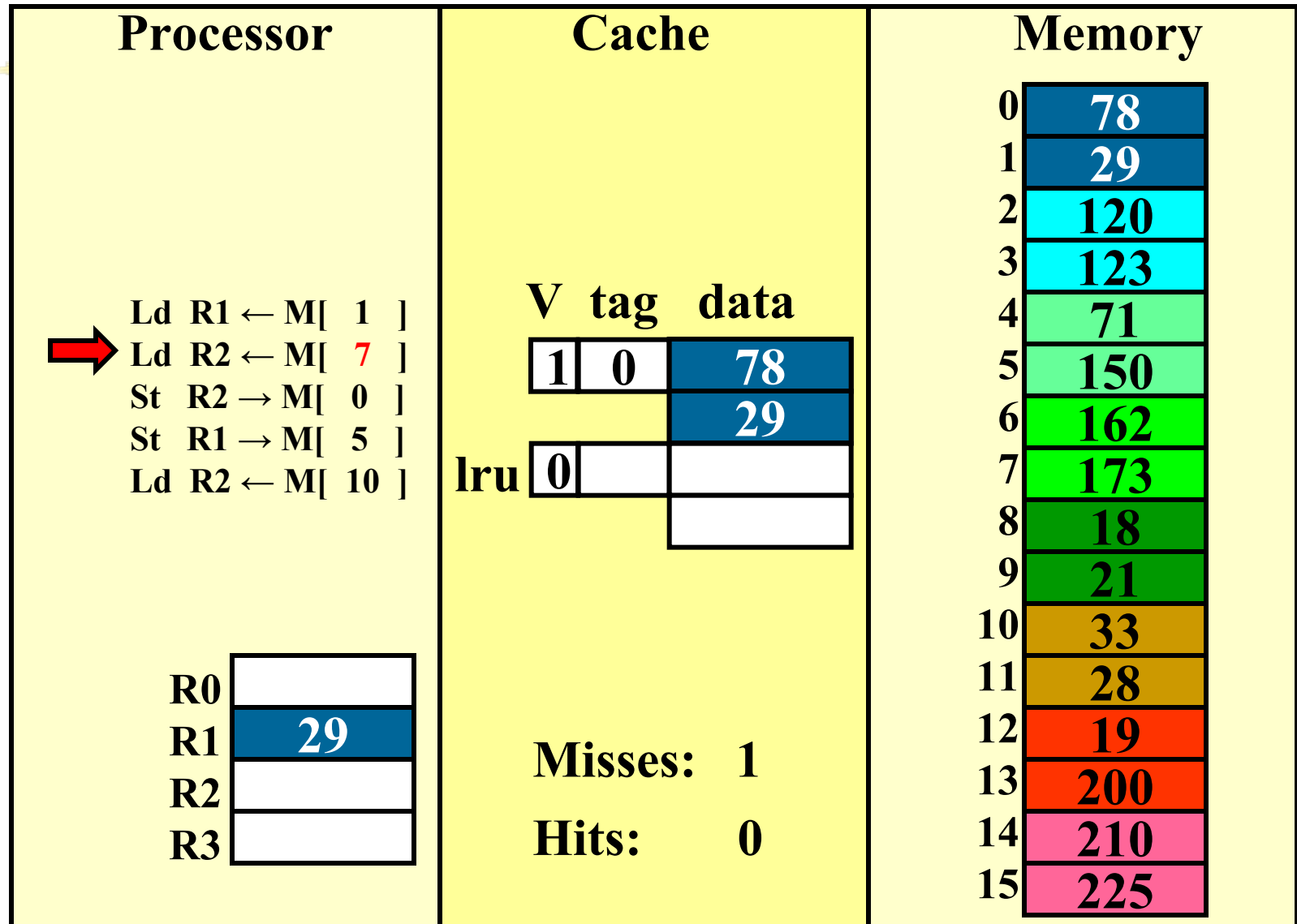
write-through (REF 1)



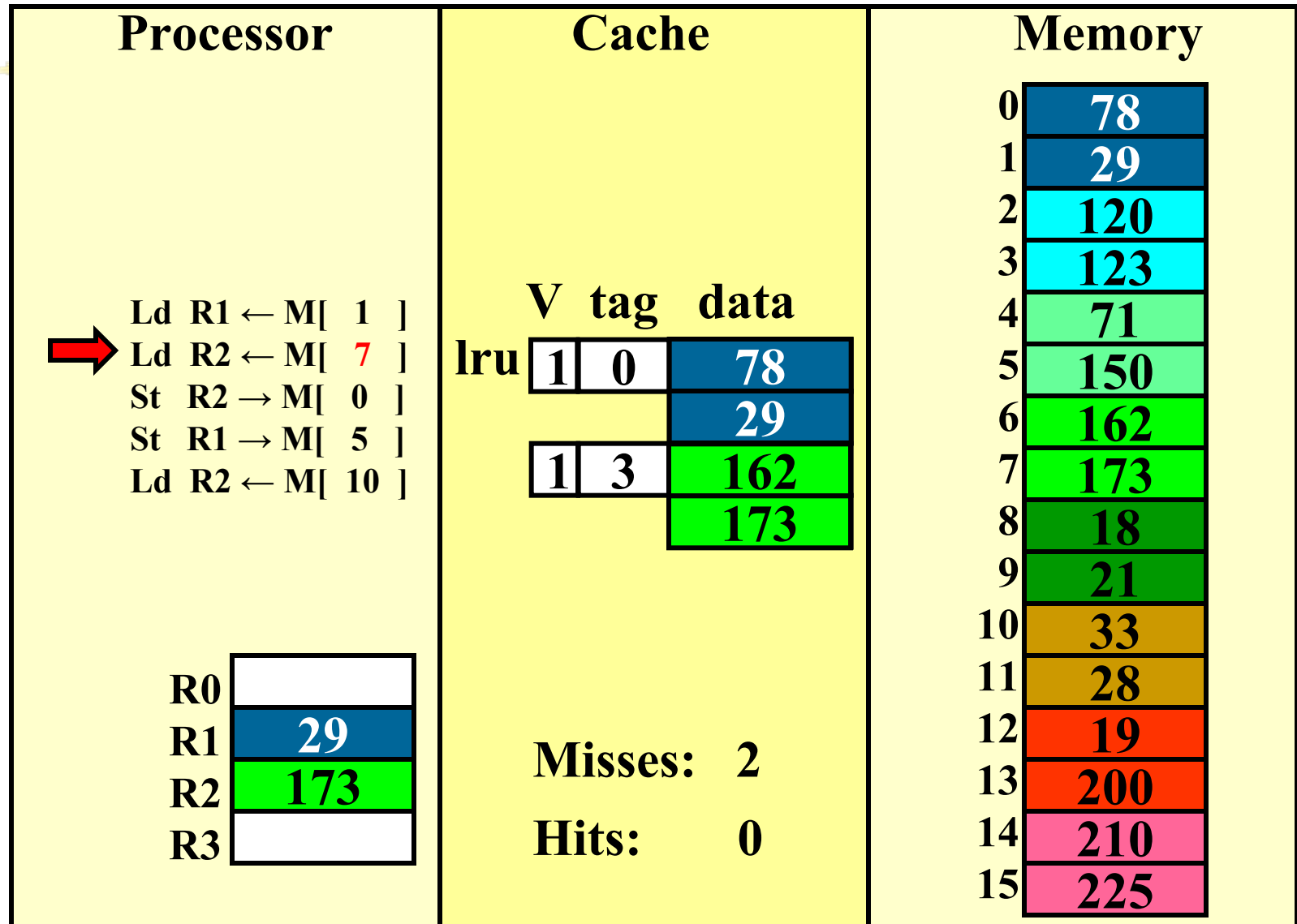
write-through (REF 1)



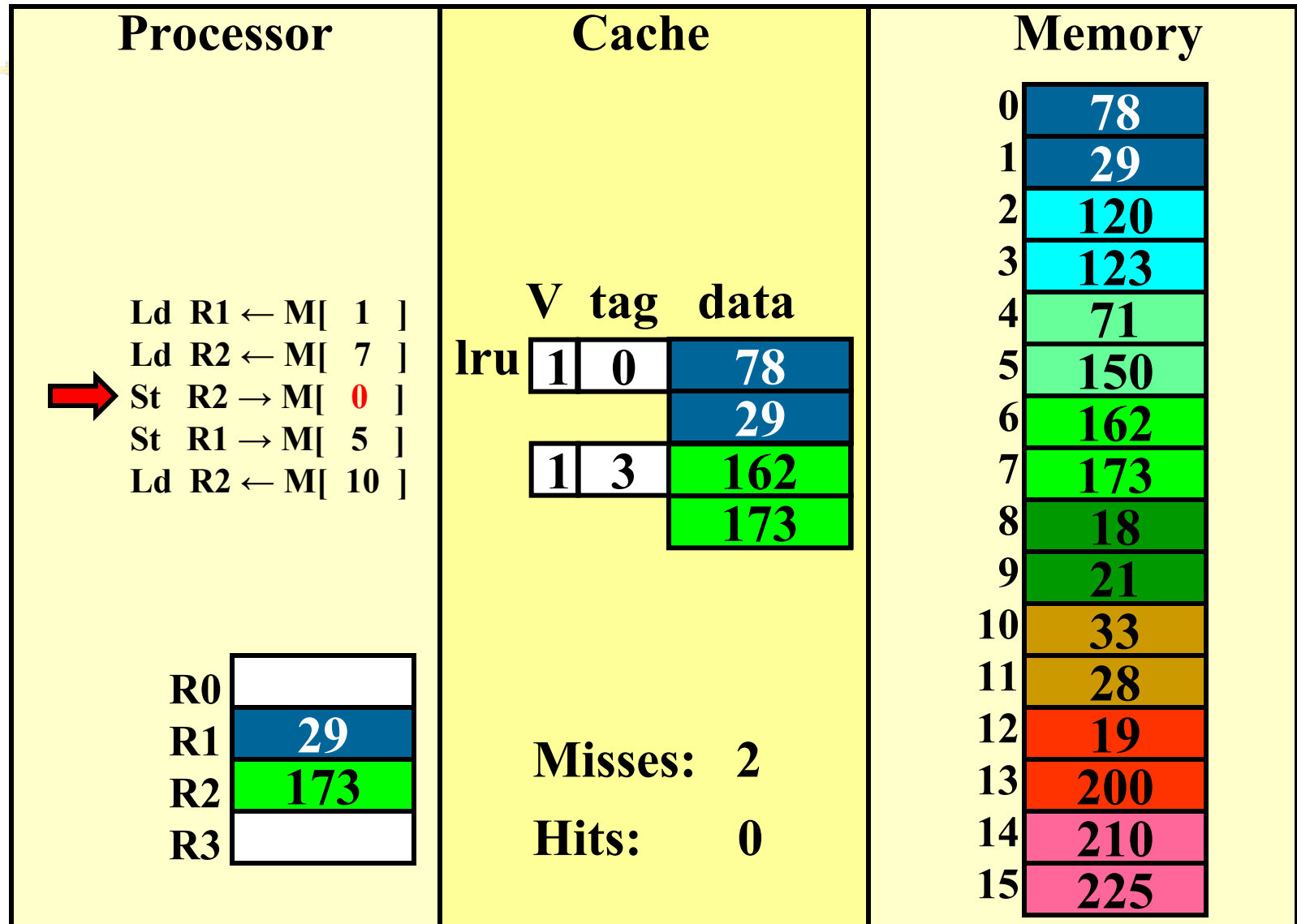
write-through (REF 2)



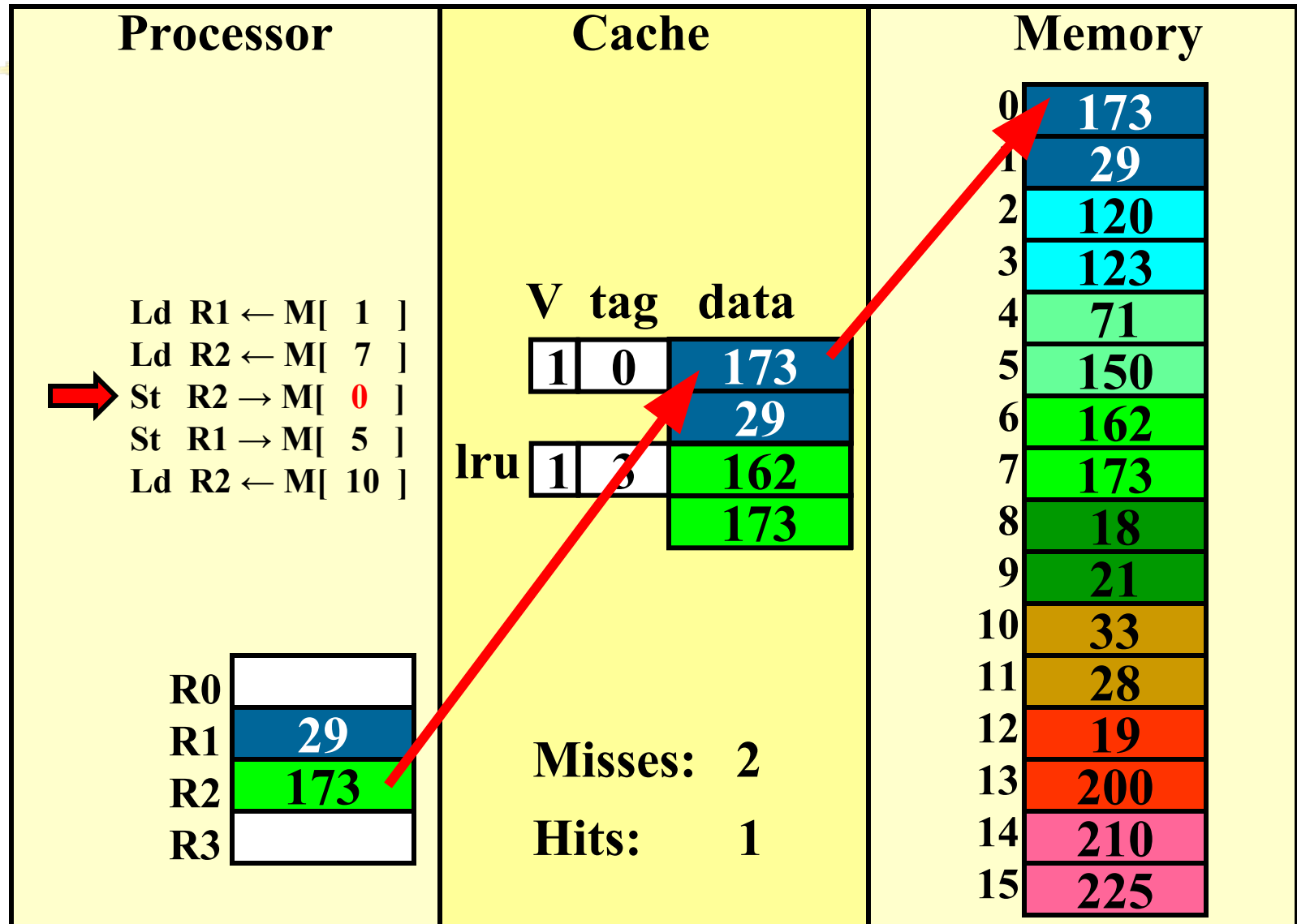
write-through (REF 2)



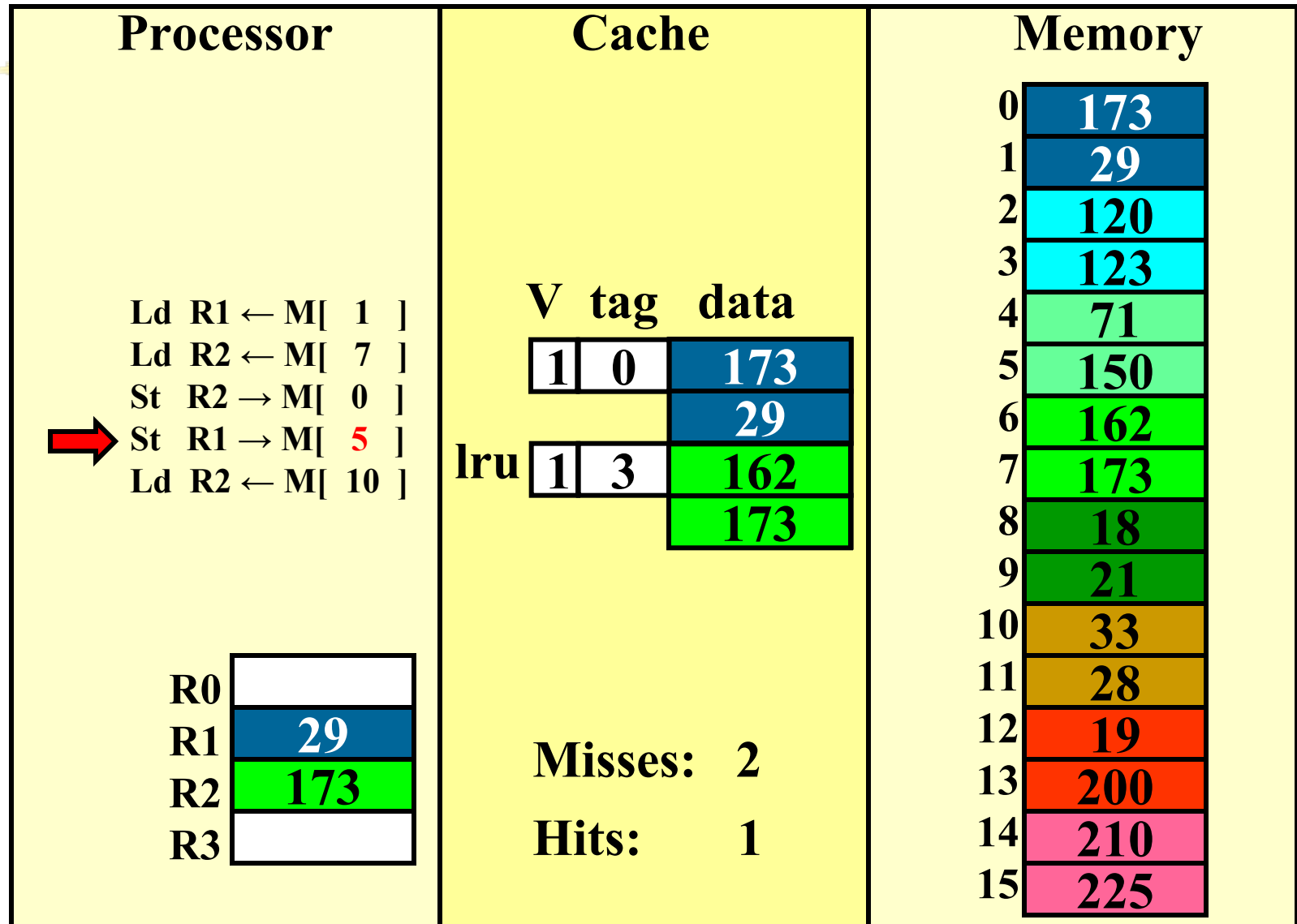
write-through (REF 3)



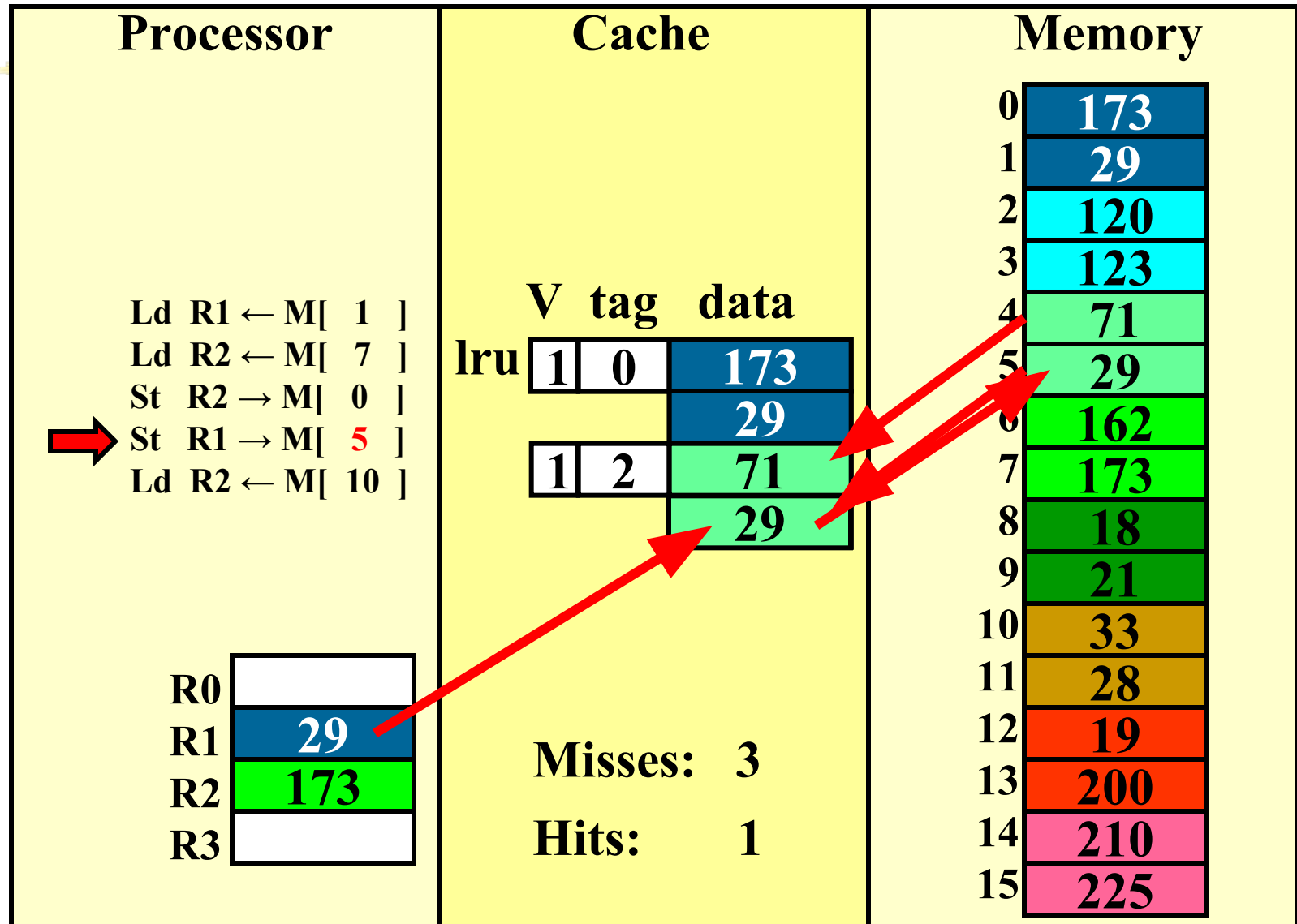
write-through (REF 3)



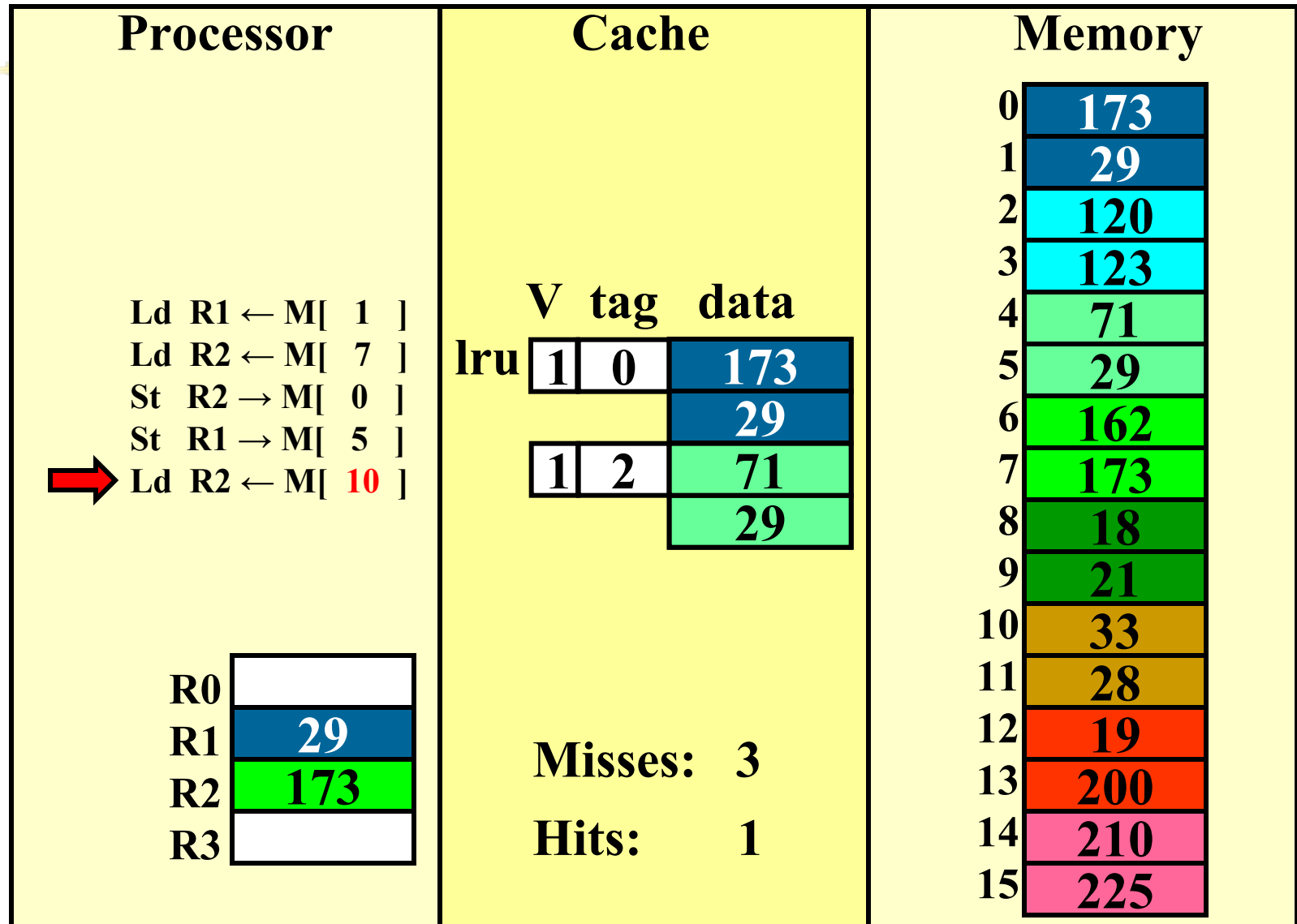
write-through (REF 4)



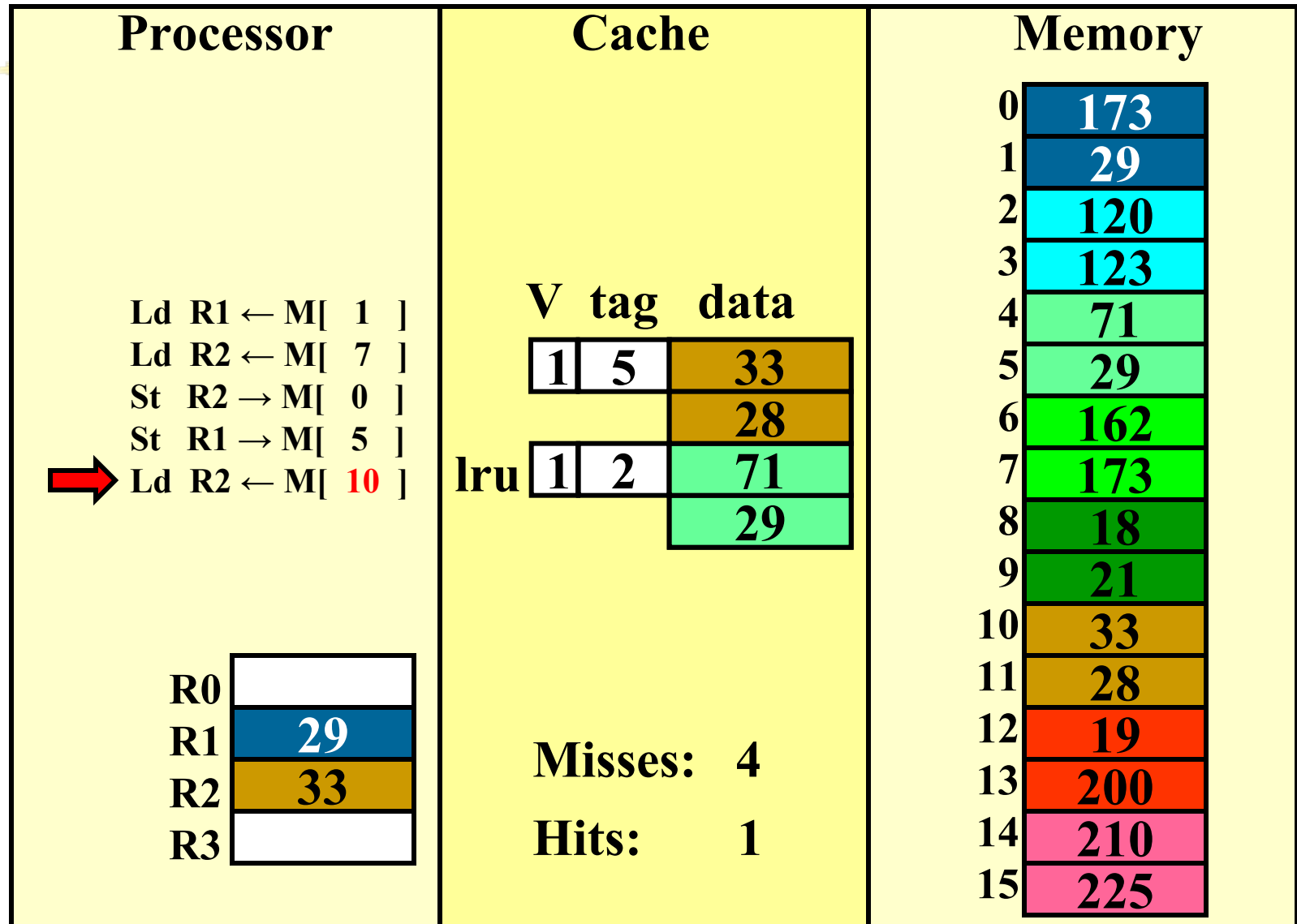
write-through (REF 4)



write-through (REF 5)



write-through (REF 5)



How many memory references?



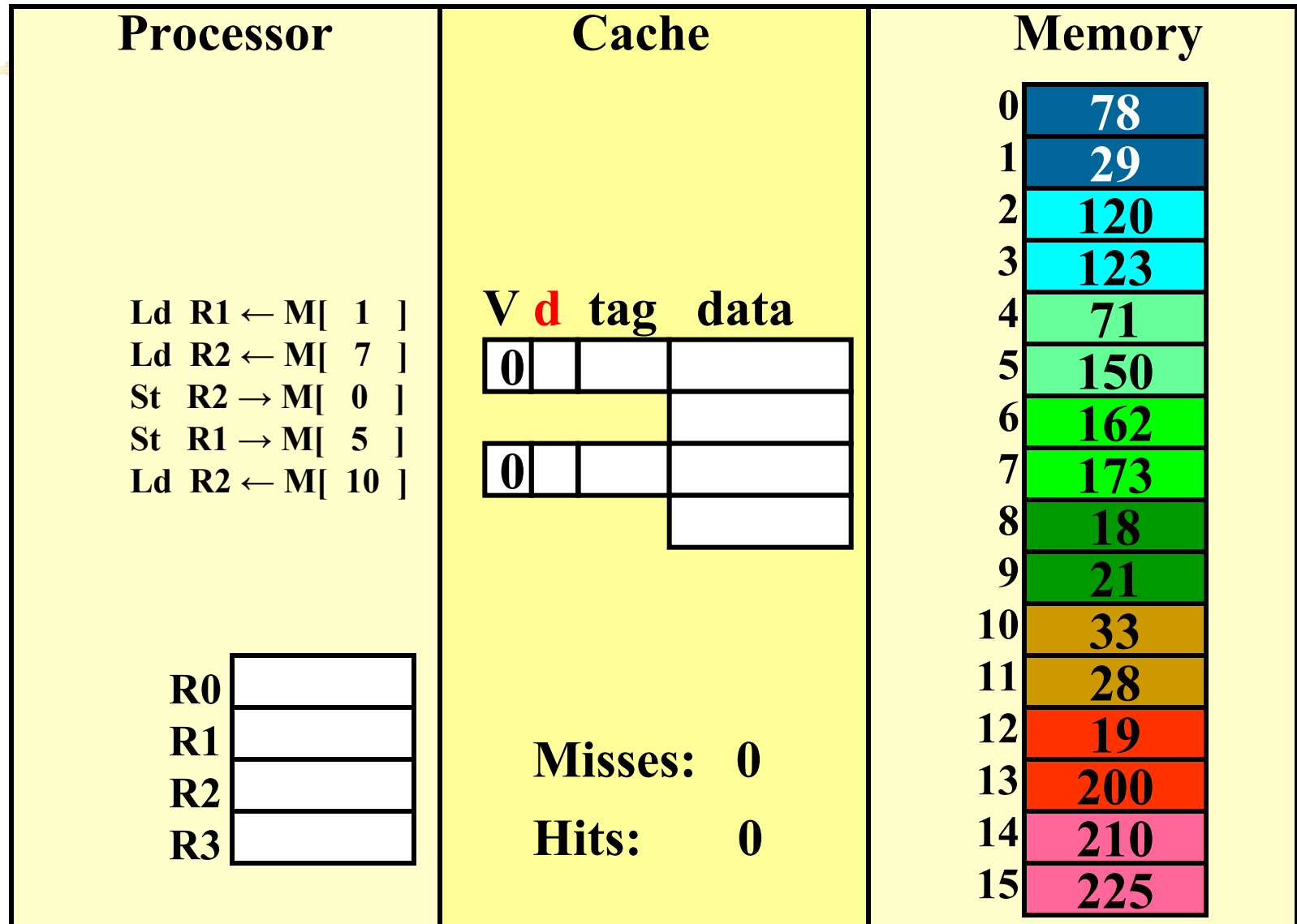
- Each miss reads a block
 - 2 bytes in this cache
- Each store writes a byte
- Total reads: 8 bytes
- Total writes: 2 bytes

but caches generally miss $< 20\%$

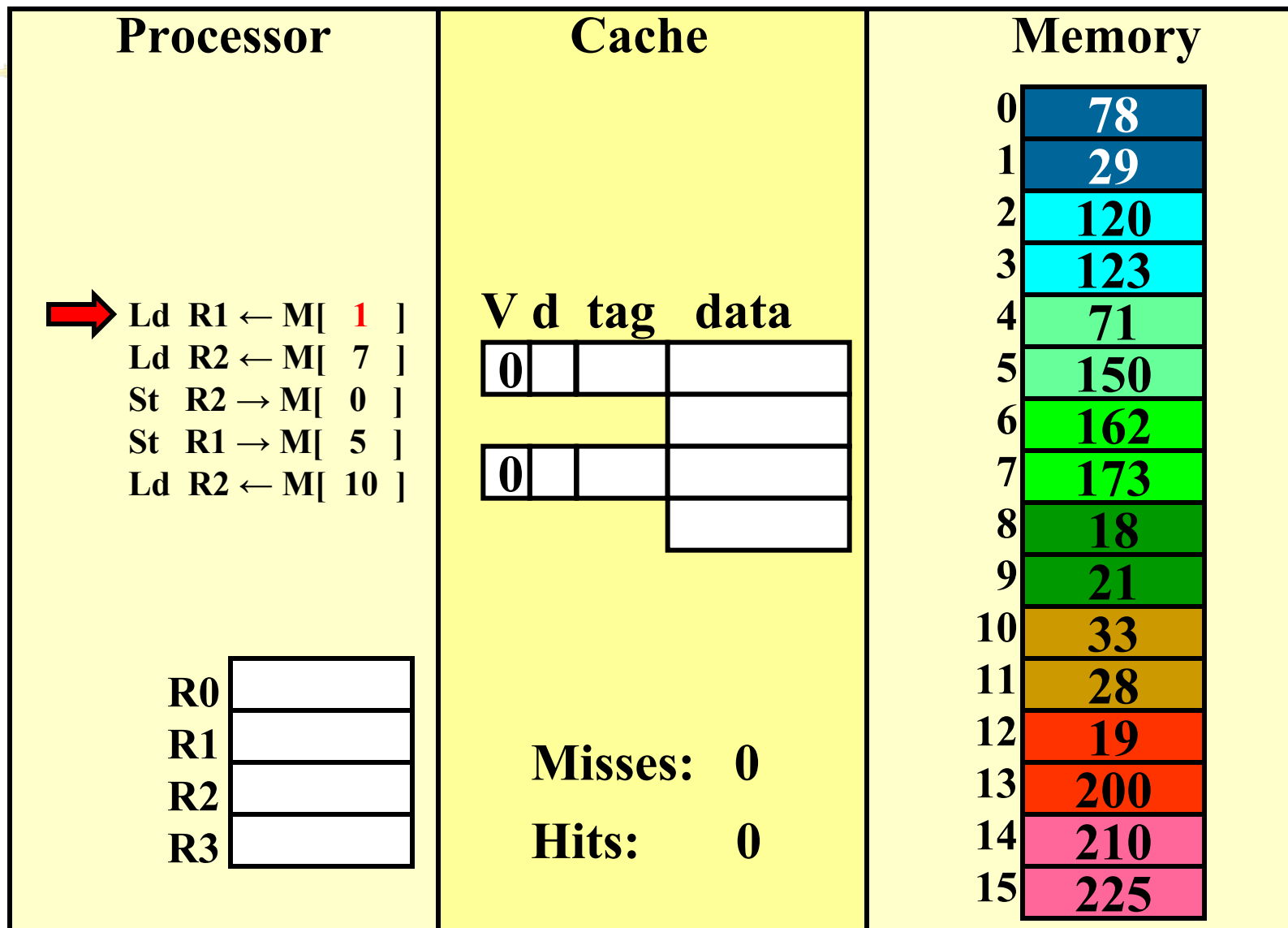
Write-through vs. write-back

- Do the cache to **NOT** write all stores to memory immediately?
 - Keep the most current copy in the cache and update the memory when that data is evicted from the cache (a **write-back** policy).
 - write-back all evicted lines?
 - No, only blocks that have been stored into
 - Keep a “**dirty bit**”, reset when the line is allocated, set when the block is stored into. If a block is “dirty” when evicted, write its data back into memory.

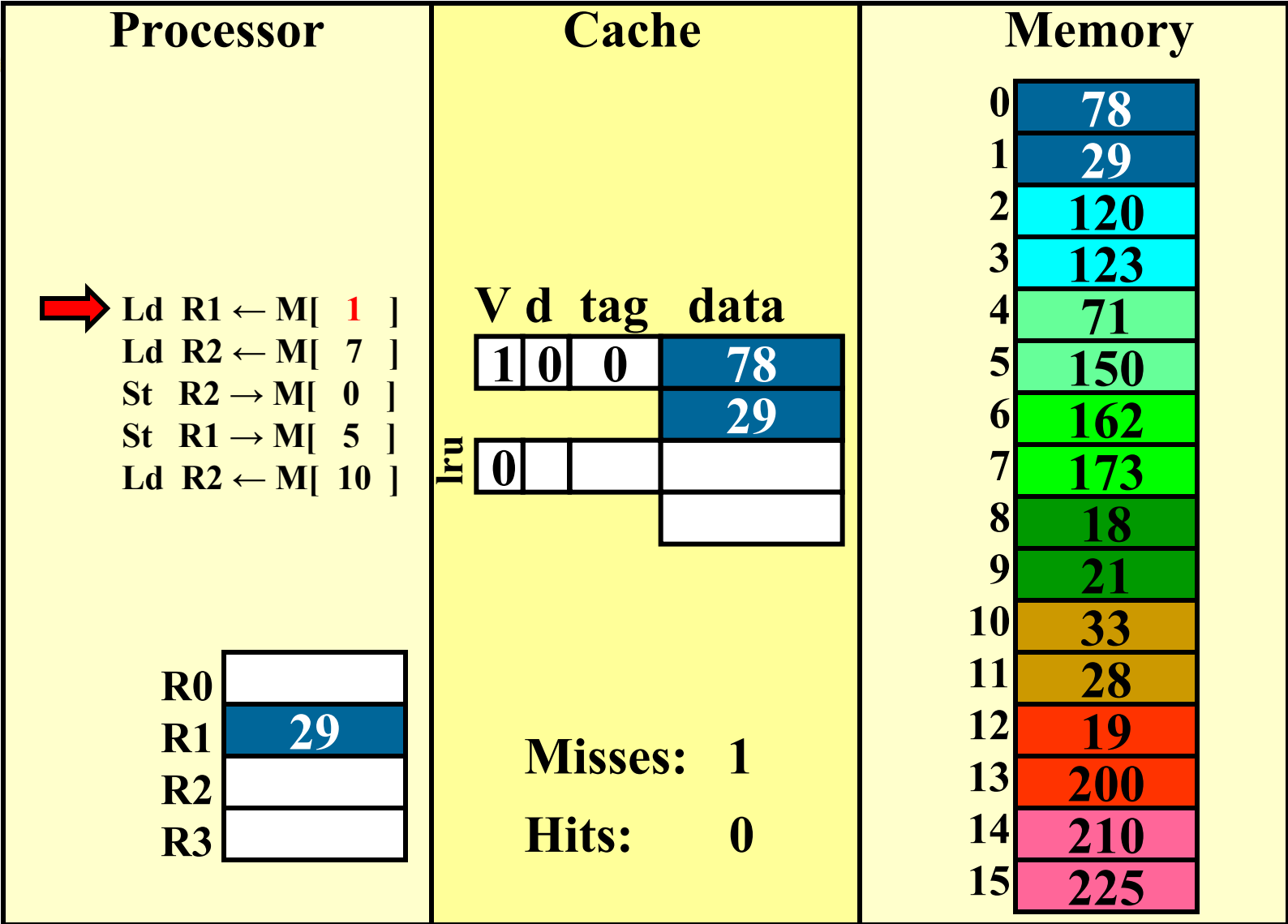
Handling stores (write-back)



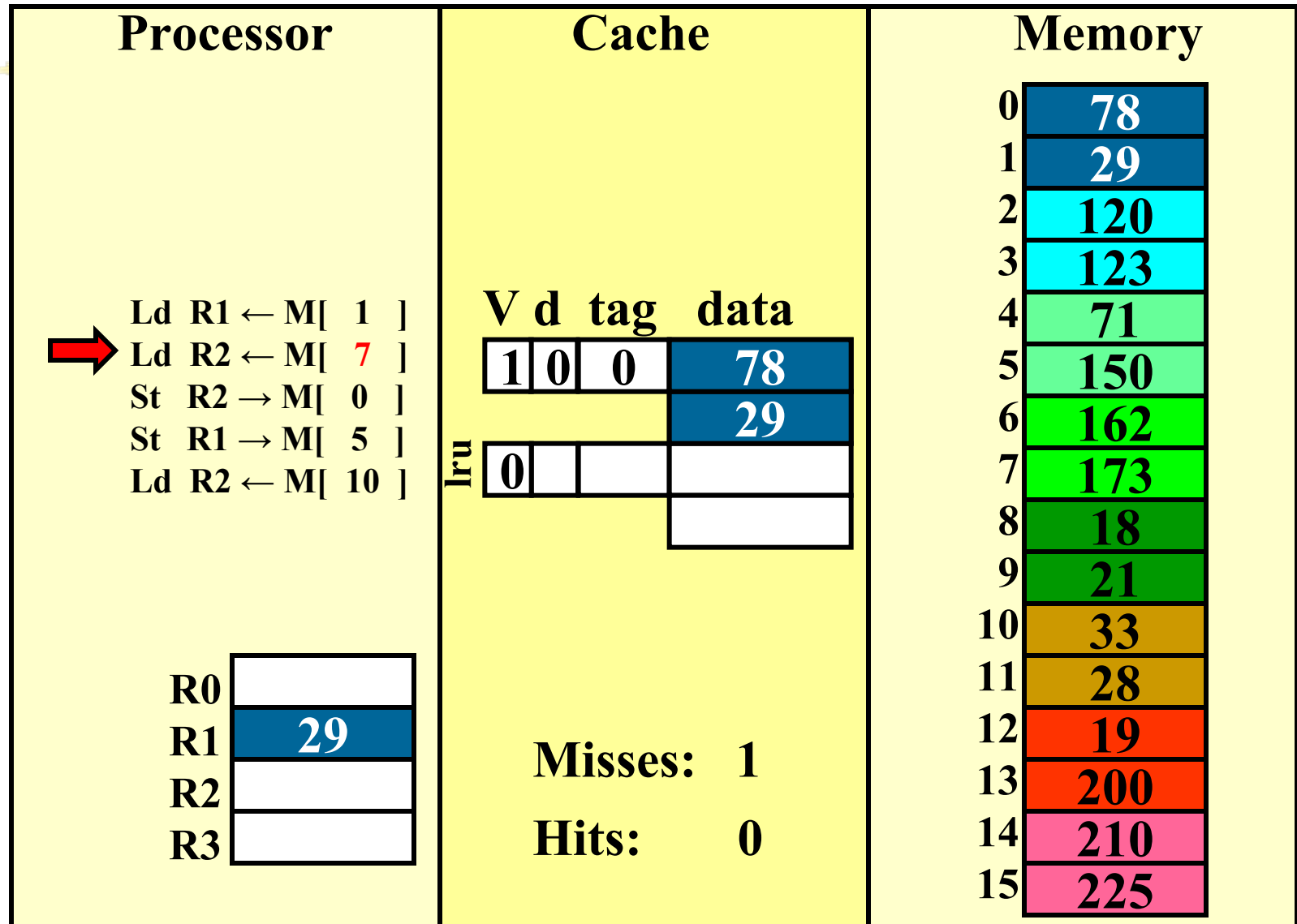
write-back (REF 1)



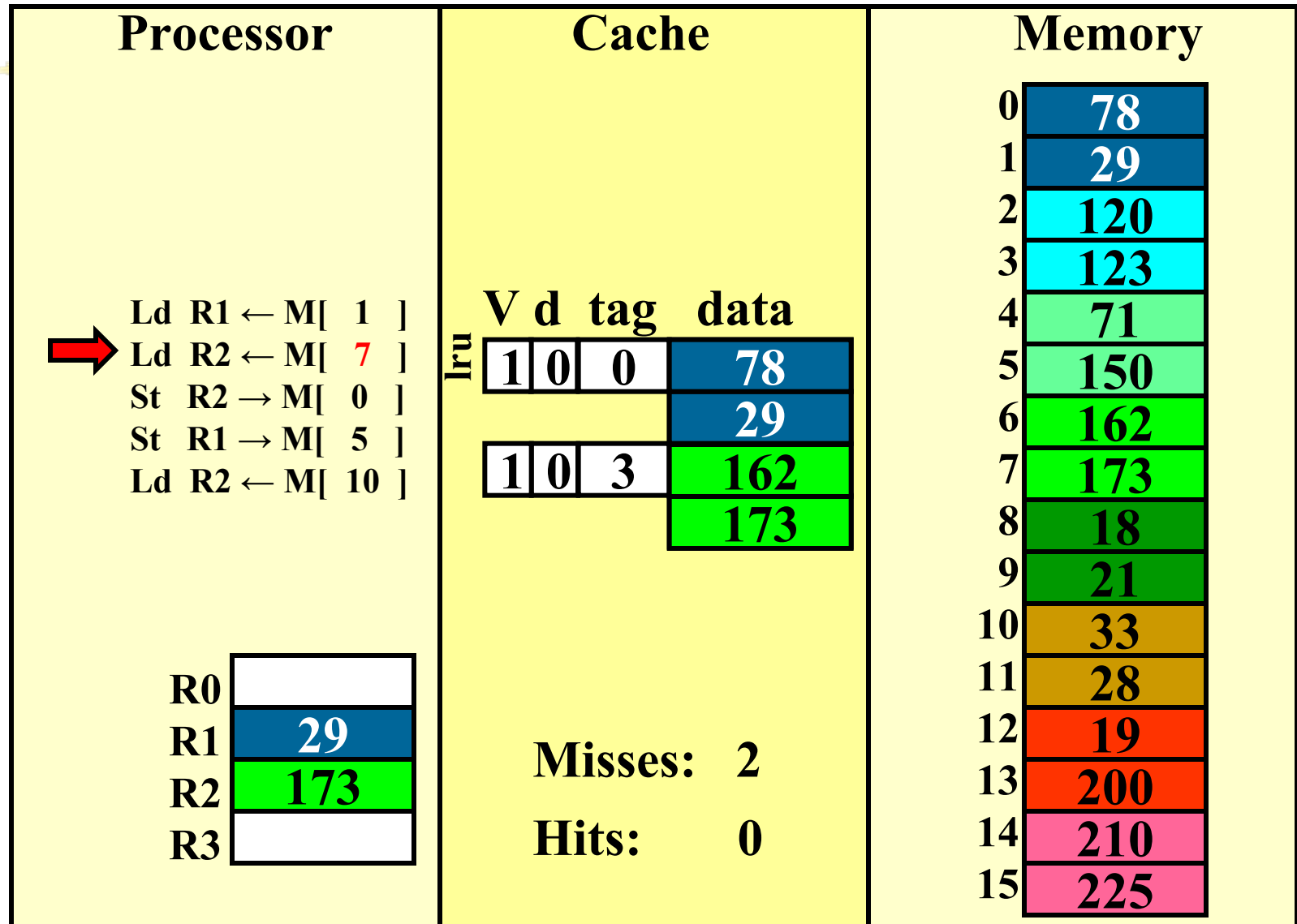
write-back (REF 1)



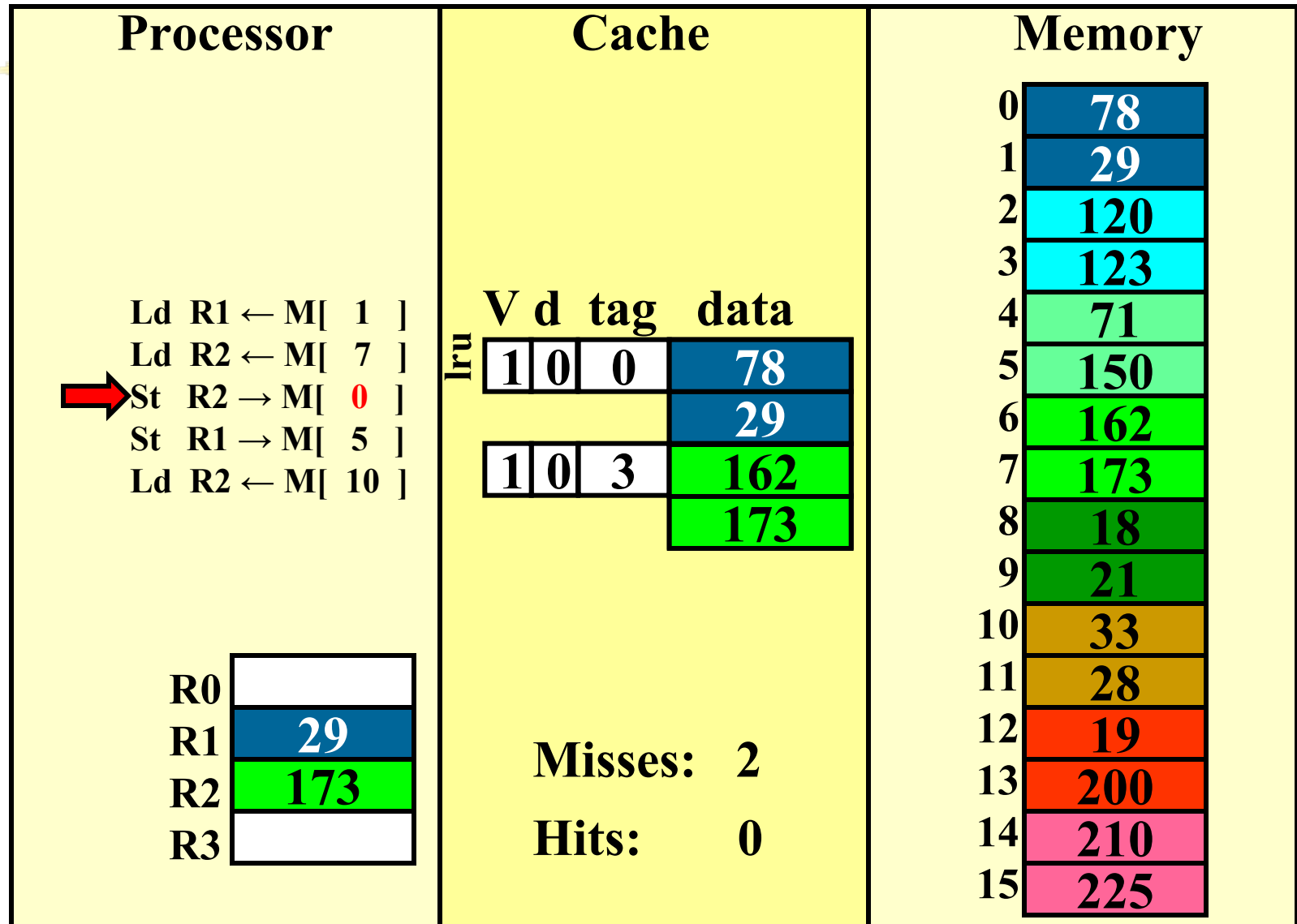
write-back (REF 2)



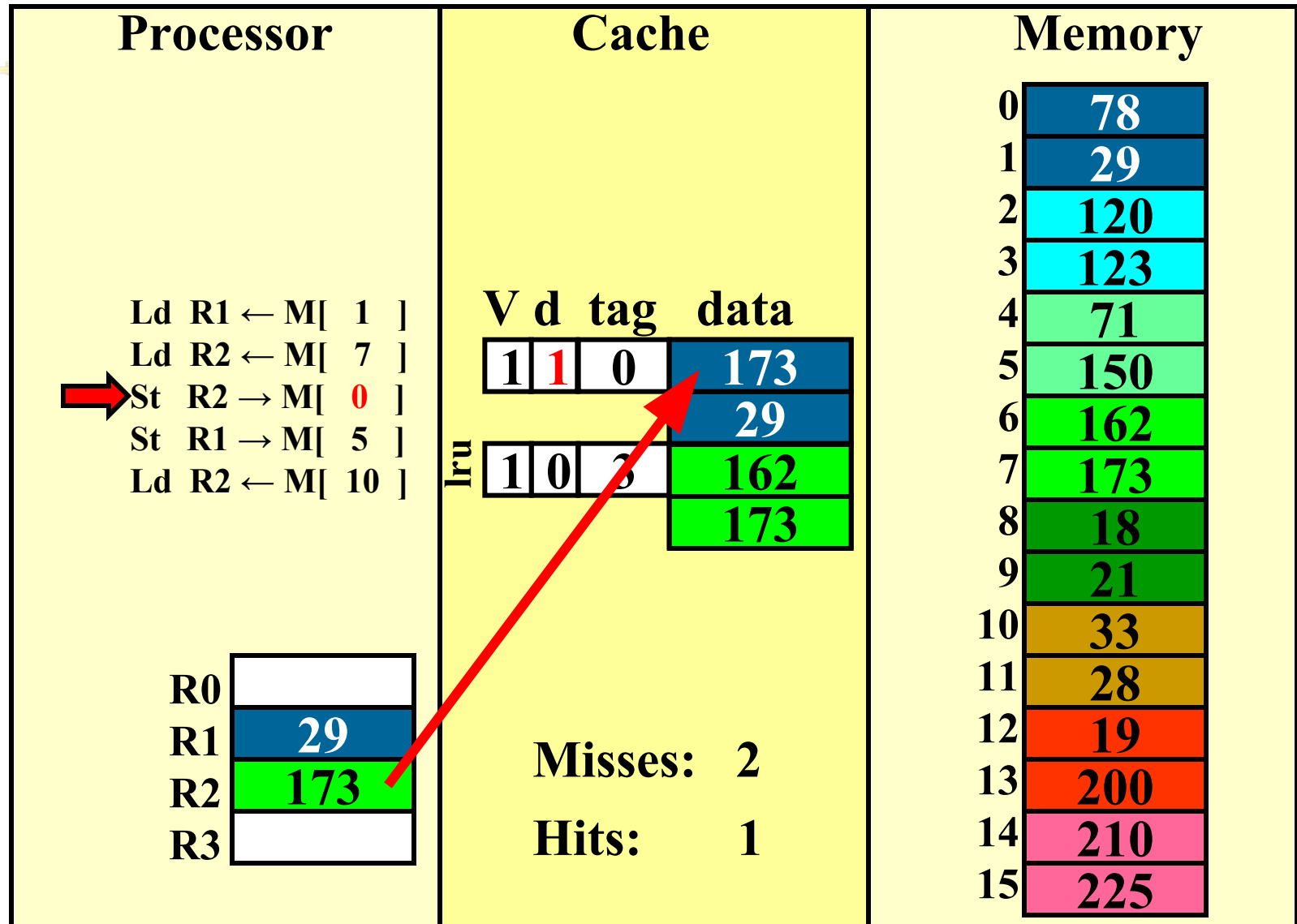
write-back (REF 2)



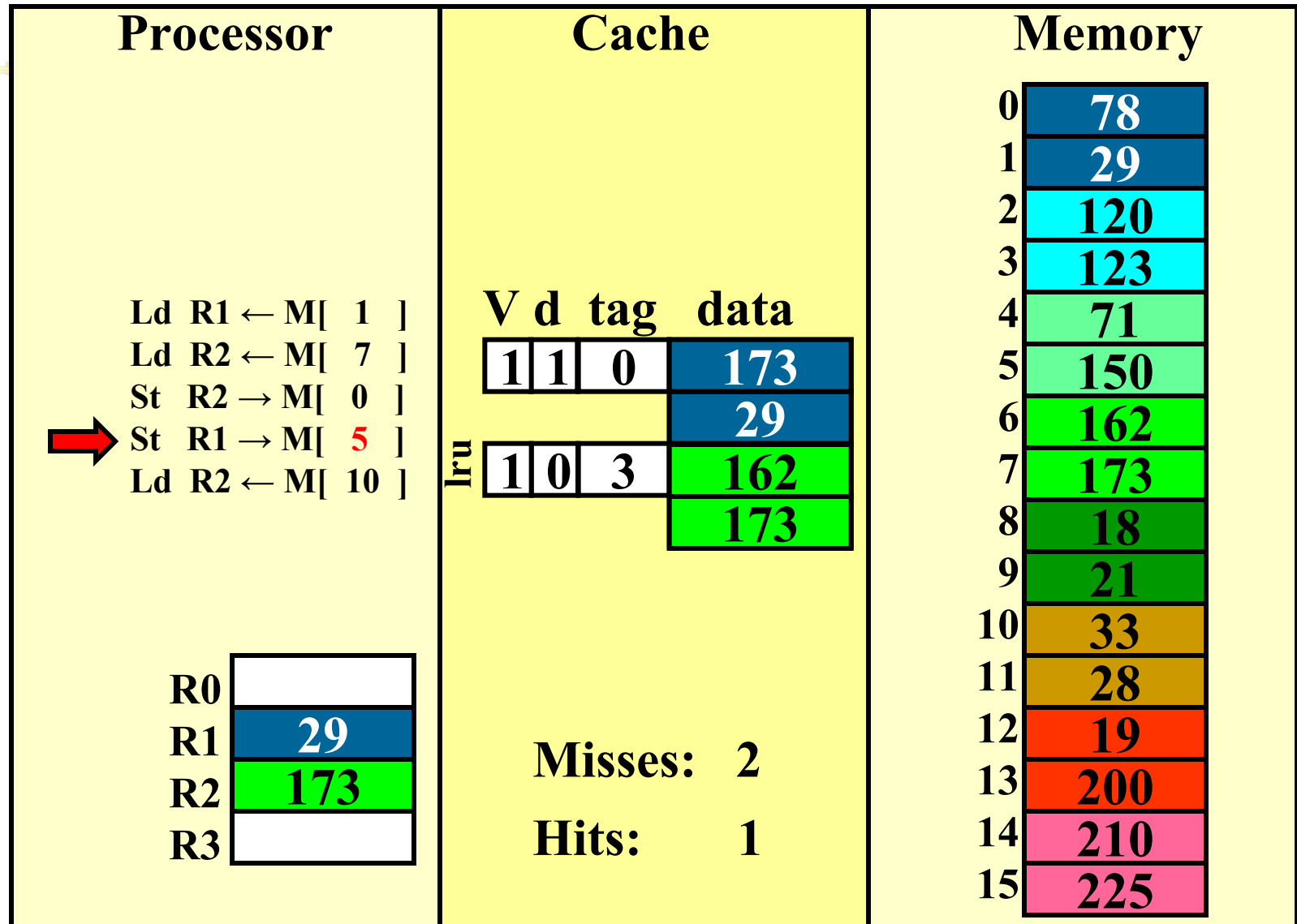
write-back (REF 3)



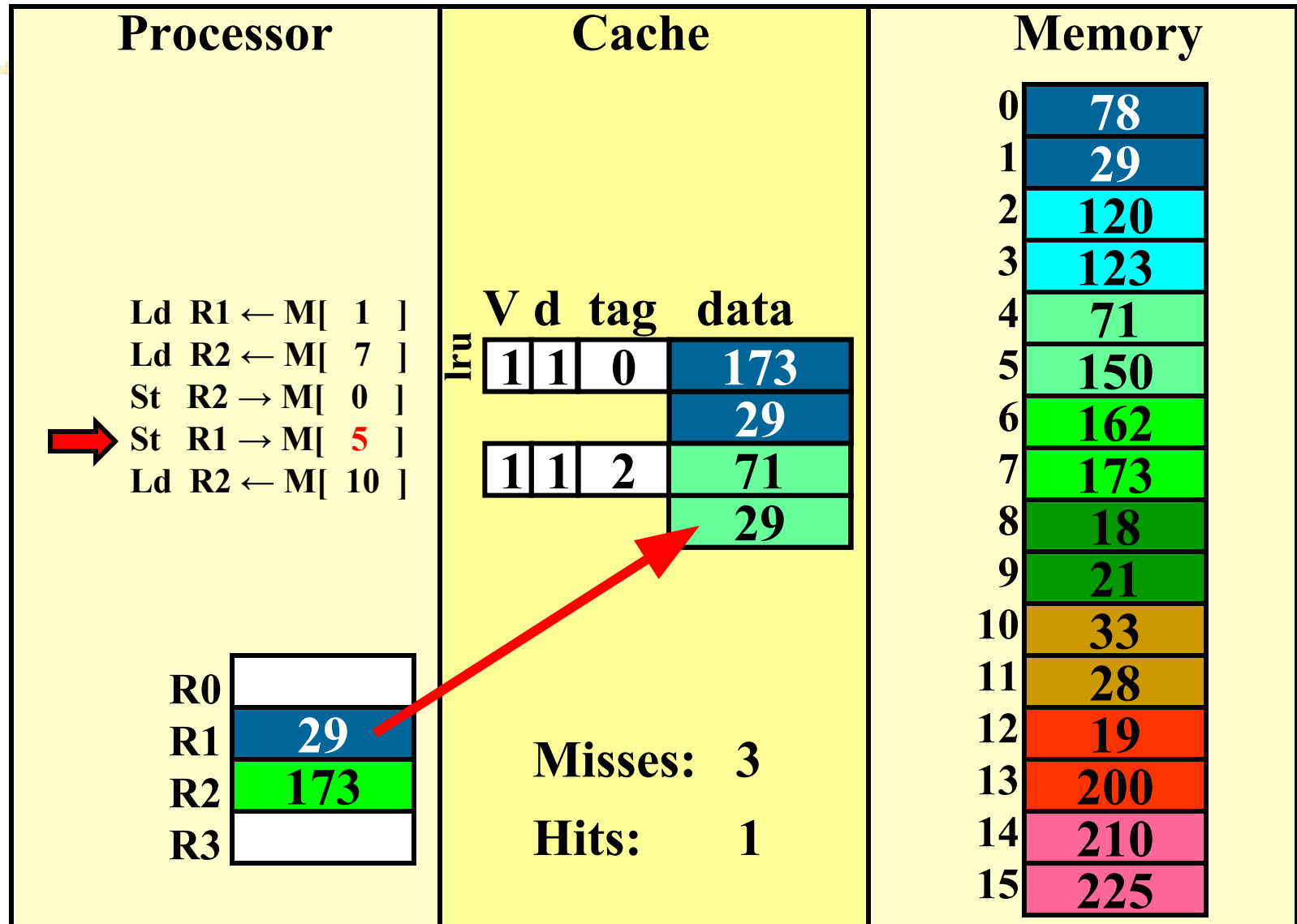
write-back (REF 3)



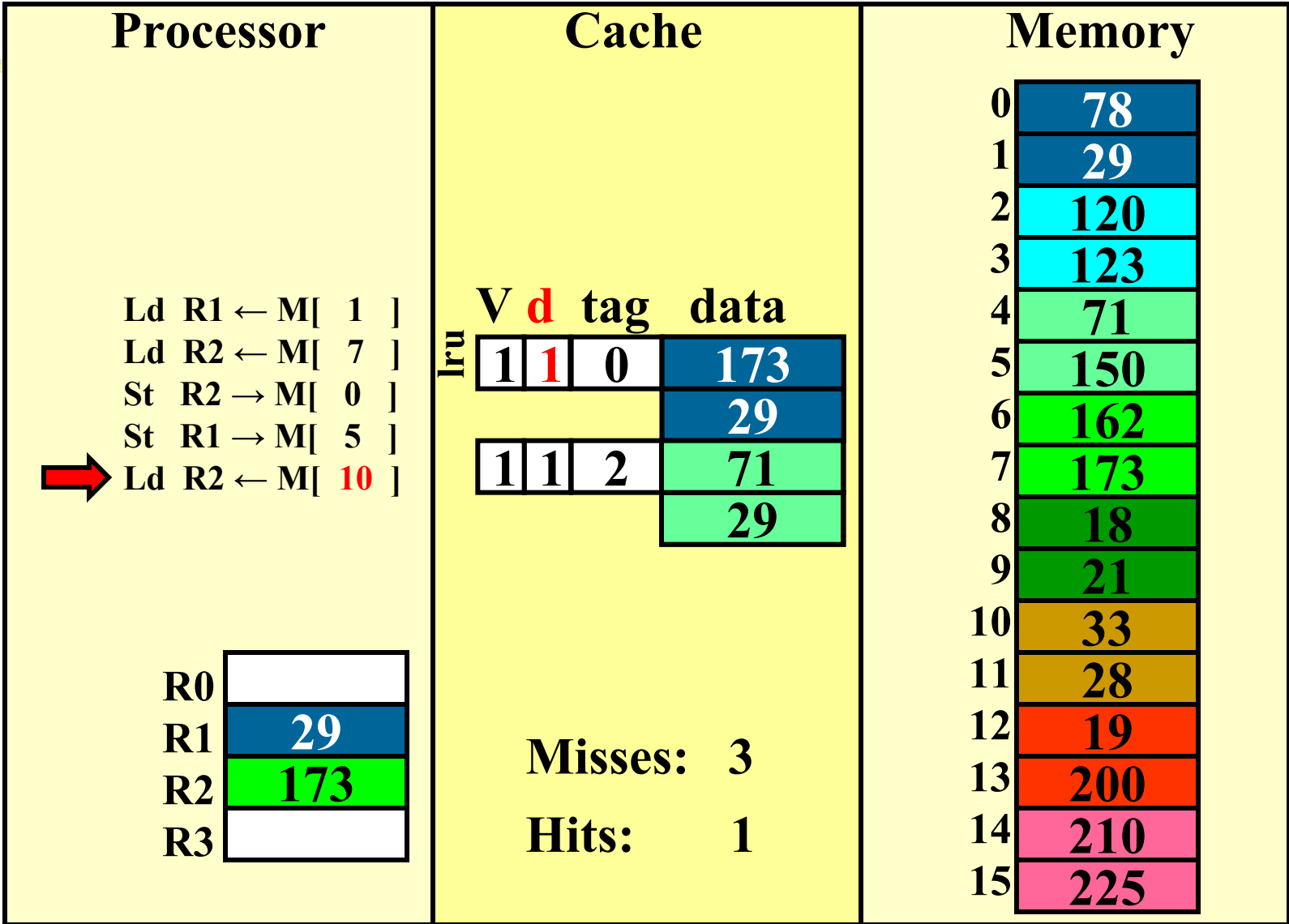
write-back (REF 4)



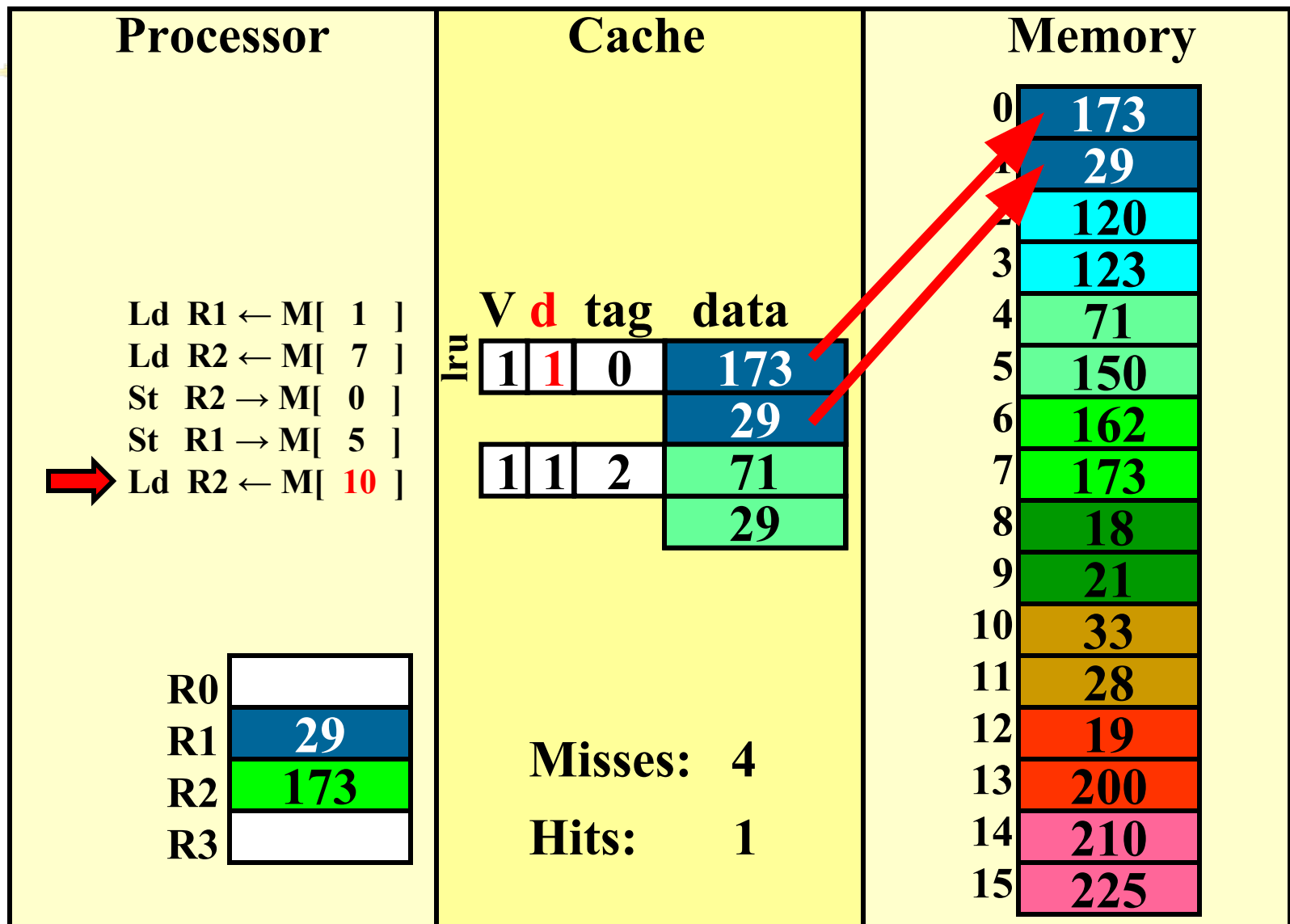
write-back (REF 4)



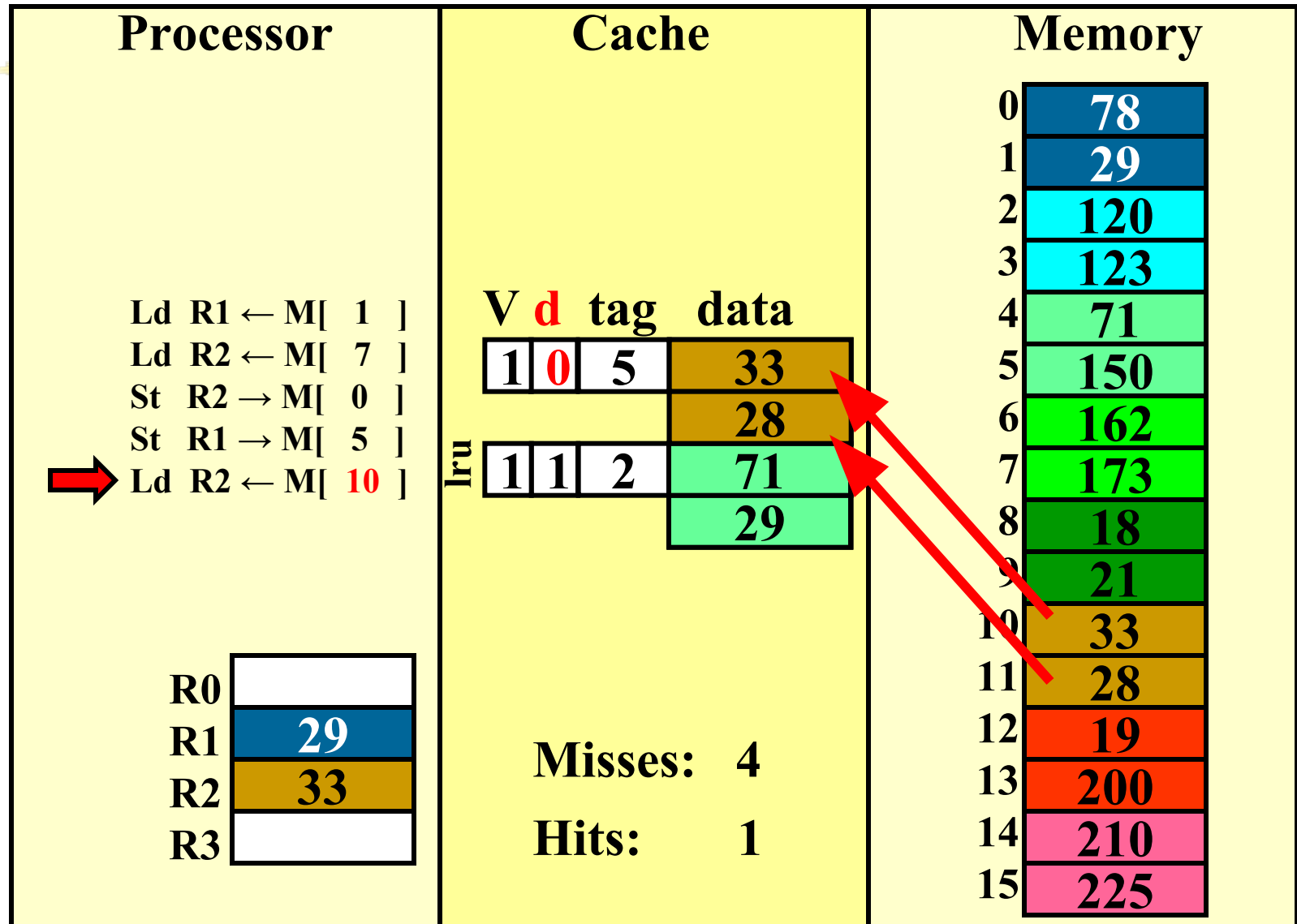
write-back (REF 5)



write-back (REF 5)



write-back (REF 5)



How many memory references?



- Each miss reads a block
 - 2 bytes in this cache
- Each evicted dirty cache line writes a block
- Total reads: 8 bytes
- Total writes: 4 bytes (after final eviction)

Choose write-back or write-through?

Review (1/2)



- Caches are NOT mandatory:
 - Processor performs arithmetic
 - Memory stores data
 - Caches simply make data transfers go *faster*
- Each level of memory hierarchy is just a subset of next higher level
- Caches speed up due to **temporal locality**: store data used recently
- Block size > 1 word speeds up due to **spatial locality**: store words adjacent to the ones used recently

Review (2/2)



- Cache design choices:
 - size of cache: speed v. capacity
 - direct-mapped v. associative
 - for N-way set assoc: choice of N
 - block replacement policy
 - 2nd level cache?
 - Write through v. write back?
- Use performance model to pick between choices, depending on programs, technology, budget, ...

Overview



- Generalized Caching
- Address Translation
- Virtual Memory / Page Tables
- TLBs
- Page Replacement Policy
- Multi-level Page Tables

Generalized Caching



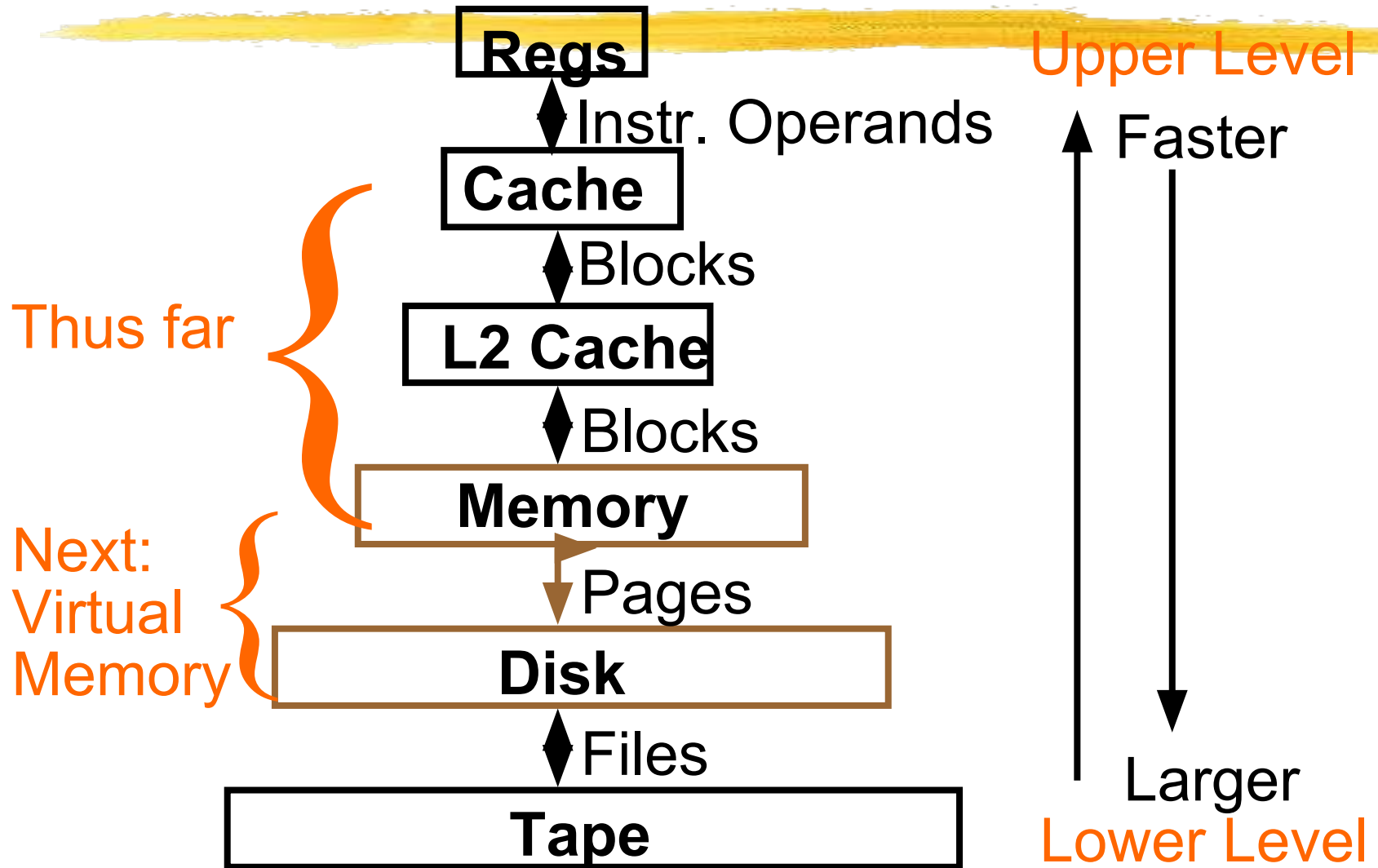
- We've discussed memory caching in detail. Caching in general shows up over and over in computer systems
 - File system cache
 - Web page cache
 - Software memorization
 - Others?
- General idea: if something is expensive but we want to do it repeatedly, just do it once and *cache* the result.

Generalized Caching



- For any caching system:
average access time =
$$\text{hit rate} * \text{hit time} + \text{miss rate} * \text{miss time}$$
- In general, hit time $\ll$ miss time so we seek to improve performance by reducing the miss rate. Miss rate is influenced by:
 - Cache size
 - Layout policy (e.g., associativity)
 - Replacement policy

Another View of the Memory Hierarchy



Memory Hierarchy Requirements



- If Principle of Locality allows caches to offer (close to) speed of cache memory with size of DRAM memory, then recursively why not use at next level to give speed of DRAM memory, size of Disk memory?
- In order to have the appearance of a large memory, we explore the relationship between main memory and hard disk.

Virtual Memory → Donanımın adaptif olmasını sağlar

With respect to large memory, not only do we want the entire virtual address space to appear to be main memory, but in most cases we would also like this complete space to appear to be available to each program that is executing. Aynı zamanda program ram'den büyük olsa da çalışır

The job of the software and hardware that implement the virtual memory is to map each **virtual address** for each program into a **physical address** in the main memory.

It is possible for a virtual address from one program and a virtual address from another program to map the same physical address.

Basic Issues in VM System Design

- size of information blocks that are transferred from secondary to main storage (M)

- block of information brought into M, and M is full, then some region of M must be released to make room for the new block --> *replacement policy*

↳ memory dolu diskteki verileri memory'ye taşıma algoritması

- which region of M is to hold the new block --> *placement policy*

↳ boş bölgeden başla

- missing item fetched from secondary memory only on the occurrence of a fault --> *demand load policy*

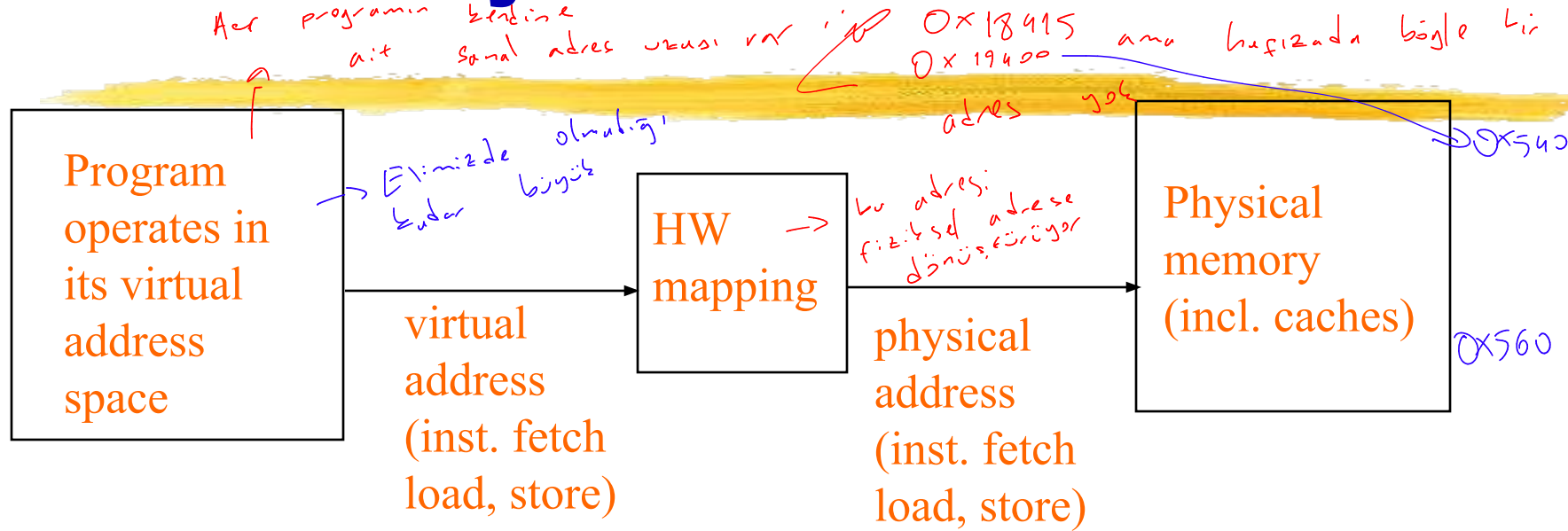
↳ ilk zaman maliyeti: hesaplaması

Memory Hierarchy Requirements



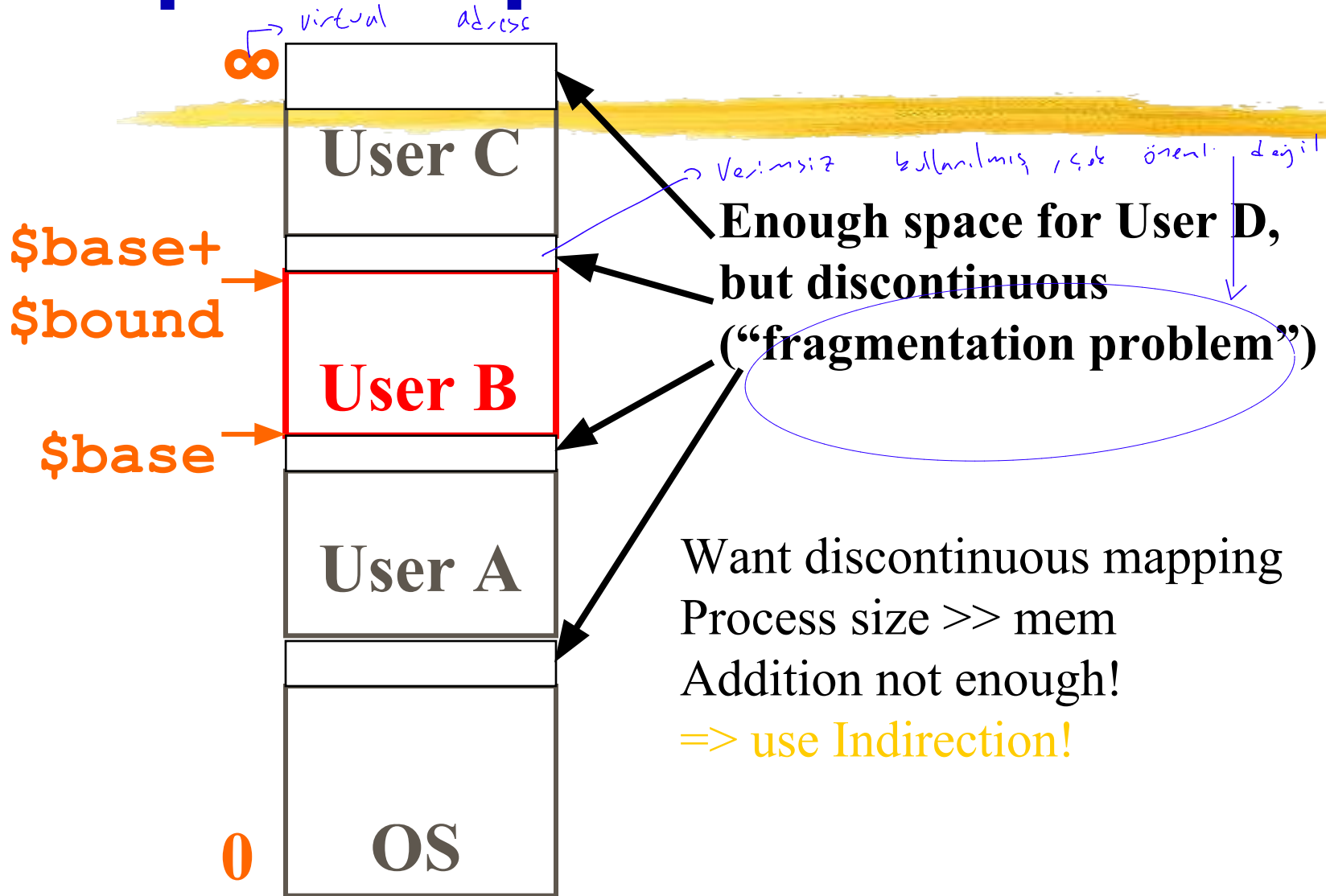
- Share memory between multiple processes but still provide protection – don't let one program read/write memory from another
- Address space – give each program the illusion that it has its own private memory
 - Suppose code starts at address 0x40000000. But different processes have different code, both residing at the same address. So each program has a different view of memory.

Virtual to Physical Addr. Translation



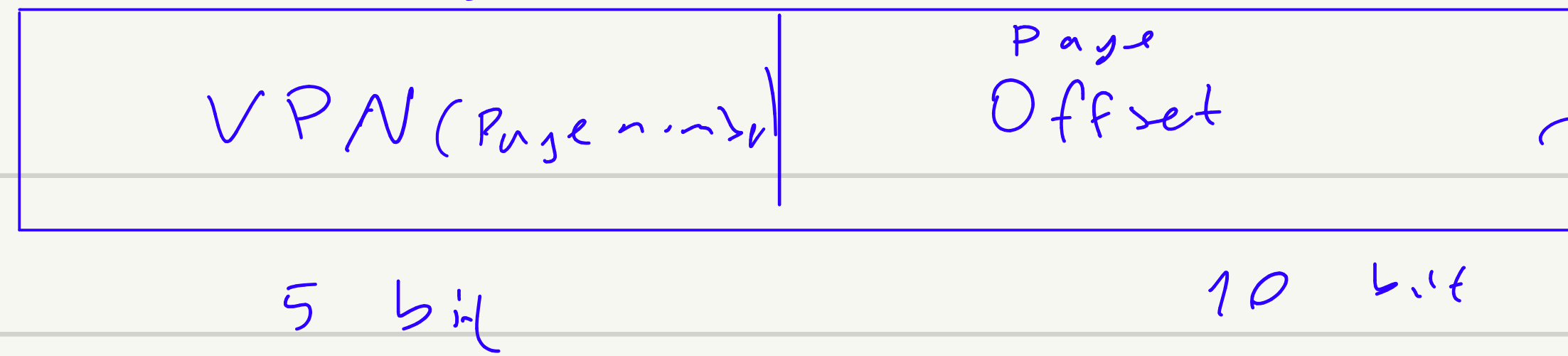
- Each program operates in its own virtual address space;
~only program running
- Each is protected from the other → Birbirine bilgi paylaşmak istemiyorsa
- OS can decide where each goes in memory
- Hardware (HW) provides virtual -> physical mapping

Simple Example: Base and Bound Reg



VA =

↓
virtual
address

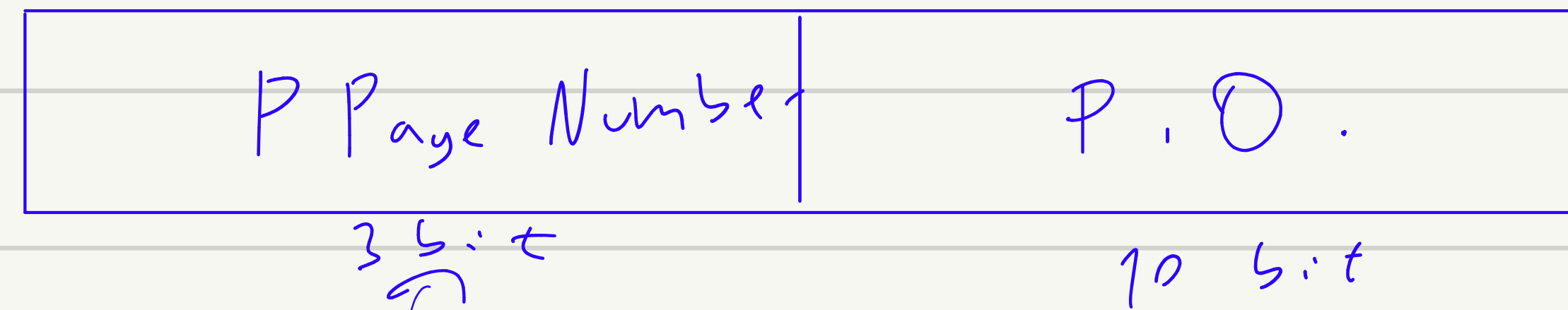


32 page

143

PA =

physical
address



8 Page ram

Page table

PPN	VPN
000	00000
100	00001

VA = 00001, 000000000010

PA = ?

100 0000000010

Uzunluğu VPN kadar burada 32

Virtual Memory Mapping Function

- Cannot have simple function to predict arbitrary mapping
- Use table lookup of mappings

Page Number	Offset
-------------	--------

Use table lookup ("Page Table") for mappings: Page number is index

Virtual Memory Mapping Function

Physical Offset = Virtual Offset

Physical Page Number

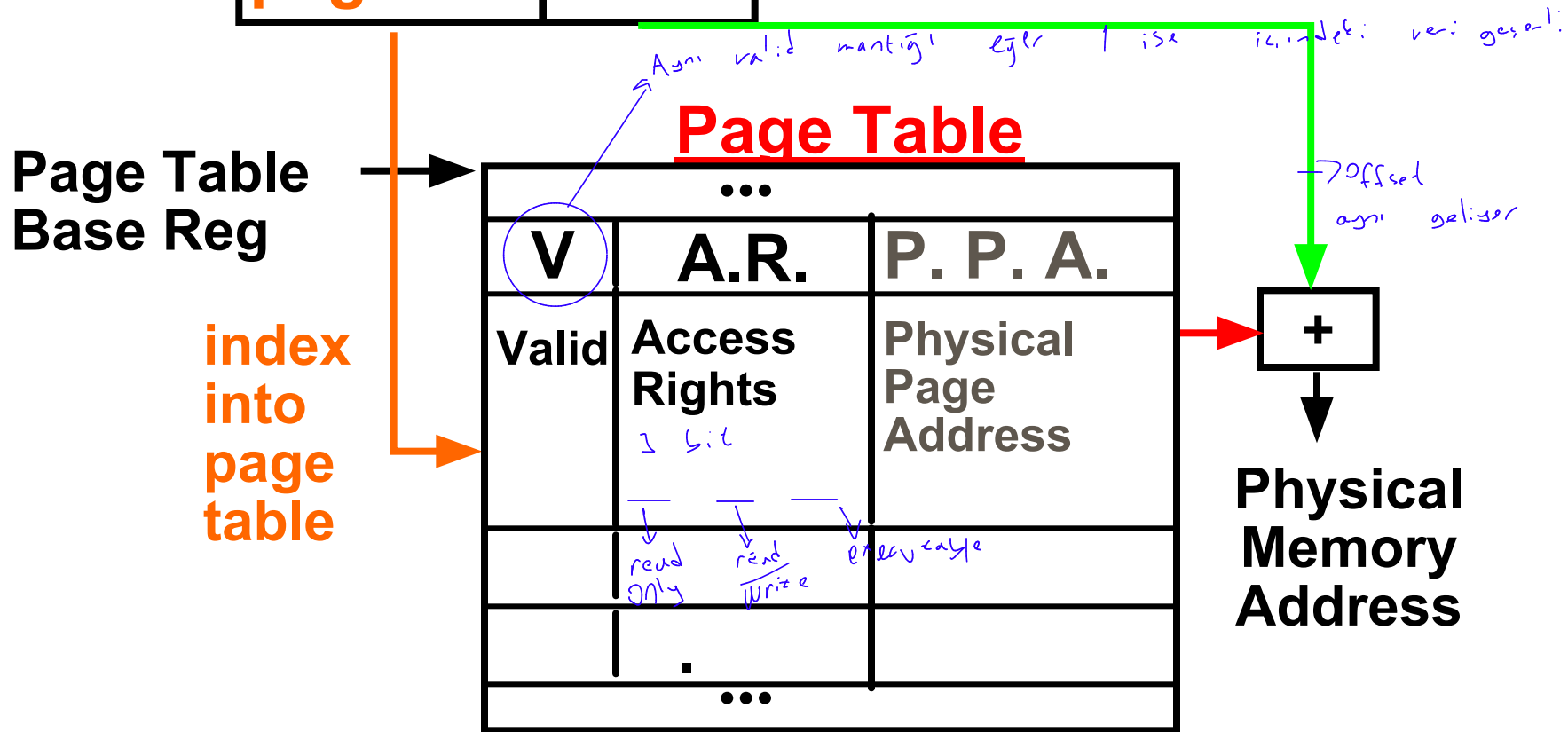
= Page Table [Virtual Page Number]

(P.P.N. also called "Page Frame")

Address Mapping: Page Table

Virtual Address:

page no. offset



Page Table located in physical memory

Page Table



- A page table is an operating system structure which contains the mapping of virtual addresses to physical locations
 - There are several different ways, all up to the operating system, to keep this data around
- Each process running in the operating system has its own page table
 - “State” of process is PC, all registers, plus page table
 - OS changes page tables by changing contents of Page Table Base Register

Requirements revisited

- Remember the motivation for VM:
- Sharing memory with protection
 - Different physical pages can be allocated to different processes (sharing)
 - A process can only touch pages in its own page table (protection)
- Separate address spaces
 - Since programs work only with virtual addresses, different programs can have different data/code at the same address!
- What about the memory hierarchy?

Page Table Entry (PTE) Format

- Contains either Physical Page Number or indication not in Main Memory
- OS maps to disk if Not Valid ($V = 0$)

Page Table

...		
V	A.R.	P. P.N.
Valid	Access Rights	Physical Page Number
V	A.R.	P. P. N.
...		

Diagram illustrating the Page Table Entry (PTE) format. The table is divided into three columns: V (Valid), A.R. (Access Rights), and P. P.N. (Physical Page Number). The middle row shows the expanded names: Valid, Access Rights, and Physical Page Number. Arrows point to the middle row from the label P.T.E. (Page Table Entry).

If valid, also check if have permission to use page: Access Rights (A.R.) may be Read Only, Read/Write, Executable

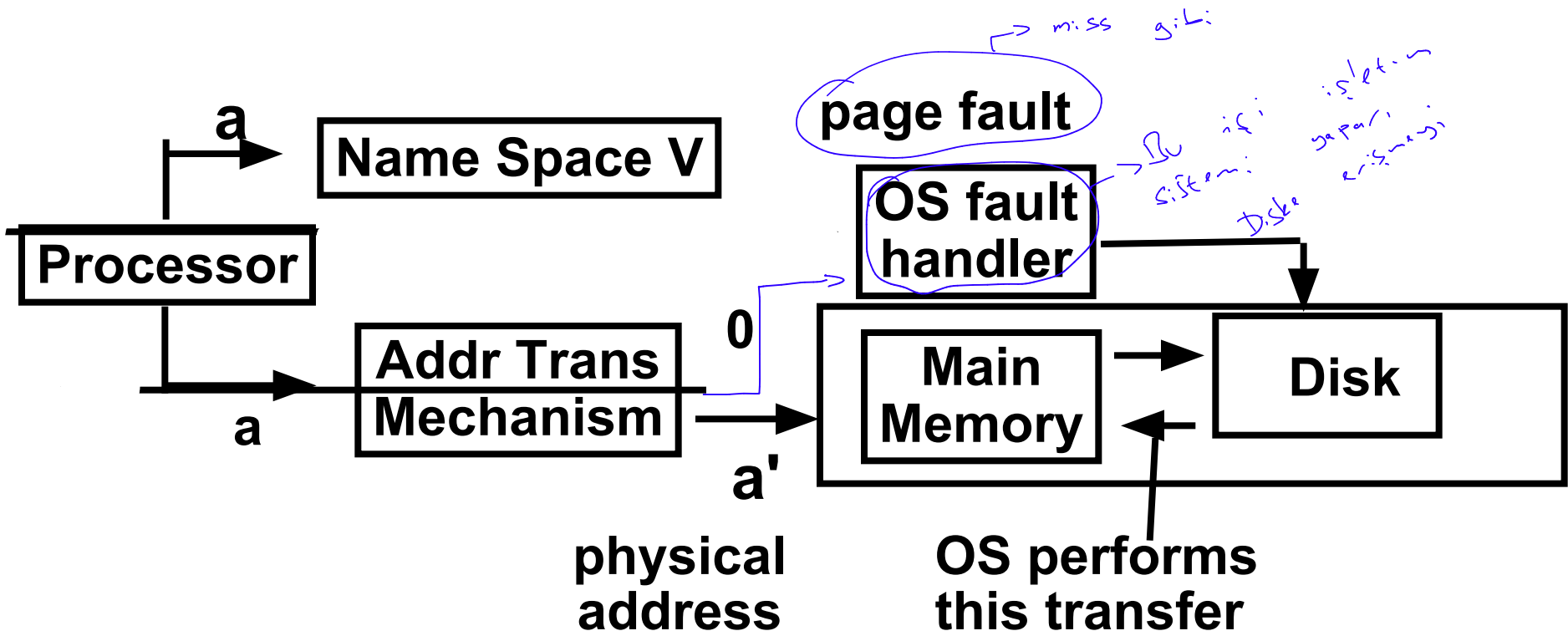
Address Map, Mathematically Speaking

$V = \{0, 1, \dots, n - 1\}$ virtual address space ($n > m$)

$M = \{0, 1, \dots, m - 1\}$ physical address space

MAP: $V \rightarrow M \cup \{\emptyset\}$ address mapping function

MAP(a) = a' if data at virtual address a is present in physical address a' and a' in M
= $\emptyset$ if data at virtual address a is not present in M



Comparing the 2 levels of hierarchy

Cache Version

Virtual Memory version

Block or Line Page

Miss Page Fault

Block Size: 32-64B Page Size: 4K-8KB

Placement: Fully Associative

Direct Mapped,
N-way Set Associative

Replacement: Least Recently Used
LRU or Random (LRU)

Write Thru or Back Write Back

Notes on Page Table

- Solves Fragmentation problem: all chunks same size, so all holes can be used
- OS must reserve "Swap Space" on disk for each process
- To grow a process, ask Operating System
 - If unused pages, OS uses them first
 - If not, OS swaps some old pages to disk
 - (Least Recently Used to pick pages to swap)
- Each process has own Page Table
- Will add details, but Page Table is essence of Virtual Memory

Translation Lookaside Buffer (TLBs)

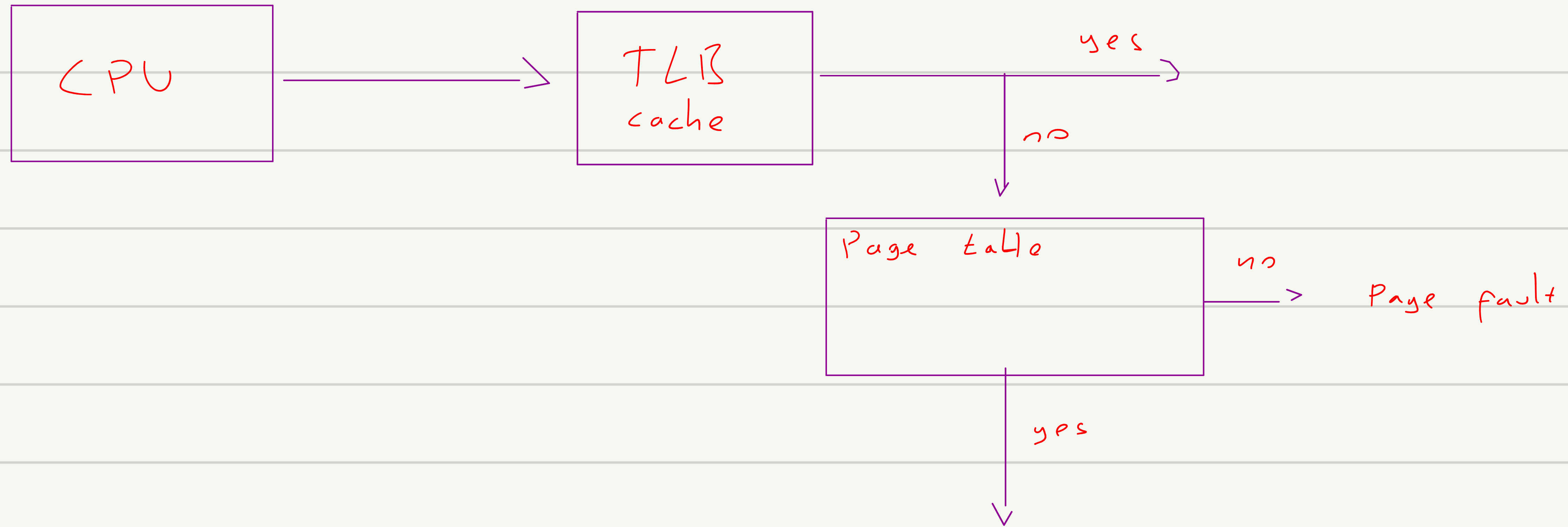
A way to speed up VM translation is to use a special cache of recently used page table entries -- this has many names, but the most frequently used is *Translation Lookaside Buffer* or *TLB*

Virtual Address	Physical Address	Dirty	Ref	Valid	Access

Virtual
page address

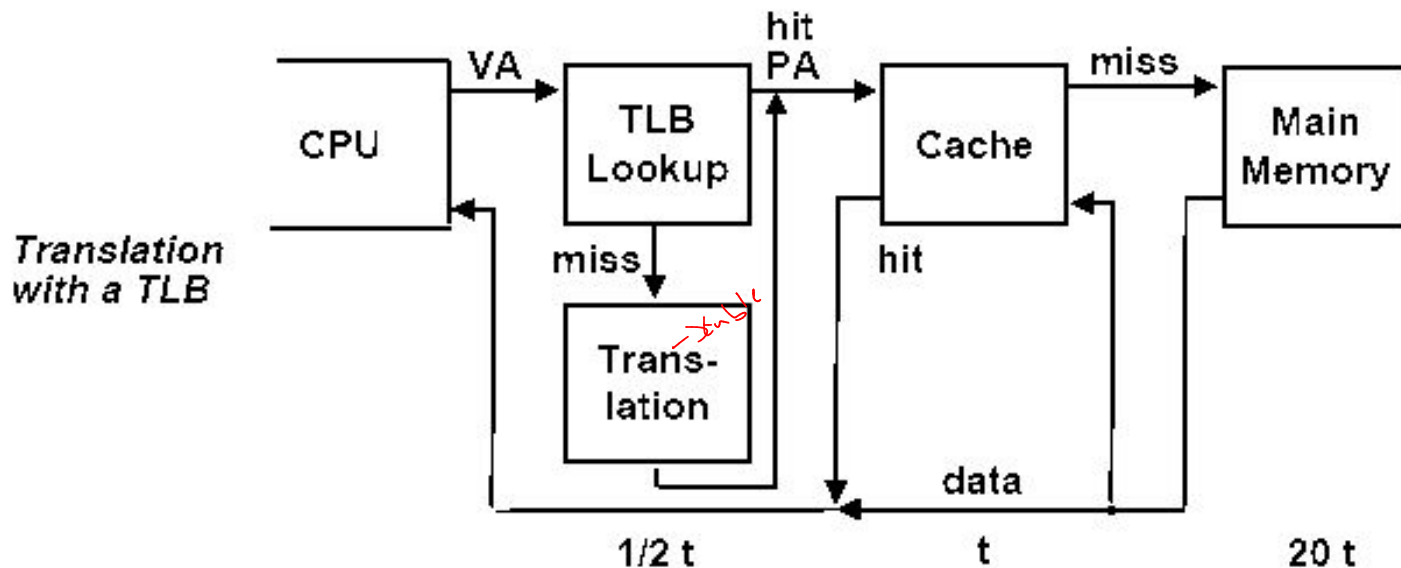
Dirty bit
Valid bit

Really just a cache on the page table mappings TLB access time comparable to cache access time (much less than main memory access time)



Translation Lookaside Buffer (TLBs)

Just like any other cache, the TLB can be organized as fully associative, set associative, or direct mapped. TLBs are usually small, typically not more than 128 - 256 entries even on high end machines. This permits fully associative lookup on these machines. Most mid-range machines use small n-way set associative organizations.



Example-1



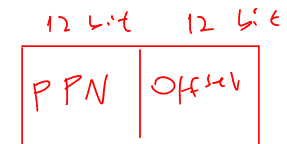
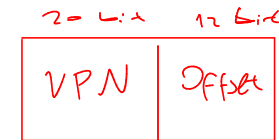
A virtual memory system uses 4KByte pages, 64-bit words, and a 48-bit virtual address. A particular program and its data require 4395 pages.

- a- What is the minimum number of page tables required?
- b-How many entries are there in the last page table.

Example-2

A small TLB has the following entries for a virtual page number of length 20 bits, a physical page number of 12 bits, and a page offset of 12 bits.

Valid-bit	Dirty-bit	Tag (Virtual Page Number)	Data (Physical Page Number)
1	1	01AF4	FFF
0	0	0E45F	E03
0	0	012FF	2F0
1	0	01A37	788
1	0	02BB4	45C
0	1	03CA0	657



The page number and offset are hexadecimal. For each of the virtual address listed, indicate a hit occurs and if it does, give the physical address : (a)

02BB4A65 (b) 0E45FB32 (c) 0D34E9DC (d) 03CA0777

45C A65

TLB miss

TLB miss

TLB miss

→ TLB miss → bu adan page tablosu
bulur not; önce valid
mi diye bakılır

Page Selection Policy



- **Demand paging:**

- Load page in response to access (page fault)
- Predominant selection policy

- **Pre-paging (prefetching):**

- Predict what pages will be accessed in near future
- Prefetch pages in advance of access
- Problems:
 - * Hard to predict accurately (trace cache)
 - * Mispredictions can cause useful pages to be replaced

- **Overlays**

- Application controls when pages loaded/replaced
- Only really relevant now for embedded/real-time systems

Page Replacement Policy



- **Random**
 - Works surprisingly well
- **FIFO (first in, first out)**
 - Throw out oldest pages
- **MIN (or OPTIMAL)**
 - Throw out page used farthest in the future
- **LRU (least recently used)**
 - Throw out page not used in longest time
- **NFU/Clock (not frequently used)**
 - Approximation to LRU _ do not throw out recently used pages

FIFO Page Replacement

FIFO: replace oldest page (first loaded)

Example:

- Memory system with three pages _ all initially free
- Reference string: A B C A B D A D B C B

	A	B	C	A	B	D	A	D	B	C	B
Frame1	(A)					(D)				(C)	
Frame2		(B)					(A)				
Frame3			(C)						(B)		

Handwritten notes: Above the reference string, arrows point to each letter with labels: miss, miss, miss, hit, hit, miss, miss, hit, miss, miss, hit. A bracket on the left labeled 'faults' spans the first three rows of the table.

Result: 7 page faults

Optimal Page Replacement

Optimal: replace page used farthest in future

– If several equal candidates, select one at random

Example:

– Memory system with three pages _ all initially free

– Reference string: A B C A B D A D B C B

	A	B	C	A	B	D	A	D	B	C	B
Frame1	A									C	
Frame2		B								↕	
Frame3			C			D				C	

Result: 5 page faults

LRU Page Replacement

Geçmişte
uzak olan

LRU: replace page used farthest in past

Example:

- Memory system with three pages _ all initially free
- Reference string: A B C A B D A D B C B

	A	B	C	A	B	D	A	D	B	C	B
Frame1	A									C	
Frame2		B									
Frame3			C			D					

Result: 5 page faults

Example-1

In a paged system, suppose the following pages are requested in the order shown;

12, 14, 2, 34, 56, 23, 14, 56, 34, 12

and the main memory partition can only hold four pages at any instant (in practice usually many more pages can be held). List the pages in the main memory after each page is transferred using each of the following replacement algorithms:

- (a) First-in-first-out replacement algorithm
- (b) Least recently used replacement algorithm
- (c) Clock replacement algorithm

Example-2

Using one use bit with each page entry, list the pages recorded in the following sequence and hence determine the pages removed using one use bit approximation to the least recently used algorithm:

13, 47, 13, 99, 47, 35, 13, 67, 47, 13, 34, 35, 99, 99, 14, 14, 47, 67

given that there are four pages in the memory partition. The use bits are only read at page fault time, and then scanned in the same order.

Suppose two use bits are provided. The first use bit is set when the page is first referenced. The second use bit is set when the page is referenced again. Deduce an algorithm to remove pages from the partition, and list the pages.

Things to Remember



- Apply Principle of Locality Recursively
- Manage memory to disk? Treat as cache
 - Included protection as bonus, now critical
 - Use Page Table of mappings vs. tag/data in cache
- Virtual Memory allows protected sharing of memory between processes with less swapping to disk, less fragmentation than always swap or base/bound